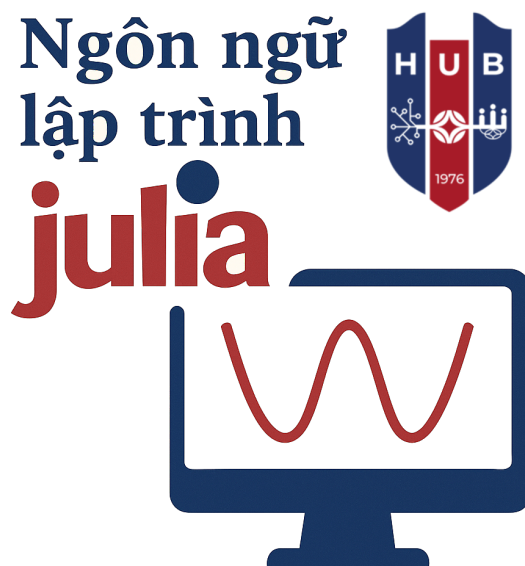


TRƯỜNG ĐẠI HỌC NGÂN HÀNG TP. HỒ CHÍ MINH

# Ngôn ngữ lập trình Julia

Sách tham khảo



Chủ biên: TS. Nguyễn Hoài Đức

Thành phố Hồ Chí Minh, 2025



# Mục lục

|          |                                           |          |
|----------|-------------------------------------------|----------|
| <b>I</b> | <b>Phần I — Nền tảng Julia</b>            | <b>1</b> |
| <b>1</b> | <b>Giới thiệu Julia và hệ sinh thái</b>   | <b>3</b> |
| 1.1      | Lịch sử và triết lý của Julia             | 3        |
| 1.1.1    | Lịch sử phát triển                        | 3        |
| 1.1.2    | Triết lý thiết kế                         | 3        |
| 1.1.3    | Thành tựu nổi bật                         | 4        |
| 1.1.4    | Bối cảnh ra đời: Vấn đề hai ngôn ngữ      | 4        |
| 1.1.5    | Tầm nhìn của các nhà sáng lập             | 4        |
| 1.1.6    | Triết lý thiết kế: Tái sử dụng code       | 5        |
| 1.1.7    | Timeline phát triển                       | 5        |
| 1.1.8    | Hệ sinh thái và cộng đồng                 | 6        |
| 1.1.9    | Ai đang sử dụng Julia?                    | 6        |
| 1.1.10   | Tại sao Julia quan trọng cho sinh viên?   | 7        |
| 1.2      | Cài đặt                                   | 7        |
| 1.2.1    | Cài đặt với juliaup (khuyến dùng)         | 7        |
| 1.2.2    | Cài đặt thủ công                          | 8        |
| 1.2.3    | Cài đặt với Docker                        | 9        |
| 1.2.4    | Kiểm tra cài đặt                          | 9        |
| 1.2.5    | Xử lý lỗi thường gặp                      | 10       |
| 1.3      | IDE và môi trường phát triển              | 11       |
| 1.3.1    | VS Code + Julia Extension (Khuyến dùng)   | 11       |
| 1.3.2    | Jupyter Notebook với IJulia               | 13       |
| 1.3.3    | Pluto.jl — Reactive Notebooks             | 14       |
| 1.3.4    | REPL — Giao diện dòng lệnh tương tác      | 15       |
| 1.3.5    | Các IDE và Editor khác                    | 16       |
| 1.4      | Package Manager                           | 17       |
| 1.4.1    | Quản lý environments                      | 17       |
| 1.4.2    | Các lệnh cơ bản                           | 17       |
| 1.4.3    | Sử dụng trong code                        | 17       |
| 1.4.4    | Registry và packages phổ biến             | 17       |
| 1.4.5    | Pkg REPL Mode chi tiết                    | 18       |
| 1.4.6    | Project.toml và Manifest.toml             | 18       |
| 1.4.7    | Environments và Isolation                 | 19       |
| 1.4.8    | Registry và Private Packages              | 20       |
| 1.4.9    | Ví dụ thực tế: Setup Data Science Project | 21       |
| 1.4.10   | Best Practices cho Package Management     | 23       |
| 1.5      | So sánh với các ngôn ngữ khác             | 23       |

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| 1.5.1    | Julia vs Python . . . . .                                 | 23        |
| 1.5.2    | Julia vs R . . . . .                                      | 23        |
| 1.5.3    | Julia vs MATLAB . . . . .                                 | 24        |
| 1.5.4    | Benchmark performance . . . . .                           | 24        |
| 1.6      | Tóm tắt chương . . . . .                                  | 24        |
| 1.7      | Bài tập . . . . .                                         | 24        |
| 1.7.1    | Bài tập cơ bản (*) . . . . .                              | 24        |
| 1.7.2    | Bài tập trung bình (**) . . . . .                         | 25        |
| 1.7.3    | Bài tập nâng cao (***) . . . . .                          | 26        |
| <b>2</b> | <b>Cú pháp cơ bản</b>                                     | <b>27</b> |
| 2.1      | Kiểu dữ liệu cơ bản . . . . .                             | 27        |
| 2.1.1    | Kiểu số nguyên (Integer Types) . . . . .                  | 27        |
| 2.1.2    | Kiểu số thực (Floating-Point Types) . . . . .             | 28        |
| 2.1.3    | Kiểu Boolean . . . . .                                    | 29        |
| 2.1.4    | Kiểu ký tự (Char) và Unicode . . . . .                    | 30        |
| 2.1.5    | Kiểu chuỗi (String) . . . . .                             | 30        |
| 2.1.6    | Hệ thống phân cấp kiểu (Type Hierarchy) . . . . .         | 31        |
| 2.1.7    | So sánh kiểu dữ liệu với Python và C++ . . . . .          | 32        |
| 2.1.8    | Các kiểu dữ liệu khác . . . . .                           | 33        |
| 2.1.9    | Chuyển đổi kiểu (Type Conversion) . . . . .               | 33        |
| 2.2      | Biến và phạm vi (Scope) . . . . .                         | 34        |
| 2.2.1    | Quy tắc đặt tên biến (Naming Conventions) . . . . .       | 34        |
| 2.2.2    | Soft Scope vs Hard Scope . . . . .                        | 35        |
| 2.2.3    | Các construct KHÔNG tạo scope mới . . . . .               | 37        |
| 2.2.4    | Vấn đề hiệu năng với Global Variables . . . . .           | 37        |
| 2.2.5    | Biến hằng (const) . . . . .                               | 38        |
| 2.2.6    | Từ khóa global . . . . .                                  | 38        |
| 2.2.7    | Let blocks . . . . .                                      | 39        |
| 2.2.8    | Scope rules trong loops . . . . .                         | 39        |
| 2.2.9    | Variable Binding và Shadowing . . . . .                   | 39        |
| 2.2.10   | Common Pitfalls và Best Practices . . . . .               | 40        |
| 2.2.11   | Debugging Scope Issues . . . . .                          | 42        |
| 2.3      | Toán tử, biểu thức và chuỗi format . . . . .              | 43        |
| 2.3.1    | Broadcasting (toán tử chấm .) . . . . .                   | 43        |
| 2.3.2    | Thứ tự ưu tiên toán tử (Operator Precedence) . . . . .    | 43        |
| 2.3.3    | Các toán tử số học (Arithmetic Operators) . . . . .       | 44        |
| 2.3.4    | Toán tử so sánh và logic (Comparison & Boolean) . . . . . | 45        |
| 2.3.5    | Xử lý chuỗi (String Operations) . . . . .                 | 45        |
| 2.3.6    | Biểu thức chính quy (Regular Expressions) . . . . .       | 47        |
| 2.3.7    | Định dạng chuỗi với Printf . . . . .                      | 48        |
| 2.4      | Tập tin, thư viện và module . . . . .                     | 49        |
| 2.4.1    | Import vs Using . . . . .                                 | 49        |
| 2.4.2    | Include files . . . . .                                   | 50        |
| 2.4.3    | Đọc file nâng cao . . . . .                               | 50        |
| 2.4.4    | Module namespacing . . . . .                              | 51        |

|           |                                                          |           |
|-----------|----------------------------------------------------------|-----------|
| 2.5       | Tóm tắt chương . . . . .                                 | 51        |
| 2.6       | Bài tập . . . . .                                        | 51        |
| 2.6.1     | Bài tập cơ bản (*) . . . . .                             | 51        |
| 2.6.2     | Bài tập trung bình (**) . . . . .                        | 52        |
| 2.6.3     | Bài tập nâng cao (***) . . . . .                         | 53        |
| <b>3</b>  | <b>Cấu trúc điều khiển và hàm</b>                        | <b>55</b> |
| 3.1       | Cấu trúc điều khiển . . . . .                            | 55        |
| 3.1.1     | Câu lệnh điều kiện if-elseif-else . . . . .              | 55        |
| 3.1.2     | Vòng lặp for . . . . .                                   | 57        |
| 3.1.3     | Vòng lặp while . . . . .                                 | 59        |
| 3.1.4     | break và continue . . . . .                              | 60        |
| 3.1.5     | Comprehensions . . . . .                                 | 60        |
| 3.1.6     | Xử lý ngoại lệ với try-catch . . . . .                   | 63        |
| 3.2       | Hàm và multiple dispatch . . . . .                       | 66        |
| 3.2.1     | Định nghĩa hàm . . . . .                                 | 66        |
| 3.2.2     | Multiple dispatch . . . . .                              | 66        |
| 3.2.3     | Optional arguments và keyword arguments . . . . .        | 67        |
| 3.2.4     | Varargs (Variable arguments) . . . . .                   | 67        |
| 3.2.5     | Do blocks . . . . .                                      | 67        |
| 3.2.6     | Return values và multiple returns . . . . .              | 68        |
| 3.2.7     | Type annotations trong functions . . . . .               | 68        |
| 3.2.8     | Splat và Slurp operators . . . . .                       | 69        |
| 3.2.9     | methods() và function introspection . . . . .            | 70        |
| 3.2.10    | Method selection algorithm . . . . .                     | 71        |
| 3.2.11    | Ambiguity resolution . . . . .                           | 72        |
| 3.2.12    | So sánh: Multiple dispatch vs OOP Polymorphism . . . . . | 73        |
| 3.3       | Closures . . . . .                                       | 74        |
| 3.3.1     | Closure cơ bản . . . . .                                 | 74        |
| 3.3.2     | Closure với mutable state . . . . .                      | 74        |
| 3.3.3     | Ứng dụng thực tế . . . . .                               | 74        |
| 3.3.4     | Performance considerations . . . . .                     | 75        |
| 3.4       | Bài tập . . . . .                                        | 75        |
| 3.4.1     | Bài tập cơ bản (*) . . . . .                             | 75        |
| 3.4.2     | Bài tập trung bình (**) . . . . .                        | 76        |
| 3.4.3     | Bài tập nâng cao (***) . . . . .                         | 78        |
| <b>II</b> | <b>Phần II — Dữ liệu và cấu trúc nâng cao</b>            | <b>81</b> |
| <b>4</b>  | <b>Cấu trúc dữ liệu và kiểu tùy chỉnh</b>                | <b>83</b> |
| 4.1       | Mảng, Tuple, Dict, Set . . . . .                         | 83        |
| 4.1.1     | Mảng (Arrays) . . . . .                                  | 83        |
| 4.1.2     | Tuple . . . . .                                          | 89        |
| 4.1.3     | Dictionary (Dict) . . . . .                              | 93        |
| 4.1.4     | Set . . . . .                                            | 100       |
| 4.1.5     | So sánh và lựa chọn . . . . .                            | 106       |

|          |                                                       |            |
|----------|-------------------------------------------------------|------------|
| 4.2      | Struct và Mutable struct . . . . .                    | 106        |
| 4.2.1    | Struct cơ bản (Immutable struct) . . . . .            | 106        |
| 4.2.2    | Mutable struct . . . . .                              | 107        |
| 4.2.3    | Constructors (Hàm khởi tạo) . . . . .                 | 107        |
| 4.2.4    | Type annotations và inspection . . . . .              | 110        |
| 4.2.5    | Nested structs . . . . .                              | 112        |
| 4.2.6    | So sánh struct vs mutable struct . . . . .            | 113        |
| 4.2.7    | So sánh với Python classes . . . . .                  | 113        |
| 4.2.8    | Struct performance và memory layout . . . . .         | 114        |
| 4.2.9    | Struct patterns và use cases . . . . .                | 116        |
| 4.3      | Abstract và Parametric type . . . . .                 | 117        |
| 4.3.1    | Abstract types (Kiểu trừu tượng) . . . . .            | 117        |
| 4.3.2    | Type hierarchy trong Julia . . . . .                  | 118        |
| 4.3.3    | Parametric types (Kiểu tham số hóa) . . . . .         | 118        |
| 4.3.4    | Type constraints (Ràng buộc kiểu) . . . . .           | 119        |
| 4.3.5    | UnionAll types . . . . .                              | 119        |
| 4.3.6    | Use cases thực tế . . . . .                           | 120        |
| 4.3.7    | So sánh: Concrete vs Abstract vs Parametric . . . . . | 121        |
| 4.3.8    | Best practices . . . . .                              | 121        |
| 4.3.9    | Mệnh đề where chi tiết . . . . .                      | 122        |
| 4.3.10   | Type invariance vs covariance . . . . .               | 123        |
| 4.3.11   | Parametric types với Collections . . . . .            | 125        |
| 4.3.12   | @code_warntype và Type Stability Analysis . . . . .   | 126        |
| 4.3.13   | Performance patterns với Parametric types . . . . .   | 128        |
| 4.3.14   | Common pitfalls và anti-patterns . . . . .            | 130        |
| 4.3.15   | Tóm tắt . . . . .                                     | 131        |
| 4.4      | Bài tập . . . . .                                     | 132        |
| 4.4.1    | Bài tập cơ bản (*) . . . . .                          | 132        |
| 4.4.2    | Bài tập trung bình (**) . . . . .                     | 133        |
| 4.4.3    | Bài tập nâng cao (***) . . . . .                      | 134        |
| <b>5</b> | <b>Lập trình hàm và đa phương thức</b>                | <b>135</b> |
| 5.1      | Multiple dispatch . . . . .                           | 135        |
| 5.1.1    | Định nghĩa và so sánh . . . . .                       | 135        |
| 5.1.2    | Ví dụ cơ bản . . . . .                                | 135        |
| 5.1.3    | Ví dụ thực tế với hình học . . . . .                  | 136        |
| 5.1.4    | Cách Julia chọn phương thức . . . . .                 | 137        |
| 5.1.5    | Tại sao multiple dispatch hữu ích? . . . . .          | 137        |
| 5.1.6    | So sánh Multiple Dispatch vs OOP . . . . .            | 137        |
| 5.1.7    | Best practices và pitfalls . . . . .                  | 138        |
| 5.1.8    | Method tables và cơ chế nội bộ . . . . .              | 138        |
| 5.1.9    | Specialization và JIT compilation . . . . .           | 140        |
| 5.1.10   | Type inference trong dispatch . . . . .               | 142        |
| 5.1.11   | Design patterns với multiple dispatch . . . . .       | 143        |
| 5.1.12   | So sánh với Visitor Pattern (OOP) . . . . .           | 146        |
| 5.1.13   | Method introspection . . . . .                        | 147        |

|          |                                           |            |
|----------|-------------------------------------------|------------|
| 5.2      | Overloading, generic programming          | 148        |
| 5.2.1    | Function overloading vs Multiple dispatch | 148        |
| 5.2.2    | Generic programming với Abstract types    | 149        |
| 5.2.3    | Parametric types (Kiểu tham số)           | 150        |
| 5.2.4    | Where clause và type constraints          | 151        |
| 5.2.5    | Extending Base functions                  | 151        |
| 5.2.6    | Type inference và performance             | 153        |
| 5.2.7    | Best practices và pitfalls                | 153        |
| 5.2.8    | Duck typing trong Julia                   | 155        |
| 5.2.9    | Informal interfaces (Giao diện ngầm định) | 156        |
| 5.2.10   | Traits pattern                            | 157        |
| 5.2.11   | Holy traits (Tim Holy's trick)            | 159        |
| 5.2.12   | Custom number types — Ví dụ thực tế       | 162        |
| 5.3      | Lambda, map/filter/reduce                 | 165        |
| 5.3.1    | Lambda và hàm ẩn danh                     | 165        |
| 5.3.2    | Closures - Hàm đóng                       | 165        |
| 5.3.3    | map, filter, reduce                       | 172        |
| 5.3.4    | mapreduce - Tối ưu hóa hiệu năng          | 173        |
| 5.3.5    | Array comprehensions - Biểu thức tạo mảng | 173        |
| 5.3.6    | Generator expressions - Biểu thức sinh    | 173        |
| 5.3.7    | Lazy vs Eager evaluation                  | 173        |
| 5.3.8    | Pipeline operator và do blocks            | 174        |
| 5.3.9    | @chain macro - Pipeline nâng cao          | 175        |
| 5.3.10   | Iterators.jl - Lazy operations            | 175        |
| 5.3.11   | So sánh với Python functional tools       | 177        |
| 5.3.12   | Best practices                            | 179        |
| 5.4      | Tóm tắt chương                            | 179        |
| 5.5      | Bài tập                                   | 179        |
| 5.5.1    | Bài tập cơ bản (*)                        | 179        |
| 5.5.2    | Bài tập trung bình (**)                   | 181        |
| 5.5.3    | Bài tập nâng cao (***)                    | 182        |
| <b>6</b> | <b>Gói và module</b>                      | <b>185</b> |
| 6.1      | Modules và tổ chức code                   | 185        |
| 6.1.1    | Module là gì?                             | 185        |
| 6.1.2    | Export và public API                      | 186        |
| 6.1.3    | Local modules vs registered packages      | 186        |
| 6.1.4    | Submodules                                | 187        |
| 6.1.5    | Relative imports trong submodules         | 187        |
| 6.1.6    | Re-exporting từ modules khác              | 189        |
| 6.1.7    | Module constants và globals pattern       | 190        |
| 6.1.8    | Import aliasing và selective imports      | 192        |
| 6.1.9    | Module introspection                      | 194        |
| 6.2      | Using vs Import                           | 195        |
| 6.2.1    | using statement                           | 195        |
| 6.2.2    | import statement                          | 196        |

|       |                                                   |     |
|-------|---------------------------------------------------|-----|
| 6.2.3 | Extending functions từ modules                    | 196 |
| 6.2.4 | Comparison table: using vs import                 | 197 |
| 6.3   | Module lifecycle và precompilation                | 197 |
| 6.3.1 | Module initialization với <code>__init__()</code> | 197 |
| 6.3.2 | Precompilation                                    | 199 |
| 6.3.3 | Best practices cho module lifecycle               | 202 |
| 6.4   | Package structure                                 | 202 |
| 6.4.1 | Package filesystem layout                         | 202 |
| 6.4.2 | Project.toml — Package metadata                   | 203 |
| 6.4.3 | File organization best practices                  | 203 |
| 6.4.4 | PkgTemplates.jl — Tự động tạo package             | 204 |
| 6.4.5 | Semantic Versioning và Compat                     | 206 |
| 6.4.6 | Documenter.jl — Documentation Generator           | 207 |
| 6.4.7 | CI/CD với GitHub Actions                          | 208 |
| 6.4.8 | Publishing package to General Registry            | 209 |
| 6.5   | Quản lý môi trường (Environments)                 | 210 |
| 6.5.1 | Environments là gì?                               | 210 |
| 6.5.2 | Project.toml vs Manifest.toml                     | 211 |
| 6.5.3 | Working với environments                          | 211 |
| 6.5.4 | Reproducibility workflow                          | 212 |
| 6.5.5 | Global environment vs Project environments        | 213 |
| 6.5.6 | Stacked Environments và <code>LOAD_PATH</code>    | 213 |
| 6.5.7 | Manifest.toml Deep Dive                           | 214 |
| 6.5.8 | Advanced Reproducibility Practices                | 215 |
| 6.5.9 | Troubleshooting Environments                      | 216 |
| 6.6   | Tạo package với PkgTemplates                      | 217 |
| 6.6.1 | PkgTemplates workflow                             | 217 |
| 6.6.2 | Generated package structure                       | 218 |
| 6.6.3 | Develop package locally                           | 219 |
| 6.7   | Semantic Versioning và Compatibility              | 219 |
| 6.7.1 | Semantic Versioning basics                        | 219 |
| 6.7.2 | 0.x versions (Pre-release)                        | 220 |
| 6.7.3 | [compat] section chi tiết                         | 220 |
| 6.7.4 | Version bounds best practices                     | 220 |
| 6.7.5 | Breaking changes và deprecation                   | 221 |
| 6.8   | Documentation với Documenter.jl                   | 221 |
| 6.8.1 | Setup Documenter.jl                               | 222 |
| 6.8.2 | Writing documentation                             | 222 |
| 6.8.3 | Docstrings for Documenter                         | 223 |
| 6.8.4 | Building documentation locally                    | 224 |
| 6.8.5 | Doctest — Testing documentation examples          | 225 |
| 6.9   | CI/CD với GitHub Actions                          | 225 |
| 6.9.1 | Basic CI workflow                                 | 225 |
| 6.9.2 | Documentation deployment                          | 226 |
| 6.9.3 | Code coverage                                     | 227 |
| 6.9.4 | TagBot — Automatic releases                       | 227 |



|          |                                         |            |
|----------|-----------------------------------------|------------|
| 6.9.5    | CompatHelper — Automatic compat updates | 228        |
| 6.10     | Publishing Package to General Registry  | 228        |
| 6.10.1   | Pre-registration checklist              | 228        |
| 6.10.2   | Registrator workflow                    | 229        |
| 6.10.3   | Version bumping workflow                | 229        |
| 6.10.4   | Package naming conventions              | 230        |
| 6.11     | Testing Best Practices                  | 230        |
| 6.11.1   | Test file organization                  | 230        |
| 6.11.2   | Test macros                             | 231        |
| 6.11.3   | Property-based testing                  | 231        |
| 6.11.4   | Test fixtures và setup                  | 232        |
| 6.11.5   | Running tests                           | 232        |
| 6.12     | Best practices                          | 232        |
| 6.12.1   | Module và package organization          | 232        |
| 6.12.2   | Environment management                  | 233        |
| 6.12.3   | Version control                         | 233        |
| 6.12.4   | Documentation                           | 234        |
| 6.12.5   | Tóm tắt                                 | 234        |
| 6.13     | Bài tập                                 | 235        |
| 6.13.1   | Bài tập cơ bản (*)                      | 235        |
| 6.13.2   | Bài tập trung bình (**)                 | 237        |
| 6.13.3   | Bài tập nâng cao (***)                  | 239        |
| <b>7</b> | <b>I/O và xử lý file</b>                | <b>241</b> |
| 7.1      | Standard I/O và REPL                    | 241        |
| 7.1.1    | Standard input và output                | 241        |
| 7.1.2    | Các hàm output cơ bản                   | 241        |
| 7.1.3    | String interpolation                    | 242        |
| 7.1.4    | Input từ user                           | 243        |
| 7.2      | File operations cơ bản                  | 243        |
| 7.2.1    | Mở và đóng files                        | 243        |
| 7.2.2    | Do-block pattern (Recommended)          | 244        |
| 7.2.3    | Reading files                           | 244        |
| 7.2.4    | Writing files                           | 245        |
| 7.2.5    | File system operations                  | 245        |
| 7.3      | CSV và DataFrames                       | 246        |
| 7.3.1    | CSV.jl - Đọc và ghi CSV files           | 246        |
| 7.3.2    | DataFrames.jl cơ bản                    | 246        |
| 7.3.3    | Data manipulation với DataFrames        | 247        |
| 7.3.4    | Missing values                          | 247        |
| 7.4      | JSON processing                         | 248        |
| 7.4.1    | JSON3.jl - High-performance JSON        | 248        |
| 7.4.2    | Converting Julia objects to JSON        | 248        |
| 7.4.3    | Structured JSON với StructTypes         | 249        |
| 7.5      | Serialization với JLD2                  | 249        |
| 7.5.1    | JLD2.jl - Julia Data Format             | 249        |

|          |                                                     |            |
|----------|-----------------------------------------------------|------------|
| 7.5.2    | JLD2 với custom types . . . . .                     | 250        |
| 7.6      | Database connections . . . . .                      | 250        |
| 7.6.1    | SQLite.jl - Embedded database . . . . .             | 250        |
| 7.6.2    | LibPQ.jl - PostgreSQL . . . . .                     | 251        |
| 7.6.3    | DBInterface - Unified database interface . . . . .  | 251        |
| 7.7      | Các format khác . . . . .                           | 252        |
| 7.7.1    | Excel files với XLSX.jl . . . . .                   | 252        |
| 7.7.2    | Parquet files với Parquet.jl . . . . .              | 252        |
| 7.7.3    | Arrow format . . . . .                              | 252        |
| 7.8      | Best practices . . . . .                            | 253        |
| 7.8.1    | Error handling cho I/O . . . . .                    | 253        |
| 7.8.2    | Working with large files . . . . .                  | 253        |
| 7.8.3    | Temporary files . . . . .                           | 253        |
| 7.8.4    | Tóm tắt . . . . .                                   | 254        |
| 7.9      | Bài tập . . . . .                                   | 254        |
| 7.9.1    | Bài tập cơ bản . . . . .                            | 254        |
| 7.9.2    | Bài tập nâng cao . . . . .                          | 254        |
| 7.9.3    | Bài tập thách thức . . . . .                        | 254        |
| <b>8</b> | <b>Metaprogramming</b>                              | <b>257</b> |
| 8.1      | Giới thiệu Metaprogramming . . . . .                | 257        |
| 8.1.1    | Tại sao cần Metaprogramming? . . . . .              | 257        |
| 8.2      | Expressions và Abstract Syntax Tree (AST) . . . . . | 258        |
| 8.2.1    | Expr - Kiểu dữ liệu Expression . . . . .            | 258        |
| 8.2.2    | Visualizing AST với dump() . . . . .                | 258        |
| 8.2.3    | Symbols . . . . .                                   | 259        |
| 8.3      | Quoting: Tạo Expressions . . . . .                  | 259        |
| 8.3.1    | Cú pháp :() . . . . .                               | 259        |
| 8.3.2    | Quote blocks . . . . .                              | 260        |
| 8.3.3    | Meta.parse() - Parse từ String . . . . .            | 260        |
| 8.4      | Interpolation với \$ . . . . .                      | 260        |
| 8.4.1    | Interpolation cơ bản . . . . .                      | 260        |
| 8.4.2    | Interpolate Expressions . . . . .                   | 261        |
| 8.4.3    | Interpolation nhiều cấp . . . . .                   | 261        |
| 8.5      | eval() và Runtime Code Generation . . . . .         | 261        |
| 8.5.1    | Sử dụng eval() . . . . .                            | 261        |
| 8.5.2    | @eval macro . . . . .                               | 262        |
| 8.5.3    | Lưu ý về eval() . . . . .                           | 262        |
| 8.6      | Macros . . . . .                                    | 262        |
| 8.6.1    | Định nghĩa Macro cơ bản . . . . .                   | 262        |
| 8.6.2    | Macro vs Function . . . . .                         | 263        |
| 8.6.3    | Xem Macro Expansion với @macroexpand . . . . .      | 263        |
| 8.6.4    | Macro Hygiene . . . . .                             | 263        |
| 8.6.5    | esc() - Escape Hygiene . . . . .                    | 264        |
| 8.6.6    | Macro với nhiều Arguments . . . . .                 | 264        |
| 8.7      | Generated Functions . . . . .                       | 265        |

|            |                                                 |            |
|------------|-------------------------------------------------|------------|
| 8.7.1      | Định nghĩa Generated Function                   | 265        |
| 8.7.2      | Use Cases cho Generated Functions               | 265        |
| 8.7.3      | Hạn chế của Generated Functions                 | 266        |
| 8.8        | Các Macros Phổ biến trong Julia                 | 266        |
| 8.8.1      | @time và @elapsed                               | 266        |
| 8.8.2      | @benchmark (từ BenchmarkTools.jl)               | 266        |
| 8.8.3      | @assert và @test                                | 267        |
| 8.8.4      | @show và @info                                  | 267        |
| 8.8.5      | @inline và @noinline                            | 267        |
| 8.8.6      | @simd và @inbounds                              | 268        |
| 8.9        | Thực hành Metaprogramming                       | 268        |
| 8.9.1      | Ví dụ: Tạo DSL cho Math Expressions             | 268        |
| 8.9.2      | Ví dụ: Macro cho Memoization                    | 269        |
| 8.9.3      | Ví dụ: Auto-generate Getters/Setters            | 270        |
| 8.9.4      | Bài tập cơ bản                                  | 270        |
| 8.9.5      | Bài tập nâng cao                                | 271        |
| 8.9.6      | Bài tập thách thức                              | 271        |
| <b>III</b> | <b>Phần III — Tính toán hiệu suất cao</b>       | <b>273</b> |
| <b>9</b>   | <b>Hiệu năng và profiling</b>                   | <b>275</b> |
| 9.1        | Benchmarking với BenchmarkTools.jl              | 275        |
| 9.1.1      | Tại sao cần benchmarking chính xác?             | 275        |
| 9.1.2      | Macro @time và các hạn chế                      | 276        |
| 9.1.3      | BenchmarkTools.jl — Benchmarking chuyên nghiệp  | 276        |
| 9.1.4      | Đọc kết quả benchmark                           | 277        |
| 9.1.5      | So sánh implementations                         | 278        |
| 9.2        | Type stability và @code_warntype                | 278        |
| 9.2.1      | Type stability là gì?                           | 278        |
| 9.2.2      | Phát hiện type instability với @code_warntype   | 279        |
| 9.2.3      | Các nguyên nhân phổ biến của type instability   | 279        |
| 9.2.4      | Strategies để đạt type stability                | 280        |
| 9.2.5      | Các công cụ phân tích khác                      | 282        |
| 9.3        | Memory management và allocation                 | 282        |
| 9.3.1      | Stack vs Heap allocation                        | 282        |
| 9.3.2      | Đo lường allocations                            | 283        |
| 9.3.3      | Garbage Collection (GC)                         | 283        |
| 9.3.4      | Strategies để giảm allocations                  | 284        |
| 9.4        | Profiling tools và techniques                   | 286        |
| 9.4.1      | Built-in profiling với @profile                 | 286        |
| 9.4.2      | ProfileView.jl — Visualization với Flame Graphs | 286        |
| 9.4.3      | ProfileSVG.jl và PProf.jl                       | 287        |
| 9.4.4      | Memory profiling với -track-allocation          | 287        |
| 9.4.5      | TimerOutputs.jl — Detailed timing               | 288        |
| 9.4.6      | Identifying và prioritizing bottlenecks         | 288        |
| 9.5        | Optimization techniques                         | 289        |

|           |                                                     |            |
|-----------|-----------------------------------------------------|------------|
| 9.5.1     | Avoiding global variables . . . . .                 | 289        |
| 9.5.2     | Loop optimization . . . . .                         | 290        |
| 9.5.3     | Multiple dispatch cho specialization . . . . .      | 291        |
| 9.5.4     | Algorithmic improvements . . . . .                  | 292        |
| 9.6       | Best practices và common pitfalls . . . . .         | 292        |
| 9.6.1     | Performance Do's and Don'ts . . . . .               | 292        |
| 9.6.2     | Performance tips quick reference . . . . .          | 293        |
| 9.6.3     | Readability vs Performance trade-offs . . . . .     | 293        |
| 9.7       | Tóm tắt chương . . . . .                            | 294        |
| 9.8       | Bài tập . . . . .                                   | 294        |
| 9.8.1     | Bài tập cơ bản . . . . .                            | 294        |
| 9.8.2     | Bài tập nâng cao . . . . .                          | 295        |
| 9.8.3     | Bài tập thách thức . . . . .                        | 296        |
| <b>10</b> | <b>Tính toán song song và GPU</b>                   | <b>297</b> |
| 10.1      | Giới thiệu parallel computing . . . . .             | 297        |
| 10.1.1    | Tại sao cần parallel computing? . . . . .           | 297        |
| 10.1.2    | Parallel computing models trong Julia . . . . .     | 297        |
| 10.1.3    | Khi nào dùng model nào? . . . . .                   | 298        |
| 10.2      | Multithreading với Threads . . . . .                | 298        |
| 10.2.1    | Setup Julia với multithreading . . . . .            | 298        |
| 10.2.2    | Threads.@threads cho parallel loops . . . . .       | 299        |
| 10.2.3    | Thread safety và race conditions . . . . .          | 300        |
| 10.2.4    | Best practices cho multithreading . . . . .         | 301        |
| 10.3      | Distributed computing . . . . .                     | 302        |
| 10.3.1    | Giới thiệu Distributed.jl . . . . .                 | 302        |
| 10.3.2    | Creating và managing workers . . . . .              | 302        |
| 10.3.3    | Remote calls và futures . . . . .                   | 303        |
| 10.3.4    | Data movement giữa workers . . . . .                | 303        |
| 10.3.5    | Parallel patterns: map-reduce . . . . .             | 304        |
| 10.4      | GPU computing với CUDA.jl . . . . .                 | 305        |
| 10.4.1    | Giới thiệu GPU programming . . . . .                | 305        |
| 10.4.2    | Setup CUDA.jl . . . . .                             | 306        |
| 10.4.3    | CuArray và basic operations . . . . .               | 306        |
| 10.4.4    | Writing custom CUDA kernels . . . . .               | 307        |
| 10.4.5    | Memory management trên GPU . . . . .                | 308        |
| 10.5      | Performance comparison . . . . .                    | 309        |
| 10.5.1    | Benchmarking parallel code . . . . .                | 309        |
| 10.5.2    | Threads vs Distributed vs GPU . . . . .             | 310        |
| 10.5.3    | Amdahl's Law và scaling limits . . . . .            | 310        |
| 10.6      | Real-world applications và best practices . . . . . | 310        |
| 10.6.1    | Case study: Monte Carlo simulation . . . . .        | 310        |
| 10.6.2    | Case study: Matrix operations . . . . .             | 311        |
| 10.6.3    | Hybrid parallelism . . . . .                        | 312        |
| 10.6.4    | Common pitfalls và debugging . . . . .              | 312        |
| 10.6.5    | Best practices summary . . . . .                    | 313        |

|                                                   |            |
|---------------------------------------------------|------------|
| 10.7 Tóm tắt chương . . . . .                     | 314        |
| 10.8 Bài tập . . . . .                            | 314        |
| 10.8.1 Bài tập cơ bản . . . . .                   | 314        |
| 10.8.2 Bài tập nâng cao . . . . .                 | 315        |
| 10.8.3 Bài tập thách thức . . . . .               | 315        |
| <b>Lời cảm ơn</b>                                 | <b>317</b> |
| <b>A Tham chiếu lệnh Julia REPL</b>               | <b>319</b> |
| A.1 Các chế độ REPL . . . . .                     | 319        |
| A.2 Phím tắt điều hướng . . . . .                 | 319        |
| A.3 Phím tắt chỉnh sửa . . . . .                  | 319        |
| A.4 Lệnh Package Manager . . . . .                | 319        |
| A.5 Biến đặc biệt REPL . . . . .                  | 320        |
| A.6 Macro hữu ích trong REPL . . . . .            | 320        |
| <b>B Packages thông dụng</b>                      | <b>321</b> |
| B.1 Xử lý dữ liệu . . . . .                       | 321        |
| B.2 Trực quan hóa . . . . .                       | 321        |
| B.3 Khoa học và toán học . . . . .                | 322        |
| B.4 Machine Learning . . . . .                    | 322        |
| B.5 Web và Networking . . . . .                   | 322        |
| B.6 Testing và Development . . . . .              | 322        |
| B.7 Parallel và GPU Computing . . . . .           | 322        |
| <b>C Ký hiệu Unicode trong Julia</b>              | <b>325</b> |
| C.1 Chữ cái Hy Lạp . . . . .                      | 325        |
| C.2 Ký hiệu toán học . . . . .                    | 325        |
| C.3 Ký hiệu quan hệ . . . . .                     | 326        |
| C.4 Mũi tên . . . . .                             | 326        |
| C.5 Subscript và Superscript . . . . .            | 326        |
| C.6 Cách nhập Unicode trong REPL . . . . .        | 326        |
| <b>D Chuyển đổi từ Python sang Julia</b>          | <b>329</b> |
| D.1 So sánh cú pháp cơ bản . . . . .              | 329        |
| D.2 Indexing và Slicing . . . . .                 | 329        |
| D.3 Control Flow . . . . .                        | 330        |
| D.4 Hàm . . . . .                                 | 330        |
| D.5 Comprehensions . . . . .                      | 330        |
| D.6 Classes vs Structs . . . . .                  | 330        |
| D.7 Sử dụng thư viện Python trong Julia . . . . . | 330        |
| D.8 Khác biệt quan trọng cần nhớ . . . . .        | 331        |



## Phần I

# Phần I — Nền tảng Julia





# Chương 1

## Giới thiệu Julia và hệ sinh thái

### Mục tiêu học tập

- Hiểu lịch sử và triết lý thiết kế của Julia
- Cài đặt và cấu hình Julia environment
- Làm quen với các IDE và công cụ phát triển
- So sánh Julia với các ngôn ngữ khác (Python, R, MATLAB)

### 1.1 Lịch sử và triết lý của Julia

#### 1.1.1 Lịch sử phát triển

Julia được khởi phát vào năm 2009 bởi Jeff Bezanson, Alan Edelman, Stefan Karpinski, và Viral Shah tại MIT [1]. Mục tiêu chính của nhóm là tạo ra một ngôn ngữ lập trình giải quyết được “vấn đề hai ngôn ngữ” (two-language problem) [2] — hiện tượng mà các nhà khoa học thường phải viết code nguyên mẫu (prototype) bằng ngôn ngữ tương tác như Python hoặc R, rồi sau đó viết lại bằng C/C++/Fortran để đạt hiệu năng cao.

Julia chính thức công bố phiên bản 1.0 vào năm 2018 [3], đánh dấu sự ổn định của API và khả năng sẵn sàng cho môi trường sản xuất (production). Từ đó, Julia phát triển nhanh chóng với hệ sinh thái phong phú, đặc biệt trong các lĩnh vực khoa học máy tính, học máy, và phân tích dữ liệu.

#### 1.1.2 Triết lý thiết kế

Julia được xây dựng trên ba trụ cột chính [2]:

**1. Multiple Dispatch (Đa phái cử):** Đây là đặc trưng cốt lõi của Julia, cho phép định nghĩa hàm dựa trên kiểu dữ liệu của **TẤT CẢ** các tham số đầu vào, không chỉ đối tượng đầu tiên như trong lập trình hướng đối tượng truyền thống. Cơ chế này giúp:

- Tái sử dụng code hiệu quả
- Kết hợp các thư viện một cách linh hoạt
- Tối ưu hóa hiệu năng cho từng kiểu dữ liệu cụ thể

**2. JIT Compilation (Biên dịch Just-In-Time):** Julia sử dụng LLVM compiler để biên dịch code thành mã máy ngay trước khi chạy. Điều này mang lại:

- Hiệu năng gần bằng C/Fortran

- Tính tương tác của Python/R
- Tuy nhiên có trade-off: thời gian biên dịch lần đầu (time to first plot)

**3. Hệ thống kiểu mạnh mẽ (Strong Type System):** Julia có hệ thống kiểu linh hoạt với:

- Kiểu động (dynamic typing) cho tính tương tác
- Kiểu tĩnh tùy chọn (optional static typing) cho hiệu năng
- Hỗ trợ đầy đủ parametric types và abstract types

### 1.1.3 Thành tựu nổi bật

Năm 2017, một chương trình Julia đạt hiệu năng đỉnh 1.54 petaflops (triệu tỷ phép tính dấu chấm động/giây) với 1.3 triệu threads khi xử lý dữ liệu ảnh thiên văn [2] — trước đó chỉ C/C++/Fortran đạt được mức này.

### 1.1.4 Bối cảnh ra đời: Vấn đề hai ngôn ngữ

Trong nhiều thập kỷ, các nhà khoa học và kỹ sư phải đối mặt với một nghịch lý trong tính toán khoa học được gọi là “**vấn đề hai ngôn ngữ**” (**two-language problem**). Quy trình làm việc điển hình bao gồm:

1. **Giai đoạn prototype:** Viết code bằng ngôn ngữ cấp cao như Python, R, hoặc MATLAB để thử nghiệm thuật toán và khám phá dữ liệu. Các ngôn ngữ này dễ học, có cú pháp đơn giản, và cho phép phát triển nhanh.
2. **Giai đoạn production:** Khi cần hiệu năng cao, phải viết lại toàn bộ code bằng C, C++, hoặc Fortran. Quá trình này tốn thời gian, dễ phát sinh lỗi, và đòi hỏi kỹ năng lập trình hệ thống.

Vấn đề này gây ra nhiều hệ quả nghiêm trọng:

- **Lãng phí thời gian:** Viết lại code có thể chiếm 50-80% thời gian dự án
- **Rủi ro lỗi:** Chuyển đổi giữa hai ngôn ngữ dễ phát sinh bug
- **Rào cản tri thức:** Nhà khoa học giỏi thuật toán nhưng không quen lập trình hệ thống
- **Chi phí bảo trì:** Phải duy trì song song hai codebase

### 1.1.5 Tầm nhìn của các nhà sáng lập

Vào năm 2009, bốn nhà nghiên cứu tại MIT bắt đầu một dự án đầy tham vọng. Trong bài blog nổi tiếng “Why We Created Julia” (2012), họ viết:

*“Chúng tôi muốn một ngôn ngữ mã nguồn mở, với giấy phép tự do. Chúng tôi muốn tốc độ của C với tính động của Ruby. Chúng tôi muốn ngôn ngữ homoiconic, với macro thực sự như Lisp, nhưng với ký hiệu toán học quen thuộc như Matlab. Chúng tôi muốn thứ có thể dùng cho lập trình tổng quát như Python, dễ dàng cho thống kê như R, tự nhiên cho xử lý chuỗi như Perl, mạnh mẽ cho đại số tuyến tính như Matlab, tốt cho việc kết nối các chương trình như shell. Thứ gì đó cực kỳ đơn giản để học, nhưng làm hài lòng cả những hacker nghiêm túc nhất. Chúng tôi muốn nó tương tác và biên dịch được.”*

Bốn nhà sáng lập đều mang đến chuyên môn riêng:

- **Jeff Bezanson:** Chuyên gia về ngôn ngữ lập trình và compiler, thiết kế core language
- **Stefan Karpinski:** Kỹ sư phần mềm, phát triển hệ sinh thái và package manager
- **Viral Shah:** Chuyên gia tính toán song song và distributed computing
- **Alan Edelman:** Giáo sư MIT, chuyên gia đại số tuyến tính và random matrix theory

### 1.1.6 Triết lý thiết kế: Tái sử dụng code

Một trong những đặc điểm quan trọng nhất của Julia là khả năng **tái sử dụng và kết hợp code** một cách tự nhiên. Như Lee Phillips mô tả trong *Practical Julia* (2024):

*“Chính đặc điểm thiết kế này cho phép mức độ tái sử dụng và tái kết hợp code chưa từng có. Điều này, cùng với bất kỳ benchmark nào, là thứ làm hài lòng các nhà nghiên cứu đã chọn Julia làm công cụ tính toán của họ. Julia đang phát triển mạnh với các nhà khoa học chủ yếu vì nó cho phép họ sử dụng code của nhau và kết hợp các thư viện để tạo ra chức năng mới theo những cách mà tác giả thư viện không hề dự tính.”*

Điều này đạt được nhờ:

- **Multiple dispatch:** Các thư viện có thể mở rộng hàm của nhau một cách tự nhiên
- **Type system:** Kiểu dữ liệu mới tự động hoạt động với code hiện có
- **Metaprogramming:** Có thể tùy biến ngôn ngữ cho từng vấn đề cụ thể

### 1.1.7 Timeline phát triển

Bảng 1.1 tóm tắt các mốc quan trọng trong lịch sử phát triển của Julia.

Bảng 1.1: Các mốc quan trọng trong lịch sử phát triển Julia

| Năm  | Phiên bản   | Sự kiện                                                   |
|------|-------------|-----------------------------------------------------------|
| 2009 | –           | Bắt đầu phát triển tại MIT                                |
| 2012 | v0.1        | Công bố công khai, blog “Why We Created Julia”            |
| 2014 | v0.3        | Cải thiện hiệu năng đáng kể, package ecosystem mở rộng    |
| 2015 | v0.4        | Multi-threading experimental, cải thiện REPL              |
| 2016 | v0.5        | Generator expressions, cải thiện type system              |
| 2017 | v0.6        | Đạt 1.54 petaflops, chuẩn bị cho v1.0                     |
| 2018 | <b>v1.0</b> | <b>Phiên bản ổn định đầu tiên</b> , API ổn định           |
| 2019 | v1.1-1.3    | Composable multi-threading, cải thiện compiler            |
| 2020 | v1.5        | Struct layout optimizations, implicit return              |
| 2021 | v1.6 LTS    | Long-term support, cải thiện thời gian load package       |
| 2022 | v1.8        | Const propagation improvements, @inline macro             |
| 2023 | v1.9-1.10   | Package extensions, cải thiện precompilation đáng kể      |
| 2024 | v1.11       | Public keyword, Memory type, cải thiện garbage collection |

### 1.1.8 Hệ sinh thái và cộng đồng

Tính đến năm 2024, Julia có hệ sinh thái phong phú với hơn **10,000 packages** đã đăng ký. Các lĩnh vực ứng dụng chính bao gồm:

- **Khoa học dữ liệu:** DataFrames.jl, CSV.jl, Query.jl
- **Học máy:** Flux.jl, MLJ.jl, Knet.jl
- **Tính toán khoa học:** DifferentialEquations.jl, JuMP.jl, Optimization.jl
- **Visualization:** Plots.jl, Makie.jl, Gadfly.jl
- **Web:** Genie.jl, HTTP.jl, JSON3.jl
- **Song song và GPU:** CUDA.jl, Distributed.jl, MPI.jl

Cộng đồng Julia được đánh giá là một trong những cộng đồng thân thiện nhất trong thế giới lập trình, với các kênh hỗ trợ:

- **Discourse:** <https://discourse.julialang.org/>
- **Slack:** Julia Slack workspace
- **JuliaCon:** Hội nghị thường niên (từ 2014)
- **Julia Community:** Local meetups trên toàn thế giới

### 1.1.9 Ai đang sử dụng Julia?

Julia được áp dụng rộng rãi trong cả học thuật và công nghiệp:

#### Học thuật:

- MIT, Stanford, Berkeley: Giảng dạy và nghiên cứu
- NASA: Mô phỏng hệ thống không gian
- Federal Reserve Bank of New York: Mô hình kinh tế vĩ mô DSGE

#### Công nghiệp:

- **Pfizer:** Phát triển vaccine COVID-19 (mô phỏng dược động học)
- **BlackRock:** Quản lý danh mục đầu tư
- **NVIDIA:** Tích hợp với CUDA cho GPU computing
- **Intel:** Tối ưu hóa cho kiến trúc x86
- **AstraZeneca:** Nghiên cứu dược phẩm

### 1.1.10 Tại sao Julia quan trọng cho sinh viên?

Đối với sinh viên đại học, Julia mang lại nhiều lợi ích:

1. **Học một lần, dùng mọi nơi:** Không cần học Python cho prototype rồi C++ cho production
2. **Cú pháp gần toán học:** Code Julia trông giống công thức toán, dễ hiểu và verify
3. **Hiệu năng ngay từ đầu:** Không cần lo về optimization khi scale up
4. **Kỹ năng được săn đón:** Julia đang phát triển nhanh trong data science và quantitative finance
5. **Cộng đồng hỗ trợ:** Dễ dàng nhận được giúp đỡ khi gặp khó khăn

## 1.2 Cài đặt

Julia có thể được cài đặt trên tất cả các hệ điều hành phổ biến. Phần này hướng dẫn chi tiết từng bước cài đặt, từ phương pháp khuyên dùng đến các lựa chọn thay thế.

### 1.2.1 Cài đặt với juliaup (khuyên dùng)

juliaup là công cụ quản lý phiên bản Julia chính thức [4], hoạt động tương tự rustup cho Rust hoặc nvm cho Node.js. Đây là phương pháp được khuyến nghị vì:

- Tự động cập nhật phiên bản mới
- Dễ dàng chuyển đổi giữa các phiên bản
- Quản lý nhiều phiên bản đồng thời
- Hỗ trợ đa nền tảng

#### 1.2.1.1 Cài đặt trên Linux/macOS

Mở Terminal và chạy lệnh sau:

```
1 curl -fsSL https://install.julialang.org | sh
```

Sau khi cài đặt, khởi động lại terminal hoặc chạy:

```
1 source ~/.bashrc      # Cho Bash
2 source ~/.zshrc       # Cho Zsh
```

#### 1.2.1.2 Cài đặt trên Windows

Có hai cách cài đặt trên Windows:

##### Cách 1: Windows Store (đơn giản nhất)

1. Mở Microsoft Store
2. Tìm kiếm “Julia”
3. Nhấn “Get” để cài đặt

##### Cách 2: Winget (dòng lệnh)

```
1 winget install julia -s msstore
```

##### Cách 3: Chocolatey (package manager)

```
1 choco install julia
```

### 1.2.1.3 Quản lý phiên bản với juliaup

Sau khi cài đặt juliaup, bạn có thể quản lý nhiều phiên bản Julia:

```
1 juliaup status           # Xem các phiên bản đã cài
2 juliaup install 1.10     # Cài phiên bản 1.10
3 juliaup install 1.9      # Cài thêm phiên bản 1.9
4 juliaup default 1.10     # Đặt 1.10 làm mặc định
5 juliaup update           # Cập nhật tất cả phiên bản
6 juliaup remove 1.9       # Gỡ phiên bản 1.9
```

Để chạy một phiên bản cụ thể mà không thay đổi mặc định:

```
1 julia +1.9               # Chạy phiên bản 1.9
2 julia +release           # Chạy phiên bản release mới nhất
3 julia +lts               # Chạy phiên bản LTS (Long Term Support)
```

## 1.2.2 Cài đặt thủ công

Nếu không muốn sử dụng juliaup, có thể tải binary trực tiếp từ <https://julialang.org/downloads/>.

### 1.2.2.1 Linux

```
1 # Tải và giải nén
2 wget
   → https://julialang-s3.julialang.org/bin/linux/x64/1.10/julia-1.10.0-linux-x86_64.tar.gz
3 tar -xvzf julia-1.10.0-linux-x86_64.tar.gz
4
5 # Di chuyển vào thư mục hệ thống
6 sudo mv julia-1.10.0 /opt/
7
8 # Tạo symbolic link
9 sudo ln -s /opt/julia-1.10.0/bin/julia /usr/local/bin/julia
```

Hoặc cài qua PPA (Ubuntu/Debian):

```
1 sudo add-apt-repository ppa:staticfloat/juliareleases
2 sudo apt-get update
3 sudo apt-get install julia
```

### 1.2.2.2 macOS

```
1 # Tải file .dmg từ julialang.org
2 # Mở file .dmg và kéo Julia vào Applications
3
4 # Tạo symbolic link để chạy từ terminal
5 sudo ln -s /Applications/Julia-1.10.app/Contents/Resources/julia/bin/julia \
6     /usr/local/bin/julia
```

Hoặc cài qua Homebrew:

```
1 brew install julia
```

### 1.2.2.3 Windows

1. Tải file .exe từ <https://julialang.org/downloads/>
2. Chạy installer, chọn “Add Julia to PATH”
3. Khởi động lại Command Prompt hoặc PowerShell

Nếu quên chọn “Add to PATH”, có thể thêm thủ công:

```
1 # Mở System Properties -> Environment Variables
2 # Thêm vào PATH: C:\Users\<username>\AppData\Local\Programs\Julia-1.10\bin
```

### 1.2.3 Cài đặt với Docker

Docker là lựa chọn tốt cho:

- Môi trường development nhất quán
- Triển khai ứng dụng
- Tránh xung đột dependencies
- CI/CD pipelines

```
1 # Kéo image Julia chính thức
2 docker pull julia:1.10
3
4 # Chạy Julia REPL trong container
5 docker run -it julia:1.10
6
7 # Chạy với volume mount để chia sẻ files
8 docker run -it -v $(pwd):/app julia:1.10
9
10 # Chạy script Julia
11 docker run -v $(pwd):/app julia:1.10 julia /app/script.jl
```

Dockerfile mẫu cho project Julia:

```
1 FROM julia:1.10
2 WORKDIR /app
3 COPY Project.toml Manifest.toml ./
4 RUN julia -e 'using Pkg; Pkg.instantiate()'
5 COPY . .
6 CMD ["julia", "main.jl"]
```

### 1.2.4 Kiểm tra cài đặt

Sau khi cài đặt, kiểm tra Julia hoạt động đúng:

```
1 # Xem phiên bản
2 julia --version
3 # Kết quả: julia version 1.10.0
4
5 # Khởi động REPL
6 julia
```

Khi REPL khởi động thành công, bạn sẽ thấy:

```

1
2      _          _ _(_) _      | Documentation: https://docs.julialang.org
3    (_)        |  ( ) ( )      |
4      _ _      | | _ _ _ _      | Type "?" for help, "]?" for Pkg help.
5    | | | | | | | / _ ` |      |
6    | | | _ | | | ( _ | |      | Version 1.10.0 (2023-12-25)
7  _/ | \ _ ' _ | _ | \ _ ' _ | | Official https://julialang.org/ release
8  | _/
9
10 julia>
```

Thử chạy một số lệnh đơn giản:

```

1 julia> 1 + 1
2 2
3
4 julia> println("Hello, Julia!")
5 Hello, Julia!
6
7 julia> versioninfo()
8 Julia Version 1.10.0
9 # ... thông tin chi tiết về hệ thống
```

## 1.2.5 Xử lý lỗi thường gặp

### 1.2.5.1 Lỗi “julia command not found”

**Nguyên nhân:** Julia chưa được thêm vào PATH.

**Giải pháp Linux/macOS:**

```

1 # Thêm vào ~/.bashrc hoặc ~/.zshrc
2 export PATH="$PATH:/path/to/julia/bin"
3 source ~/.bashrc
```

**Giải pháp Windows:**

1. Mở “Edit environment variables for your account”
2. Chọn PATH, nhấn Edit
3. Thêm đường dẫn đến thư mục bin của Julia

### 1.2.5.2 Lỗi SSL/TLS khi cài packages

**Nguyên nhân:** Firewall hoặc proxy chặn kết nối.

**Giải pháp:**

```

1 # Đặt biến môi trường proxy
2 ENV["HTTP_PROXY"] = "http://proxy:port"
3 ENV["HTTPS_PROXY"] = "http://proxy:port"
```



### 1.2.5.3 Lỗi precompilation chậm

**Nguyên nhân:** Lần đầu load package, Julia cần compile.

**Giải pháp:** Sử dụng PackageCompiler.jl để tạo sysimage:

```
1 using PackageCompiler
2 create_sysimage([:Plots, :DataFrames];
3   sysimage_path="custom_sysimage.so")
```

### 1.2.5.4 Lỗi BoundsError

**Nguyên nhân:** Truy cập array với index không hợp lệ (Julia index từ 1).

**Ví dụ:**

```
1 arr = [1, 2, 3]
2 arr[0] # ERROR: BoundsError - Julia index từ 1, không phải 0
3 arr[1] # Đúng: trả về 1
```

### 1.2.5.5 Lỗi DomainError

**Nguyên nhân:** Gọi hàm với argument ngoài domain.

**Ví dụ:**

```
1 sqrt(-1) # ERROR: DomainError
2 sqrt(Complex(-1)) # Đúng: 0.0 + 1.0im
3 log(-1) # ERROR: DomainError
4 log(Complex(-1)) # Đúng: 0.0 + 3.14159...im
```

## 1.3 IDE và môi trường phát triển

Julia hỗ trợ nhiều môi trường phát triển khác nhau, từ IDE đầy đủ tính năng đến notebook tương tác. Phần này giới thiệu các lựa chọn phổ biến nhất và hướng dẫn cài đặt chi tiết.

### 1.3.1 VS Code + Julia Extension (Khuyến dùng)

Visual Studio Code với Julia extension là môi trường phát triển được khuyến dùng nhất cho Julia, với hơn 2 triệu lượt tải. Extension này cung cấp trải nghiệm phát triển chuyên nghiệp:

#### 1.3.1.1 Tính năng chính

- **IntelliSense:** Tự động hoàn thành code thông minh, hiểu context
- **Linting:** Phát hiện lỗi cú pháp và cảnh báo trong thời gian thực
- **Debugger tích hợp:** Breakpoints, step through, watch variables
- **Inline evaluation:** Chạy code và xem kết quả ngay trong editor
- **Plot pane:** Hiển thị đồ thị inline, hỗ trợ zoom và export
- **Workspace explorer:** Xem và quản lý các biến đang hoạt động
- **Notebook support:** Mở và chỉnh sửa .ipynb files
- **Git integration:** Version control tích hợp sẵn
- **Documentation hover:** Xem docs bằng cách hover chuột

### 1.3.1.2 Cài đặt chi tiết

#### Bước 1: Cài đặt VS Code

Tải từ <https://code.visualstudio.com/Download> và cài đặt theo hướng dẫn hệ điều hành.

#### Bước 2: Cài Julia Extension

1. Mở VS Code
2. Nhấn `Ctrl+Shift+X` (Windows/Linux) hoặc `Cmd+Shift+X` (macOS)
3. Tìm “Julia” trong ô tìm kiếm
4. Chọn extension có tên “Julia” (`julialang.language-julia`)
5. Nhấn “Install”
6. Khởi động lại VS Code

#### Bước 3: Cấu hình Julia path (nếu cần)

Nếu Julia không được phát hiện tự động:

1. Mở Settings (`Ctrl+,`)
2. Tìm “julia executable”
3. Nhập đường dẫn đến Julia executable

### 1.3.1.3 Các phím tắt quan trọng

- `Ctrl+Enter`: Chạy dòng/block hiện tại
- `Shift+Enter`: Chạy và di chuyển đến dòng tiếp theo
- `Ctrl+Shift+Enter`: Chạy toàn bộ file
- `Ctrl+Shift+P` → “Julia: Start REPL”: Mở REPL
- `Ctrl+Shift+P` → “Julia: Execute File”: Chạy file
- `Ctrl+Shift+P` → “Julia: Change Current Environment”: Đổi environment

### 1.3.1.4 Cấu hình khuyến dùng

Thêm vào `settings.json`:

```

1 {
2   "julia.executablePath": "/usr/local/bin/julia",
3   "julia.NumThreads": 4,
4   "julia.enableTelemetry": false,
5   "julia.execution.resultType": "inline",
6   "julia.execution.codeInREPL": true,
7   "editor.formatOnSave": true
8 }
```

### 1.3.2 Jupyter Notebook với IJulia

Jupyter Notebook là môi trường tương tác phổ biến trong nghiên cứu khoa học và giảng dạy. Julia tích hợp với Jupyter thông qua kernel IJulia.

#### 1.3.2.1 Ưu điểm của Jupyter

- Kết hợp code, văn bản, công thức toán, và đồ thị trong một document
- Dễ chia sẻ và tái sử dụng
- Hỗ trợ nhiều ngôn ngữ (Python, R, Julia, ...)
- Phù hợp cho exploratory data analysis
- Tích hợp tốt với các hệ thống cloud (Google Colab, Binder)

#### 1.3.2.2 Cài đặt IJulia

```

1 # Trong Julia REPL
2 using Pkg
3 Pkg.add("IJulia")
4
5 # Lần đầu tiên sử dụng, IJulia sẽ tự động cài Jupyter nếu chưa có
6 using IJulia
7 notebook() # Mở Jupyter Notebook trong browser
8
9 # Hoặc sử dụng JupyterLab (giao diện mới hơn)
10 jupyterlab()
```

#### 1.3.2.3 Sử dụng Jupyter với Julia đã cài sẵn

Nếu đã có Jupyter (từ Anaconda hoặc pip):

```

1 # Cài IJulia kernel
2 using IJulia
3 IJulia.installkernel("Julia")
4
5 # Sau đó mở Jupyter từ terminal
6 # $ jupyter notebook
7 # Chọn kernel "Julia 1.10" khi tạo notebook mới
```

#### 1.3.2.4 Tips sử dụng Jupyter với Julia

- Sử dụng Shift+Enter để chạy cell và di chuyển xuống
- Sử dụng Ctrl+Enter để chạy cell và ở lại
- Thêm ; cuối dòng để ẩn output
- Sử dụng ?function\_name để xem documentation
- Sử dụng magic command %time để đo thời gian (cần PyCall)

### 1.3.3 Pluto.jl — Reactive Notebooks

Pluto.jl là notebook environment được thiết kế riêng cho Julia, với tính năng **reactive** — mọi thay đổi tự động lan truyền đến các cells liên quan.

#### 1.3.3.1 Điểm khác biệt so với Jupyter

- **Reactive:** Thay đổi một cell tự động cập nhật tất cả cells phụ thuộc
- **Reproducible:** Thứ tự cells không quan trọng, luôn cho kết quả nhất quán
- **Pure Julia:** Không phụ thuộc Python hay Conda
- **Nhẹ:** File .jl thuần túy, dễ version control
- **Package management:** Tự động quản lý dependencies cho mỗi notebook

#### 1.3.3.2 Cài đặt và sử dụng

```
1 # Cài đặt Pluto
2 using Pkg
3 Pkg.add("Pluto")
4
5 # Khởi động Pluto
6 using Pluto
7 Pluto.run()
8 # Browser sẽ tự động mở tại http://localhost:1234
```

#### 1.3.3.3 Tính năng nổi bật

```
1 # Trong Pluto, mọi cell đều reactive
2 x = 5 # Cell 1
3 y = x^2 # Cell 2: Tự động cập nhật khi x thay đổi
4 z = x + y # Cell 3: Tự động cập nhật khi x hoặc y thay đổi
5
6 # Khi thay đổi x = 10, cả y và z đều tự động tính lại!
```

#### 1.3.3.4 Sử dụng Pluto cho giảng dạy

Pluto đặc biệt phù hợp cho:

- Tài liệu giảng dạy interactive
- Bài tập có thể tự kiểm tra
- Visualization động
- Exploratory programming

```
1 # Ví dụ: Interactive slider
2 using PlutoUI
3
4 @bind n Slider(1:100, default=10)
5 # Thay đổi slider, plot tự động cập nhật
6
7 plot(1:n, (1:n).^2, label="y = x^2")
```

### 1.3.4 REPL — Giao diện dòng lệnh tương tác

Julia REPL (Read-Eval-Print Loop) là môi trường tương tác mạnh mẽ, đi kèm sẵn với Julia. REPL có nhiều chế độ đặc biệt, giúp tăng năng suất đáng kể.

#### 1.3.4.1 Các chế độ REPL

- **Julia mode** (mặc định, prompt: `julia>`): Chạy code Julia
- **Help mode** (nhấn `?`, prompt: `help?>`): Xem documentation
- **Shell mode** (nhấn `;`, prompt: `shell>`): Chạy lệnh shell
- **Package mode** (nhấn `]`, prompt: `(@v1.10) pkg>`): Quản lý packages

#### 1.3.4.2 Tips và tricks trong REPL

##### Tab completion:

```
1 julia> pri<Tab>
2 # Hiện: print, println, printstyled, ...
3
4 julia> \alpha<Tab>
5 # Chuyển thành: α (Unicode character)
6
7 julia> DataF<Tab>
8 # Hiện: DataFrames (nếu đã install)
```

##### Xem documentation:

```
1 help?> sin
2 # Hiện thị documentation của hàm sin
3
4 help?> @time
5 # Documentation của macro @time
```

##### Shell commands:

```
1 shell> ls
2 # Liệt kê files trong thư mục hiện tại
3
4 shell> pwd
5 # Hiện thị đường dẫn hiện tại
6
7 shell> cd /path/to/project
8 # Thay đổi thư mục
```

##### Package management:

```
1 (@v1.10) pkg> add DataFrames Plots
2 # Cài đặt packages
3
4 (@v1.10) pkg> status
5 # Xem packages đã cài
6
7 (@v1.10) pkg> update
8 # Cập nhật tất cả packages
```

### 1.3.4.3 Phím tắt REPL hữu ích

- Ctrl+C: Hủy lệnh đang chạy
- Ctrl+D: Thoát REPL
- Ctrl+L: Xóa màn hình
- ↑/↓: Duyệt lịch sử lệnh
- Ctrl+R: Tìm kiếm trong lịch sử
- Backspace (trong special mode): Quay về Julia mode

### 1.3.4.4 Startup file

Julia load file `~/.julia/config/startup.jl` mỗi khi khởi động. Sử dụng để:

```
1 # ~/.julia/config/startup.jl
2
3 # Load packages thường dùng
4 using Revise # Hot-reload code changes
5
6 # Thiết lập môi trường
7 ENV["JULIA_NUM_THREADS"] = "auto"
8
9 # Alias tiện ích
10 macro tic()
11     return :(start_time = time())
12 end
13 macro toc()
14     return :(println("Elapsed: ", time() - start_time, " seconds"))
15 end
16
17 # Thông báo khi REPL sẵn sàng
18 println("Julia is ready!")
```

### 1.3.5 Các IDE và Editor khác

Ngoài VS Code, Julia còn được hỗ trợ bởi nhiều editor khác:

- **Vim/Neovim:** Plugin `julia-vim` với syntax highlighting và REPL integration
- **Emacs:** `julia-mode` với ESS (Emacs Speaks Statistics)
- **Sublime Text:** Julia package với syntax và build system
- **Atom** (discontinued): Juno IDE — trước đây là IDE chính thức
- **IntelliJ IDEA:** Julia plugin với IntelliSense cơ bản

*Khuyến nghị:* Đối với sinh viên mới bắt đầu, nên sử dụng VS Code + Julia Extension. Khi đã quen, có thể thử Pluto.jl cho các bài tập interactive.

## 1.4 Package Manager

Julia có hệ thống quản lý package tích hợp sẵn (`Pkg`), với dependency resolution tự động và version control chặt chẽ.

### 1.4.1 Quản lý environments

Julia sử dụng “environments” — bộ packages độc lập cho mỗi dự án, tương tự Python `virtualenv` nhưng tích hợp sâu hơn:

- `Project.toml`: Danh sách packages trực tiếp
- `Manifest.toml`: Danh sách đầy đủ dependencies (tự động sinh)

### 1.4.2 Các lệnh cơ bản

**Chuyển sang Package mode:** Nhấn `]` trong REPL

```

1  # Tạo environment mới
2  (@v1.10) pkg> activate .
3    Activating new project at `./current/directory`
4
5  # Thêm package
6  (project) pkg> add DataFrames Plots
7
8  # Xem trạng thái
9  (project) pkg> status
10
11 # Cập nhật packages
12 (project) pkg> update
13
14 # Xóa package
15 (project) pkg> rm Plots
16
17 # Quay lại Julia mode
18 (project) pkg> <Backspace>
```

### 1.4.3 Sử dụng trong code

```

1  using Pkg
2
3  Pkg.activate(".")           # Kích hoạt environment
4  Pkg.add("DataFrames")      # Thêm package
5  Pkg.instantiate()          # Cài đặt từ Manifest.toml
```

### 1.4.4 Registry và packages phổ biến

Julia General Registry [4]: <https://github.com/JuliaRegistries/General>  
 Packages phổ biến:

- `DataFrames.jl`: Xử lý dữ liệu dạng bảng
- `Plots.jl`: Trực quan hóa dữ liệu

- `DifferentialEquations.jl`: Giải phương trình vi phân
- `Flux.jl`: Deep learning framework
- `JuMP.jl`: Mathematical optimization

### 1.4.5 Pkg REPL Mode chi tiết

Khi nhấn phím `]` trong Julia REPL, bạn chuyển sang Pkg mode với prompt thay đổi:

```
1 julia> ]
2 (@v1.10) pkg>
```

Prefix `(@v1.10)` cho biết bạn đang ở default environment của Julia 1.10. Các lệnh quan trọng trong Pkg mode:

| Lệnh                                     | Mô tả                                                           |
|------------------------------------------|-----------------------------------------------------------------|
| <code>status</code> hoặc <code>st</code> | Hiển thị packages đã cài trong environment hiện tại             |
| <code>add Package</code>                 | Cài đặt package từ General Registry                             |
| <code>add Package@version</code>         | Cài đặt phiên bản cụ thể (vd: <code>add DataFrames@1.6</code> ) |
| <code>rm Package</code>                  | Gỡ bỏ package khỏi environment                                  |
| <code>update</code> hoặc <code>up</code> | Cập nhật tất cả packages lên phiên bản mới nhất                 |
| <code>update Package</code>              | Cập nhật package cụ thể                                         |
| <code>activate .</code>                  | Kích hoạt project environment trong thư mục hiện tại            |
| <code>activate</code>                    | Quay về default environment                                     |
| <code>instantiate</code>                 | Cài đặt packages từ Manifest.toml (reproducibility)             |
| <code>resolve</code>                     | Giải quyết dependencies và cập nhật Manifest.toml               |
| <code>gc</code>                          | Garbage collection: xóa packages không còn sử dụng              |
| <code>pin Package</code>                 | Ghim package ở phiên bản hiện tại, không cho update             |
| <code>free Package</code>                | Bỏ ghim, cho phép update lại                                    |
| <code>develop Package</code>             | Clone package để phát triển locally                             |
| <code>test Package</code>                | Chạy test suite của package                                     |

Nhấn Backspace hoặc `Ctrl+C` để thoát Pkg mode và quay lại Julia REPL.

### 1.4.6 Project.toml và Manifest.toml

Hai file này là trái tim của Julia package management, đảm bảo reproducibility:

#### 1.4.6.1 Project.toml

File này định nghĩa **direct dependencies**—packages mà project của bạn sử dụng trực tiếp:

```
1 # Project.toml
2 name = "MyProject"
3 uuid = "12345678-1234-1234-1234-123456789012"
4 authors = ["Nguyen Van A <a@email.com>"]
5 version = "0.1.0"
6
7 [deps]
8 CSV = "336ed68f-0bac-5ca0-87d4-7b16caf5d00b"
```



```

9 DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
10 Plots = "91a5bcdd-55d7-5caf-9e0b-520d859cae80"
11
12 [compat]
13 julia = "1.9"
14 DataFrames = "1.6"

```

Các thành phần:

- name: Tên project (bắt buộc cho packages, optional cho scripts)
- uuid: Unique identifier, tự động sinh khi tạo package
- [deps]: Danh sách dependencies với UUID từ registry
- [compat]: Ràng buộc phiên bản để đảm bảo tương thích

#### 1.4.6.2 Manifest.toml

File này ghi lại **toàn bộ dependency graph**—bao gồm cả indirect dependencies:

```

1 # Manifest.toml (tự động sinh, KHÔNG nên edit thủ công)
2 julia_version = "1.10.0"
3 manifest_format = "2.0"
4
5 [[deps.CSV]]
6 deps = ["CodecZlib", "Dates", "FilePathsBase", ...]
7 git-tree-sha1 = "..."
8 uuid = "336ed68f-0bac-5ca0-87d4-7b16caf5d00b"
9 version = "0.10.12"
10
11 [[deps.DataFrames]]
12 deps = ["Compat", "DataAPI", "Future", ...]
13 git-tree-sha1 = "..."
14 uuid = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
15 version = "1.6.1"

```

#### Reproducibility workflow:

1. Developer A commit cả Project.toml và Manifest.toml vào Git
2. Developer B clone repository
3. Developer B chạy `pkg> instantiate`
4. Julia cài đặt **chính xác** các phiên bản trong Manifest.toml

#### 1.4.7 Environments và Isolation

Julia environments cho phép mỗi project có bộ packages riêng biệt, tránh xung đột phiên bản:

```

1 # Tạo project mới với environment riêng
2 julia> mkdir("MyDataProject")
3 julia> cd("MyDataProject")
4

```

```

5 julia> ]
6 (@v1.10) pkg> activate .
7   Activating new project at `~/path/to/MyDataProject`
8
9 (MyDataProject) pkg> add DataFrames CSV Plots
10   Resolving package versions...
11   Installed DataFrames v1.6.1
12   Installed CSV v0.10.12
13   Installed Plots v1.39.0
14   Updating `Project.toml`
15   Updating `Manifest.toml`

```

Sau khi activate, prompt thay đổi thành (MyDataProject) thay vì (@v1.10), cho thấy bạn đang làm việc trong project-specific environment.

### 1.4.7.1 Stacked Environments

Julia hỗ trợ stacked environments—environment có thể “kế thừa” packages từ environment khác:

```

1 # LOAD_PATH xác định thứ tự tìm kiếm packages
2 julia> LOAD_PATH
3 3-element Vector{String}:
4  "@"           # Project environment (Project.toml)
5  "@v#.#"       # Default environment
6  "@stdlib"     # Standard library

```

Thứ tự ưu tiên: Project → Default → Stdlib. Package trong project environment được ưu tiên hơn default.

### 1.4.7.2 Shared Environments

Tạo shared environment cho các công cụ development:

```

1 # Tạo shared environment cho dev tools
2 (@v1.10) pkg> activate --shared devtools
3   Activating new project at `~/.julia/environments/devtools`
4
5 (devtools) pkg> add Revise BenchmarkTools JuliaFormatter
6
7 # Sau này, thêm vào startup.jl để tự động load
8 # File: ~/.julia/config/startup.jl
9 using Revise # Hot-reload code changes

```

## 1.4.8 Registry và Private Packages

### 1.4.8.1 General Registry

Julia General Registry là kho packages chính thức, hosted tại GitHub. Khi bạn chạy add DataFrames, Julia:

1. Tìm package trong General Registry
2. Tải về thông tin phiên bản và dependencies

3. Resolve dependency graph

4. Download và cài đặt packages

### 1.4.8.2 Private Registry

Cho doanh nghiệp hoặc team cần packages nội bộ:

```
1 # Thêm private registry
2 (@v1.10) pkg> registry add https://github.com/mycompany/MyRegistry
3
4 # Sau đó có thể add packages từ private registry
5 (@v1.10) pkg> add MyInternalPackage
```

### 1.4.8.3 Cài đặt từ Git URL

Cho packages chưa đăng ký hoặc đang phát triển:

```
1 # Từ GitHub
2 pkg> add https://github.com/username/UnregisteredPackage.jl
3
4 # Từ branch cụ thể
5 pkg> add https://github.com/username/Package.jl#develop
6
7 # Từ commit cụ thể
8 pkg> add https://github.com/username/Package.jl#abc123
9
10 # Từ Git tag
11 pkg> add https://github.com/username/Package.jl@v1.0.0
```

### 1.4.8.4 Local Development

Phát triển package locally với develop:

```
1 # Clone package để phát triển
2 pkg> develop DataFrames
3 # Package được clone vào ~/.julia/dev/DataFrames
4
5 # Hoặc develop từ local path
6 pkg> develop /path/to/MyLocalPackage
7
8 # Quay lại sử dụng phiên bản từ registry
9 pkg> free DataFrames
```

## 1.4.9 Ví dụ thực tế: Setup Data Science Project

Workflow hoàn chỉnh để tạo project data science từ đầu:

```
1 # 1. Tạo thư mục project
2 shell> mkdir DataAnalysisProject
3 shell> cd DataAnalysisProject
4
5 # 2. Khởi tạo Git repository
6 shell> git init
```

```

7
8 # 3. Mở Julia và activate environment
9 julia> ]
10 (@v1.10) pkg> activate .
11   Activating new project at `/path/to/DataAnalysisProject`
12
13 # 4. Thêm packages cần thiết
14 (DataAnalysisProject) pkg> add DataFrames CSV Statistics Plots
15
16 # 5. Thêm compat bounds cho reproducibility
17 (DataAnalysisProject) pkg> compat DataFrames 1.6
18 (DataAnalysisProject) pkg> compat julia 1.9
19
20 # 6. Kiểm tra status
21 (DataAnalysisProject) pkg> status
22 Project DataAnalysisProject v0.1.0
23 Status `/path/to/DataAnalysisProject/Project.toml`
24   [336ed68f] CSV v0.10.12
25   [a93c6f00] DataFrames v1.6.1
26   [91a5bcd] Plots v1.39.0
27   [10745b16] Statistics v1.10.0
28
29 # 7. Thoát Pkg mode và bắt đầu code
30 (DataAnalysisProject) pkg> <Backspace>
31
32 julia> using DataFrames, CSV, Plots
33
34 julia> # Bắt đầu phân tích dữ liệu...

```

### Cấu trúc thư mục sau khi setup:

```

DataAnalysisProject/
├── Project.toml      # Direct dependencies
├── Manifest.toml    # Full dependency graph (auto-generated)
├── src/              # Source code
│   └── analysis.jl
├── data/             # Data files
│   └── input.csv
├── notebooks/       # Jupyter notebooks
│   └── exploration.ipynb
└── README.md

```

### Git workflow:

```

1 # Commit project files
2 shell> git add Project.toml Manifest.toml src/ README.md
3 shell> git commit -m "Initial project setup with Julia dependencies"
4
5 # Collaborator clones và instantiates
6 shell> git clone <repo-url>
7 shell> cd DataAnalysisProject
8 julia> ]

```

```

9 pkg> activate .
10 pkg> instantiate # Cài đặt chính xác các phiên bản từ Manifest.toml

```

#### 1.4.10 Best Practices cho Package Management

1. **Luôn sử dụng project-specific environments:** Tránh cài packages vào default environment
2. **Commit cả Project.toml và Manifest.toml:** Đảm bảo reproducibility hoàn toàn
3. **Sử dụng compat bounds:** Tránh breaking changes khi dependencies update
4. **Chạy pkg> gc định kỳ:** Dọn dẹp packages không sử dụng, tiết kiệm disk
5. **Pin packages trong production:** Tránh unexpected updates
6. **Test sau mỗi lần update:** Chạy pkg> test để verify compatibility
7. **Sử dụng Revise.jl trong development:** Auto-reload code changes mà không cần restart Julia

#### Ghi chú

**Lưu ý quan trọng:** Manifest.toml có thể rất lớn (hàng nghìn dòng) vì chứa toàn bộ dependency tree. Đây là điều bình thường và cần thiết cho reproducibility. Không nên edit file này thủ công.

## 1.5 So sánh với các ngôn ngữ khác

### 1.5.1 Julia vs Python

| Tiêu chí          | Julia                           | Python                         |
|-------------------|---------------------------------|--------------------------------|
| Hiệu năng         | Gần C/Fortran (JIT compilation) | Chậm hơn 10-100x (interpreter) |
| Hệ sinh thái      | Đang phát triển (~10k packages) | Rất lớn (>400k packages)       |
| Học máy           | Flux.jl, MLJ.jl                 | PyTorch, TensorFlow (dominant) |
| Cú pháp           | Gần Python, nhưng 1-indexed     | Pythonic, 0-indexed            |
| Multiple dispatch | Core feature                    | Single dispatch (OOP)          |

**Khi nào dùng Julia:** Scientific computing, phương trình vi phân, optimization, cần viết custom algorithms với performance cao.

**Khi nào dùng Python:** Web development, automation, hệ sinh thái ML/DL mature.

### 1.5.2 Julia vs R

| Tiêu chí            | Julia                        | R                               |
|---------------------|------------------------------|---------------------------------|
| Hiệu năng           | 10-100x nhanh hơn R          | Chậm (interpreter)              |
| Thống kê            | Đầy đủ nhưng ít hơn R        | Dominant (ggplot2, dplyr, etc.) |
| General programming | Ngôn ngữ đa năng             | Chủ yếu thống kê                |
| DataFrames          | DataFrames.jl (syntax gần R) | Tidyverse (mature ecosystem)    |

**Khi nào dùng Julia:** Cần performance + statistical computing, large-scale simulations.

**Khi nào dùng R:** Statistical analysis, biostatistics, academic research với ggplot2.

### 1.5.3 Julia vs MATLAB

| Tiêu chí          | Julia                      | MATLAB                       |
|-------------------|----------------------------|------------------------------|
| Giá               | Miễn phí, open-source      | Trả phí (expensive)          |
| Hiệu năng         | Tương đương hoặc nhanh hơn | Nhanh nhưng đắt              |
| Cú pháp           | Gần MATLAB (dễ chuyển đổi) | Proprietary syntax           |
| Hệ sinh thái      | Đang phát triển            | Mature (Simulink, toolboxes) |
| Matrix operations | Native support (tốt)       | Excellent (core strength)    |

**Khi nào dùng Julia:** Thay thế MATLAB miễn phí, research không yêu cầu Simulink.

**Khi nào dùng MATLAB:** Industry standard, Simulink required, budget sẵn có.

### 1.5.4 Benchmark performance

Theo Computer Language Benchmarks Game và nghiên cứu của Julia creators:

- Julia: 1-2x chậm hơn C (nhưng gần như tương đương)
- Python: 10-100x chậm hơn Julia
- R: 20-100x chậm hơn Julia
- MATLAB: Tương đương Julia trong matrix operations, chậm hơn trong general code

**Lưu ý:** Performance phụ thuộc nhiều vào cách viết code. Julia cho phép viết code high-level với performance low-level.

## 1.6 Tóm tắt chương

- **[Lịch sử Julia]:** [TODO: Tóm tắt lịch sử và mục tiêu thiết kế]
- **[Multiple Dispatch]:** [TODO: Key point về multiple dispatch]
- **[JIT Compilation]:** [TODO: Key point về performance]
- **[Cài đặt]:** [TODO: Các phương pháp cài đặt Julia]
- **[So sánh]:** [TODO: Julia vs Python/R/MATLAB]

## 1.7 Bài tập

### 1.7.1 Bài tập cơ bản (\*)

#### Bài tập: Cài đặt Julia

Cài đặt Julia trên máy tính sử dụng juliaup. Sau khi cài đặt, xác nhận bằng các lệnh:

```
1 julia --version
2 # Trong REPL:
3 julia> VERSION
4 julia> Sys.MACHINE
```

**Bài tập: Làm quen REPL**

Thực hành trong Julia REPL: (1) Tính  $2 + 3 \times 4$ , (2) Gán biến  $x = 10, y = 20$ , (3) Sử dụng Help mode với `?println`, (4) Sử dụng Shell mode với `;pwd`.

**Bài tập: Các phép toán cơ bản**

Tính trong REPL: (a)  $\sqrt{144} + 2^{10}$ , (b)  $\sin(\pi/4) \times \cos(\pi/4)$ , (c)  $(3+4i) \times (1-2i)$ , (d)  $\log_{10}(1000)$ .

**Bài tập: Unicode và ký tự đặc biệt**

Julia hỗ trợ Unicode. Thử: `\alpha + Tab`  $\rightarrow \alpha$ , `\sum + Tab`  $\rightarrow \sum$ . Tạo 3 biến với ký tự Hy Lạp.

**Bài tập: Khám phá Package Manager**

Mở Pkg mode (`]` ) và thực hiện: `status`, `add Example`, `rm Example`, `? (help)`.

**1.7.2 Bài tập trung bình (\*\*)****Bài tập: So sánh performance với Python**

Viết hàm tính tổng từ 1 đến  $n$  bằng Julia và Python, so sánh thời gian với `@time`. Ghi nhận speedup ratio.

```

1 function sum_to_n(n)
2     total = 0
3     for i in 1:n
4         total += i
5     end
6     return total
7 end
8 @time sum_to_n(100_000_000)
```

**Bài tập: Tạo Project Environment**

Tạo project mới: `mkdir MyProject && cd MyProject`, `julia --project=.`, trong REPL: `pkg> activate .`, `pkg> add Statistics`. Kiểm tra `Project.toml`.

**Bài tập: Sử dụng Documentation**

Tra cứu và sử dụng: `rand()`, `collect()`, `enumerate()`, `zip()`. Ghi lại cú pháp và ví dụ cho mỗi hàm.

### 1.7.3 Bài tập nâng cao (\*\*\*)

#### Bài tập: Benchmark với BenchmarkTools

Cài và sử dụng BenchmarkTools.jl để so sánh 3 cách tính tổng:

```
1 using BenchmarkTools
2 data = rand(1_000_000)
3 @btime sum($data)
4 @btime reduce(+, $data)
5 @btime foldl(+, $data)
```

Giải thích tại sao `sum()` nhanh nhất.

#### Bài tập: Mini-project: Fibonacci comparison

Implement tính Fibonacci thứ  $n$  bằng 3 cách: (1) đệ quy naive, (2) memoization, (3) iterative. So sánh performance với  $n = 35$ . Viết báo cáo ngắn về độ phức tạp và ưu điểm của Julia.

## Đọc thêm

- Bezanson et al. (2017): "Julia: A Fresh Approach to Numerical Computing"
- Julia Documentation: <https://docs.julialang.org>
- Thêm tài liệu tham khảo]



## Chương 2

# Cú pháp cơ bản

### Mục tiêu học tập

- Nắm vững các kiểu dữ liệu cơ bản trong Julia
- Hiểu scope và phạm vi biến
- Sử dụng toán tử và broadcasting
- Làm việc với files, modules và packages

### 2.1 Kiểu dữ liệu cơ bản

Julia cung cấp nhiều kiểu dữ liệu tích hợp:

- `Int` — số nguyên có dấu (tùy hệ thống: 32 hoặc 64 bit).
- `Float64` — số thực dấu chấm động chính xác kép.
- `Bool` — giá trị logic `true` hoặc `false`.
- `String` — chuỗi Unicode.

```
1 a::Int = 42
2 b::Float64 = 3.1415
3 flag::Bool = true
4 s::String = "Xin chào Julia!"
```

Các hàm kiểm tra kiểu:

```
1 typeof(a) # Int64
2 isa(b, Float64) # true
```

#### 2.1.1 Kiểu số nguyên (Integer Types)

Julia cung cấp nhiều kiểu số nguyên với độ rộng bit khác nhau, cho phép tối ưu hóa bộ nhớ và hiệu năng:

**Signed integers (có dấu):**

| Kiểu   | Bits  | Phạm vi                                                | Ghi chú              |
|--------|-------|--------------------------------------------------------|----------------------|
| Int8   | 8     | −128 đến 127                                           | Tiết kiệm bộ nhớ     |
| Int16  | 16    | −32,768 đến 32,767                                     |                      |
| Int32  | 32    | $\approx -2.1 \times 10^9$ đến $2.1 \times 10^9$       | Mặc định trên 32-bit |
| Int64  | 64    | $\approx -9.2 \times 10^{18}$ đến $9.2 \times 10^{18}$ | Mặc định trên 64-bit |
| Int128 | 128   | $\approx \pm 1.7 \times 10^{38}$                       | Số rất lớn           |
| BigInt | Tùy ý | Không giới hạn                                         | Độ chính xác tùy ý   |

### Unsigned integers (không dấu):

```
1 using CUDA
2 A = CuArray(rand(1000,1000))
3 B = A * A' # Matrix multiplication on GPU
```

### Tạo số với kiểu cụ thể:

```
1 a = Int8(100) # Chuyển đổi tường minh
2 b = 0x42 # UInt8 (literal hex)
3 c = 0x42000000 # UInt32 (tự động chọn size)
4
5 # Kiểm tra giá trị min/max
6 typemax(Int64) # 9223372036854775807
7 typemin(Int8) # -128
```

### BigInt cho số cực lớn:

```
1 # Tính factorial lớn — Int64 sẽ tràn số
2 factorial(20) # 2432902008176640000 (OK)
3 factorial(21) # ERROR: OverflowError
4
5 # Dùng BigInt để tránh overflow
6 factorial(big(21)) # 51090942171709440000
7 big(2)^1000 # Số có hơn 300 chữ số
```

### Lưu ý quan trọng:

- Mặc định Julia dùng Int64 (hoặc Int32 trên hệ thống 32-bit)
- Chỉ dùng kiểu nhỏ hơn (Int8, Int16) khi cần tiết kiệm bộ nhớ với dữ liệu lớn
- BigInt chậm hơn native integers — chỉ dùng khi thực sự cần

## 2.1.2 Kiểu số thực (Floating-Point Types)

Julia tuân theo chuẩn IEEE 754 cho số thực dấu phẩy động:

| Kiểu     | Bits  | Độ chính xác | Phạm vi (xấp xỉ)                         |
|----------|-------|--------------|------------------------------------------|
| Float16  | 16    | Half         | $\pm 6.55 \times 10^4$ (3-4 chữ số)      |
| Float32  | 32    | Single       | $\pm 3.4 \times 10^{38}$ (7 chữ số)      |
| Float64  | 64    | Double       | $\pm 1.8 \times 10^{308}$ (15-16 chữ số) |
| BigFloat | Tùy ý | Arbitrary    | Độ chính xác tùy ý                       |

### Tạo số thực với kiểu cụ thể:

```

1 a = 3.14           # Float64 (mặc định)
2 b = 3.14f0         # Float32 (hậu tố f)
3 c = Float16(3.14)  # Float16
4
5 # Kiểm tra giá trị min/max hữu hạn
6 floatmax(Float64)  # 1.7976931348623157e308
7 floatmin(Float64)  # 2.2250738585072014e-308

```

#### Giá trị đặc biệt:

```

1 Inf           # Vô cực dương
2 -Inf          # Vô cực âm
3 NaN           # Not a Number (0/0, Inf-Inf)
4
5 1/0           # Inf
6 0/0           # NaN
7 isinf(1/0)    # true
8 isnan(0/0)    # true
9 isfinite(3.14) # true

```

#### BigFloat cho độ chính xác cao:

```

1 # Float64 có sai số làm tròn
2 0.1 + 0.2           # 0.30000000000000004
3
4 # BigFloat với precision mặc định (256 bits)
5 big(0.1) + big(0.2) # 0.30000000000000000...
6
7 # Thay đổi precision
8 setprecision(512)    # Tăng precision toàn cục
9 BigFloat(π)          # 3.1415926... (nhiều chữ số hơn)

```

#### Lưu ý về hiệu năng:

- Float64 nhanh nhất trên CPU hiện đại (phần cứng hỗ trợ)
- Float32 nhanh gấp 2 lần trên GPU — dùng cho machine learning
- BigFloat chậm hơn đáng kể — chỉ dùng khi cần độ chính xác cao

### 2.1.3 Kiểu Boolean

Boolean trong Julia là kiểu Bool với hai giá trị: true và false.

#### Toán tử logic:

```

1 # AND, OR, NOT
2 true && false      # false (AND)
3 true || false     # true (OR)
4 !true             # false (NOT)
5
6 # Short-circuit evaluation
7 x = 5
8 x > 0 && println("Dương") # In "Dương"
9 x < 0 || println("Không âm") # In "Không âm"

```

#### So sánh và Boolean:

```

1  5 > 3                # true
2  5 == 5               # true
3  5 != 3               # true
4
5  # Chuỗi so sánh (chained comparisons)
6  1 < 5 < 10           # true
7  0 <= x <= 100        # Kiểm tra x trong [0, 100]

```

**Boolean là số:**

```

1  true + true          # 2 (true = 1)
2  3 * false            # 0 (false = 0)
3  sum([true, true, false]) # 2

```

### 2.1.4 Kiểu ký tự (Char) và Unicode

Ký tự trong Julia dùng Char — hỗ trợ đầy đủ Unicode:

```

1  c1 = 'A'              # Char
2  c2 = 'π'              # Unicode char
3  c3 = '□'              # Tiếng Trung
4  c4 = '👉'              # Emoji
5
6  typeof('A')           # Char

```

**Char là số nguyên Unicode:**

```

1  Int('A')              # 65 (mã ASCII)
2  Int('π')              # 960 (mã Unicode)
3  Char(65)              # 'A'
4  Char(0x03A3)          # 'Σ' (Greek Sigma)
5
6  # Toán số học với Char
7  'A' + 1               # 'B'
8  'Z' - 'A'             # 25

```

**Kiểm tra loại ký tự:**

```

1  isdigit('7')          # true
2  isletter('A')          # true
3  isuppercase('A')       # true
4  islowercase('a')       # true
5  isspace(' ')           # true

```

### 2.1.5 Kiểu chuỗi (String)

String trong Julia là chuỗi UTF-8 immutable:

**Tạo chuỗi:**

```

1  s1 = "Hello"           # Chuỗi cơ bản
2  s2 = "Xin chào ☐☐ ☐" # Unicode đầy đủ
3
4  # Chuỗi nhiều dòng
5  s3 = """
6      Dòng 1

```

```

7     Dòng 2
8     ""
9
10    # Raw string (không xử lý escape)
11    s4 = raw"C:\Users\path" # Giữ nguyên backslash

```

### String interpolation:

```

1    name = "Julia"
2    version = 1.10
3
4    "Ngôn ngữ $name phiên bản $version"      # Biến đơn
5    "1 + 2 = $(1 + 2)"                       # Biểu thức
6    "Pi = $(round( $\pi$ , digits=4))"        # Hàm

```

### Nối chuỗi:

```

1    "Hello" * " " * "World" # "Hello World" (dùng *)
2    string("a", "b", 123)   # "ab123"
3    join(["a", "b", "c"], "-") # "a-b-c"

```

### Truy cập và thao tác:

```

1    s = "Julia"
2    s[1]           # 'J' (index 1-based)
3    s[end]         # 'a'
4    s[1:3]         # "Jul" (substring)
5    length(s)      # 5
6
7    # Với Unicode – cẩn thận với index
8    s2 = "Việt Nam"
9    length(s2)     # 8 (ký tự)
10   ncodeunits(s2) # 10 (byte UTF-8)
11   s2[1]          # 'V'
12   # s2[3]        # ERROR – byte thứ 3 không phải char hợp lệ
13   collect(s2)[3] # 'ệ' (dùng collect để iterate an toàn)

```

### Tìm kiếm và thay thế:

```

1    s = "Hello Julia World"
2    occursin("Julia", s)      # true
3    findfirst("Julia", s)     # 7:11 (range)
4    replace(s, "World" => "!") # "Hello Julia !"

```

## 2.1.6 Hệ thống phân cấp kiểu (Type Hierarchy)

Julia có hệ thống kiểu phân cấp mạnh mẽ:

```

1    # Kiểm tra subtype
2    Int64 <: Integer      # true
3    Integer <: Number    # true
4    Float64 <: Real      # true
5    Real <: Number       # true
6
7    # Cây phân cấp số

```

```

8  # Number
9  # └─ Real
10 # └─ Integer
11 # └─ Signed (Int8, Int16, Int32, Int64, Int128, BigInt)
12 # └─ Unsigned (UInt8, UInt16, ...)
13 # └─ AbstractFloat (Float16, Float32, Float64, BigFloat)
14 # └─ Complex

```

### Hàm typeof() và supertypes:

```

1  typeof(42)           # Int64
2  typeof(3.14)         # Float64
3  typeof("hello")     # String
4
5  # Xem cây cha
6  supertype(Int64)     # Signed
7  supertype(Signed)    # Integer
8  supertypes(Int64)    # (Int64, Signed, Integer, Real, Number, Any)

```

### Hàm isa() và dispatch:

```

1  isa(42, Int64)       # true
2  isa(42, Integer)     # true (vì Int64 <: Integer)
3  isa(42, Number)      # true
4
5  # Hữu ích cho type dispatch
6  function process(x::Integer)
7      println("Số nguyên: $x")
8  end
9  function process(x::AbstractFloat)
10     println("Số thực: $x")
11 end

```

## 2.1.7 So sánh kiểu dữ liệu với Python và C++

| Julia       | Python             | C++             | Ghi chú                    |
|-------------|--------------------|-----------------|----------------------------|
| Int64       | int                | int64_t/long    | Julia có nhiều sizes       |
| Float64     | float              | double          | Python float = 64-bit      |
| Bool        | bool               | bool            | Tương tự                   |
| Char        | (không có)         | char            | Python dùng str[0]         |
| String      | str                | std::string     | Julia immutable, UTF-8     |
| BigInt      | int (tự động)      | (thư viện)      | Python int tự động mở rộng |
| Complex{T}  | complex            | std::complex<T> | Julia tham số hóa kiểu     |
| Rational{T} | fractions.Fraction | (thư viện)      | Julia built-in             |

### Điểm khác biệt chính:

- **Python:** Dynamic typing, int tự động thành big int khi cần
- **C++:** Static typing, phải khai báo kiểu, không có BigInt built-in
- **Julia:** Static typing với type inference, nhiều numeric types, parametric types

### Ví dụ so sánh overflow:

```

1 # Julia - phải explicit dùng BigInt
2 typemax(Int64) + 1 # Overflow! (số âm)
3 big(typemax(Int64)) + 1 # OK, BigInt
4
5 # Python - tự động mở rộng
6 # >>> 2**63 - 1 + 1 # OK, Python xử lý tự động

```

### 2.1.8 Các kiểu dữ liệu khác

Julia cung cấp nhiều kiểu dữ liệu phong phú hơn:

**Char** — ký tự đơn:

```

1 c::Char = 'A'
2 unicode_char = 'π'
3 typeof('7') # Char (khác với "7" là String)

```

**Complex** — số phức:

```

1 z1 = 2 + 3im # Complex{Int64}
2 z2 = 2.0 + 3.0im # ComplexF64 (alias cho Complex{Float64})
3 abs(z1) # 3.605... (modulus)
4 angle(z2) # 0.982... (argument/phase)

```

**Rational** — phân số:

```

1 r = 3//4 # Rational{Int64}
2 r + 1//2 # 5//4 (tự động rút gọn)
3 float(r) # 0.75 (chuyển sang Float64)

```

### 2.1.9 Chuyển đổi kiểu (Type Conversion)

Julia hỗ trợ chuyển đổi kiểu tường minh và tự động:

**Chuyển đổi tường minh:**

```

1 # Float -> Int: PHẢI dùng trunc/floor/ceil
2 trunc(Int, 3.14) # 3 (truncate, không làm tròn)
3 floor(Int, 3.14) # 3 (làm tròn xuống)
4 ceil(Int, 3.14) # 4 (làm tròn lên)
5
6 # Int -> Float
7 Float64(42) # 42.0
8
9 # Number -> String
10 string(123) # "123"
11
12 # String -> Number
13 parse(Int, "456") # 456

```

**Chuyển đổi tự động (type promotion):**

```

1 1 + 2.5 # 3.5 (Int promote thành Float64)
2 2 + 3im # 2 + 3im (Int promote thành Complex{Int64})

```

**Kiểm tra tương thích kiểu:**

```
1 Int64 <: Integer      # true (Int64 là subtype của Integer)
2 String <: Integer     # false
3 Float64 <: Real       # true
```

**Lưu ý:** Chuyển đổi không hợp lệ sẽ báo lỗi:

```
1 Int64(3.14)           # ERROR: InexactError (mất thông tin)
2 Int64("hello")        # ERROR: MethodError
3 parse{Int, "3.14"}     # ERROR: ArgumentError (không phải integer string)
```

Julia yêu cầu explicit conversion khi có khả năng mất thông tin (ví dụ: float→int). Điều này giúp tránh bugs từ implicit conversions.

## 2.2 Biến và phạm vi (Scope)

Phạm vi (scope) xác định nơi biến có hiệu lực:

- **Global scope:** định nghĩa ngoài hàm hoặc module.
- **Local scope:** định nghĩa trong hàm, vòng lặp, hoặc block.

```
1 x = 10 # global
2
3 function demo()
4     x = 5 # local
5     println("Trong hàm: x = $x")
6 end
7
8 demo()
9 println("Bên ngoài: x = $x")
```

Khi viết script, tránh thay đổi biến toàn cục trong hàm — sẽ làm giảm hiệu năng.

### 2.2.1 Quy tắc đặt tên biến (Naming Conventions)

Julia có quy tắc đặt tên linh hoạt, hỗ trợ Unicode đầy đủ:

**Quy tắc cơ bản:**

- Tên biến phân biệt hoa/thường (x khác X)
- Bắt đầu bằng chữ cái (a-z, A-Z), underscore (`_`), hoặc Unicode > 00A0
- Có thể chứa số, `!`, và các ký tự Unicode
- Không được trùng với từ khóa (`if`, `for`, `function`,...)

**Hỗ trợ Unicode — điểm mạnh của Julia:**

```
1 # Ký tự đặc biệt trong tên biến
2 α = 0.5           # Greek letters
3 β□ = 0.9          # Subscripts
4 x̄ = [1, 2, 3]      # Accent marks
5 ∑ = sum            # Symbols
6
7 # Ví dụ thực tế: công thức toán học
```



```

8  ε = 0.0001           # Epsilon cho độ chính xác
9  Δt = 0.01           # Delta time
10 θ = π/4             # Góc theta
11
12 # Dùng trong công thức
13 f(x) = sin(θ) * x + ε

```

### Cách gõ ký tự Unicode trong REPL:

```

1  # Gõ \alpha rồi nhấn Tab → α
2  # Gõ \epsilon rồi nhấn Tab → ε
3  # Gõ \_1 rồi nhấn Tab → 1 (subscript)
4  # Gõ \hat{x} rồi nhấn Tab →  $\hat{x}$ 
5
6  # Kiểm tra cách gõ: vào help mode (?) và paste ký tự
7  # help?> ε
8  # "ε" can be typed by \varepsilon<tab>

```

### Conventions theo cộng đồng Julia:

| Loại              | Convention                                    | Ví dụ                                       |
|-------------------|-----------------------------------------------|---------------------------------------------|
| Biến thông thường | lowercase, có thể dùng <code>_</code>         | <code>my_var</code> , <code>count</code>    |
| Hằng số           | UPPERCASE với <code>_</code>                  | <code>MAX_SIZE</code> , <code>PI</code>     |
| Kiểu/Struct       | UpperCamelCase                                | <code>MyType</code> , <code>Point2D</code>  |
| Hàm               | lowercase, không dùng <code>_</code>          | <code>sqrt</code> , <code>pushfirst!</code> |
| Hàm mutating      | Kết thúc bằng <code>!</code>                  | <code>push!</code> , <code>sort!</code>     |
| Hàm predicate     | Kết thúc bằng <code>?</code> (không bắt buộc) | <code>isempty</code> , <code>haskey</code>  |

**Lưu ý:** Unicode rất hữu ích khi viết code khoa học — code giống công thức trong tài liệu. Tuy nhiên, cần nhắc khả năng đọc cho team.

### 2.2.2 Soft Scope vs Hard Scope

Julia có hai loại scope khác biệt quan trọng:

**Hard scope** — tạo bởi:

- Functions và anonymous functions
- do blocks
- Struct definitions
- Modules

**Soft scope** — tạo bởi:

- for và while loops
- try-catch blocks
- Comprehensions

#### Sự khác biệt quan trọng:

Trong **hard scope**, biến local luôn được tạo mới:

```

1 x = 10
2 function foo()
3     x = 5      # Tạo local x MỚI, không ảnh hưởng global
4     return x
5 end
6 foo()          # 5
7 println(x)     # 10 (global x không đổi)

```

Trong **soft scope**, hành vi KHÁC NHAU giữa REPL và script:

```

1 # === TRONG REPL (interactive mode) ===
2 x = 10
3 for i in 1:3
4     x = i      # Gán vào global x (soft scope)
5 end
6 println(x)     # 3 (global x đã thay đổi!)
7
8 # === TRONG SCRIPT (.jl file) ===
9 x = 10
10 for i in 1:3
11     x = i      # Tạo local x MỚI!
12 end
13 println(x)     # 10 (global x không đổi)

```

Tại sao có sự khác biệt này?

- **REPL:** Tiện lợi cho thử nghiệm nhanh — không cần global
- **Script:** An toàn hơn — tránh vô tình thay đổi global state

Best practice để code chạy đúng ở cả hai môi trường:

```

1 # Cách 1: Dùng global keyword (tuồng minh)
2 x = 0
3 for i in 1:5
4     global x = i
5 end
6
7 # Cách 2: Đưa code vào function (recommended)
8 function update_x()
9     local x = 0
10    for i in 1:5
11        x = i
12    end
13    return x
14 end
15
16 # Cách 3: Dùng let block
17 x = let x = 0
18     for i in 1:5
19         x = i
20     end
21     x
22 end

```

### 2.2.3 Các construct KHÔNG tạo scope mới

Quan trọng: if-else và begin-end KHÔNG tạo scope mới:

```

1 function example()
2     if true
3         y = 10      # y được tạo trong function scope
4     end
5     return y        # OK! y accessible ở đây
6 end
7
8 # begin-end cũng không tạo scope
9 begin
10     z = 100
11 end
12 println(z)         # 100 (z "leak" ra ngoài)

```

### 2.2.4 Vấn đề hiệu năng với Global Variables

Tại sao global variables làm chậm code?

Julia compiler cần biết kiểu dữ liệu để tối ưu hóa. Với global variables:

- Kiểu có thể thay đổi bất cứ lúc nào bởi code khác
- Compiler không thể infer type → không thể tối ưu
- Mỗi lần access phải check type → overhead lớn

Ví dụ so sánh hiệu năng:

```

1 # CHẬM - dùng global variable
2 data = rand(1000000)
3
4 function sum_global()
5     s = 0.0
6     for x in data      # data là global!
7         s += x
8     end
9     return s
10 end
11
12 # NHANH - truyền qua parameter
13 function sum_param(arr)
14     s = 0.0
15     for x in arr        # arr là local parameter
16         s += x
17     end
18     return s
19 end
20
21 # Benchmark: sum_param(data) nhanh hơn 10-100x!

```

Giải pháp cho global variables:

1. Dùng const cho constants:

```
1 const GLOBAL_DATA = rand(1000000)  # Type-stable
2
3 function process()
4     s = 0.0
5     for x in GLOBAL_DATA  # Compiler biết type
6         s += x
7     end
8     return s
9 end
```

## 2. Type annotation (Julia 1.8+):

```
1 # Julia 1.8+ cho phép annotate type cho global
2 global_data::Vector{Float64} = rand(1000000)
```

## 3. Truyền qua function arguments (recommended):

```
1 function main()
2     data = rand(1000000)  # local
3     result = process(data) # truyền vào
4     return result
5 end
```

### Quy tắc vàng:

- Dùng global variables cho **config/constants** — đánh dấu **const**
- Truyền data qua **function arguments** — không dùng global
- Trong performance-critical code — **KHÔNG BAO GIỜ** dùng untyped globals

### 2.2.5 Biến hằng (const)

Khai báo biến hằng giúp compiler tối ưu hóa:

```
1 const PI_VALUE = 3.14159
2 const MAX_USERS = 1000
3
4 # Cố gắng thay đổi sẽ cảnh báo (nhưng vẫn cho phép)
5 PI_VALUE = 3.14  # WARNING: redefining constant
```

**const** chủ yếu dùng cho global constants. Trong hàm không cần thiết vì Julia đã tối ưu local variables.

### 2.2.6 Từ khóa global

Khi cần sửa đổi biến global trong hàm/loop:

```
1 counter = 0
2
3 function increment()
4     global counter  # Bắt buộc khi muốn thay đổi global variable
5     counter += 1
6 end
7
8 increment()
9 println(counter)  # 1
```

**Best practice:** Tránh global variables — truyền qua tham số thay vì dùng global.

### 2.2.7 Let blocks

let tạo scope cục bộ mới, hữu ích khi cần biến tạm:

```

1 x = 10
2
3 let x = 5
4     println("Trong let: x = $x") # 5
5 end
6
7 println("Bên ngoài: x = $x")    # 10

```

Ứng dụng phổ biến: tránh biến tạm "leak" ra ngoài scope:

```

1 # Không dùng let - biến i sẽ tồn tại sau loop
2 for i in 1:3
3     # ...
4 end
5 println(i) # 3 (biến i vẫn tồn tại)
6
7 # Dùng let - biến i chỉ tồn tại trong let block
8 let
9     for i in 1:3
10        # ...
11    end
12    # i không tồn tại ở đây
13 end

```

### 2.2.8 Scope rules trong loops

Julia có quy tắc scope khác nhau giữa REPL và script files:

- **Trong REPL:** Loop variables tự động là global nếu đã tồn tại
- **Trong files:** Loop variables luôn là local (cần `global` để modify global vars)

Ví dụ trong file `.jl`:

```

1 x = 0
2 for i in 1:5
3     x = i # Tạo local x mới, không thay đổi global x
4 end
5 println(x) # 0 (global x không đổi)
6
7 # Muốn thay đổi global x:
8 x = 0
9 for i in 1:5
10    global x = i
11 end
12 println(x) # 5

```

### 2.2.9 Variable Binding và Shadowing

Trong Julia, toán tử = thực hiện **binding** (gắn kết) chứ không phải **assignment** theo nghĩa truyền thống:

**Binding vs Assignment:**

```

1  # Binding: gán tên 'x' vào giá trị 2
2  x = 2
3  # x bây giờ "trò" đến giá trị 2
4
5  # Rebinding: gán lại tên 'x' vào giá trị mới
6  x = x + 3
7  # x bây giờ "trò" đến giá trị 5
8
9  # Với mutable objects
10 arr = [1, 2, 3]
11 arr2 = arr          # arr2 và arr trỏ cùng một array
12 arr[1] = 100        # Thay đổi arr cũng thay đổi arr2
13 println(arr2)       # [100, 2, 3]

```

### Variable Shadowing — che khuất biến:

```

1  x = 10  # global x
2
3  function outer()
4      x = 20  # local x che khuất global x
5
6      function inner()
7          x = 30  # local x che khuất outer's x
8          println("inner: x = $x")  # 30
9      end
10
11     inner()
12     println("outer: x = $x")  # 20
13 end
14
15 outer()
16 println("global: x = $x")  # 10

```

### Cảnh báo shadowing trong loops:

```

1  # Khi chạy script file, Julia cảnh báo về ambiguity
2  x = 0
3  for i in 1:3
4      x = i  # WARNING: Assignment to 'x' in soft scope...
5  end
6
7  # Giải pháp: tường minh với local hoặc global
8  for i in 1:3
9      local x = i  # Rõ ràng: tạo local mới
10 end
11
12 for i in 1:3
13     global x = i  # Rõ ràng: dùng global
14 end

```

## 2.2.10 Common Pitfalls và Best Practices

### Pitfall 1: Quên khác biệt REPL vs Script

```

1  # Code này hoạt động trong REPL
2  sum = 0
3  for i in 1:10
4      sum += i
5  end
6  println(sum) # 55 trong REPL, 0 trong script!
7
8  # Fix: đưa vào function
9  function calculate_sum()
10     sum = 0
11     for i in 1:10
12         sum += i
13     end
14     return sum
15 end

```

### Pitfall 2: Performance trap với globals

```

1  # BAD: type-unstable global
2  data = [1, 2, 3] # Julia không biết type lúc compile
3
4  function slow_sum()
5      s = 0
6      for x in data # Phải check type mỗi iteration
7          s += x
8      end
9      return s
10 end
11
12 # GOOD: type-stable với const hoặc parameter
13 const DATA = [1, 2, 3] # Type known at compile time
14
15 function fast_sum(arr) # Hoặc truyền parameter
16     s = 0
17     for x in arr
18         s += x
19     end
20     return s
21 end

```

### Pitfall 3: Closure capturing

```

1  # Tạo nhiều functions, mỗi function "nhớ" i
2  functions = []
3  for i in 1:3
4      push!(functions, () -> i)
5  end
6
7  # Kết quả có thể bất ngờ!
8  [f() for f in functions] # [3, 3, 3] trong một số phiên bản
9
10 # Fix: dùng let để capture giá trị
11 functions = []
12 for i in 1:3

```

```

13     let i = i
14     push!(functions, () -> i)
15 end
16 end
17 [f() for f in functions] # [1, 2, 3]

```

### Best Practices tổng kết:

1. **Đặt code trong functions:** Tránh hoàn toàn vấn đề scope và tối ưu hiệu năng
2. **Dùng const cho global constants:** Compiler biết type → nhanh hơn
3. **Tường minh local/global:** Tránh ambiguity warnings
4. **Truyền data qua parameters:** Không dùng global variables cho data
5. **Dùng let cho closures:** Đảm bảo capture đúng giá trị
6. **Test cả REPL và script:** Hành vi có thể khác nhau

## 2.2.11 Debugging Scope Issues

### Kiểm tra scope của biến:

```

1 # Dùng @code_warntype để phát hiện type instability
2 @code_warntype slow_sum()
3 # Nếu thấy Any hoặc Union → có vấn đề với global variables
4
5 # Kiểm tra biến nào là global
6 function debug_scope()
7     x = 1
8     @show typeof(x)
9     @show isdefined(Main, :x) # Kiểm tra có global x không
10 end

```

### Dùng modules để tổ chức scope:

```

1 module MyModule
2     export my_function
3
4     # Constants trong module scope
5     const CONFIG = Dict{"max" => 100}
6
7     # Private functions
8     helper() = println("internal")
9
10    # Exported functions
11    function my_function()
12        helper()
13        return CONFIG["max"]
14    end
15 end
16
17 using .MyModule
18 my_function() # Works
19 # helper()    # ERROR: not exported

```



## 2.3 Toán tử, biểu thức và chuỗi format

Các toán tử phổ biến: +, -, \*, /,  $\square$ , %, ==, !=, <, >, <=, >=, &&, ||, !.

```
1 x, y = 3, 4
2 z = x^2 + y^2
3 println("z = \$z") # nội suy chuỗi
4 @printf("Giá trị trung bình: %.2f\n", (x+y)/2)
```

### 2.3.1 Broadcasting (toán tử chấm .)

Broadcasting [4] cho phép áp dụng phép toán element-wise trên mảng:

**Cú pháp:** Thêm dấu chấm . trước operator hoặc function

```
1 # Không broadcast - lỗi vì [1,2,3] + 10 không hợp lệ
2 a = [1, 2, 3]
3 # a + 10 # ERROR: MethodError
4
5 # Broadcast - cộng 10 vào từng phần tử
6 b = a .+ 10 # [11, 12, 13]
7 c = a .* 2 # [2, 4, 6]
8 d = a .^ 2 # [1, 4, 9]
```

**Broadcasting với functions:**

```
1 sqrt([1, 4, 9]) # [1.0, 2.0, 3.0]
2 sin([0,  $\pi/2$ ,  $\pi$ ]) # [0.0, 1.0, 0.0]
3
4 # Tương đương với map:
5 map(sqrt, [1, 4, 9]) # [1.0, 2.0, 3.0]
```

**Broadcasting với nhiều mảng:**

```
1 x = [1, 2, 3]
2 y = [4, 5, 6]
3 x .+ y # [5, 7, 9] (element-wise addition)
4
5 # Broadcasting với size khác nhau (Hadamard product)
6 A = [1 2; 3 4] # 2x2 matrix
7 v = [10, 20] # 2-element vector
8 A .* v # [10 20; 60 80] (broadcast v theo columns)
```

**Macro @. để broadcast tất cả operators:**

```
1 @. a + 10 * sin(a) # Tương đương: a .+ 10 .* sin.(a)
```

### 2.3.2 Thứ tự ưu tiên toán tử (Operator Precedence)

Julia tuân theo thứ tự ưu tiên toán học chuẩn:

| Nhóm                     | Operators                                     |
|--------------------------|-----------------------------------------------|
| Cao nhất: Exponentiation | <code>^</code>                                |
| Unary operators          | <code>+, -, !, ~</code>                       |
| Multiplication/Division  | <code>*, /, %, ÷, //</code>                   |
| Addition/Subtraction     | <code>+, -</code>                             |
| Bitwise shift            | <code>&lt;&lt;, &gt;&gt;</code>               |
| Comparison               | <code>&lt;, &lt;=, &gt;, &gt;=, ==, !=</code> |
| Logical AND              | <code>&amp;&amp;</code>                       |
| Logical OR               | <code>  </code>                               |
| Thấp nhất: Assignment    | <code>=, +=, -=, *=</code>                    |

Ví dụ:

```
1 3 + 2 * 5           # 13 (nhân trước, cộng sau)
2 2^3^2               # 512 (lũy thừa kết hợp phải: 2^(3^2))
3 true || false && false # true (&& trước ||)
```

Bitwise operators:

```
1 5 & 3               # 1 (bitwise AND: 101 & 011 = 001)
2 5 | 3               # 7 (bitwise OR: 101 | 011 = 111)
3 5 ⊕ 3               # 6 (bitwise XOR: 101 ⊕ 011 = 110)
4 ~5                  # -6 (bitwise NOT)
5 5 << 1              # 10 (shift left: 101 -> 1010)
```

2.3.3 Các toán tử số học (Arithmetic Operators)

Julia hỗ trợ đầy đủ các toán tử số học cơ bản và mở rộng:

| Toán tử         | Ví dụ                      | Mô tả                  |
|-----------------|----------------------------|------------------------|
| <code>+</code>  | <code>3 + 2 → 5</code>     | Cộng                   |
| <code>-</code>  | <code>3 - 2 → 1</code>     | Trừ                    |
| <code>*</code>  | <code>3 * 2 → 6</code>     | Nhân                   |
| <code>/</code>  | <code>7 / 2 → 3.5</code>   | Chia (kết quả Float64) |
| <code>÷</code>  | <code>7 ÷ 2 → 3</code>     | Chia nguyên (div)      |
| <code>%</code>  | <code>7 % 2 → 1</code>     | Phần dư (mod)          |
| <code>^</code>  | <code>2 ^ 3 → 8</code>     | Lũy thừa               |
| <code>//</code> | <code>3 // 4 → 3//4</code> | Phân số (Rational)     |

Toán tử gán kết hợp (Compound Assignment):

```
1 x = 10
2 x += 5           # x = x + 5 → 15
3 x -= 3           # x = x - 3 → 12
4 x *= 2           # x = x * 2 → 24
5 x /= 4           # x = x / 4 → 6.0
6 x ^= 2           # x = x ^ 2 → 36.0
```

Phân số và số phức:

```
1 # Rational numbers (phân số chính xác)
2 r = 1//3 + 1//6           # 1//2 (tự động rút gọn)
3 r * 6                     # 3//1
```

```

4
5 # Complex numbers (số phức)
6 z = 3 + 4im           # Complex{Int64}
7 abs(z)                # 5.0 (module)
8 angle(z)              # 0.9272... (góc pha)
9 real(z), imag(z)      # (3, 4)
10 conj(z)              # 3 - 4im (liên hợp)

```

### 2.3.4 Toán tử so sánh và logic (Comparison & Boolean)

**Toán tử so sánh:**

| Toán tử | Ví dụ               | Mô tả                   |
|---------|---------------------|-------------------------|
| ==      | 3 == 3 → true       | Bằng (so sánh giá trị)  |
| !=, ≠   | 3 != 4 → true       | Không bằng              |
| <       | 2 < 5 → true        | Nhỏ hơn                 |
| <=, ≤   | 3 <= 3 → true       | Nhỏ hơn hoặc bằng       |
| >       | 5 > 2 → true        | Lớn hơn                 |
| >=, ≥   | 3 >= 3 → true       | Lớn hơn hoặc bằng       |
| ===     | [1] === [1] → false | Đồng nhất (cùng object) |

**So sánh chuỗi (Chained Comparisons):**

```

1 # Julia cho phép so sánh chuỗi – rất thuận tiện!
2 x = 5
3 1 < x < 10           # true (tương đương 1 < x && x < 10)
4 0 <= x <= 100        # true
5 1 < 2 < 3 < 4        # true (đánh giá từ trái sang phải)

```

**Toán tử logic:**

```

1 # AND, OR, NOT
2 true && false         # false (AND)
3 true || false        # true  (OR)
4 !true                # false (NOT)
5
6 # Short-circuit evaluation (đánh giá ngắn mạch)
7 x = 5
8 x > 0 && println("x dương") # In "x dương"
9 x < 0 || println("x không âm") # In "x không âm"
10
11 # Ternary operator (toán tử ba ngôi)
12 result = x > 0 ? "dương" : "không dương" # "dương"

```

### 2.3.5 Xử lý chuỗi (String Operations)

Chuỗi trong Julia là immutable và được đặt trong dấu ngoặc kép "...".

**Tạo và nối chuỗi:**

```

1 s1 = "Hello"
2 s2 = "Julia"
3
4 # Nối chuỗi bằng * (không phải +)
5 greeting = s1 * " " * s2           # "Hello Julia"

```

```

6
7 # Lặp chuỗi bằng ^
8 stars = "*" ^ 10           # "*****"
9
10 # Nội suy chuỗi (string interpolation)
11 name = "Đức"
12 age = 25
13 msg = "Xin chào $name, bạn $age tuổi"
14 calc = "2 + 3 = $(2 + 3)"  # "2 + 3 = 5"

```

### Các hàm xử lý chuỗi thường dùng:

```

1 s = "  Julia Programming  "
2
3 # Độ dài và truy cập
4 length(s)           # 22
5 sizeof(s)           # 22 (bytes)
6 s[3]                # 'J' (Char)
7 s[3:7]              # "Julia"
8
9 # Cắt khoảng trắng
10 strip(s)            # "Julia Programming"
11 lstrip(s)           # "Julia Programming "
12 rstrip(s)           # "  Julia Programming"
13
14 # Chuyển đổi case
15 uppercase("julia")  # "JULIA"
16 lowercase("JULIA")  # "julia"
17 titlecase("hello world") # "Hello World"
18
19 # Kiểm tra chuỗi
20 startswith("Julia", "Jul") # true
21 endswith("Julia", "lia")   # true
22 contains("Julia", "li")    # true
23 occursin("li", "Julia")    # true (thứ tự ngược)

```

### Tách và nối chuỗi:

```

1 # split - tách chuỗi thành mảng
2 words = split("apple,banana,cherry", ",") # ["apple", "banana", "cherry"]
3 parts = split(" a b c ")                  # ["a", "b", "c"] (bỏ khoảng
   ↪   trắng)
4
5 # join - nối mảng thành chuỗi
6 join(["A", "B", "C"], "-")                # "A-B-C"
7 join(1:5, ", ")                          # "1, 2, 3, 4, 5"
8
9 # replace - thay thế
10 replace("Hello World", "World" => "Julia") # "Hello Julia"
11 replace("aabbcc", "b" => "X")              # "aaXXcc"

```

### Tìm kiếm trong chuỗi:

```

1 s = "Julia is great for data science"
2

```

```

3 # findfirst, findlast - vị trí ký tự/substring
4 findfirst('i', s)           # 4 (vị trí 'i' đầu tiên)
5 findlast('a', s)           # 29
6 findfirst("data", s)       # 20:23 (range)
7
8 # findall - tất cả vị trí
9 findall('a', s)            # [5, 15, 20, 29]

```

### 2.3.6 Biểu thức chính quy (Regular Expressions)

Julia sử dụng PCRE (Perl-Compatible Regular Expressions) với cú pháp `r"pattern"`.

#### Cú pháp cơ bản:

```

1 # Tạo regex
2 rx = r"\d+"                # Khớp 1+ chữ số
3
4 # occursin - kiểm tra khớp
5 occursin(r"\d+", "abc123") # true
6 occursin(r"^\d+$", "abc123") # false (phải toàn số)
7
8 # match - tìm khớp đầu tiên
9 m = match(r"\d+", "foo 123 bar 456")
10 m.match                    # "123"
11 m.offset                   # 5 (vị trí bắt đầu)

```

#### Capturing groups:

```

1 # Dùng () để capture
2 rx = r"(\w+)@(\w+)\.(\w+)"
3 m = match(rx, "user@example.com")
4 m.captures                  # ["user", "example", "com"]
5 m[1], m[2], m[3]           # "user", "example", "com"
6
7 # Named captures
8 rx = r"(?<name>\w+)@(?<domain>\w+\.\w+)"
9 m = match(rx, "john@gmail.com")
10 m[:name]                   # "john"
11 m[:domain]                 # "gmail.com"

```

#### Tìm tất cả và thay thế:

```

1 text = "Call 0901234567 or 0912345678"
2
3 # eachmatch - iterator qua tất cả matches
4 for m in eachmatch(r"\d{10}", text)
5     println(m.match)
6 end
7 # Output: 0901234567, 0912345678
8
9 # collect all matches
10 phones = [m.match for m in eachmatch(r"\d{10}", text)]
11 # ["0901234567", "0912345678"]
12
13 # replace với regex

```

```
14 replace(text, r"\d{10}" => "XXX-XXX-XXXX")
15 # "Call XXX-XXX-XXXX or XXX-XXX-XXXX"
16
17 # replace với capture groups
18 replace("John Smith", r"(\w+) (\w+)" => s"\2, \1")
19 # "Smith, John"
```

2.3.7 Định dạng chuỗi với Printf

Module Printf cung cấp các hàm format theo kiểu C.

@printf và @sprintf:

```
1 using Printf
2
3 # @printf - in ra stdout
4 @printf("Số nguyên: %d\n", 42)
5 @printf("Số thực: %.2f\n", 3.14159)
6 @printf("Chuỗi: %s\n", "Julia")
7
8 # @sprintf - trả về String
9 s = @sprintf("Pi = %.4f", pi) # "Pi = 3.1416"
```

Format specifiers:

| Specifier | Ví dụ           | Mô tả                     |
|-----------|-----------------|---------------------------|
| %d, %i    | %d → 42         | Số nguyên                 |
| %f        | %.2f → 3.14     | Số thực (fixed)           |
| %e        | %.2e → 3.14e+00 | Ký hiệu khoa học          |
| %g        | %g              | Tự động (f hoặc e)        |
| %s        | %s → hello      | Chuỗi                     |
| %c        | %c → A          | Ký tự                     |
| %x, %X    | %x → 2a         | Hex (lowercase/uppercase) |
| %o        | %o → 52         | Octal                     |
| %%        | %% → %          | Ký tự %                   |

Padding và alignment:

```
1 # Width và precision
2 @printf("|%10d|", 42) # |          42| (right-align, width 10)
3 @printf("|%-10d|", 42) # |42          | (left-align)
4 @printf("|%010d|", 42) # |0000000042| (zero-padding)
5
6 # Float precision
7 @printf("|%10.3f|", pi) # |      3.142|
8 @printf("|%-10.3f|", pi) # |3.142      |
9
10 # Dynamic width/precision với *
11 width = 8
12 prec = 3
13 @printf("%*.*f", width, prec, pi) # "      3.142"
```

Ví dụ thực tế — Bảng dữ liệu:

```

1  # In bảng có format
2  data = [("Alice", 25, 85.5), ("Bob", 30, 92.3), ("Charlie", 22, 78.9)]
3
4  println("=" ^ 35)
5  @printf("| %-10s | %4s | %6s |\n", "Tên", "Tuổi", "Điểm")
6  println("-" ^ 35)
7  for (name, age, score) in data
8      @printf("| %-10s | %4d | %6.1f |\n", name, age, score)
9  end
10 println("=" ^ 35)
11 # Output:
12 # =====
13 # | Tên          | Tuổi | Điểm |
14 # -----
15 # | Alice        | 25   | 85.5 |
16 # | Bob          | 30   | 92.3 |
17 # | Charlie      | 22   | 78.9 |
18 # =====

```

## 2.4 Tập tin, thư viện và module

Đọc / ghi tập tin:

```

1  open("output.txt", "w") do f
2      write(f, "Xin chào, Julia!\n")
3  end
4
5  content = read("output.txt", String)
6  println(content)

```

Thư viện và module:

```

1  using Statistics, LinearAlgebra
2
3  m = [1 2; 3 4]
4  println(det(m)) # Định thức ma trận

```

### 2.4.1 Import vs Using

Julia có hai cách load modules: using và import

**using** — import tất cả exported names:

```

1  using Statistics          # Tất cả exported functions: mean, std, var,...
2  mean([1, 2, 3])          # 2.0 (dùng trực tiếp)
3
4  # using với specific names:
5  using Statistics: mean, median
6  mean([1,2,3])            # OK
7  std([1,2,3])             # ERROR: std not imported

```

**import** — chỉ import module name:

```

1  import LinearAlgebra
2  LinearAlgebra.det([1 2; 3 4]) # -2.0 (phải dùng prefix)

```

```
3
4  # import với specific names:
5  import LinearAlgebra: det, norm
6  det([1 2; 3 4])          # -2.0 (dùng trực tiếp)
```

#### Khác biệt chính:

- using: Dùng cho standard libraries, dễ đọc hơn
- import: Dùng khi muốn extend functions hoặc tránh name conflicts

### 2.4.2 Include files

include để load code từ file khác:

```
1 include("helper_functions.jl")  # Load và execute file
2 include("config.jl")           # Thường dùng cho config
```

Lưu ý: include khác using — nó copy-paste code vào vị trí hiện tại.

### 2.4.3 Đọc file nâng cao

Đọc từng dòng với readline:

```
1 io = open("data.txt")
2 while !eof(io)
3     line = readline(io)
4     println(line)
5 end
6 close(io)
```

Đọc tất cả dòng với readlines:

```
1 lines = readlines("data.txt")  # Vector{String}
2 for line in lines
3     println(line)
4 end
```

Đọc file CSV thủ công:

```
1 # Ví dụ file CSV: name,age,score
2 io = open("data.csv")
3 header = readline(io)          # Đọc dòng đầu
4 header_cols = split(header, ',')  # ["name", "age", "score"]
5
6 while !eof(io)
7     line = readline(io)
8     values = split(line, ',')
9     name, age, score = values[1], parse{Int}(values[2]), parse{Float64}(
        ↪ values[3])
10    println("$name: $age tuổi, điểm $score")
11 end
12 close(io)
```

Best practice — dùng do blocks:



```

1 # Auto-close file sau khi xong
2 open("data.txt") do io
3     for line in eachline(io)
4         println(line)
5     end
6 end # File tự động close

```

#### 2.4.4 Module namespacing

Khi có name conflicts, dùng qualified names:

```

1 using PackageA
2 using PackageB
3
4 # Nếu cả hai có function foo():
5 PackageA.foo(x)      # Gọi foo từ PackageA
6 PackageB.foo(x)      # Gọi foo từ PackageB

```

Alias modules:

```

1 using LinearAlgebra as LA
2 LA.det([1 2; 3 4])    # -2.0

```

## 2.5 Tóm tắt chương

- [Kiểu dữ liệu]: [TODO: Các kiểu cơ bản - Int, Float, String, Bool]
- [Scope]: [TODO: Global, local, const scope rules]
- [Broadcasting]: [TODO: Toán tử chấm .() cho vectorization]
- [Files & Modules]: [TODO: Import, using, include patterns]

## 2.6 Bài tập

### 2.6.1 Bài tập cơ bản (\*)

Bài tập: Type conversion

Chuyển đổi giữa các kiểu dữ liệu:

```

1 x = 42
2 Float64(x)      # Int -> Float
3 string(x)       # Int -> String
4 parse{Int, "123"} # String -> Int

```

Thực hành: chuyển "3.14" → Float64, true → Int.

Bài tập: Broadcasting với dot syntax

Áp dụng hàm cho từng phần tử với dot syntax:

```

1 arr = [1, 4, 9, 16, 25]
2 sqrt.(arr)      # [1.0, 2.0, 3.0, 4.0, 5.0]
3 arr .+ 10       # Cộng 10 vào mỗi phần tử
4 sin.(arr) .* cos.(arr) # Kết hợp nhiều operations

```

## Bài tập: Kiểu số và precision

So sánh các kiểu số:

```
1 a = 1/3                # Float64
2 b = big"1" / big"3"    # BigFloat
3 c = 1//3               # Rational
```

Tính  $\sum_{i=1}^{100} 1/i$  với Float64 và BigFloat. So sánh kết quả.

## Bài tập: String interpolation và operations

Thực hành với strings:

```
1 name = "Julia"
2 version = 1.10
3 "Ngôn ngữ $name phiên bản $version"
4 "2 + 2 = $(2 + 2)"
5 repeat("Ha", 3)          # "HaHaHa"
6 join(["a", "b", "c"], ", ") # "a, b, c"
```

## Bài tập: Boolean và comparison

Viết expressions cho: (a) kiểm tra x chia hết cho 3 và 5, (b) kiểm tra string bắt đầu bằng "Julia", (c) kiểm tra array có phần tử > 100.

## 2.6.2 Bài tập trung bình (\*\*)

## Bài tập: Scope rules

Giải thích output của code sau:

```
1 x = 10
2 function test()
3     x = 20
4     println("Inside: $x")
5 end
6 test()
7 println("Outside: $x")
```

Sửa để hàm thay đổi giá trị global x.

**Bài tập: File I/O với CSV**

Đọc file CSV và tính tổng kê:

```

1  # Tạo file test
2  open("scores.csv", "w") do io
3      println(io, "name,score")
4      println(io, "An,85")
5      println(io, "Binh,92")
6      println(io, "Cuong,78")
7  end
8
9  # Đọc và tính điểm trung bình
10 # TODO: implement

```

**Bài tập: Comprehensions nâng cao**

Sử dụng comprehensions để: (a) tạo ma trận  $5 \times 5$  với  $a_{ij} = i + j$ , (b) lọc số nguyên tố từ 1-100, (c) tạo dictionary từ hai arrays.

**2.6.3 Bài tập nâng cao (\*\*\*)****Bài tập: Module design**

Tạo module MathUtils với:

```

1  module MathUtils
2  export factorial_recursive, is_prime, gcd_euclidean
3
4  function factorial_recursive(n)
5      # TODO: implement
6  end
7
8  function is_prime(n)
9      # TODO: implement
10 end
11
12 function gcd_euclidean(a, b)
13     # TODO: implement
14 end
15
16 end # module

```

**Bài tập: Mini-project: Text analyzer**

Viết chương trình đọc file text và trả về: (a) số từ, (b) số dòng, (c) 10 từ xuất hiện nhiều nhất, (d) độ dài trung bình của từ. Sử dụng Dict để đếm từ.

**Đọc thêm**

- Julia Documentation: Types - <https://docs.julialang.org/en/v1/manual/types/>
- Julia Documentation: Scope - <https://docs.julialang.org/en/v1/manual/variables-and-scope/>

- Thêm tài liệu tham khảo]

## Chương 3

# Cấu trúc điều khiển và hàm

### Mục tiêu học tập

- Sử dụng các cấu trúc điều khiển (if, for, while)
- Định nghĩa và sử dụng functions
- Hiểu multiple dispatch cơ bản
- Làm việc với closures và do blocks

### 3.1 Cấu trúc điều khiển

Julia cung cấp các cấu trúc điều khiển (control flow structures) tiêu chuẩn tương tự các ngôn ngữ lập trình hiện đại, bao gồm câu lệnh điều kiện (if-elseif-else), vòng lặp (for, while), và xử lý ngoại lệ (try-catch). Các cấu trúc này đều sử dụng từ khóa `end` để đánh dấu kết thúc block, giúp code dễ đọc và rõ ràng hơn so với cú pháp dùng dấu ngoặc nhọn.

#### 3.1.1 Câu lệnh điều kiện if-elseif-else

Câu lệnh `if` cho phép thực thi các khối code khác nhau dựa trên điều kiện Boolean. Julia hỗ trợ đầy đủ các toán tử so sánh (`<`, `>`, `<=`, `>=`, `==`, `!=`) và toán tử logic (`&&`, `||`, `!`).

```
1 # Ví dụ câu lệnh if-elseif-else
2 x = 10
3 if x > 0
4     println("x = $x là số dương")
5 elseif x == 0
6     println("x = $x bằng 0")
7 else
8     println("x = $x là số âm")
9 end
```

#### Giải thích:

- Điều kiện được đánh giá từ trên xuống - nếu `x > 0` là `true`, các điều kiện sau không được kiểm tra
- `elseif` có thể sử dụng nhiều lần để kiểm tra nhiều điều kiện
- Block `else` là tùy chọn, chỉ thực thi khi tất cả điều kiện trước đó đều `false`

### 3.1.1.1 Toán tử ba ngôi (Ternary operator)

Julia hỗ trợ toán tử ba ngôi với cú pháp `condition ? value_if_true : value_if_false`, cho phép viết câu lệnh điều kiện ngắn gọn trên một dòng [2]:

```

1  # Ternary operator cơ bản
2  x = 10
3  result = x > 0 ? "dương" : "không dương"
4  # → "dương"
5
6  # So sánh với if-else tương đương
7  result = if x > 0
8            "dương"
9  else
10           "không dương"
11 end
12
13 # Ternary lồng nhau (nested)
14 grade = 85
15 status = grade >= 90 ? "Xuất sắc" :
16             grade >= 80 ? "Giỏi" :
17             grade >= 70 ? "Khá" : "Trung bình"
18 # → "Giỏi"
```

#### Lưu ý quan trọng:

- Cả hai nhánh `true` và `false` đều phải có giá trị (không thể để trống)
- Ternary operator trả về giá trị, có thể gán trực tiếp cho biến
- Với logic phức tạp (>2 điều kiện), nên dùng `if-elseif-else` để code dễ đọc hơn
- Khác với C/C++, Julia không cho phép side effects trong ternary (như `x > 0 ? println("yes") : println("no")`)

### 3.1.1.2 Short-circuit evaluation

Julia hỗ trợ short-circuit evaluation với `&&` (and) và `||` (or), thường được sử dụng như một dạng rút gọn của câu lệnh điều kiện:

```

1  # && - thực thi về phải nếu về trái true
2  x = 5
3  x > 0 && println("x là số dương")  # In ra: x là số dương
4
5  # || - thực thi về phải nếu về trái false
6  y = -3
7  y > 0 || println("y không dương")  # In ra: y không dương
8
9  # Ứng dụng: kiểm tra điều kiện trước khi thực thi
10 function safe_divide(a, b)
11     b == 0 && return nothing  # Return sớm nếu b = 0
12     return a / b
13 end
14
```

```

15 safe_divide(10, 2)    # → 5.0
16 safe_divide(10, 0)    # → nothing

```

### Pattern thường gặp:

- `condition && action` — thực thi action nếu condition là true
- `condition || action` — thực thi action nếu condition là false
- Thường dùng cho validation, early return, hoặc default values

### 3.1.2 Vòng lặp for

Vòng lặp `for` được sử dụng để duyệt qua các collection (mảng, tuple, ranges, v.v.). Julia sử dụng cú pháp `for item in collection` trực quan và dễ hiểu.

```

1  # Vòng lặp với range
2  for i in 1:5
3      println("i = $i")
4  end
5
6  # Vòng lặp với array
7  fruits = ["táo", "chuối", "cam"]
8  for fruit in fruits
9      println(fruit)
10 end

```

### Đặc điểm quan trọng:

- Biến lặp (`i`, `fruit`) tự động được tạo trong scope của vòng lặp
- Vòng lặp `for` trong Julia là iterator-based, hoạt động với bất kỳ iterable object nào
- Hỗ trợ nested loops và multiple iteration variables

#### 3.1.2.1 enumerate và zip

Hàm `enumerate` trả về cặp (`index`, `value`) khi duyệt collection, hữu ích khi cần cả vị trí và giá trị [2]:

```

1  # enumerate - lấy cả index và value
2  fruits = ["táo", "chuối", "cam", "xoài"]
3  for (i, fruit) in enumerate(fruits)
4      println("$i: $fruit")
5  end
6  # Output:
7  # 1: táo
8  # 2: chuối
9  # 3: cam
10 # 4: xoài
11
12 # So sánh với previous value (pattern từ DKM)
13 values = [10, 5, 15, 8, 20]
14 for (i, val) in enumerate(values)

```

```
15     if i > 1 && val < values[i-1]
16         println("Giảm tại vị trí $i: $(values[i-1]) → $val")
17     end
18 end
19 # → Giảm tại vị trí 2: 10 → 5
20 # → Giảm tại vị trí 4: 15 → 8
```

Hàm zip cho phép duyệt song song nhiều collections cùng lúc:

```
1 # zip - duyệt song song nhiều collections
2 names = ["An", "Bình", "Chi"]
3 ages = [20, 22, 21]
4 cities = ["HCM", "Hà Nội", "Đà Nẵng"]
5
6 for (name, age, city) in zip(names, ages, cities)
7     println("$name, $age tuổi, ở $city")
8 end
9 # Output:
10 # An, 20 tuổi, ở HCM
11 # Bình, 22 tuổi, ở Hà Nội
12 # Chi, 21 tuổi, ở Đà Nẵng
13
14 # Kết hợp enumerate và zip
15 for (i, (name, age)) in enumerate(zip(names, ages))
16     println("$i. $name ($age)")
17 end
```

**Lưu ý:** zip dừng khi collection ngắn nhất hết phần tử. Nếu cần xử lý đầy đủ, dùng package `Iterators.jl` với hàm `zip_longest`.

### 3.1.2.2 Nested loops và multiple iteration

Julia hỗ trợ cú pháp gọn cho nested loops trên cùng một dòng:

```
1 # Nested loops truyền thống
2 for i in 1:3
3     for j in 1:2
4         println("($i, $j)")
5     end
6 end
7
8 # Cú pháp gọn - tương đương nested loops
9 for i in 1:3, j in 1:2
10     println("($i, $j)")
11 end
12 # Output: (1,1), (1,2), (2,1), (2,2), (3,1), (3,2)
13
14 # Ví dụ: tạo bảng cửu chương
15 for i in 2:5, j in 1:10
16     println("$i × $j = $(i*j)")
17 end
18
19 # Với Cartesian product
```



```

20 using Base.Iterators: product
21 for (x, y) in product(1:3, ['a', 'b'])
22     println("($x, $y)")
23 end

```

### 3.1.2.3 Iterating với eachindex và pairs

Julia cung cấp các hàm chuyên biệt để iterate hiệu quả:

```

1  # eachindex - iterate qua indices (tối ưu cho mọi array type)
2  A = [10, 20, 30, 40]
3  for i in eachindex(A)
4      A[i] *= 2 # Modify in-place
5  end
6  # A = [20, 40, 60, 80]
7
8  # pairs - trả về key-value pairs
9  dict = Dict{"a" => 1, "b" => 2, "c" => 3}
10 for (key, value) in pairs(dict)
11     println("$key => $value")
12 end
13
14 # pairs với array
15 arr = ["first", "second", "third"]
16 for (idx, val) in pairs(arr)
17     println("Index $idx: $val")
18 end

```

#### Best practices:

- Dùng eachindex thay vì 1:length(A) để code hoạt động với mọi array type (bao gồm OffsetArrays)
- Dùng pairs khi cần cả key và value từ Dict hoặc array
- Dùng enumerate khi cần index bắt đầu từ 1

### 3.1.3 Vòng lặp while

Vòng lặp while thực thi block code cho đến khi điều kiện trở thành false. Khác với for, while không biết trước số lần lặp.

```

1  # Vòng lặp while
2  i = 1
3  while i <= 5
4      println("i = $i")
5      i += 1
6  end

```

#### Lưu ý về scope:

- Khi chạy trong file .jl, cần sử dụng global keyword để modify biến global: `global i`  
`+= 1`
- Trong REPL, scope rules linh hoạt hơn và không yêu cầu global
- Best practice: tránh dùng global variables, ưu tiên truyền qua tham số hàm

### 3.1.4 break và continue

Julia hỗ trợ hai từ khóa để kiểm soát vòng lặp:

- `break`: thoát khỏi vòng lặp ngay lập tức
- `continue`: bỏ qua phần còn lại của iteration hiện tại, chuyển sang iteration tiếp theo

```
1  # continue: bỏ qua số lẻ
2  for i in 1:5
3      if isodd(i)
4          continue
5      end
6      println("$i là số chẵn")
7  end
8
9  # break: dừng khi i > 3
10 for i in 1:10
11     if i > 3
12         break
13     end
14     println("i = $i")
15 end
```

#### 3.1.4.1 return trong vòng lặp

Từ khóa `return` có thể được sử dụng bên trong vòng lặp để thoát khỏi hàm ngay lập tức:

```
1  # Tìm phần tử đầu tiên thỏa điều kiện
2  function find_first_even(arr)
3      for x in arr
4          if iseven(x)
5              return x  # Thoát hàm ngay khi tìm thấy
6          end
7      end
8      return nothing  # Không tìm thấy
9  end
10
11 find_first_even([1, 3, 5, 4, 7])  # → 4
12 find_first_even([1, 3, 5, 7])    # → nothing
13
14 # Pattern: early return với validation
15 function process_data(data)
16     isempty(data) && return "Empty data"
17     length(data) > 1000 && return "Data too large"
18
19     # Xử lý data...
20     return sum(data)
21 end
```

### 3.1.5 Comprehensions

Comprehensions là cú pháp mạnh mẽ để tạo collections (array, dict, generator) một cách ngắn gọn và hiệu quả. Đây là feature quan trọng kế thừa từ Python nhưng với hiệu năng cao hơn nhờ type inference [2].

### 3.1.5.1 Array comprehension

Array comprehension tạo array từ một biểu thức lặp:

```

1  # Cú pháp cơ bản: [expression for var in collection]
2  squares = [x^2 for x in 1:5]
3  # → [1, 4, 9, 16, 25]
4
5  # So sánh với for loop truyền thống
6  squares_loop = Int[]
7  for x in 1:5
8      push!(squares_loop, x^2)
9  end
10 # squares_loop = [1, 4, 9, 16, 25]
11
12 # Với điều kiện (filter)
13 evens = [x for x in 1:10 if iseven(x)]
14 # → [2, 4, 6, 8, 10]
15
16 # Kết hợp expression và filter
17 even_squares = [x^2 for x in 1:10 if iseven(x)]
18 # → [4, 16, 36, 64, 100]

```

### 3.1.5.2 Multi-dimensional comprehension

Comprehension có thể tạo array nhiều chiều:

```

1  # Matrix 3x4 từ comprehension
2  matrix = [i + j for i in 1:3, j in 1:4]
3  # 3x4 Matrix{Int64}:
4  #  2  3  4  5
5  #  3  4  5  6
6  #  4  5  6  7
7
8  # Bảng cửu chương
9  mult_table = [i * j for i in 1:9, j in 1:9]
10
11 # Nested comprehension (flatten)
12 pairs = [(i, j) for i in 1:3 for j in 1:2]
13 # → [(1,1), (1,2), (2,1), (2,2), (3,1), (3,2)]

```

**Lưu ý cú pháp:**

- [... for i in A, j in B] — tạo matrix (outer product)
- [... for i in A for j in B] — tạo vector (flatten)

### 3.1.5.3 Dictionary comprehension

Tạo Dict từ comprehension với cú pháp key => value:

```

1  # Dict comprehension cơ bản
2  word_lengths = Dict{word => length(word) for word in ["julia", "python",
    ↪ "rust"]}

```

```

3  # Dict("julia" => 5, "python" => 6, "rust" => 4)
4
5  # Đảo ngược key-value của dict
6  original = Dict{"a" => 1, "b" => 2, "c" => 3}
7  reversed = Dict{v => k for (k, v) in original}
8  # Dict{1 => "a", 2 => "b", 3 => "c"}
9
10 # Với filter
11 long_words = Dict{w => length(w) for w in ["hi", "hello", "hey"] if length(w)
    ↪ > 2}
12 # Dict{"hello" => 5, "hey" => 3}

```

#### 3.1.5.4 Generator expressions

Generator không tạo array ngay lập tức mà tạo từng phần tử khi cần (lazy evaluation), tiết kiệm memory:

```

1  # Generator expression - dùng () thay vì []
2  gen = (x^2 for x in 1:1000000)
3  # Base.Generator{...} - không chiếm memory
4
5  # Sử dụng với hàm aggregate
6  sum(x^2 for x in 1:100)      # → 338350
7  maximum(x^2 for x in 1:10)  # → 100
8
9  # Generator có thể iterate
10 for val in (x^2 for x in 1:5)
11     println(val)
12 end
13
14 # So sánh memory usage
15 # Array: [x^2 for x in 1:10^8] # ~800MB
16 # Generator: (x^2 for x in 1:10^8) # ~0MB (lazy)

```

#### Khi nào dùng Generator:

- Data lớn không cần lưu toàn bộ trong memory
- Chỉ cần iterate một lần
- Dùng với các hàm như sum, maximum, any, all

#### 3.1.5.5 Typed comprehension

Có thể chỉ định kiểu dữ liệu cho comprehension:

```

1  # Chỉ định kiểu Float64
2  floats = Float64[x for x in 1:5]
3  # → [1.0, 2.0, 3.0, 4.0, 5.0]
4
5  # Complex numbers
6  complexes = Complex{Float64}[x + im*x for x in 1:3]
7  # → [1.0+1.0im, 2.0+2.0im, 3.0+3.0im]

```

### 3.1.6 Xử lý ngoại lệ với try-catch

Julia sử dụng cơ chế exception handling với try-catch-finally để xử lý lỗi runtime. Đây là cách tiêu chuẩn để handle các tình huống lỗi không mong đợi.

```

1  # try-catch cơ bản
2  try
3      result = 10 / 0  # DivideError (nhưng Julia trả về Inf)
4      println("Result: $result")
5  catch e
6      println("Lỗi xảy ra: $e")
7  end
8
9  # try-catch-finally
10 try
11     x = parse{Int, "abc"}  # ArgumentError
12 catch e
13     println("Không thể parse: $e")
14 finally
15     println("Finally block luôn chạy")
16 end

```

**Đặc điểm:**

- Block catch nhận exception object e chứa thông tin lỗi
- Block finally (tùy chọn) luôn được thực thi dù có lỗi hay không, thường dùng để cleanup resources
- Julia có nhiều built-in exception types: DivideError, ArgumentError, DomainError, UndefVarError, etc.

#### 3.1.6.1 Phân biệt loại exception

Có thể kiểm tra loại exception cụ thể trong block catch để xử lý phù hợp:

```

1  function safe_operation(x)
2      try
3          result = sqrt(x)
4          return result
5      catch e
6          if isa(e, DomainError)
7              println("Lỗi: không thể tính căn của số âm")
8              return NaN
9          elseif isa(e, MethodError)
10             println("Lỗi: kiểu dữ liệu không hợp lệ")
11             return nothing
12          else
13              rethrow(e)  # Ném lại exception không xử lý được
14          end
15      end
16  end
17
18  safe_operation(4)  # → 2.0

```

```
19 safe_operation(-1)    # → NaN (DomainError handled)
20 safe_operation("a")   # → nothing (MethodError handled)
```

### 3.1.6.2 throw và rethrow

Hàm throw tạo exception mới, rethrow ném lại exception đang xử lý:

```
1  # throw - tạo exception
2  function validate_age(age)
3      age < 0 && throw(DomainError(age, "Tuổi không thể âm"))
4      age > 150 && throw(DomainError(age, "Tuổi không hợp lệ"))
5      return age
6  end
7
8  try
9      validate_age(-5)
10 catch e
11     println("Validation failed: ", e.msg)
12 end
13 # → Validation failed: Tuổi không thể âm
14
15 # rethrow - ném lại sau khi log
16 function process_with_logging(data)
17     try
18         # Xử lý data
19         return parse{Int, data}
20     catch e
21         @error "Lỗi khi xử lý: $data"
22         rethrow(e) # Ném lại để caller xử lý
23     end
24 end
```

### 3.1.6.3 Custom exceptions

Có thể định nghĩa exception types riêng bằng cách kế thừa từ Exception:

```
1  # Định nghĩa custom exception
2  struct ValidationError <: Exception
3      field::String
4      message::String
5  end
6
7  # Override hiển thị lỗi
8  Base.showerror(io::IO, e::ValidationError) =
9      print(io, "ValidationError in '$(e.field)': $(e.message)")
10
11 # Sử dụng custom exception
12 function validate_user(name, email)
13     isempty(name) && throw(ValidationError("name", "Tên không được để trống"))
14     !occursin("@", email) && throw(ValidationError("email", "Email không hợp
15     ⇨ lệ"))
16     return true
17 end
```

```

17
18 try
19     validate_user("", "test@example.com")
20 catch e
21     if isa(e, ValidationError)
22         println("Lỗi validation: $(e.field) - $(e.message)")
23     end
24 end
25 # → Lỗi validation: name - Tên không được để trống

```

### 3.1.6.4 Built-in exception types

Julia cung cấp nhiều exception types có sẵn:

| Exception          | Mô tả                         |
|--------------------|-------------------------------|
| ArgumentError      | Tham số không hợp lệ          |
| BoundsError        | Index ngoài phạm vi           |
| DomainError        | Giá trị ngoài domain của hàm  |
| DivideError        | Chia cho 0 (integer division) |
| ErrorException     | Exception chung               |
| KeyError           | Key không tồn tại trong Dict  |
| MethodError        | Không tìm thấy method phù hợp |
| OutOfMemoryError   | Hết bộ nhớ                    |
| StackOverflowError | Tràn stack (đệ quy quá sâu)   |
| TypeError          | Kiểu dữ liệu không khớp       |
| UndefVarError      | Biến chưa được định nghĩa     |

Bảng 3.1: Các exception types phổ biến trong Julia

### 3.1.6.5 Best practices cho exception handling

- **Không bắt mọi exception:** Chỉ catch những exception bạn biết cách xử lý. Các lỗi như `OutOfMemoryError` thường không nên catch vì chương trình không thể tiếp tục một cách đáng tin cậy.
- **Dùng finally cho cleanup:** Luôn giải phóng resources (đóng file, connection) trong block `finally`.
- **Log trước khi rethrow:** Nếu cần ném lại exception, log thông tin debug trước.
- **Ưu tiên return value:** Với những lỗi thường xuyên (như parse failure), cân nhắc dùng `tryparse` trả về `nothing` thay vì exception.

```

1 # Pattern: tryparse thay vì try-catch
2 input = "not a number"
3
4 # Không khuyến khích - dùng exception cho control flow
5 result = try
6     parse{Int}(input)
7 catch
8     0
9 end
10
11 # Khuyến khích - dùng tryparse

```

```

12 result = something(tryparse{Int, input}, 0)
13 # → 0 (default value nếu parse thất bại)

```

## 3.2 Hàm và multiple dispatch

Hàm (functions) là đối tượng cơ bản (first-class objects) trong Julia, có thể được truyền như tham số, trả về từ hàm khác, và gán cho biến. Đặc điểm nổi bật nhất của hàm trong Julia là cơ chế **đa phái cử** (multiple dispatch) [2] - khả năng định nghĩa nhiều phương thức (methods) cho cùng một hàm, với hành vi khác nhau tùy theo kiểu dữ liệu của *tất cả* các tham số.

### 3.2.1 Định nghĩa hàm

Julia hỗ trợ nhiều cú pháp để định nghĩa hàm:

#### 1. Named function (hàm có tên):

```

1 function add(a::Int, b::Int)
2     return a + b
3 end
4
5 result = add(3, 4) # 7

```

#### 2. Short-form function (cú pháp ngắn gọn):

```

1 square(x) = x^2
2 result = square(5) # 25

```

#### 3. Anonymous function (lambda):

```

1 double = x -> 2x
2 result = double(5) # 10

```

### 3.2.2 Multiple dispatch

Multiple dispatch [2] là đặc trưng cốt lõi của Julia, cho phép định nghĩa nhiều methods cho cùng một function name, phân biệt bởi kiểu dữ liệu của *tất cả* các tham số (không chỉ tham số đầu tiên như OOP truyền thống).

```

1 # Method 1: Hai tham số kiểu String và Int
2 function greet(name::String, age::Int)
3     return "Xin chào $name, bạn $age tuổi"
4 end
5
6 # Method 2: Chỉ một tham số kiểu String
7 function greet(name::String)
8     return "Xin chào $name"
9 end
10
11 # Julia tự động chọn method phù hợp
12 greet("An", 20) # → "Xin chào An, bạn 20 tuổi" (Method 1)
13 greet("Bình") # → "Xin chào Bình" (Method 2)

```

#### Lợi ích của multiple dispatch:

- **Code reuse:** Định nghĩa hành vi tối ưu cho từng kiểu dữ liệu cụ thể



- **Composability:** Các packages khác nhau có thể extend functions cho custom types
- **Performance:** Compiler có thể chọn method chính xác nhất và tối ưu hóa

### 3.2.3 Optional arguments và keyword arguments

#### 1. Optional arguments (tham số tùy chọn):

```

1 function power(x, n=2) # n mặc định là 2
2     return x^n
3 end
4
5 power(3)          # 9 (n=2)
6 power(3, 3)       # 27 (n=3)

```

#### 2. Keyword arguments (tham số có tên):

```

1 function describe_person(; name::String, age::Int, city::String="TP.HCM")
2     return "$name, $age tuổi, sống ở $city"
3 end
4
5 # Phải dùng tên tham số khi gọi
6 describe_person(name="An", age=25)
7 # → "An, 25 tuổi, sống ở TP.HCM"
8
9 describe_person(name="Bình", age=30, city="Hà Nội")
10 # → "Bình, 30 tuổi, sống ở Hà Nội"

```

**Lưu ý:** Dấu chấm phẩy ; trong signature phân tách positional arguments và keyword arguments.

### 3.2.4 Varargs (Variable arguments)

Julia hỗ trợ varargs (số lượng tham số thay đổi) với cú pháp `args...`:

```

1 function sum_all(args...)
2     return sum(args)
3 end
4
5 sum_all(1, 2, 3)          # 6
6 sum_all(1, 2, 3, 4, 5)    # 15

```

### 3.2.5 Do blocks

Do blocks là cú pháp đặc biệt để truyền anonymous function như tham số đầu tiên, thường dùng với higher-order functions như `map`, `filter`:

```

1 result = map([1, 2, 3, 4, 5]) do x
2     x^2 + 1
3 end
4 # → [2, 5, 10, 17, 26]

```

Cú pháp này tương đương với:

```

1 result = map(x -> x^2 + 1, [1, 2, 3, 4, 5])

```

### 3.2.6 Return values và multiple returns

Julia functions có thể trả về nhiều giá trị cùng lúc thông qua tuple[?]. Đây là tính năng rất tiện lợi khi cần trả về nhiều kết quả từ một hàm.

#### 1. Return đơn giản:

```
1 function square(x)
2     return x^2
3 end
4
5 # Hoặc implicit return (giá trị cuối cùng)
6 square_v2(x) = x^2
```

#### 2. Multiple returns (trả về tuple):

```
1 function divrem_custom(a, b)
2     quotient = div(a, b)
3     remainder = a % b
4     return quotient, remainder # Trả về tuple
5 end
6
7 q, r = divrem_custom(17, 5) # q=3, r=2
```

#### 3. Destructuring assignment:

```
1 function minmax_custom(arr)
2     return minimum(arr), maximum(arr)
3 end
4
5 min_val, max_val = minmax_custom([3, 1, 4, 1, 5, 9, 2, 6])
6 # min_val=1, max_val=9
7
8 # Bỏ qua giá trị không cần với underscore
9 _, max_only = minmax_custom([1, 2, 3])
```

#### 4. Named tuple returns (Julia 1.5+):

```
1 function analyze_data(data)
2     return (
3         mean = sum(data) / length(data),
4         count = length(data),
5         total = sum(data)
6     )
7 end
8
9 result = analyze_data([10, 20, 30])
10 result.mean # 20.0
11 result.count # 3
12 result.total # 60
```

### 3.2.7 Type annotations trong functions

Type annotations giúp tăng performance và làm rõ ý định của code[?]. Julia cho phép annotate cả input parameters và return type.

#### 1. Parameter type annotations:

```

1  # Cụ thể kiểu
2  function add_integers(a::Int64, b::Int64)
3      return a + b
4  end
5
6  # Abstract type (flexible hơn)
7  function add_numbers(a::Number, b::Number)
8      return a + b
9  end
10
11 # Parametric type
12 function first_element(arr::Vector{T}) where T
13     return arr[1]
14 end

```

## 2. Return type annotation:

```

1  function safe_divide(a::Number, b::Number)::Float64
2      return a / b
3  end
4
5  # Julia sẽ convert kết quả về Float64
6  safe_divide(10, 3)  # 3.3333333333333335

```

## 3. Union types cho flexibility:

```

1  function find_value(dict::Dict, key)::Union{Int, Nothing}
2      return get(dict, key, nothing)
3  end
4
5  d = Dict("a" => 1, "b" => 2)
6  find_value(d, "a")  # 1
7  find_value(d, "c")  # nothing

```

## Lưu ý về performance:

- Type annotations trên parameters KHÔNG tăng performance (chỉ dispatch)
- Type stability (hàm luôn trả về cùng kiểu) mới quan trọng cho performance
- Dùng @code\_warntype để kiểm tra type inference

### 3.2.8 Splat và Slurp operators

Julia sử dụng toán tử ... (ba chấm) với hai vai trò khác nhau tùy context[?]:

#### 1. Slurp (trong function definition) - Thu gom arguments:

```

1  function show_all(first, rest...)
2      println("First: ", first)
3      println("Rest: ", rest)
4      println("Type of rest: ", typeof(rest))
5  end
6
7  show_all(1, 2, 3, 4, 5)
8  # First: 1
9  # Rest: (2, 3, 4, 5)
10 # Type of rest: NTuple{4, Int64}

```

## 2. Splat (trong function call) - Mở rộng collection:

```

1  function add_three(a, b, c)
2      return a + b + c
3  end
4
5  numbers = [10, 20, 30]
6  add_three(numbers...)  # 60 (splat array thành 3 arguments)
7
8  # Splat với tuple
9  t = (1, 2, 3)
10 add_three(t...)  # 6

```

## 3. Kết hợp Slurp và Splat:

```

1  function wrap_print(args...)
2      println("=== START ===")
3      println(args...)  # Splat để pass sang println
4      println("=== END ===")
5  end
6
7  wrap_print("Hello", " ", "World", "!")
8  # === START ===
9  # Hello World!
10 # === END ===

```

## 4. Keyword argument splatting:

```

1  function configure(; host="localhost", port=8080, debug=false)
2      println("Host: $host, Port: $port, Debug: $debug")
3  end
4
5  settings = Dict(:host => "192.168.1.1", :port => 3000)
6  configure(; settings...)  # Splat Dict vào keyword arguments
7  # Host: 192.168.1.1, Port: 3000, Debug: false

```

### 3.2.9 methods() và function introspection

Julia cung cấp các công cụ mạnh mẽ để khám phá methods của một function[?]:

#### 1. Liệt kê tất cả methods:

```

1  # Xem methods của hàm +
2  methods(+)
3  # [1] +(x::Bool, z::Complex{Bool}) ...
4  # [2] +(x::Float32, y::Float32) ...
5  # ... (hàng trăm methods)
6
7  # Đếm số methods
8  length(methods(+))  # > 200 methods!

```

#### 2. Tìm method cụ thể với @which:

```

1  @which 1 + 2
2  # +(x::T, y::T) where T<:Union{Int128, Int16, ...} in Base at int.jl:87
3

```

```

4 @which 1.0 + 2.0
5 # +(x::Float64, y::Float64) in Base at float.jl:408
6
7 @which "hello" * " world"
8 # *(s1::Union{...String...}, ss::Union{...String...}...) in Base at
   ↪ strings/basic.jl:260

```

### 3. Xem method signatures:

```

1 # Định nghĩa custom function với nhiều methods
2 function process(x::Int)
3     println("Processing integer: ", x)
4 end
5
6 function process(x::Float64)
7     println("Processing float: ", x)
8 end
9
10 function process(x::String)
11     println("Processing string: ", x)
12 end
13
14 methods(process)
15 # 3 methods for generic function "process":
16 # [1] process(x::Int64)
17 # [2] process(x::Float64)
18 # [3] process(x::String)

```

### 4. methodswith() - Tìm methods cho một type:

```

1 # Tìm tất cả methods nhận String làm argument
2 methodswith(String)
3
4 # Tìm methods trong module cụ thể
5 methodswith(String, Base)

```

#### 3.2.10 Method selection algorithm

Khi gọi một function, Julia chọn method phù hợp nhất theo thuật toán sau<sup>[2]</sup>:

##### 1. Specificity ordering:

```

1 # Julia chọn method cụ thể nhất (most specific)
2 foo(x::Any) = "any"
3 foo(x::Number) = "number"
4 foo(x::Integer) = "integer"
5 foo(x::Int64) = "int64"
6
7 foo(1)          # "int64" (most specific)
8 foo(1.0)        # "number" (Float64 <: Number)
9 foo("hi")       # "any" (String không <: Number)

```

##### 2. Multiple arguments - Cartesian product:

```

1 bar(x::Int, y::Int) = "both int"
2 bar(x::Int, y::Float64) = "int, float"

```

```

3 bar(x::Float64, y::Int) = "float, int"
4 bar(x::Number, y::Number) = "both number"
5
6 bar(1, 2)          # "both int"
7 bar(1, 2.0)        # "int, float"
8 bar(1.0, 2)        # "float, int"
9 bar(1.0, 2.0)      # "both number" (Float64 <: Number)

```

### 3. Dispatch rules:

- Julia dispatch trên **tất cả** arguments (không chỉ argument đầu)
- Method cụ thể hơn (more specific) được ưu tiên
- Nếu không có method exact match, Julia tìm method với supertype
- Nếu nhiều methods cùng mức độ cụ thể → MethodError (ambiguity)

#### 3.2.11 Ambiguity resolution

Khi Julia không thể quyết định method nào cụ thể hơn, sẽ xảy ra ambiguity error[?]:

##### 1. Ví dụ ambiguity:

```

1 f(x::Int, y::Any) = "int, any"
2 f(x::Any, y::Int) = "any, int"
3
4 f(1, "hello")    # "int, any" - OK
5 f("hello", 1)   # "any, int" - OK
6 f(1, 2)          # ERROR: MethodError - ambiguous!

```

##### 2. Giải quyết ambiguity - Thêm method cụ thể:

```

1 # Thêm method cho trường hợp cả hai là Int
2 f(x::Int, y::Int) = "both int"
3
4 f(1, 2)          # "both int" - OK!

```

##### 3. Kiểm tra ambiguities trong module:

```

1 # Phát hiện ambiguities
2 using Test
3 @test isempty(Test.detect_ambiguities(MyModule))

```

##### 4. Best practices để tránh ambiguity:

- Luôn định nghĩa method cho "diagonal cases" (cả hai arguments cùng type)
- Sử dụng abstract types thay vì Any khi có thể
- Test với detect\_ambiguities() trong unit tests
- Prefer specificity: viết methods từ specific → general

### 3.2.12 So sánh: Multiple dispatch vs OOP Polymorphism

Multiple dispatch của Julia khác biệt cơ bản với polymorphism trong OOP truyền thống[2]:

| Tiêu chí           | OOP (Single Dispatch)                | Julia (Multiple Dispatch)         |
|--------------------|--------------------------------------|-----------------------------------|
| Dispatch dựa trên  | Type của object receiver (this/self) | Types của <b>tất cả</b> arguments |
| Method definition  | Bên trong class definition           | Độc lập với type definition       |
| Extensibility      | Cần modify class hoặc inheritance    | Có thể add methods từ bên ngoài   |
| Expression problem | Khó thêm operations mới              | Dễ thêm cả types và operations    |
| Runtime behavior   | vtable lookup trên receiver          | Type-based method selection       |

Ví dụ so sánh:

```

1 # Julia - Multiple dispatch
2 # Các types được định nghĩa riêng
3 struct Dog end
4 struct Cat end
5
6 # Methods được định nghĩa riêng, dispatch trên TẤT CẢ arguments
7 interact(::Dog, ::Dog) = "Dogs play together"
8 interact(::Cat, ::Cat) = "Cats ignore each other"
9 interact(::Dog, ::Cat) = "Dog chases cat"
10 interact(::Cat, ::Dog) = "Cat hisses at dog"
11
12 interact(Dog(), Cat()) # "Dog chases cat"
13 interact(Cat(), Dog()) # "Cat hisses at dog"

```

Trong OOP truyền thống (Python-like pseudocode):

```

1 # OOP - Chỉ dispatch trên self
2 # class Dog:
3 #     def interact(self, other):
4 #         if isinstance(other, Dog):
5 #             return "Dogs play"
6 #         elif isinstance(other, Cat):
7 #             return "Dog chases cat"
8 #
9 # Vấn đề: Logic interact phải nằm trong class Dog
10 # Không thể extend từ bên ngoài mà không modify class

```

Lợi ích của Multiple Dispatch:

1. **Separation of concerns:** Types và behaviors được tách biệt
2. **Extensibility:** Package A có thể định nghĩa method cho types từ Package B
3. **Performance:** Compiler biết exact method tại compile time (khi type stable)
4. **Clean binary operations:** `a + b` không cần `a.__add__(b)` hay `a.__radd__(b)`

### 3.3 Closures

Closure là một hàm (inner function) được trả về bởi hàm khác (outer function), với khả năng *capture* (bắt giữ) các biến từ scope của outer function. Đây là kỹ thuật lập trình mạnh mẽ cho phép tạo ra function factories và maintain state giữa các lần gọi.

#### 3.3.1 Closure cơ bản

```
1 function multiplier(k)
2     return x -> k * x # Inner function captures k
3 end
4
5 double = multiplier(2) # double là closure với k=2
6 triple = multiplier(3) # triple là closure với k=3
7
8 double(10) # 20
9 triple(10) # 30
```

##### Giải thích:

- Hàm multiplier trả về một anonymous function  $x \rightarrow k * x$
- Anonymous function này *captures* biến k từ outer scope
- Mỗi closure (double, triple) có giá trị k riêng biệt

#### 3.3.2 Closure với mutable state

Closures có thể capture biến mutable, cho phép maintain state giữa các lần gọi:

```
1 function counter()
2     count = 0
3     return function()
4         count += 1
5         return count
6     end
7 end
8
9 c = counter()
10 c() # 1
11 c() # 2
12 c() # 3
```

Mỗi lần gọi `c()`, biến `count` được increment và giữ nguyên giá trị giữa các lần gọi.

#### 3.3.3 Ứng dụng thực tế

##### 1. Function factories:

```
1 function make_adder(x)
2     return y -> x + y
3 end
4
5 add5 = make_adder(5)
```



```

6 add10 = make_adder(10)
7
8 add5(3)    # 8
9 add10(3)   # 13

```

## 2. Custom filters:

```

1 function make_filter(threshold)
2     return x -> x > threshold
3 end
4
5 filter_gt_5 = make_filter(5)
6 numbers = [1, 3, 5, 7, 9, 2, 4, 6, 8, 10]
7 filtered = filter(filter_gt_5, numbers)
8 # → [7, 9, 6, 8, 10]

```

### Lưu ý quan trọng:

- Closures trong Julia không phải là anonymous functions - chúng là anonymous functions có khả năng capture variables
- Captured variables được shared giữa closure và outer function
- Closures có thể capture nhiều variables cùng lúc

### 3.3.4 Performance considerations

Closures có thể ảnh hưởng đến performance nếu capture quá nhiều biến hoặc capture biến kiểu không ổn định (type-unstable). Best practices:

- Minimize số lượng captured variables
- Đảm bảo captured variables có kiểu ổn định (stable types)
- Sử dụng @code\_warntype để kiểm tra type inference

## 3.4 Bài tập

### 3.4.1 Bài tập cơ bản (\*)

#### Bài tập: If-elseif-else

Viết hàm `grade(score)` nhận điểm số (0-100) và trả về xếp loại:

- A: 90-100, B: 80-89, C: 70-79, D: 60-69, F: < 60

```

1 function grade(score)
2     if score >= 90
3         return "A"
4     elseif score >= 80
5         return "B"
6     # TODO: complete the function
7     end
8 end

```

**Bài tập: For loop cơ bản**

Sử dụng for loop để: (a) tính tổng các số từ 1 đến 100, (b) in bảng cửu chương 7, (c) tìm số lớn nhất trong array [3, 1, 4, 1, 5, 9, 2, 6].

**Bài tập: While loop**

Viết chương trình đoán số: máy chọn random 1-100, người dùng đoán, thông báo "cao hơn" hoặc "thấp hơn" đến khi đoán đúng.

```

1  function guessing_game()
2      secret = rand(1:100)
3      attempts = 0
4      while true
5          print("Đoán số (1-100): ")
6          guess = parse{Int, readline()}
7          attempts += 1
8          # TODO: implement logic
9      end
10 end

```

**Bài tập: Định nghĩa function**

Viết các hàm: (a) `is_even(n)` kiểm tra số chẵn, (b) `factorial(n)` tính giai thừa, (c) `fibonacci(n)` trả về số Fibonacci thứ n.

**Bài tập: Multiple return values**

Viết hàm `minmax(arr)` trả về cả giá trị nhỏ nhất và lớn nhất của array:

```

1  function minmax(arr)
2      return minimum(arr), maximum(arr)
3  end
4  min_val, max_val = minmax([3, 1, 4, 1, 5, 9])

```

Mở rộng để trả về cả index của min và max.

**3.4.2 Bài tập trung bình (\*\*)****Bài tập: Multiple dispatch**

Tạo hàm `area` với multiple dispatch để tính diện tích:

```

1  # Hình tròn (1 argument: radius)
2  area(r) = π * r^2
3
4  # Hình chữ nhật (2 arguments: width, height)
5  area(w, h) = w * h
6
7  # Hình tam giác (3 arguments: base, height, :triangle)
8  area(b, h, ::Val{:triangle}) = 0.5 * b * h

```

Thêm method cho hình thang (4 arguments).

### Bài tập: Higher-order functions

Sử dụng map, filter, reduce để:

```

1 numbers = 1:20
2 # (a) Bình phương các số chẵn
3 map(x -> x^2, filter(iseven, numbers))
4
5 # (b) Tích của các số lẻ
6 reduce(*, filter(isodd, numbers))
7
8 # (c) Tổng các số chia hết cho 3
9 # TODO: implement

```

### Bài tập: Do block syntax

Chuyển đổi các expressions sau sang do-block syntax:

```

1 # Từ:
2 map(x -> x^2 + 2x + 1, 1:5)
3
4 # Sang:
5 map(1:5) do x
6     x^2 + 2x + 1
7 end
8
9 # Thực hành với filter và reduce

```

### 3.4.3 Bài tập nâng cao (\*\*\*)

#### Bài tập: Closure và state

Tạo module BankAccount sử dụng closures để quản lý state:

```
1  function create_account(initial_balance)
2      balance = initial_balance
3
4      function deposit(amount)
5          balance += amount
6          return balance
7      end
8
9      function withdraw(amount)
10         if amount <= balance
11             balance -= amount
12             return balance
13         else
14             error("Insufficient funds")
15         end
16     end
17
18     function get_balance()
19         return balance
20     end
21
22     return {deposit=deposit, withdraw=withdraw, balance=get_balance}
23 end
24
25 acc = create_account(1000)
26 acc.deposit(500)      # 1500
27 acc.withdraw(200)    # 1300
28 acc.balance()        # 1300
```

Thêm chức năng: transaction history, interest calculation.

### Bài tập: Mini-project: Expression evaluator

Viết evaluator cho biểu thức toán học đơn giản:

```

1 function evaluate(expr::String)
2     # Parse và evaluate expressions như:
3     # "2 + 3" → 5
4     # "10 - 4 * 2" → 2 (respecting precedence)
5     # "(2 + 3) * 4" → 20
6     # TODO: implement using recursion and pattern matching
7 end
8
9 # Hints:
10 # 1. Tokenize string into numbers and operators
11 # 2. Handle operator precedence (* / before + -)
12 # 3. Handle parentheses with recursion

```

Bonus: hỗ trợ functions như sin, cos, sqrt.

## Đọc thêm

- Julia Documentation: Control Flow - <https://docs.julialang.org/en/v1/manual/control-flow/>
- Julia Documentation: Functions - <https://docs.julialang.org/en/v1/manual/functions/>
- Julia Documentation: Methods - <https://docs.julialang.org/en/v1/manual/methods/>



## Phần II

# Phần II — Dữ liệu và cấu trúc nâng cao





## Chương 4

# Cấu trúc dữ liệu và kiểu tùy chỉnh

### Mục tiêu học tập

- Sử dụng Arrays, Tuples, Dicts, Sets hiệu quả
- Thiết kế custom types với struct và mutable struct
- Hiểu abstract types và type hierarchy
- Áp dụng parametric types cho generic programming

### 4.1 Mảng, Tuple, Dict, Set

Julia cung cấp một hệ thống cấu trúc dữ liệu (data structures) phong phú và hiệu năng cao, bao gồm các kiểu collection cơ bản như mảng (arrays), tuple, dictionary, và set. Mỗi cấu trúc dữ liệu có đặc điểm và use case riêng, được tối ưu cho các tình huống sử dụng khác nhau.

#### 4.1.1 Mảng (Arrays)

Mảng là cấu trúc dữ liệu quan trọng nhất trong Julia [4], được thiết kế đặc biệt cho tính toán số học hiệu năng cao. Julia arrays là **column-major** (giống Fortran và MATLAB), khác với Python (row-major), điều này ảnh hưởng đến performance khi duyệt mảng.

##### 1. Tạo mảng:

```
1 # Array 1D từ literal
2 A = [1, 2, 3, 4, 5]
3
4 # Array 2D (matrix)
5 M = [1 2 3; 4 5 6] # 2x3 matrix
6
7 # Khởi tạo với zeros/ones
8 zeros_arr = zeros(5) # [0.0, 0.0, 0.0, 0.0, 0.0]
9 ones_arr = ones(3, 3) # 3x3 matrix toàn 1
10 identity_mat = Matrix{Float64}(I, 3, 3) # Ma trận đơn vị
11
12 # Range và collect
13 range_arr = collect(1:10) # [1, 2, ..., 10]
```

##### 2. Indexing và slicing:

```

1 A = [10, 20, 30, 40, 50]
2
3 # Indexing (1-based - bắt đầu từ 1)
4 first = A[1]      # 10
5 last = A[end]     # 50
6 second_last = A[end-1] # 40
7
8 # Slicing
9 subset = A[2:4]    # [20, 30, 40]
10 every_two = A[1:2:end] # [10, 30, 50]
11
12 # Boolean indexing
13 filtered = A[A .> 25] # [30, 40, 50]

```

**Lưu ý quan trọng:** Julia sử dụng **1-based indexing** (index bắt đầu từ 1), khác với Python/C/Java (0-based). Đây là quyết định thiết kế để gần hơn với ký hiệu toán học.

### 3. Array operations:

```

1 # Element-wise operations (broadcasting với dấu chấm .)
2 A = [1, 2, 3, 4]
3 B = [10, 20, 30, 40]
4
5 A .+ B      # [11, 22, 33, 44]
6 A .* 2      # [2, 4, 6, 8]
7 A .^ 2      # [1, 4, 9, 16]
8
9 # Các hàm tổng hợp
10 sum(A)      # 10
11 mean(A)     # 2.5 (cần: using Statistics)
12 maximum(A)  # 4
13 minimum(A)  # 1
14
15 # Push/pop (modify in-place)
16 push!(A, 5) # A = [1, 2, 3, 4, 5]
17 pop!(A)     # Trả về 5, A = [1, 2, 3, 4]

```

### Đặc điểm quan trọng:

- Arrays trong Julia là **mutable** - có thể thay đổi elements in-place
- Hỗ trợ arrays nhiều chiều (N-dimensional arrays)
- Type-stable arrays (tất cả elements cùng type) có performance tốt nhất
- Broadcasting với dấu chấm (.) cho phép vectorize operations

#### 4.1.1.1 Mảng đa chiều (Multidimensional Arrays)

Julia hỗ trợ arrays N chiều một cách tự nhiên, đặc biệt quan trọng cho tính toán khoa học và machine learning. Các hàm inspection giúp hiểu cấu trúc array:

```

1 # Tạo matrix 3x4
2 M = [1 2 3 4; 5 6 7 8; 9 10 11 12]
3

```

```

4 # Inspection functions
5 length(M)      # 12 - tổng số elements
6 ndims(M)       # 2 - số chiều
7 size(M)        # (3, 4) - tuple kích thước mỗi chiều
8 size(M, 1)     # 3 - số hàng (chiều 1)
9 size(M, 2)     # 4 - số cột (chiều 2)
10
11 # Kiểm tra type
12 typeof(M)      # Matrix{Int64} (alias cho Array{Int64, 2})
13 eltype(M)      # Int64 - kiểu của elements

```

### Indexing matrix:

```

1 M = [1 2 3 4; 5 6 7 8; 9 10 11 12]
2
3 # Truy cập element: M[row, col]
4 M[1, 1]        # 1 (top-left)
5 M[3, 4]        # 12 (bottom-right)
6 M[end, end]    # 12 (dùng end keyword)
7
8 # Slicing hàng và cột
9 M[1, :]        # [1, 2, 3, 4] - hàng đầu tiên (Vector)
10 M[:, 2]        # [2, 6, 10] - cột thứ hai (Vector)
11 M[1:2, 2:3]    # 2x2 submatrix
12
13 # Linear indexing (column-major order)
14 M[5]           # 2 (element thứ 5 theo column-major)
15 # Thứ tự: 1,5,9,2,6,10,3,7,11,4,8,12

```

**Column-major order:** Julia lưu arrays theo column-major (như Fortran, MATLAB), nghĩa là các phần tử cùng cột nằm liên tiếp trong memory. Điều này ảnh hưởng performance khi duyệt mảng:

```

1 M = rand(1000, 1000)
2
3 # CHẬM - duyệt theo hàng (cache miss)
4 for i in 1:1000
5     for j in 1:1000
6         M[i, j] += 1 # Không tối ưu
7     end
8 end
9
10 # NHANH - duyệt theo cột (cache friendly)
11 for j in 1:1000
12     for i in 1:1000
13         M[i, j] += 1 # Tối ưu memory access
14     end
15 end

```

### Reshape và Permutedims:

```

1 A = collect(1:12) # Vector 1D: [1,2,3,...,12]
2
3 # Reshape: thay đổi shape mà không copy data

```

```

4 B = reshape(A, 3, 4) # 3x4 matrix
5 C = reshape(A, 2, 2, 3) # 3D array 2x2x3
6
7 # Permutedims: hoán vị chiều (như transpose nhưng tổng quát hơn)
8 D = permutedims(B, (2, 1)) # Transpose: hoán đổi chiều 1 và 2
9
10 # vec: flatten về 1D
11 vec(B) # [1, 2, 3, ..., 12]

```

### Khởi tạo arrays đặc biệt:

```

1 using LinearAlgebra
2
3 # Ma trận đơn vị
4 I3 = Matrix{Float64}(I, 3, 3) # 3x3 identity
5
6 # Ma trận đường chéo
7 D = Matrix{Float64}(Diagonal{Float64}([1, 2, 3])) # 3x3 diagonal
8
9 # Random matrix
10 R = rand{Float64}(3, 4) # Float64 uniform [0,1)
11 Rn = randn{Float64}(3, 4) # Float64 normal distribution
12 Ri = rand{Int64}(1:100, 3, 4) # Int random từ 1-100
13
14 # Fill với giá trị
15 F = fill{Float64}(42, 3, 4) # 3x4 matrix toàn 42
16
17 # Similar: tạo array cùng type và size
18 S = similar{Float64}(R) # Uninitialized, cùng shape với R

```

#### 4.1.1.2 Broadcasting chi tiết

Broadcasting là cơ chế mạnh mẽ cho phép apply operations lên arrays với shapes khác nhau. Julia tự động “mở rộng” dimensions khi cần thiết.

##### Quy tắc broadcasting:

1. Dimensions được so sánh từ phải sang trái
2. Dimension size 1 được “broadcast” (mở rộng) để match
3. Dimension không tồn tại được coi như size 1

```

1 # Scalar broadcast
2 A = [1, 2, 3]
3 A .+ 10 # [11, 12, 13]
4 A .^ 2 # [1, 4, 9]
5
6 # Vector + Matrix broadcast
7 M = [1 2 3; 4 5 6] # 2x3 matrix
8 v = [10, 20] # 2-element vector (column)
9
10 M .+ v # Broadcast v theo cột
11 # [11 12 13; 24 25 26]
12

```

```

13 # Row vector broadcast
14 r = [100 200 300] # 1x3 row vector
15 M .+ r # Broadcast r theo hàng
16 # [101 202 303; 104 205 306]

```

### Dot syntax cho functions:

```

1 # Elementwise function application
2 x = [0,  $\pi/4$ ,  $\pi/2$ ]
3 sin.(x) # [0.0, 0.707..., 1.0]
4 sqrt.(1:4) # [1.0, 1.414..., 1.732..., 2.0]
5
6 # Chained broadcasting với @. macro
7 y = [1, 2, 3, 4]
8 @. sin(y)^2 + cos(y)^2 # Tự động thêm . cho tất cả
9 # Tương đương: sin.(y).^2 .+ cos.(y).^2

```

### Broadcast fusion - Tối ưu hóa tự động:

Julia “fuses” các broadcast operations liên tiếp thành một loop duy nhất, tránh tạo temporary arrays:

```

1 A = rand(1000)
2
3 # Có fusion (1 loop, không temps)
4 result = @. sin(A) + cos(A) + tan(A)
5
6 # Hoặc viết explicit
7 result = sin.(A) .+ cos.(A) .+ tan.(A)
8 # Julia fuse thành: [sin(a) + cos(a) + tan(a) for a in A]

```

### Custom broadcast với hàm riêng:

```

1 # Định nghĩa hàm scalar
2 clamp01(x) = clamp(x, 0, 1)
3
4 # Apply elementwise
5 data = [-0.5, 0.3, 0.7, 1.5]
6 clamp01.(data) # [0.0, 0.3, 0.7, 1.0]
7
8 # Broadcast với multiple arguments
9 f(x, y, z) = x + 2y + 3z
10 f.([1, 2], [10, 20], 100) # [321, 342]

```

#### 4.1.1.3 Views vs Copies

Khi slicing arrays, Julia mặc định tạo **copy** của data. Để tránh copy (tiết kiệm memory và time), sử dụng **views**.

#### Copy (mặc định):

```

1 A = [1, 2, 3, 4, 5]
2
3 # Slicing tạo copy
4 B = A[2:4] # B = [2, 3, 4] - COPY
5 B[1] = 100 # B = [100, 3, 4]
6 A # [1, 2, 3, 4, 5] - A không đổi!

```

**View (không copy):**

```
1 A = [1, 2, 3, 4, 5]
2
3 # @view tạo view (không copy)
4 C = @view A[2:4] # C là view vào A
5 C[1] = 100      # Thay đổi qua view
6 A              # [1, 100, 3, 4, 5] - A bị thay đổi!
7
8 # typeof cho view
9 typeof(C)      # SubArray{Int64, 1, ...}
```

**@views macro - Apply cho block code:**

```
1 M = rand(1000, 1000)
2
3 # Chuyển tất cả slicing thành views trong block
4 @views begin
5     row1 = M[1, :]      # View, không copy
6     col1 = M[:, 1]      # View, không copy
7     block = M[1:100, 1:100] # View, không copy
8 end
9
10 # Hoặc inline
11 @views M[1:10, :] .= 0 # Modify in-place qua view
```

**Khi nào dùng view vs copy?**

- Dùng view khi: Xử lý array lớn, chỉ đọc hoặc modify in-place
- Dùng copy khi: Cần data độc lập, array nhỏ, không muốn side effects

**4.1.1.4 Array Comprehensions**

Julia hỗ trợ comprehensions để tạo arrays một cách ngắn gọn và hiệu quả:

```
1 # Basic comprehension
2 squares = [x^2 for x in 1:5] # [1, 4, 9, 16, 25]
3
4 # Với condition (filter)
5 evens = [x for x in 1:10 if iseven(x)] # [2, 4, 6, 8, 10]
6
7 # 2D comprehension
8 table = [i * j for i in 1:3, j in 1:4] # 3x4 multiplication table
9
10 # Nested comprehension
11 coords = [(x, y) for x in 1:3 for y in 1:2]
12 # [(1,1), (1,2), (2,1), (2,2), (3,1), (3,2)]
13
14 # Typed comprehension
15 floats = Float64[x^2 for x in 1:5] # Vector{Float64}
```

**Generator expressions (lazy evaluation):**

```

1 # Generator không tạo array ngay
2 gen = (x^2 for x in 1:1000000)
3
4 # Chỉ tính khi iterate
5 sum(gen)          # Không cần 1M array trong memory
6
7 # Collect để materialize
8 first_10 = collect(Iterators.take(gen, 10))

```

### 4.1.2 Tuple

Tuple là collection **immutable** (không thể thay đổi sau khi tạo), thường dùng để nhóm các giá trị liên quan với types khác nhau. Tuples có performance tốt hơn arrays cho dữ liệu nhỏ và không đổi.

#### 1. Tạo tuple:

```

1 # Tuple cơ bản
2 t = (1, "hai", 3.0)
3
4 # Named tuple (có tên cho từng field)
5 person = (name="An", age=25, city="TP.HCM")
6
7 # Single-element tuple (cần dấu phẩy)
8 single = (42,) # Tuple với 1 phần tử

```

#### 2. Truy cập elements:

```

1 t = (1, "hai", 3.0)
2
3 # Indexing (1-based)
4 first = t[1]      # 1
5 second = t[2]     # "hai"
6
7 # Unpacking
8 x, y, z = t       # x=1, y="hai", z=3.0
9
10 # Named tuple access
11 person = (name="An", age=25)
12 name = person.name # "An"
13 age = person[2]    # 25 (cũng có thể dùng index)

```

#### 3. Use cases:

```

1 # Trả về nhiều giá trị từ function
2 function stats(arr)
3     return (mean=mean(arr), std=std(arr), n=length(arr))
4 end
5
6 result = stats([1, 2, 3, 4, 5])
7 # result.mean, result.std, result.n
8
9 # Swap values (không cần biến tạm)
10 a, b = 10, 20
11 a, b = b, a # a=20, b=10

```

#### Đặc điểm:

- **Immutable:** Không thể thay đổi elements sau khi tạo
- Có thể chứa các types khác nhau
- Named tuples cung cấp truy cập bằng tên field (giống struct)
- Performance tốt cho small, fixed-size collections

#### 4.1.2.1 NamedTuples nâng cao

NamedTuples kết hợp lợi ích của tuples (immutable, performant) với khả năng truy cập bằng tên như struct. Chúng đặc biệt hữu ích cho function returns và data records.

##### Tạo và truy cập:

```

1  # NamedTuple với type annotation
2  person = (name="Minh", age=25, gpa=3.5)
3
4  # Truy cập bằng dot notation
5  person.name      # "Minh"
6  person.age       # 25
7
8  # Truy cập bằng Symbol
9  person[:name]     # "Minh"
10
11 # Lấy tất cả keys và values
12 keys(person)      # (:name, :age, :gpa)
13 values(person)     # ("Minh", 25, 3.5)
14 pairs(person)      # pairs iterator

```

##### Merge và update:

```

1  # Merge hai named tuples
2  base = (a=1, b=2)
3  extra = (c=3, d=4)
4  merged = merge(base, extra)  # (a=1, b=2, c=3, d=4)
5
6  # Override values (tạo tuple mới)
7  updated = merge(base, (b=20,))  # (a=1, b=20)
8
9  # Spread syntax (Julia 1.7+)
10 combined = (; base..., c=3)  # (a=1, b=2, c=3)

```

##### NamedTuple type syntax:

```

1  # Định nghĩa type cho NamedTuple
2  const PersonType = NamedTuple{(:name, :age), Tuple{String, Int}}
3
4  # Tạo instance
5  p1 = PersonType(("An", 30))
6  p2 = (name="Bình", age=25)::PersonType  # Type assertion
7
8  # Destructuring với tên
9  (; name, age) = p1
10 println(name)  # "An"

```



**Trong function signatures:**

```

1  # Function trả về NamedTuple
2  function analyze(data)
3      return (
4          count = length(data),
5          sum = sum(data),
6          mean = sum(data) / length(data),
7          range = (minimum(data), maximum(data))
8      )
9  end
10
11 result = analyze([1, 2, 3, 4, 5])
12 result.mean      # 3.0
13 result.range     # (1, 5)
14
15 # Function nhận NamedTuple
16 function print_person(p::NamedTuple{(:name, :age)})
17     println("$ (p.name) is $ (p.age) years old")
18 end

```

**4.1.2.2 Tuple vs NamedTuple vs Struct**

So sánh các cấu trúc dữ liệu tương tự:

| Đặc điểm        | Tuple    | NamedTuple | Struct        |
|-----------------|----------|------------|---------------|
| Mutable         | Không    | Không      | Tùy chọn      |
| Named fields    | Không    | Có         | Có            |
| Type parameters | Có       | Có         | Có            |
| Methods         | Không    | Không      | Có            |
| Inheritance     | Không    | Không      | Abstract only |
| Performance     | Tốt nhất | Tốt        | Tốt           |

**Khi nào dùng gì?**

- **Tuple:** Trả về 2-3 giá trị đơn giản từ function, không cần đặt tên
- **NamedTuple:** Trả về nhiều giá trị có ý nghĩa, cần truy cập bằng tên
- **Struct:** Cần methods, complex behavior, hoặc data lớn/phức tạp

**4.1.2.3 Destructuring và Pattern Matching**

Julia hỗ trợ destructuring (unpacking) linh hoạt cho tuples và arrays:

**Basic destructuring:**

```

1  # Tuple unpacking
2  point = (3, 4, 5)
3  x, y, z = point    # x=3, y=4, z=5
4
5  # Bỏ qua giá trị với _
6  first, _, last = (1, 2, 3)  # first=1, last=3
7
8  # Swap values

```

```

9  a, b = 10, 20
10 a, b = b, a      # a=20, b=10

```

### Nested destructuring:

```

1  # Nested tuple
2  data = ((1, 2), (3, 4))
3  ((a, b), (c, d)) = data    # a=1, b=2, c=3, d=4
4
5  # Mixed với NamedTuple
6  record = (info=(name="An", age=25), scores=[90, 85, 92])
7  (; info, scores) = record
8  (; name, age) = info
9  println(name)      # "An"

```

### Trong function parameters:

```

1  # Destructure trong signature
2  function distance((x1, y1), (x2, y2))
3      return sqrt((x2-x1)^2 + (y2-y1)^2)
4  end
5
6  p1 = (0, 0)
7  p2 = (3, 4)
8  distance(p1, p2)    # 5.0
9
10 # Với NamedTuple
11 function greet(; name, age)
12     println("Hello $name, you are $age years old")
13 end
14
15 greet((name="Minh", age=30))

```

### Splatting với ...:

```

1  # Collect remaining elements
2  first, rest... = [1, 2, 3, 4, 5]
3  # first = 1, rest = [2, 3, 4, 5]
4
5  # Trong function calls
6  args = (1, 2, 3)
7  f(args...)      # Tương đương f(1, 2, 3)
8
9  # Combine arrays
10 a = [1, 2]
11 b = [3, 4]
12 combined = [a..., 0, b...] # [1, 2, 0, 3, 4]

```

#### 4.1.2.4 Tổng kết Tuples

| Khái niệm      | Mô tả                                                                            |
|----------------|----------------------------------------------------------------------------------|
| Tuple cơ bản   | Collection immutable, heterogeneous, access bằng index                           |
| NamedTuple     | Tuple với named fields, access bằng <code>.name</code> hoặc <code>[:name]</code> |
| Single-element | Cần trailing comma: <code>(42,)</code>                                           |
| Deconstructing | Unpack values: <code>a, b = tuple</code>                                         |
| Splatting      | Spread với <code>...: f(args...)</code>                                          |

#### 4.1.3 Dictionary (Dict)

Dictionary là cấu trúc dữ liệu key-value, cho phép truy cập dữ liệu theo key với độ phức tạp  $O(1)$  trung bình. Dict trong Julia sử dụng hash table implementation.

##### 1. Tạo dictionary:

```

1 # Dict literal
2 scores = Dict{"An" => 85, "Bình" => 90, "Cường" => 78}
3
4 # Empty dict với type annotation
5 grades = Dict{String, Int}()
6
7 # Từ pairs
8 pairs = ["a" => 1, "b" => 2, "c" => 3]
9 d = Dict(pairs)
```

##### 2. Truy cập và thao tác:

```

1 scores = Dict{"An" => 85, "Bình" => 90}
2
3 # Truy cập
4 an_score = scores["An"] # 85
5
6 # Kiểm tra key tồn tại
7 haskey(scores, "An")    # true
8 haskey(scores, "Dũng")  # false
9
10 # Get với default value (tránh KeyError)
11 dung_score = get(scores, "Dũng", 0) # 0
12
13 # Thêm/update
14 scores["Dũng"] = 88      # Thêm mới
15 scores["An"] = 87        # Update existing
16
17 # Xóa
18 delete!(scores, "Bình")
19
20 # Duyệt qua dict
21 for (name, score) in scores
22     println("$name: $score")
23 end
```

##### 3. Các operations hữu ích:

```
1 scores = Dict("An" => 85, "Bình" => 90, "Cường" => 78)
2
3 # Keys và values
4 names = keys(scores)      # Dict keys
5 grades = values(scores)   # Dict values
6
7 # Merge dicts
8 more_scores = Dict("Dũng" => 92, "Em" => 88)
9 all_scores = merge(scores, more_scores)
10
11 # Filter
12 high_scorers = filter(p -> p.second >= 85, scores)
```

### Đặc điểm:

- Mutable - có thể thêm/xóa/sửa key-value pairs
- Keys phải là immutable types (String, Int, Symbol, Tuple, etc.)
- Không đảm bảo thứ tự insertion (dùng OrderedDict nếu cần)
- Average  $O(1)$  lookup time

#### 4.1.3.1 get! và get với default function

Hàm get trả về giá trị mặc định khi key không tồn tại, nhưng không thay đổi dictionary. Hàm get! thì sẽ thêm key-value vào dictionary nếu key chưa tồn tại.

```
1 cache = Dict{String, Int}()
2
3 # get! thêm key nếu chưa tồn tại và trả về giá trị
4 val1 = get!(cache, "count", 0) # cache = Dict("count" => 0), val1 = 0
5 val2 = get!(cache, "count", 100) # cache không đổi, val2 = 0 (đã tồn tại)
6
7 # get! với function để tạo default value (lazy evaluation)
8 expensive_cache = Dict{String, Vector{Int}}()
9 result = get!(expensive_cache, "data") do
10     println("Computing expensive value...")
11     collect(1:1000) # Chỉ chạy khi key chưa tồn tại
12 end
13
14 # get với function (không modify dict)
15 config = Dict("timeout" => 30)
16 port = get(config, "port") do
17     8080 # Default nếu không có "port"
18 end
19 # config vẫn chỉ có "timeout", không thêm "port"
```

### So sánh get vs get!:

#### 4.1.3.2 merge và mergewith

merge kết hợp nhiều dictionaries, với các key trùng sẽ lấy giá trị từ dictionary sau cùng. mergewith cho phép custom logic khi merge keys trùng.

| Hàm                   | Modify Dict?  | Use case             |
|-----------------------|---------------|----------------------|
| get(d, k, default)    | Không         | Read-only lookup     |
| get!(d, k, default)   | Có (thêm key) | Memoization, caching |
| get(d, k) do ... end  | Không         | Lazy default         |
| get!(d, k) do ... end | Có            | Lazy initialization  |

```

1 d1 = Dict{"a" => 1, "b" => 2}
2 d2 = Dict{"b" => 3, "c" => 4}
3 d3 = Dict{"c" => 5, "d" => 6}
4
5 # merge - giá trị sau ghi đè giá trị trước
6 merged = merge(d1, d2, d3)
7 # Dict{"a" => 1, "b" => 3, "c" => 5, "d" => 6}
8
9 # merge! - modify d1 in-place
10 merge!(d1, d2)
11 # d1 = Dict{"a" => 1, "b" => 3, "c" => 4}
12
13 # mergewith - custom combine function
14 counts1 = Dict{"apple" => 5, "banana" => 3}
15 counts2 = Dict{"apple" => 2, "cherry" => 7}
16
17 # Cộng các giá trị khi key trùng
18 total = mergewith(+, counts1, counts2)
19 # Dict{"apple" => 7, "banana" => 3, "cherry" => 7}
20
21 # Lấy max khi trùng key
22 best_scores = mergewith(max,
23     Dict{"An" => 85, "Bình" => 90},
24     Dict{"An" => 92, "Bình" => 88}
25 )
26 # Dict{"An" => 92, "Bình" => 90}
27
28 # String concatenation
29 messages = mergewith(*,
30     Dict{"greeting" => "Hello "},
31     Dict{"greeting" => "World!"}
32 )
33 # Dict{"greeting" => "Hello World!"}

```

### 4.1.3.3 Duyệt Dictionary nâng cao

Có nhiều cách duyệt qua dictionary với các hàm tiện ích khác nhau.

```

1 scores = Dict{"An" => 85, "Bình" => 90, "Cường" => 78, "Dũng" => 92}
2
3 # pairs() - iterator trả về key => value pairs
4 for p in pairs(scores)
5     println("$p.first): $p.second)")
6 end
7
8 # Destructuring trong for loop
9 for (name, score) in scores

```

```
10     println("$name đạt $score điểm")
11 end
12
13 # keys() và values() riêng biệt
14 all_names = collect(keys(scores)) # ["An", "Bình", "Cường", "Dũng"]
15 all_scores = collect(values(scores)) # [85, 90, 78, 92]
16
17 # eachindex - duyệt theo indices (cho Dict là keys)
18 for k in eachindex(scores)
19     println("Key: $k, Value: $(scores[k])")
20 end
21
22 # map trên pairs
23 doubled = Dict{k => v * 2 for (k, v) in scores}
24 # Dict("An" => 170, "Bình" => 180, ...)
25
26 # filter với predicate
27 high_achievers = filter(p -> p.second >= 85, scores)
28 # Dict("An" => 85, "Bình" => 90, "Dũng" => 92)
29
30 # Chỉ lọc keys
31 passing_names = [k for (k, v) in scores if v >= 60]
```

#### 4.1.3.4 Dict Comprehension chi tiết

Dict comprehension là cách ngắn gọn và hiệu quả để tạo dictionary từ iterable.

```
1 # Cơ bản: key => value từ iteration
2 squares = Dict{x => x^2 for x in 1:5}
3 # Dict{1 => 1, 2 => 4, 3 => 9, 4 => 16, 5 => 25}
4
5 # Với điều kiện filter
6 even_squares = Dict{x => x^2 for x in 1:10 if iseven(x)}
7 # Dict{2 => 4, 4 => 16, 6 => 36, 8 => 64, 10 => 100}
8
9 # Transform từ existing dict
10 original = Dict{"a" => 1, "b" => 2, "c" => 3}
11 uppercased = Dict{uppercase(k) => v * 10 for (k, v) in original}
12 # Dict{"A" => 10, "B" => 20, "C" => 30}
13
14 # Đảo ngược key-value (chỉ khi values unique)
15 flipped = Dict{v => k for (k, v) in original}
16 # Dict{1 => "a", 2 => "b", 3 => "c"}
17
18 # Từ hai arrays song song
19 keys_arr = ["name", "age", "city"]
20 vals_arr = ["Julia", 12, "MIT"]
21 person = Dict{k => v for (k, v) in zip(keys_arr, vals_arr)}
22 # Dict{"name" => "Julia", "age" => 12, "city" => "MIT"}
23
24 # Nested comprehension
25 matrix_coords = Dict{(i, j) => i * j for i in 1:3 for j in 1:3}
26 # Dict{(1,1) => 1, (1,2) => 2, ..., (3,3) => 9}
```

```

27
28 # Từ enumerate
29 indexed = Dict{i => char for (i, char) in enumerate("ABCD")}
30 # Dict{1 => 'A', 2 => 'B', 3 => 'C', 4 => 'D'}

```

#### 4.1.3.5 Symbol keys vs String keys

Trong Julia, sử dụng Symbol làm keys thường hiệu quả hơn String cho các use cases cố định.

```

1 # String keys - linh hoạt nhưng chậm hơn
2 config_str = Dict{"host" => "localhost", "port" => 8080}
3
4 # Symbol keys - nhanh hơn cho fixed keys
5 config_sym = Dict{:host => "localhost", :port => 8080}
6
7 # Symbols được intern - so sánh nhanh O(1)
8 # Strings cần so sánh từng character O(n)
9
10 # Benchmark example:
11 # using BenchmarkTools
12 # d_str = Dict{"key$i" => i for i in 1:1000}
13 # d_sym = Dict{Symbol("key$i") => i for i in 1:1000}
14 # @btime d_str["key500"] # ~15 ns
15 # @btime d_sym[:key500] # ~8 ns
16
17 # Quy tắc chọn:
18 # Symbol: config keys, fixed options, API fields
19 # String: user input, dynamic keys, external data

```

#### So sánh Symbol vs String keys:

| Tiêu chí     | Symbol               | String                   |
|--------------|----------------------|--------------------------|
| Lookup speed | Nhanh hơn (interned) | Chậm hơn (so sánh chars) |
| Memory       | Tốt hơn (shared)     | Mỗi string riêng biệt    |
| Creation     | Chậm hơn (intern)    | Nhanh hơn                |
| Use case     | Fixed keys, configs  | Dynamic, user input      |
| Syntax       | :symbol              | "string"                 |

#### 4.1.3.6 OrderedDict và DefaultDict

Thư viện DataStructures.jl cung cấp các Dict variants hữu ích.

```

1 using DataStructures
2
3 # OrderedDict - giữ thứ tự insertion
4 od = OrderedDict{"first" => 1, "second" => 2, "third" => 3}
5 for (k, v) in od
6     println("$k: $v") # Luôn in theo thứ tự insertion
7 end
8 # first: 1
9 # second: 2
10 # third: 3
11

```

```

12 # DefaultDict - tự động default value cho missing keys
13 # Hữu ích cho counting, grouping
14 word_count = defaultdict(String, Int)(0)
15 for word in ["apple", "banana", "apple", "cherry", "apple"]
16     word_count[word] += 1 # Không cần check haskey
17 end
18 # DefaultDict("apple" => 3, "banana" => 1, "cherry" => 1)
19
20 # DefaultDict với function
21 list_dict = defaultdict(String, Vector{Int}){Vector{Int}}
22 push!(list_dict["group_a"], 1)
23 push!(list_dict["group_a"], 2)
24 push!(list_dict["group_b"], 10)
25 # DefaultDict("group_a" => [1, 2], "group_b" => [10])
26
27 # SortedDict - keys luôn được sort
28 sd = SortedDict("charlie" => 3, "alpha" => 1, "bravo" => 2)
29 collect(keys(sd)) # ["alpha", "bravo", "charlie"]

```

Khi nào dùng Dict variant nào:

- **Dict**: Default choice, fastest lookup
- **OrderedDict**: Cần giữ thứ tự insertion (JSON output, configs)
- **DefaultDict**: Counting, grouping, avoid haskey checks
- **SortedDict**: Cần keys luôn sorted

#### 4.1.3.7 IdDict - Identity-based Dict

IdDict so sánh keys bằng identity (===) thay vì equality (==). Hữu ích khi làm việc với mutable objects.

```

1 # Với Dict thông thường - so sánh bằng ==
2 d = Dict{Int, String}([1, 2] => "a")
3 d[[1, 2]] = "b" # Ghi đè vì [1,2] == [1,2]
4 # Dict{Int, String}([1, 2] => "b")
5
6 # Với IdDict - so sánh bằng === (identity)
7 id = IdDict{Vector{Int}, String}()
8 arr1 = [1, 2]
9 arr2 = [1, 2] # Cùng giá trị nhưng khác object
10
11 id[arr1] = "first"
12 id[arr2] = "second"
13 # Có 2 entries vì arr1 !== arr2
14
15 id[arr1] # "first"
16 id[arr2] # "second"
17
18 # Use case: tracking object instances
19 # objects_metadata = IdDict{Any, Dict{Symbol, Any}}()

```



#### 4.1.3.8 WeakKeyDict - Garbage-collectable keys

WeakKeyDict cho phép keys được garbage collected khi không còn reference khác. Hữu ích cho caching và object tracking.

```

1  # WeakKeyDict - keys có thể bị garbage collected
2  cache = WeakKeyDict{Vector{Int}, Int}()
3
4  function process(data)
5      # Nếu đã cache, return ngay
6      haskey(cache, data) && return cache[data]
7
8      # Tính toán và cache
9      result = sum(data) # Expensive computation
10     cache[data] = result
11     return result
12 end
13
14 arr = [1, 2, 3, 4, 5]
15 process(arr) # Tính và cache
16 process(arr) # Lấy từ cache
17
18 # Khi arr không còn được reference
19 arr = nothing
20 GC.gc() # Key trong cache sẽ bị xóa tự động

```

#### 4.1.3.9 Dict Performance Tips

```

1  # 1. Preallocate với sizehint! nếu biết trước số lượng keys
2  d = Dict{String, Int}()
3  sizehint!(d, 10000) # Tránh rehashing
4
5  # 2. Dùng get thay vì haskey + access
6  # Bad:
7  if haskey(d, "key")
8      val = d["key"]
9  end
10 # Good:
11 val = get(d, "key", nothing)
12
13 # 3. Dùng Symbol keys cho fixed/known keys
14 # Symbol comparison là O(1), String là O(n)
15
16 # 4. Tránh type instability
17 # Bad: Dict() - Any keys, Any values
18 # Good: Dict{String, Int}() - typed
19
20 # 5. Dùng empty! thay vì tạo dict mới
21 empty!(d) # Reuse allocated memory
22
23 # 6. Lock cho concurrent access
24 const shared_dict = Dict{String, Int}()
25 const dict_lock = ReentrantLock()

```

```

26
27 function safe_update!(d, k, v)
28     lock(dict_lock) do
29         d[k] = v
30     end
31 end

```

### Big-O Complexity của Dict operations:

| Operation                    | Average | Worst Case |
|------------------------------|---------|------------|
| d[key] (access)              | O(1)    | O(n)       |
| d[key] = val (insert/update) | O(1)    | O(n)       |
| delete!(d, key)              | O(1)    | O(n)       |
| haskey(d, key)               | O(1)    | O(n)       |
| keys(d), values(d)           | O(1)    | O(1)       |
| length(d)                    | O(1)    | O(1)       |
| merge(d1, d2)                | O(n)    | O(n)       |
| Iteration                    | O(n)    | O(n)       |

Worst case O(n) xảy ra khi nhiều keys có cùng hash value (hash collision). Trong thực tế, với hash function tốt, hầu hết operations là O(1).

#### 4.1.4 Set

Set là collection chỉ chứa các phần tử **unique** (không trùng lặp), hỗ trợ các phép toán tập hợp (union, intersection, difference). Set dùng hash table nên có O(1) lookup time.

##### 1. Tạo set:

```

1  # Set từ array (tự động loại duplicates)
2  s = Set([1, 2, 2, 3, 3, 3])  # Set([1, 2, 3])
3
4  # Empty set với type
5  s_int = Set{Int}()
6
7  # Set từ range
8  s_range = Set(1:10)

```

##### 2. Thao tác cơ bản:

```

1  s = Set([1, 2, 3])
2
3  # Thêm element
4  push!(s, 4)      # s = Set([1, 2, 3, 4])
5  push!(s, 2)      # Không thay đổi (2 đã tồn tại)
6
7  # Xóa element
8  delete!(s, 3)    # s = Set([1, 2, 4])
9
10 # Kiểm tra membership
11 2 in s           # true
12 5 in s           # false
13
14 # Số lượng elements
15 length(s)        # 3

```

### 3. Phép toán tập hợp:

```

1 A = Set([1, 2, 3, 4])
2 B = Set([3, 4, 5, 6])
3
4 # Union (hợp)
5 union(A, B)          # Set([1, 2, 3, 4, 5, 6])
6 A ∪ B                # Ký hiệu Unicode (gõ: \cup + Tab)
7
8 # Intersection (giao)
9 intersect(A, B)       # Set([3, 4])
10 A ∩ B               # Ký hiệu Unicode (gõ: \cap + Tab)
11
12 # Difference (hiệu)
13 setdiff(A, B)         # Set([1, 2])
14
15 # Symmetric difference (hiệu đối xứng)
16 symdiff(A, B)         # Set([1, 2, 5, 6])
17
18 # Subset check
19 Set([1, 2]) ⊆ A       # true (gõ: \subseq + Tab)

```

#### Use cases:

```

1 # 1. Loại bỏ duplicates
2 words = ["apple", "banana", "apple", "cherry", "banana"]
3 unique_words = Set(words) # Set(["apple", "banana", "cherry"])
4
5 # 2. Membership testing (fast)
6 allowed_users = Set(["admin", "user1", "user2"])
7 if "user1" in allowed_users
8     println("Access granted")
9 end
10
11 # 3. Find common elements
12 students_math = Set(["An", "Bình", "Cường"])
13 students_physics = Set(["Bình", "Dũng", "Em"])
14 both_classes = intersect(students_math, students_physics)
15 # Set(["Bình"])

```

#### Đặc điểm:

- Chỉ chứa unique elements (tự động loại duplicates)
- Không đảm bảo thứ tự elements
- O(1) membership testing (rất nhanh)
- Hỗ trợ đầy đủ các phép toán tập hợp
- Elements phải là immutable và hashable

#### 4.1.4.1 Subset và Superset operations

Julia hỗ trợ đầy đủ các phép kiểm tra quan hệ giữa các tập hợp.

```

1  A = Set([1, 2, 3])
2  B = Set([1, 2, 3, 4, 5])
3  C = Set([1, 2, 3])
4
5  # issubset - kiểm tra tập con
6  issubset(A, B)      # true - A là subset của B
7  A ⊆ B               # true (gõ: \subseq + Tab)
8  A ⊂ B               # true - proper subset (gõ: \subset + Tab)
9
10 # issetequal - hai set có cùng elements
11 issetequal(A, C)    # true
12 A == C              # true
13
14 # Superset (tập cha)
15 B ⊇ A               # true (gõ: \supseteq + Tab)
16 issuperset(B, A)    # true
17
18 # Disjoint - không có element chung
19 isdisjoint(Set([1, 2]), Set([3, 4])) # true
20 isdisjoint(Set([1, 2]), Set([2, 3])) # false (có 2 chung)
21
22 # Empty set check
23 isempty(Set())      # true
24 isempty(A)          # false

```

#### 4.1.4.2 Set Comprehension

Tương tự array và dict comprehension, có thể tạo set từ comprehension.

```

1  # Set comprehension cơ bản
2  squares = Set(x^2 for x in 1:10)
3  # Set([1, 4, 9, 16, 25, 36, 49, 64, 81, 100])
4
5  # Với filter condition
6  even_squares = Set(x^2 for x in 1:10 if iseven(x))
7  # Set([4, 16, 36, 64, 100])
8
9  # Từ string (unique characters)
10 unique_chars = Set(c for c in "hello world" if c != ' ')
11 # Set(['h', 'e', 'l', 'o', 'w', 'r', 'd'])
12
13 # Từ multiple ranges
14 pairs_sum = Set(i + j for i in 1:3 for j in 1:3)
15 # Set([2, 3, 4, 5, 6])
16
17 # Transform elements
18 lowercased = Set(lowercase(s) for s in ["Hello", "WORLD", "Julia"])
19 # Set(["hello", "world", "julia"])

```

#### 4.1.4.3 In-place Set modifications

Các phép toán tập hợp có phiên bản in-place (modify set gốc).

```

1 A = Set([1, 2, 3])
2 B = Set([3, 4, 5])
3
4 # union! - thêm elements từ B vào A
5 union!(A, B)
6 # A = Set([1, 2, 3, 4, 5])
7
8 # intersect! - giữ lại chỉ elements chung
9 C = Set([1, 2, 3, 4, 5])
10 D = Set([3, 4, 5, 6, 7])
11 intersect!(C, D)
12 # C = Set([3, 4, 5])
13
14 # setdiff! - loại bỏ elements có trong set khác
15 E = Set([1, 2, 3, 4, 5])
16 setdiff!(E, Set([2, 4]))
17 # E = Set([1, 3, 5])
18
19 # symdiff! - symmetric difference in-place
20 F = Set([1, 2, 3])
21 symdiff!(F, Set([2, 3, 4]))
22 # F = Set([1, 4])
23
24 # filter! - loại bỏ elements không thỏa condition
25 G = Set([1, 2, 3, 4, 5, 6])
26 filter!(iseven, G)
27 # G = Set([2, 4, 6])

```

#### 4.1.4.4 BitSet - Optimized Integer Set

BitSet là specialized set cho integers, sử dụng bit vector representation. Rất hiệu quả về memory và performance cho dense integer sets.

```

1 # Tạo BitSet
2 bs = BitSet([1, 5, 10, 100, 1000])
3
4 # Tất cả set operations đều hoạt động
5 push!(bs, 50)
6 2 in bs           # false
7 50 in bs          # true
8
9 # Range
10 bs_range = BitSet(1:100)
11
12 # Set operations với BitSet
13 bs1 = BitSet([1, 2, 3, 4, 5])
14 bs2 = BitSet([4, 5, 6, 7, 8])
15
16 union(bs1, bs2)    # BitSet([1, 2, 3, 4, 5, 6, 7, 8])
17 intersect(bs1, bs2) # BitSet([4, 5])
18
19 # So sánh với Set{Int}:
20 # BitSet: ~1 bit per potential element, O(1) operations

```

```
21 # Set{Int}: ~64 bytes per element, O(1) amortized
22
23 # Use case: tracking seen indices, dense integer collections
24 seen_pages = BitSet()
25 push!(seen_pages, 1)
26 push!(seen_pages, 5)
27 push!(seen_pages, 3)
28 # Iterate in sorted order (automatic)
29 collect(seen_pages) # [1, 3, 5]
```

#### Khi nào dùng BitSet vs SetInt:

- **BitSet**: Dense integers, small range, need sorted iteration
- **Set{Int}**: Sparse integers, large range, mixed types

#### 4.1.4.5 Set với Custom Types

Để sử dụng custom types trong Set, cần implement hash và ==.

```
1 struct Point
2     x::Float64
3     y::Float64
4 end
5
6 # Default: mỗi instance là unique (by identity)
7 p1 = Point(1.0, 2.0)
8 p2 = Point(1.0, 2.0)
9 Set([p1, p2]) # Set với 2 elements (dù cùng giá trị)
10
11 # Để so sánh bằng value, cần define hash và ==
12 import Base: hash, ==
13
14 ==(a::Point, b::Point) = a.x == b.x && a.y == b.y
15 hash(p::Point, h::UInt) = hash(p.y, hash(p.x, h))
16
17 # Sau khi define
18 Set([p1, p2]) # Set với 1 element (p1 == p2)
19
20 # Practical example: unique coordinates
21 struct Coord
22     lat::Float64
23     lon::Float64
24 end
25
26 ==(a::Coord, b::Coord) = a.lat == b.lat && a.lon == b.lon
27 hash(c::Coord, h::UInt) = hash(c.lon, hash(c.lat, h))
28
29 locations = Set{Coord}()
30 push!(locations, Coord(10.5, 106.7))
31 push!(locations, Coord(10.5, 106.7)) # Không thêm (duplicate)
32 push!(locations, Coord(21.0, 105.8))
33 length(locations) # 2
```

## 4.1.4.6 Set Patterns và Use Cases

```

1  # 1. Counting unique elements
2  words = ["apple", "banana", "apple", "cherry", "banana", "apple"]
3  unique_count = length(Set(words))  # 3
4
5  # 2. Find duplicates
6  function find_duplicates(arr)
7      seen = Set{eltype(arr)}{ }
8      duplicates = Set{eltype(arr)}{ }
9      for x in arr
10         if x in seen
11             push!(duplicates, x)
12         else
13             push!(seen, x)
14         end
15     end
16     return duplicates
17 end
18 find_duplicates([1, 2, 3, 2, 4, 3])  # Set{2, 3}
19
20 # 3. Two-set analysis (Venn diagram logic)
21 registered = Set(["An", "Bình", "Cường", "Dũng"])
22 attended = Set(["Bình", "Dũng", "Em"])
23
24 present_registered = intersect(registered, attended)  # Đăng ký và có mặt
25 absent_registered = setdiff(registered, attended)    # Đăng ký nhưng vắng
26 unregistered_attendees = setdiff(attended, registered) # Có mặt nhưng không
27               ↪ đăng ký
28
29 # 4. Efficient lookup table
30 valid_commands = Set(["start", "stop", "pause", "resume", "status"])
31 function execute(cmd::String)
32     if cmd in valid_commands
33         println("Executing: $cmd")
34     else
35         println("Unknown command: $cmd")
36     end
37 end
38
39 # 5. Remove duplicates while preserving first occurrence order
40 function unique_ordered(arr)
41     seen = Set{eltype(arr)}{ }
42     result = eltype(arr){ }
43     for x in arr
44         if !(x in seen)
45             push!(seen, x)
46             push!(result, x)
47         end
48     end
49     return result
50 end

```

```
50 unique_ordered([3, 1, 4, 1, 5, 9, 2, 6, 5]) # [3, 1, 4, 5, 9, 2, 6]
```

#### 4.1.4.7 Set Performance và Big-O

Big-O Complexity của Set operations:

| Operation                        | Average        | Worst Case     |
|----------------------------------|----------------|----------------|
| <code>x in S</code> (membership) | $O(1)$         | $O(n)$         |
| <code>push!(S, x)</code>         | $O(1)$         | $O(n)$         |
| <code>delete!(S, x)</code>       | $O(1)$         | $O(n)$         |
| <code>length(S)</code>           | $O(1)$         | $O(1)$         |
| <code>union(A, B)</code>         | $O(m+n)$       | $O(m+n)$       |
| <code>intersect(A, B)</code>     | $O(\min(m,n))$ | $O(\min(m,n))$ |
| <code>setdiff(A, B)</code>       | $O(m)$         | $O(m)$         |
| <code>issubset(A, B)</code>      | $O(m)$         | $O(m)$         |
| Iteration                        | $O(n)$         | $O(n)$         |

Với  $m = |A|$ ,  $n = |B|$ . Worst case  $O(n)$  xảy ra khi có nhiều hash collisions.

#### 4.1.5 So sánh và lựa chọn

| Cấu trúc | Mutable? | Ordered? | Use case                        |
|----------|----------|----------|---------------------------------|
| Array    | Có       | Có       | Numeric computation, sequences  |
| Tuple    | Không    | Có       | Fixed data, function returns    |
| Dict     | Có       | Không    | Key-value lookups               |
| Set      | Có       | Không    | Unique elements, set operations |

Nguyên tắc lựa chọn:

- Dùng **Array** cho: tính toán số học, sequences có thứ tự, numeric data
- Dùng **Tuple** cho: dữ liệu nhỏ không đổi, return multiple values, coordinates
- Dùng **Dict** cho: key-value mapping, fast lookups, configurations
- Dùng **Set** cho: unique collections, membership testing, set operations

## 4.2 Struct và Mutable struct

Trong Julia, **struct** (còn gọi là composite types - kiểu dữ liệu tổng hợp) cho phép bạn tự định nghĩa kiểu dữ liệu riêng bằng cách nhóm các trường (fields) liên quan lại với nhau. Đây là cơ chế quan trọng để tổ chức và cấu trúc hóa dữ liệu trong chương trình.

### 4.2.1 Struct cơ bản (Immutable struct)

Mặc định, struct trong Julia là **immutable** (bất biến) - nghĩa là sau khi tạo một instance, bạn không thể thay đổi giá trị các trường của nó. Thiết kế này giúp code ít lỗi hơn và hiệu năng tốt hơn.

1. Định nghĩa struct:

```
1 struct Point
2     x::Float64
3     y::Float64
4 end
```



Trong định nghĩa trên:

- `Point` là tên của kiểu dữ liệu mới
- `x` và `y` là các trường (fields)
- `::Float64` là type annotation - chỉ định kiểu dữ liệu của mỗi trường

## 2. Tạo instances:

```
1 # Tạo Point với constructor mặc định
2 p1 = Point(3.0, 4.0)
3 p2 = Point(0.0, 0.0)
4
5 # Truy cập các trường
6 println(p1.x) # 3.0
7 println(p1.y) # 4.0
```

## Lưu ý về immutability:

```
1 p1.x = 5.0 # ERROR: setfield! immutable struct cannot be changed
```

Bạn không thể thay đổi giá trị trường sau khi tạo. Nếu cần sửa đổi, phải tạo instance mới:

```
1 # Tạo Point mới với giá trị khác
2 p1_new = Point(5.0, p1.y)
```

### 4.2.2 Mutable struct

Nếu bạn cần khả năng thay đổi các trường sau khi tạo, sử dụng từ khóa `mutable` trước `struct`:

```
1 mutable struct Particle
2     pos::Point # Vị trí
3     vel::Point # Vận tốc
4     mass::Float64 # Khối lượng
5 end
```

## Thao tác với mutable struct:

```
1 # Tạo particle
2 particle = Particle(Point(0.0, 0.0), Point(1.0, 2.0), 5.0)
3
4 # Có thể thay đổi các trường
5 particle.mass = 10.0
6 particle.pos = Point(5.0, 5.0)
7
8 println(particle.mass) # 10.0
```

### 4.2.3 Constructors (Hàm khởi tạo)

Julia tự động tạo constructor mặc định cho mọi struct, nhưng bạn có thể tự định nghĩa constructors để thêm logic hoặc validation.

#### 1. Outer constructors (bên ngoài struct definition):

```

1  struct Rectangle
2      width::Float64
3      height::Float64
4  end
5
6  # Outer constructor để tạo hình vuông tiện hơn
7  Rectangle(side::Float64) = Rectangle(side, side)
8
9  r1 = Rectangle(10.0, 5.0)  # Hình chữ nhật
10 r2 = Rectangle(7.0)        # Hình vuông 7x7

```

## 2. Inner constructors (bên trong struct definition):

Inner constructors cho phép validation và custom logic khi tạo instances:

```

1  struct Circle
2      radius::Float64
3
4      # Inner constructor với validation
5      function Circle(r::Number)
6          if r <= 0
7              error("Radius must be positive")
8          end
9          new(Float64(r))  # new() tạo instance
10     end
11 end
12
13 c1 = Circle(5.0)        # OK
14 c2 = Circle(-1.0)       # ERROR: Radius must be positive

```

**Quan trọng:** Inner constructor sử dụng hàm đặc biệt `new()` để tạo instance.

### 4.2.3.1 Hàm `new()` chi tiết

Hàm `new()` là một hàm đặc biệt chỉ có thể sử dụng bên trong inner constructor. Nó có một số đặc điểm quan trọng:

```

1  mutable struct Tank
2      drymass::Float64
3      totalmass::Float64
4      propellant::Float64
5
6      # Inner constructor với new()
7      function Tank(drymass::Number, totalmass::Number)
8          # new() tự động tính propellant
9          new(drymass, totalmass, totalmass - drymass)
10     end
11 end
12
13 tank = Tank(5, 50)
14 # Tank(5.0, 50.0, 45.0) - propellant được tính tự động

```

**Đặc điểm của `new()`:**

- Chỉ khả dụng bên trong inner constructor

- Có thể nhận ít arguments hơn số fields (fields còn lại sẽ có giá trị không xác định)
- Không thể nhận nhiều arguments hơn số fields
- Inner constructor thay thế hoàn toàn constructor mặc định của Julia

```

1  # Ví dụ với partial initialization
2  struct Partial
3      a::Int
4      b::Int
5      c::Int
6
7      function Partial(a::Int)
8          x = new(a)  # b và c chưa được khởi tạo
9          x  # Nguy hiểm! Không nên dùng
10     end
11 end

```

#### 4.2.3.2 Best practices cho constructors

Theo Julia documentation, nên tuân thủ nguyên tắc sau:

“Provide as few inner constructor methods as possible: only those taking all arguments explicitly and enforcing essential error checking and transformation.”

##### Guidelines:

1. Ưu tiên outer constructors cho convenience methods
2. Chỉ dùng inner constructor khi cần:
  - Validation bắt buộc (ví dụ: kiểm tra giá trị dương)
  - Computed fields (tính toán field từ các tham số khác)
  - Invariant enforcement (đảm bảo tính nhất quán dữ liệu)
3. Không dùng inner constructor chỉ để đổi signature

```

1  # Không nên: dùng inner constructor chỉ để đổi order
2  struct BadExample
3      x::Int
4      y::Int
5      BadExample(y, x) = new(x, y)  # Gây confusion
6  end
7
8  # Nên: dùng outer constructor cho convenience
9  struct GoodExample
10     x::Int
11     y::Int
12 end
13 GoodExample(; x=0, y=0) = GoodExample(x, y)  # Keyword constructor

```

### 4.2.3.3 Multiple constructors pattern

Bạn có thể kết hợp inner và outer constructors:

```
1  struct ComplexNum
2      real::Float64
3      imag::Float64
4
5      # Inner: validation
6      function ComplexNum(r::Real, i::Real)
7          new(Float64(r), Float64(i))
8      end
9  end
10
11 # Outer: convenience methods
12 ComplexNum(r::Real) = ComplexNum(r, 0.0)
13 ComplexNum() = ComplexNum(0.0, 0.0)
14
15 # Outer: từ polar coordinates
16 function ComplexNum(magnitude::Real, angle::Real, ::Val{:polar})
17     ComplexNum(magnitude * cos(angle), magnitude * sin(angle))
18 end
19
20 # Sử dụng
21 c1 = ComplexNum(3, 4)           # Cartesian
22 c2 = ComplexNum(5)              # Real only
23 c3 = ComplexNum(5, pi/4, Val{:polar}) # Polar
```

### 4.2.4 Type annotations và inspection

#### 1. Field names và types:

```
1  struct Person
2      name::String
3      age::Int
4      height::Float64
5  end
6
7  person = Person("An", 25, 1.75)
8
9  # Xem field names
10 fieldnames(Person) # (:name, :age, :height)
11
12 # Xem field type
13 fieldtype(Person, :age) # Int64
```

#### 2. Default values với @kwdef macro:

```
1  Base.@kwdef struct Config
2      host::String = "localhost"
3      port::Int = 8080
4      timeout::Float64 = 30.0
5      debug::Bool = false
6  end
```

```

7
8 # Tạo với defaults
9 config1 = Config()
10
11 # Override một số giá trị
12 config2 = Config(port=9000, debug=true)

```

Macro @kwdef tự động tạo keyword argument constructor với default values.

#### 4.2.4.1 getfield và setfield!

Julia cung cấp các hàm low-level để truy cập fields:

```

1 struct Point3D
2     x::Float64
3     y::Float64
4     z::Float64
5 end
6
7 p = Point3D(1.0, 2.0, 3.0)
8
9 # getfield - truy cập field bằng Symbol
10 getfield(p, :x)      # 1.0
11 getfield(p, :y)      # 2.0
12
13 # Equivalent với dot syntax
14 p.x == getfield(p, :x) # true
15
16 # Lấy tất cả fields
17 [getfield(p, f) for f in fieldnames(Point3D)]
18 # [1.0, 2.0, 3.0]

```

setfield! với mutable struct:

```

1 mutable struct MutablePoint
2     x::Float64
3     y::Float64
4 end
5
6 mp = MutablePoint(1.0, 2.0)
7 setfield!(mp, :x, 10.0) # Thay đổi x thành 10.0
8 mp.x # 10.0
9
10 # Equivalent với
11 mp.y = 20.0 # Cũng dùng setfield! bên dưới

```

#### 4.2.4.2 getproperty và setproperty! (Custom access)

Bạn có thể override cách truy cập properties bằng cách định nghĩa getproperty và setproperty!:

```

1 struct Vector2D
2     data::Vector{Float64}
3 end
4

```

```
5  # Custom getproperty cho x, y access
6  function Base.getproperty(v::Vector2D, name::Symbol)
7      if name === :x
8          return getfield(v, :data)[1]
9      elseif name === :y
10         return getfield(v, :data)[2]
11     elseif name === :magnitude
12         d = getfield(v, :data)
13         return sqrt(d[1]^2 + d[2]^2)
14     else
15         return getfield(v, name)
16     end
17 end
18
19 v = Vector2D([3.0, 4.0])
20 v.x          # 3.0
21 v.y          # 4.0
22 v.magnitude   # 5.0 (computed property)
23 v.data       # [3.0, 4.0]
```

#### Lưu ý quan trọng:

- Bên trong `getproperty`, phải dùng `getfield` để tránh infinite recursion
- `propertynames()` có thể override để hỗ trợ tab-completion
- Pattern này hữu ích cho computed properties và lazy evaluation

```
1  # Override propertynames cho tab-completion
2  function Base.propertynames(v::Vector2D, private::Bool=false)
3      return (:x, :y, :magnitude, :data)
4  end
```

#### 4.2.5 Nested structs

Bạn có thể lồng structs trong structs:

```
1  struct Address
2      street::String
3      city::String
4      zipcode::String
5  end
6
7  struct Employee
8      name::String
9      age::Int
10     address::Address
11 end
12
13 # Tạo nested struct
14 emp = Employee(
15     "Nguyễn Văn A",
16     30,
17     Address("123 Đường ABC", "TP.HCM", "700000")
18 )
```

```

18 )
19
20 # Truy cập nested fields
21 println(emp.address.city) # TP.HCM

```

#### 4.2.6 So sánh struct vs mutable struct

| Đặc điểm        | struct (immutable)        | mutable struct        |
|-----------------|---------------------------|-----------------------|
| Thay đổi fields | Không                     | Có                    |
| Performance     | Nhanh hơn                 | Chậm hơn              |
| Memory          | Ít hơn                    | Nhiều hơn             |
| Thread-safety   | An toàn                   | Cẩn thận              |
| Use case        | Data records, coordinates | State objects, caches |

##### Best practices:

- Ưu tiên **immutable struct** - dễ reasoning, ít bugs, performance tốt
- Chỉ dùng mutable khi *thực sự cần* thay đổi state
- Immutable types: Int, Float64, String, Tuple, NamedTuple, struct
- Mutable types: Array, Dict, mutable struct
- Khi thay đổi type definition, cần restart Julia REPL

#### 4.2.7 So sánh với Python classes

Để giúp người đọc đã quen với Python hiểu rõ hơn, đây là bảng so sánh:

| Khái niệm               | Python                       | Julia                                    |
|-------------------------|------------------------------|------------------------------------------|
| Định nghĩa class/struct | class Point:                 | struct Point                             |
| Mutable by default      | Có                           | Không (cần mutable)                      |
| Constructor             | <code>__init__</code>        | Inner constructor với <code>new()</code> |
| Instance variable       | <code>self.x</code>          | Field <code>x::Type</code>               |
| Class methods           | @classmethod                 | Outer constructor                        |
| Properties              | @property                    | getproperty override                     |
| Inheritance             | class B(A):                  | struct B <: A (chỉ abstract)             |
| Private fields          | <code>_x</code> (convention) | Không có (dùng modules)                  |

##### Ví dụ so sánh cụ thể:

```

1 # Python
2 # class Point:
3 #     def __init__(self, x, y):
4 #         self.x = x
5 #         self.y = y
6 #
7 #     def distance(self):
8 #         return (self.x**2 + self.y**2)**0.5
9
10 # Julia equivalent
11 struct Point
12     x::Float64
13     y::Float64
14 end

```

```

15
16 # Methods tách biệt khỏi struct definition
17 distance(p::Point) = sqrt(p.x^2 + p.y^2)
18
19 p = Point(3.0, 4.0)
20 distance(p) # 5.0

```

### Điểm khác biệt chính:

1. **Julia không có class methods:** Methods được định nghĩa bên ngoài struct và dispatch dựa trên types của arguments (multiple dispatch)
2. **Julia không có inheritance cho concrete types:** Chỉ abstract types có thể là supertype
3. **Julia structs không có mutable state by default:** Phải explicitly khai báo mutable
4. **Julia không có private/protected:** Dùng modules và export để kiểm soát visibility

## 4.2.8 Struct performance và memory layout

Julia structs có memory layout tối ưu giúp tăng performance:

### 4.2.8.1 Stack vs Heap allocation

```

1 # Immutable struct - có thể stack-allocated
2 struct StackPoint
3     x::Float64
4     y::Float64
5 end
6
7 # Mutable struct - luôn heap-allocated
8 mutable struct HeapPoint
9     x::Float64
10    y::Float64
11 end
12
13 # So sánh performance
14 using BenchmarkTools
15
16 function sum_points_immutable(n)
17     total = 0.0
18     for i in 1:n
19         p = StackPoint(i, i) # Stack allocation
20         total += p.x + p.y
21     end
22     total
23 end
24
25 function sum_points_mutable(n)
26     total = 0.0
27     for i in 1:n
28         p = HeapPoint(i, i) # Heap allocation
29         total += p.x + p.y
30     end

```



```

31     total
32 end
33
34 # @btime sum_points_immutable(1000000) # Nhanh hơn
35 # @btime sum_points_mutable(1000000)   # Chậm hơn do GC

```

#### 4.2.8.2 Type stability cho performance

Để có performance tốt nhất, luôn khai báo concrete types cho fields:

```

1  # Không tối ưu - abstract type field
2  struct BadContainer
3      value::Any # Compiler không biết type
4  end
5
6  # Tốt hơn - parametric type
7  struct GoodContainer{T}
8      value::T # Type cụ thể khi instantiate
9  end
10
11 # Tốt nhất - concrete type
12 struct BestContainer
13     value::Float64 # Compiler biết chính xác
14 end
15
16 # So sánh
17 @code_warntype (x -> x.value) (BestContainer{1.0})
18 # Kết quả: Body::Float64 (type stable)

```

#### 4.2.8.3 Memory layout và isbits types

```

1  # isbits type - stored inline, no pointers
2  struct IsBitsPoint
3      x::Float64
4      y::Float64
5  end
6
7  isbitstype(IsBitsPoint) # true
8  sizeof(IsBitsPoint)    # 16 bytes (2 * 8)
9
10 # Không phải isbits - có reference type
11 struct NotIsBits
12     x::Float64
13     name::String # String là reference type
14 end
15
16 isbitstype(NotIsBits) # false
17
18 # Trong Array, isbits types stored contiguously
19 arr = [IsBitsPoint(i, i) for i in 1:1000]
20 # Memory: [x1, y1, x2, y2, x3, y3, ...]
21 # Tốt cho cache locality!

```

## 4.2.9 Struct patterns và use cases

### 4.2.9.1 Immutable wrapper pattern

```
1  # Wrapper cho unit safety
2  struct Meters
3      value::Float64
4  end
5
6  struct Seconds
7      value::Float64
8  end
9
10 # Type-safe operations
11 Base.+(a::Meters, b::Meters) = Meters(a.value + b.value)
12 Base.:(a::Meters, b::Seconds) = a.value / b.value # Velocity
13
14 d1 = Meters(100.0)
15 d2 = Meters(50.0)
16 t = Seconds(10.0)
17
18 d1 + d2      # Meters(150.0)
19 d1 / t       # 10.0 (m/s)
20 # d1 + t     # MethodError - không thể cộng meters với seconds
```

### 4.2.9.2 Builder pattern với mutable struct

```
1  mutable struct QueryBuilder
2      table::String
3      columns::Vector{String}
4      conditions::Vector{String}
5      limit::Union{Int, Nothing}
6  end
7
8  QueryBuilder(table::String) = QueryBuilder(table, String[], String[], nothing)
9
10 function select!(qb::QueryBuilder, cols...)
11     append!(qb.columns, cols)
12     qb
13 end
14
15 function where!(qb::QueryBuilder, condition::String)
16     push!(qb.conditions, condition)
17     qb
18 end
19
20 function limit!(qb::QueryBuilder, n::Int)
21     qb.limit = n
22     qb
23 end
24
25 function to_sql(qb::QueryBuilder)
26     cols = isempty(qb.columns) ? "*" : join(qb.columns, ", ")
```

```

27     sql = "SELECT $cols FROM $(qb.table)"
28     if !isempty(qb.conditions)
29         sql *= " WHERE " * join(qb.conditions, " AND ")
30     end
31     if qb.limit != nothing
32         sql *= " LIMIT $(qb.limit)"
33     end
34     sql
35 end
36
37 # Fluent API
38 query = QueryBuilder("users") |>
39     qb -> select!(qb, "name", "email") |>
40     qb -> where!(qb, "active = true") |>
41     qb -> limit!(qb, 10)
42
43 to_sql(query)
44 # "SELECT name, email FROM users WHERE active = true LIMIT 10"

```

#### 4.2.9.3 Singleton pattern

```

1 # Singleton với struct không có fields
2 struct DatabaseConnection end
3
4 # Chỉ có một instance duy nhất
5 const DB = DatabaseConnection()
6
7 # Methods cho singleton
8 connect(::DatabaseConnection) = println("Connecting...")
9 query(::DatabaseConnection, sql) = println("Query: $sql")
10
11 connect(DB)
12 query(DB, "SELECT * FROM users")

```

## 4.3 Abstract và Parametric type

### 4.3.1 Abstract types (Kiểu trừu tượng)

Abstract types trong Julia [2, 4] là các kiểu không thể tạo instances trực tiếp, nhưng có thể được dùng để tổ chức hệ thống phân cấp kiểu (type hierarchy) và viết generic code.

**Định nghĩa abstract type:**

```

1 # Định nghĩa abstract type cơ bản
2 abstract type Shape end
3
4 # Abstract type với subtyping
5 abstract type Polygon <: Shape end

```

**Giải thích:**

- Shape là abstract type không có parent type cụ thể (mặc định là Any)
- Polygon là subtype của Shape (sử dụng operator <:)

- Không thể tạo: `Shape()`  $\rightarrow$  Error
- Chỉ dùng để định nghĩa interface và hierarchy

### 4.3.2 Type hierarchy trong Julia

Julia có hệ thống phân cấp kiểu rất rõ ràng, với `Any` là kiểu ở đỉnh và `Union{}` (empty type) ở đáy.

**Hệ thống kiểu số (Number hierarchy):**

```

1 # Khám phá type hierarchy với supertype() và subtypes()
2 julia> supertype(Int64)
3 # Output: Signed
4
5 julia> supertype(Signed)
6 # Output: Integer
7
8 julia> subtypes(Real)
9 # Output: [AbstractFloat, AbstractIrrational, Integer, Rational]
10
11 julia> subtypes(Integer)
12 # Output: [Bool, Signed, Unsigned]
```

**Cấu trúc phân cấp:**

```

Any
└─ Number
    └─ Real
        ├── AbstractFloat (Float16, Float32, Float64)
        ├── Integer
        │   ├── Signed (Int8, Int16, Int32, Int64, Int128)
        │   └── Unsigned (UInt8, UInt16, UInt32, UInt64, UInt128)
        │       └─ Bool
        └─ Rational
```

### 4.3.3 Parametric types (Kiểu tham số hóa)

Parametric types [2] cho phép định nghĩa generic types với type parameters, tương tự templates trong C++ hoặc generics trong Java.

**Định nghĩa parametric struct:**

```

1 # Parametric type với 1 type parameter
2 struct Point{T}
3     x::T
4     y::T
5 end
6
7 # Sử dụng
8 p1 = Point{Float64}(1.0, 2.0) # Tường minh
9 p2 = Point(3, 4)             # Julia tự suy luận: Point{Int64}
10
11 println(typeof(p1)) # Output: Point{Float64}
12 println(typeof(p2)) # Output: Point{Int64}
```

**Lợi ích của parametric types:**

1. **Type safety:** Đảm bảo x và y cùng kiểu
2. **Performance:** Compiler tối ưu hóa cho từng concrete type
3. **Code reuse:** Một definition cho nhiều types
4. **Memory efficiency:** Packed representation

**4.3.4 Type constraints (Ràng buộc kiểu)**

Type constraints sử dụng operator `<:` để giới hạn type parameter trong một subtype hierarchy nhất định.

**Ví dụ với constraints:**

```

1  # T phải là subtype của Real (Int, Float64, Rational, etc.)
2  struct Circle{T<:Real} <: Shape
3      radius::T
4  end
5
6  # Hợp lệ
7  c1 = Circle(5)           # Circle{Int64}
8  c2 = Circle(3.14)        # Circle{Float64}
9  c3 = Circle(2//3)        # Circle{Rational{Int64}}
10
11 # Không hợp lệ
12 # c4 = Circle("text") # ERROR: type String is not a subtype of Real

```

**Multiple type parameters:**

```

1  # Container với 2 type parameters
2  struct Pair{T,S}
3      first::T
4      second::S
5  end
6
7  # Flexible - các fields có thể khác kiểu
8  p1 = Pair(1, "hello")    # Pair{Int64, String}
9  p2 = Pair(3.14, true)    # Pair{Float64, Bool}
10 p3 = Pair("a", "b")      # Pair{String, String}

```

**4.3.5 UnionAll types**

Khi bạn viết `Point{T}`, đây là một UnionAll type - đại diện cho tất cả các `Point{T}` với mọi giá trị của T.

```

1  # Point{T} là UnionAll type
2  julia> Point{T} where T
3  # Output: Point
4
5  # Point{Float64} là concrete type
6  julia> isconcretetype(Point{Float64})
7  # Output: true

```

```
8
9  # Point (không có {T}) cũng là UnionAll
10 julia> Point === (Point{T} where T)
11 # Output: true
```

### 4.3.6 Use cases thực tế

#### 1. Generic container với type safety:

```
1  struct Container{T}
2      value::T
3  end
4
5  # Type safety - không thể mix types
6  c1 = Container{Int}(42)
7  c1.value = 3.14 # ERROR: type Float64 không phải Int
8
9  # Generic function hoạt động với mọi Container
10 function unwrap(c::Container{T}) where T
11     return c.value::T # Type assertion
12 end
```

#### 2. Abstract type cho polymorphism:

```
1  abstract type Animal end
2
3  struct Dog <: Animal
4      name::String
5  end
6
7  struct Cat <: Animal
8      name::String
9  end
10
11 # Generic function cho tất cả Animal
12 function speak(a::Animal)
13     if a isa Dog
14         println("$ (a.name): Woof!")
15     elseif a isa Cat
16         println("$ (a.name): Meow!")
17     end
18 end
19
20 dog = Dog("Buddy")
21 cat = Cat("Whiskers")
22 speak(dog) # Output: Buddy: Woof!
23 speak(cat) # Output: Whiskers: Meow!
```

#### 3. Parametric types với constraints trong data structures:

```
1  # Stack chỉ chấp nhận Number types
2  struct Stack{T<:Number}
3      items::Vector{T}
4  end
```

```

5
6 Stack{T}() where {T<:Number} = Stack{T}(T[])
7
8 function push!(s::Stack{T}, item::T) where T
9     push!(s.items, item)
10 end
11
12 # Sử dụng
13 s = Stack{Int64}()
14 push!(s, 10)
15 push!(s, 20)
16 # push!(s, "text") # ERROR: không thể push String

```

#### 4.3.7 So sánh: Concrete vs Abstract vs Parametric

| Đặc điểm        | Concrete      | Abstract  | Parametric          |
|-----------------|---------------|-----------|---------------------|
| Có instances    |               |           | (concrete)          |
| Có fields       |               |           |                     |
| Có subtypes     |               |           | (nếu concrete)      |
| Type parameters |               |           |                     |
| Generic code    | Limited       |           |                     |
| Performance     | Best          | N/A       | Best (khi concrete) |
| Use case        | Specific data | Hierarchy | Generic data        |

Bảng 4.1: So sánh các loại types trong Julia

#### 4.3.8 Best practices

##### 1. Prefer parametric over abstract fields:

```

1 # ❌ Bad - abstract field (type instability)
2 struct BadContainer
3     value::Any # hoặc ::Real, ::Number
4 end
5
6 # ✅ Good - parametric type
7 struct GoodContainer{T}
8     value::T
9 end

```

##### 2. Use constraints khi cần:

```

1 # Chỉ ràng buộc khi thực sự cần
2 struct Numeric{T<:Number} # Cần operations như +, *, etc.
3     value::T
4 end

```

##### 3. Leverage type hierarchy:

```

1 # Viết generic functions cho abstract types
2 function process(x::AbstractVector)
3     # Hoạt động với Vector, SubArray, Range, etc.
4     return sum(x)
5 end

```

#### 4. Type stability quan trọng:

```
1  # □ Type unstable
2  function bad(x)
3      if x > 0
4          return x          # Int or Float
5      else
6          return "negative" # String
7      end
8  end
9
10 # □ Type stable
11 function good(x::T) where T<:Number
12     if x > 0
13         return x          # Always T
14     else
15         return zero(T)    # Always T
16     end
17 end
```

##### 4.3.9 Mệnh đề where chi tiết

Mệnh đề where [2] là cách Julia định nghĩa type parameters trong functions và types. Hiểu rõ where giúp viết generic code hiệu quả.

###### 4.3.9.1 Cú pháp cơ bản của where

```
1  # Cú pháp 1: where sau function signature
2  function identity(x::T) where T
3      return x
4  end
5
6  # Cú pháp 2: where với constraints
7  function double(x::T) where T<:Number
8      return 2x
9  end
10
11 # Cú pháp 3: Multiple type parameters
12 function combine(a::T, b::S) where {T, S}
13     return (a, b)
14 end
15
16 # Cú pháp 4: Nested where
17 function nested(x::Vector{T}) where T where S
18     # T là element type của Vector
19     return first(x)
20 end
```

###### 4.3.9.2 where với multiple constraints

```
1  # T và S đều phải là Number
2  function add_pair(a::T, b::S) where {T<:Number, S<:Number}
3      return a + b
```



```

4  end
5
6  # T phải là subtype của S
7  function upcast(x::T) where {T, S>:T}
8      return x::S
9  end
10
11 # Bounded type parameters
12 function bounded(x::T) where {T<:Real, T>:Integer}
13     # T phải nằm giữa Integer và Real trong hierarchy
14     return x
15 end
16
17 # Sử dụng
18 julia> add_pair(1, 2.5)
19 # Output: 3.5
20
21 julia> add_pair(1+2im, 3.0)
22 # Output: 4.0 + 2.0im

```

#### 4.3.9.3 UnionAll và where equivalence

```

1  # Các cách viết tương đương
2  Vector{T} where T           # UnionAll notation
3  Vector{T} where {T}         # Explicit braces
4  Vector{T} where T<:Any      # With constraint
5
6  # Tất cả đều tương đương với kiểu Vector (abstract)
7  julia> Vector === (Vector{T} where T)
8  # Output: true
9
10 # Constrained UnionAll
11 julia> Vector{T} where T<:Number
12 # Output: Vector{T} where T<:Number
13
14 # Điều này KHÔNG phải là subtype của Vector{Number}!
15 julia> Vector{Int} <: Vector{Number}
16 # Output: false # Type invariance!
17
18 julia> Vector{Int} <: (Vector{T} where T<:Number)
19 # Output: true # Nhưng UnionAll hoạt động

```

#### 4.3.10 Type invariance vs covariance

Một trong những khái niệm quan trọng nhất khi làm việc với parametric types là hiểu về type invariance của Julia [2].

##### 4.3.10.1 Type invariance trong Julia

```

1  # Julia's parametric types are INVARIANT
2  julia> Int64 <: Number
3  # Output: true
4

```

```

5  julia> Vector{Int64} <: Vector{Number}
6  # Output: false    # Invariance!
7
8  # Tại sao? Vì nếu covariant, code sau sẽ unsafe:
9  function unsafe!(v::Vector{Number})
10     push!(v, 3.14)  # Thêm Float64
11 end
12
13 v = [1, 2, 3]  # Vector{Int64}
14 # Nếu Vector{Int64} <: Vector{Number}, thì:
15 # unsafe!(v)  # Sẽ thêm Float64 vào Vector{Int64}!
16 # v[4]        # Runtime error!

```

#### 4.3.10.2 Giải pháp: Abstract type annotations

```

1  # □ Không hoạt động với invariance
2  function process_numbers(v::Vector{Number})
3     return sum(v)
4  end
5  # process_numbers([1, 2, 3])  # ERROR!
6
7  # □ Giải pháp 1: UnionAll với where
8  function process_numbers(v::Vector{T}) where T<:Number
9     return sum(v)
10 end
11
12 # □ Giải pháp 2: AbstractVector
13 function process_numbers(v::AbstractVector{<:Number})
14     return sum(v)
15 end
16
17 # □ Giải pháp 3: Shorthand syntax
18 function process_numbers(v::Vector{<:Number})
19     return sum(v)
20 end
21
22 # Tất cả 3 cách đều hoạt động
23 julia> process_numbers([1, 2, 3])
24 # Output: 6

```

#### 4.3.10.3 Covariance với Tuple

```

1  # Tuple là COVARIANT (exception trong Julia)
2  julia> Tuple{Int} <: Tuple{Number}
3  # Output: true
4
5  julia> Tuple{Int, String} <: Tuple{Number, Any}
6  # Output: true
7
8  # Tại sao? Tuple là immutable, không thể modify
9  t = (1, 2, 3)  # Tuple{Int64, Int64, Int64}
10 # t[1] = 3.14  # ERROR: Tuple is immutable

```

### 4.3.11 Parametric types với Collections

Julia's collection types sử dụng parametric types để đảm bảo type safety và performance [2].

#### 4.3.11.1 Array{T,N} chi tiết

```

1  # Array có 2 type parameters: element type T và dimensionality N
2  julia> typeof([1, 2, 3])
3  # Output: Vector{Int64} # Alias cho Array{Int64, 1}
4
5  julia> typeof([1 2; 3 4])
6  # Output: Matrix{Int64} # Alias cho Array{Int64, 2}
7
8  julia> typeof(rand(2, 3, 4))
9  # Output: Array{Float64, 3}
10
11 # Type aliases
12 Vector{T} = Array{T, 1}
13 Matrix{T} = Array{T, 2}
14
15 # Tạo typed arrays
16 zeros(Float32, 3) # Vector{Float32} với 3 zeros
17 ones(Int8, 2, 2) # Matrix{Int8} 2x2 với ones
18 Array{Complex{Float64}}(undef, 5) # Uninitialized

```

#### 4.3.11.2 Dict{K,V} và Set{T}

```

1  # Dict với explicit type parameters
2  d1 = Dict{String, Int}()
3  d1["one"] = 1
4  # d1["two"] = "2" # ERROR: String không phải Int
5
6  # Dict với inferred types
7  d2 = Dict{"a" => 1, "b" => 2} # Dict{String, Int64}
8
9  # Type widening
10 d3 = Dict{Any, Any}() # Flexible nhưng slower
11 d3["name"] = "Julia"
12 d3["version"] = 1.9
13 d3["stable"] = true
14
15 # Set với type parameter
16 s1 = Set{Int}()
17 push!(s1, 1)
18 push!(s1, 2)
19 # push!(s1, "3") # ERROR
20
21 # Set với inferred type
22 s2 = Set([1, 2, 3]) # Set{Int64}

```

### 4.3.11.3 Parametric Pair và conversion

```
1 # Pair{A,B} là parametric type
2 p1 = Pair{String, Int}("age", 25)
3 p2 = "name" => "Julia" # Pair{String, String}
4
5 # Type của Pair xác định key-value types trong Dict
6 julia> typeof("a" => 1)
7 # Output: Pair{String, Int64}
8
9 # Conversion với parametric types
10 convert(Vector{Float64}, [1, 2, 3]) # Vector{Float64}
11 convert(Dict{Symbol, Any}, Dict("a" => 1)) # Symbol keys
12
13 # promote_type cho collections
14 julia> promote_type(Int, Float64)
15 # Output: Float64
16
17 julia> [1, 2.0, 3] # Promoted to Vector{Float64}
```

### 4.3.12 @code\_warntype và Type Stability Analysis

@code\_warntype là công cụ quan trọng nhất để phân tích type stability và tối ưu performance trong Julia [2].

#### 4.3.12.1 Sử dụng @code\_warntype

```
1 # Function type unstable
2 function bad_abs(x)
3     if x < 0
4         return -x
5     else
6         return x
7     end
8 end
9
10 # Kiểm tra với @code_warntype
11 julia> @code_warntype bad_abs(-5)
12 # Body::Int64 # OK - màu xanh
13 # ...
14 # return %3::Int64 # Cụ thể
15
16 # Function type unstable
17 function really_bad(x)
18     if x > 0
19         return x
20     else
21         return "negative"
22     end
23 end
24
25 julia> @code_warntype really_bad(5)
26 # Body::Union{Int64, String} # WARNING - màu đỏ!
```

```

27 # ...
28 # return %4::Union{Int64, String} # Type không xác định

```

#### 4.3.12.2 Đọc output của @code\_warntype

```

1 # Color coding trong @code_warntype:
2 # - Xanh lá (green): Concrete type - TỐT
3 # - Đỏ (red): Union hoặc Any - XẤU
4 # - Vàng (yellow): Small union - CÓ THỂ CHẤP NHẬN
5
6 # Ví dụ với struct
7 struct Container{T}
8     value::T
9 end
10
11 function get_value(c::Container{T}) where T
12     return c.value
13 end
14
15 julia> @code_warntype get_value(Container{42})
16 # Body::Int64 # Concrete - PERFECT!
17
18 # So với abstract field
19 struct BadContainer
20     value::Any
21 end
22
23 julia> @code_warntype (c -> c.value) (BadContainer{42})
24 # Body::Any # Type không biết - SLOW!

```

#### 4.3.12.3 Patterns gây type instability

```

1 # Pattern 1: Abstract return types
2 function unstable1(x::Int)
3     return x > 0 ? x : nothing # Union{Int, Nothing}
4 end
5
6 # Pattern 2: Global variables
7 global_config = Dict{:size} => 100
8
9 function unstable2()
10     return global_config[:size] # Any (không biết type)
11 end
12
13 # Pattern 3: Abstract field access
14 mutable struct Unstable
15     data::Any
16 end
17
18 function unstable3(u::Unstable)
19     return u.data + 1 # Any + Int = Any
20 end
21

```

```
22 # FIX: Sử dụng type assertions hoặc parametric types
23 function stable1(x::Int)
24     result = x > 0 ? x : nothing
25     return something(result, 0)::Int # Always Int
26 end
27
28 function stable2()
29     return global_config[:size]::Int # Type assertion
30 end
31
32 mutable struct Stable{T}
33     data::T
34 end
35
36 function stable3(s::Stable{Int})
37     return s.data + 1 # Int + Int = Int
38 end
```

### 4.3.13 Performance patterns với Parametric types

Parametric types là chìa khóa để đạt performance cao trong Julia [2].

#### 4.3.13.1 Specialization và JIT compilation

```
1 # Julia compiles separate version cho mỗi concrete type
2 function square(x::T) where T<:Number
3     return x * x
4 end
5
6 # Khi gọi:
7 square(5) # Compile version cho Int64
8 square(3.14) # Compile version cho Float64
9 square(2+3im) # Compile version cho Complex{Int64}
10
11 # Mỗi version được tối ưu cho type cụ thể
12 # → Performance như viết code chuyên biệt
13
14 # Kiểm tra số methods
15 julia> methods(square)
16 # 1 method for generic function "square":
17 # [1] square(x::T) where T<:Number at REPL[1]:2
18
19 # Nhưng có nhiều compiled specializations!
```

#### 4.3.13.2 Memory layout optimization

```
1 # Parametric struct với concrete type → inline storage
2 struct Point{T}
3     x::T
4     y::T
5 end
6
7 # Memory layout:
```

```

8 # Point{Float64}: [x::Float64, y::Float64] = 16 bytes
9 # Point{Int32}:    [x::Int32, y::Int32]      = 8 bytes
10
11 # Vector của Point được stored inline
12 julia> sizeof(Point{Float64}(0,0))
13 # Output: 16
14
15 julia> sizeof(Vector{Point{Float64}}(undef, 100))
16 # Output: 1600 # 100 * 16 bytes, contiguous memory
17
18 # So với abstract type
19 struct BadPoint
20     x::Number # Pointer + type tag
21     y::Number
22 end
23
24 julia> sizeof(BadPoint(0, 0))
25 # Output: 16 # But each field is a pointer!
26 # Actual memory usage: 16 + 2*(16) = 48 bytes
27 # Plus: cache misses, GC pressure

```

#### 4.3.13.3 Function barriers

```

1 # Problem: Container với unknown element type
2 function process(container)
3     total = 0
4     for item in container
5         total += item # Type của item không biết!
6     end
7     return total
8 end
9
10 # Solution: Function barrier
11 function process_barrier(container)
12     T = eltype(container)
13     return _process_inner(container, T)
14 end
15
16 function _process_inner(container, ::Type{T}) where T
17     total = zero(T) # Typed zero
18     for item in container
19         total += item::T # Type assertion
20     end
21     return total
22 end
23
24 # Performance: Inner function được specialized cho T
25 # Outer function chỉ dispatch 1 lần

```

### 4.3.14 Common pitfalls và anti-patterns

#### 4.3.14.1 Pitfall 1: Type parameter confusion

```

1  # ❌ Sai: T không được sử dụng đúng cách
2  function wrong(x::T, y::T) where T
3      return x + y
4  end
5
6  julia> wrong(1, 2.0)
7  # ERROR: Int64 ≠ Float64, T không khớp
8
9  # ✅ Đúng: Cho phép khác type
10 function correct(x::T, y::S) where {T, S}
11     return x + y
12 end
13
14 julia> correct(1, 2.0)
15 # Output: 3.0

```

#### 4.3.14.2 Pitfall 2: Invariance surprise

```

1  # ❌ Sai: Không hiểu invariance
2  function sum_numbers(v::Vector{Number})
3      return sum(v)
4  end
5
6  julia> sum_numbers([1, 2, 3])
7  # ERROR: Vector{Int64} không phải Vector{Number}
8
9  # ✅ Đúng: Sử dụng where hoặc <:
10 function sum_numbers(v::Vector{<:Number})
11     return sum(v)
12 end
13
14 # hoặc
15 function sum_numbers(v::Vector{T}) where T<:Number
16     return sum(v)
17 end

```

#### 4.3.14.3 Pitfall 3: Overconstraining

```

1  # ❌ Quá restrictive
2  function overconstrained(x::Float64)
3      return x * 2
4  end
5  # Không hoạt động với Int, Float32, etc.
6
7  # ✅ Đúng mức constraint
8  function balanced(x::T) where T<:Real
9      return x * 2
10 end
11 # Hoạt động với Int, Float64, Rational, etc.
12

```



```

13 # □ Không constraint khi cần
14 function underconstrained(x)
15     return x + 1 # x có thể là String!
16 end
17
18 # □ Constraint khi cần arithmetic
19 function properly_constrained(x::Number)
20     return x + 1
21 end

```

#### 4.3.14.4 Pitfall 4: Abstract field types

```

1 # □ Performance killer
2 struct Slow
3     values::Vector{Any}
4     config::Dict{String, Any}
5 end
6
7 # □ Parametric alternative
8 struct Fast{T, V}
9     values::Vector{T}
10    config::Dict{String, V}
11 end
12
13 # □ Hoặc với constraints
14 struct Medium{T<:Number}
15     values::Vector{T}
16     config::Dict{String, Int}
17 end
18
19 # Benchmark difference: 10-100x faster với parametric types!

```

#### 4.3.15 Tóm tắt

- **Abstract types:** Tổ chức type hierarchy, không có instances
- **Parametric types:** Generic types với type parameters (T, S)
- **Type constraints:** T<:Real giới hạn T trong hierarchy
- **Subtyping:** Operator <: định nghĩa quan hệ parent-child
- **Performance:** Parametric types được specialized tại compile-time
- **Best practice:** Prefer parametric types over abstract fields

## 4.4 Bài tập

### 4.4.1 Bài tập cơ bản (\*)

#### Bài tập: Tạo và thao tác mảng

Viết các đoạn code thực hiện:

1. Tạo vector 10 số nguyên từ 1 đến 10
2. Tạo ma trận 3x3 với các phần tử là bình phương của chỉ số
3. Lấy phần tử hàng 2, cột 3 của ma trận
4. Slice lấy hàng đầu tiên và cột cuối cùng

# ỢGi ý

```
v = collect(1:10)
```

```
m = [i^2 + j for i in 1:3, j in 1:3]
```

#### Bài tập: Comprehension và broadcasting

Sử dụng comprehension và broadcasting để:

1. Tạo vector các số chẵn từ 2 đến 20
2. Tính bình phương của tất cả phần tử trong vector
3. Tạo ma trận 5x5 với  $m[i, j] = i * j$
4. So sánh hiệu năng giữa loop và broadcasting

#### Bài tập: Dictionary cơ bản

Tạo dictionary lưu thông tin sinh viên:

1. Keys: tên sinh viên, values: điểm trung bình
2. Thêm 5 sinh viên vào dictionary
3. Tìm sinh viên có điểm cao nhất
4. Xóa sinh viên có điểm thấp nhất
5. Liệt kê tất cả sinh viên có điểm  $\geq 8.0$

#### Bài tập: Set operations

Cho hai tập hợp môn học:

```
mon_hoc_a = Set(["Toán", "Lý", "Hóa", "Sinh"])
```

```
mon_hoc_b = Set(["Toán", "ãVn", "ửS", "Đị"a"])
```

Tính: (a) Hợp, (b) Giao, (c) Hiệu A - B, (d) Hiệu đối xứng

**Bài tập: Tuple và NamedTuple**

Viết function trả về thông kê của một vector:

```
function thong_ke(v::Vector)
    # Trả về NamedTuple ở vi:
    # min, max, mean, std, count
end
# Ví dụ: thong_ke([1,2,3,4,5])
# => (min=1, max=5, mean=3.0, std=1.58, count=5)
```

**4.4.2 Bài tập trung bình (\*\*)****Bài tập: Struct cơ bản - SinhVien**

Định nghĩa struct SinhVien với:

- Fields: mssv (String), ten (String), diem (Vector{Float64})
- Method: diem\_tb(sv) tính điểm trung bình
- Method: xep\_loai(sv) trả về "Giỏi/Khá/TB/Yếu"
- Tạo 5 sinh viên và sắp xếp theo điểm TB giảm dần

**Bài tập: Mutable struct - TaiKhoanNganHang**

Tạo mutable struct TaiKhoanNganHang:

```
mutable struct TaiKhoanNganHang
    so_tai_khoan::String
    chu_tai_khoan::String
    so_du::Float64
end
# Implement: nap_tien!, rut_tien!, chuyen_tien!
# Kiểm tra số dư trước khi rút/ủy chuyển
```

**Bài tập: Sparse matrix**

Tạo sparse matrix 1000x1000 với:

1. Chỉ 100 phần tử khác 0 (ngẫu nhiên)
2. So sánh memory usage với dense matrix
3. Thực hiện nhân ma trận sparse với vector
4. Đo thời gian so với dense matrix

```
using SparseArrays
# ỢGi ý: sparse(), nnz(), @allocated
```

## 4.4.3 Bài tập nâng cao (\*\*\*)

## Bài tập: Parametric struct - Container

Thiết kế generic container:

```
struct Stack{T}
    items::Vector{T}
end
# Implement: push!, pop!, peek, isempty, size
# Test ới Stack{Int}, Stack{String}, Stack{Any}
```

Mở rộng: Tạo Queue{T} với enqueue!/dequeue!

## Bài tập: Type hierarchy - Hình học

Thiết kế hệ thống type cho hình học 2D:

```
abstract type HìnhHoc end
abstract type HìnhPhang <: HìnhHoc end

struct HìnhTron <: HìnhPhang
    ban_kinh::Float64
end

struct HìnhChuNhat <: HìnhPhang
    chieu_dai::Float64
    chieu_rong::Float64
end

# Implement: dien_tich(), chu_vi() cho ỗi type
# ửS ụng multiple dispatch

Thêm: HìnhVuong, HìnhTamGiac, HìnhThoi
```

## Đọc thêm

- Julia Documentation: Arrays - <https://docs.julialang.org/en/v1/manual/arrays/>
- Julia Documentation: Types - <https://docs.julialang.org/en/v1/manual/types/>
- Bezanson et al. (2017): Julia's type system design

## Chương 5

# Lập trình hàm và đa phương thức

### Mục tiêu học tập

- Nắm vững multiple dispatch và method specialization
- Hiểu overloading và generic programming
- Áp dụng lambda functions và functional programming
- Sử dụng higher-order functions và function composition

### 5.1 Multiple dispatch

**Multiple dispatch** (đa phái cử) là một trong những tính năng cốt lõi và đặc trưng nhất của Julia, cho phép một hàm có nhiều phương thức (methods) khác nhau, và phương thức được chọn dựa trên *kiểu dữ liệu của tất cả các đối số* truyền vào hàm. Theo Bezanson và cộng sự [2], "nếu có một chủ đề xuyên suốt trong thiết kế của ngôn ngữ Julia, đó chính là multiple dispatch".

#### 5.1.1 Định nghĩa và so sánh

**Multiple dispatch** là quá trình chọn phương thức (method) để thực thi tại runtime dựa trên kiểu (type) của tất cả các tham số của hàm. Điều này khác biệt rõ rệt với các mô hình lập trình truyền thống:

- **Single dispatch (OOP):** Phương thức được chọn chỉ dựa trên kiểu của đối tượng gọi phương thức (receiver), thường là đối tượng trước dấu chấm trong cú pháp `object.method()`.
- **Function overloading (C++, Java):** Quyết định phương thức được thực hiện tại compile-time dựa trên kiểu khai báo, không phải kiểu thực tế tại runtime.
- **Multiple dispatch (Julia):** Phương thức được chọn tại runtime dựa trên kiểu thực tế của *tất cả* các đối số, không chỉ một đối số.

Sự khác biệt quan trọng là: dispatch là một quá trình runtime, trong khi method overloading là khái niệm compile-time.

#### 5.1.2 Ví dụ cơ bản

##### Ví dụ 1: Single argument dispatch

```
1 # Định nghĩa hàm với nhiều phương thức
2 describe(x::Int) = "Đây là số nguyên: $x"
3 describe(x::Float64) = "Đây là số thực: $x"
4 describe(x::String) = "Đây là chuỗi: $x"
5
6 # Julia tự động chọn phương thức phù hợp
7 println(describe(42))           # Đây là số nguyên: 42
8 println(describe(3.14))        # Đây là số thực: 3.14
9 println(describe("Julia"))     # Đây là chuỗi: Julia
```

## Ví dụ 2: Multiple arguments dispatch

```
1 # Hàm tính diện tích với nhiều phương thức
2 area(r::Real) =  $\pi$  * r^2           # Diện tích hình tròn
3 area(w::Real, h::Real) = w * h     # Diện tích hình chữ nhật
4
5 # Julia chọn phương thức dựa trên số lượng và kiểu đối số
6 println(area(5.0))              # 78.53981633974483 (hình tròn)
7 println(area(4.0, 6.0))        # 24.0 (hình chữ nhật)
```

### 5.1.3 Ví dụ thực tế với hình học

```
1 # Định nghĩa kiểu trừu tượng
2 abstract type Shape end
3
4 # Các kiểu cụ thể
5 struct Circle <: Shape
6     radius::Float64
7 end
8
9 struct Rectangle <: Shape
10     width::Float64
11     height::Float64
12 end
13
14 # Multiple dispatch với abstract type
15 area(c::Circle) =  $\pi$  * c.radius^2
16 area(r::Rectangle) = r.width * r.height
17
18 # Hàm kiểm tra giao nhau - multiple dispatch với 2 arguments
19 intersect(c1::Circle, c2::Circle) =
20     "Kiểm tra giao nhau giữa 2 hình tròn"
21
22 intersect(r1::Rectangle, r2::Rectangle) =
23     "Kiểm tra giao nhau giữa 2 hình chữ nhật"
24
25 intersect(c::Circle, r::Rectangle) =
26     "Kiểm tra giao nhau giữa hình tròn và hình chữ nhật"
27
28 # Tận dụng symmetry
29 intersect(r::Rectangle, c::Circle) = intersect(c, r)
30
31 # Sử dụng
```

```

32 c1 = Circle(2.0)
33 r1 = Rectangle(3.0, 4.0)
34
35 println(area(c1))           # 12.566370614359172
36 println(area(r1))           # 12.0
37 println(intersect(c1, r1))  # Kiểm tra giao nhau...

```

#### 5.1.4 Cách Julia chọn phương thức

Julia áp dụng nguyên tắc **narrowest type matching** (khớp kiểu hẹp nhất): khi có nhiều phương thức phù hợp với các đối số truyền vào, Julia sẽ chọn phương thức có kiểu đối số *cụ thể nhất*.

```

1 # Ví dụ về narrowest type matching
2 process(x::Number) = "Number: $x"
3 process(x::Real) = "Real: $x"
4 process(x::Int) = "Int: $x"
5
6 # Julia chọn phương thức cụ thể nhất
7 println(process(42))      # "Int: 42" (cụ thể nhất)
8 println(process(3.14))    # "Real: 3.14" (Float64 <: Real)
9 println(process(2+3im))   # "Number: 2+3im" (Complex <: Number)

```

Hệ thống phân cấp kiểu: `Int <: Real <: Number`. Julia chọn kiểu cụ thể nhất có sẵn.

#### 5.1.5 Tại sao multiple dispatch hữu ích?

Multiple dispatch đặc biệt hữu ích trong lập trình khoa học và toán học vì:

- **Không có đối số "đặc biệt"**: Trong phép toán toán học như  $+(a, b)$ , không có đối số nào quan trọng hơn đối số kia. Multiple dispatch xem xét tất cả các đối số một cách công bằng.
- **Code linh hoạt và tái sử dụng**: Kết hợp với hệ thống kiểu linh hoạt, multiple dispatch cho phép viết code tổng quát mà vẫn đảm bảo type safety.
- **Mở rộng dễ dàng**: Có thể thêm phương thức mới cho hàm có sẵn mà không cần sửa code gốc.
- **Hiệu năng cao**: Julia compiler tối ưu hóa từng phương thức cho từng kiểu cụ thể.

#### 5.1.6 So sánh Multiple Dispatch vs OOP

| Đặc điểm     | Multiple Dispatch (Julia)     | Single Dispatch (OOP)          |
|--------------|-------------------------------|--------------------------------|
| Cơ chế chọn  | Dựa trên <i>tất cả</i> đối số | Dựa trên receiver object       |
| Tổ chức code | Methods thuộc functions       | Methods thuộc classes          |
| Cú pháp      | <code>func(a, b)</code>       | <code>a.func(b)</code>         |
| Mở rộng      | Thêm method mới dễ dàng       | Phải modify class hoặc inherit |
| Use case     | Toán học, khoa học            | Business logic, OOP            |

Bảng 5.1: So sánh Multiple Dispatch và Single Dispatch

### 5.1.7 Best practices và pitfalls

#### Best practices:

- **Đặt tên hàm chung, tách behavior theo kiểu:** Dùng một tên hàm như `area`, `process`, với nhiều methods cho các kiểu khác nhau.
- **Sử dụng abstract types để nhóm kiểu liên quan:** Giúp viết methods tổng quát cho cả nhóm kiểu.
- **Viết methods cho kiểu cụ thể trước:** Julia ưu tiên narrow types, nên định nghĩa methods cụ thể trước khi định nghĩa methods tổng quát.
- **Tài liệu hóa rõ ràng:** Mỗi method nên có docstring giải thích behavior cho kiểu cụ thể đó.

#### Common pitfalls (lỗi thường gặp):

- **Thiếu methods cho trường hợp cần thiết:** Gây lỗi `MethodError` khi gọi hàm với kiểu chưa được hỗ trợ.
- **Ambiguous methods:** Định nghĩa hai methods mà Julia không thể quyết định method nào cụ thể hơn.
- **Nhầm dispatch với value-based dispatch:** Julia dispatch dựa trên *type*, không phải *value*. Không thể dispatch dựa trên giá trị cụ thể của đối số.
- **Không tận dụng type hierarchy:** Bỏ lỡ cơ hội viết code tổng quát bằng cách dùng abstract types.

### 5.1.8 Method tables và cơ chế nội bộ

Để hiểu sâu về multiple dispatch, cần nắm cách Julia lưu trữ và tìm kiếm methods tại runtime. Julia sử dụng cấu trúc gọi là **method table** (bảng phương thức) cho mỗi generic function[2].

#### 5.1.8.1 Cấu trúc của method table

Mỗi hàm trong Julia là một **generic function** chứa một method table với các methods được đăng ký. Method table lưu trữ:

- **Method signatures:** Tuple các kiểu tham số mà method chấp nhận
- **Method definitions:** Code thực thi cho mỗi signature
- **Specializations cache:** Cache các compiled versions cho concrete type combinations

#### Xem method table với hàm `methods()`:

```
1 # Định nghĩa hàm với nhiều methods
2 greet(x::Int) = "Hello number $x"
3 greet(x::String) = "Hello $x"
4 greet(x::Float64) = "Hello float $x"
5
6 # Xem tất cả methods của hàm greet
7 methods(greet)
8 # 3 methods for generic function "greet":
```



```

9  # greet(x::Float64) in Main at REPL[3]:1
10 # greet(x::Int64) in Main at REPL[1]:1
11 # greet(x::String) in Main at REPL[2]:1
12
13 # Số lượng methods
14 length(methods(greet)) # 3

```

### 5.1.8.2 Quá trình dispatch tại runtime

Khi gọi một hàm, Julia thực hiện **method lookup** theo các bước:

1. **Xác định kiểu của TẤT CẢ arguments:** Julia xem xét kiểu thực tế (concrete type) của mọi đối số, không chỉ kiểu khai báo.
2. **Tìm methods phù hợp:** Duyệt qua method table để tìm các methods có signature khớp với tuple kiểu của arguments.
3. **Chọn method cụ thể nhất:** Nếu có nhiều methods phù hợp, Julia áp dụng nguyên tắc **most specific matching** — chọn method có kiểu tham số cụ thể nhất (hẹp nhất trong type hierarchy).
4. **Cache và thực thi:** Method được biên dịch (nếu chưa có trong cache) và thực thi.

Ví dụ quá trình dispatch:

```

1  # Type hierarchy: Int64 <: Integer <: Real <: Number <: Any
2  process(x::Number) = "Processing Number"
3  process(x::Real) = "Processing Real"
4  process(x::Integer) = "Processing Integer"
5
6  # Gọi với Int64
7  process(42) # "Processing Integer"
8
9  # Tại sao? Julia tìm thấy 3 methods phù hợp:
10 # - process(::Number) khớp vì Int64 <: Number
11 # - process(::Real) khớp vì Int64 <: Real
12 # - process(::Integer) khớp vì Int64 <: Integer
13 # Julia chọn Integer vì cụ thể nhất

```

### 5.1.8.3 Ambiguity và MethodError

Khi Julia không thể xác định method nào cụ thể nhất, **MethodError** với thông báo "ambiguous" sẽ được ném:

```

1  # Định nghĩa tạo ambiguity
2  combine(x::Int, y::Number) = x + y
3  combine(x::Number, y::Int) = x * y
4
5  # Gọi với (Int, Int) - cả 2 methods đều khớp!
6  combine(2, 3) # ERROR: MethodError: combine is ambiguous
7
8  # Giải pháp: thêm method cụ thể cho trường hợp (Int, Int)
9  combine(x::Int, y::Int) = x - y # Giải quyết ambiguity
10 combine(2, 3) # -1 (dùng method mới)

```

### Nguyên tắc tránh ambiguity:

- Định nghĩa method cho **mọi intersection** của các type combinations
- Sử dụng `@which` để kiểm tra method nào được gọi
- Dùng `Base.method_ambiguities` để phát hiện ambiguities trong module

## 5.1.9 Specialization và JIT compilation

Một trong những lý do Julia đạt hiệu năng cao là nhờ cơ chế **specialization** — Julia biên dịch version riêng của mỗi method cho từng combination của concrete types.

### 5.1.9.1 Cách Julia compile methods

Julia sử dụng **Just-In-Time (JIT) compilation** với LLVM backend. Khi một method được gọi lần đầu với concrete types cụ thể:

1. **Type inference:** Julia suy luận kiểu của tất cả variables trong method body
2. **Code generation:** Tạo mã LLVM IR chuyên biệt cho concrete types đó
3. **Compilation:** LLVM biên dịch thành native machine code
4. **Caching:** Kết quả được lưu trong specializations cache

Xem quá trình với các introspection macros:

```

1 function add_numbers(x, y)
2     return x + y
3 end
4
5 # Xem lowered code (sau khi parse, trước optimize)
6 @code_lowered add_numbers(1, 2)
7
8 # Xem typed code (sau type inference)
9 @code_typed add_numbers(1, 2) # Specialized cho (Int64, Int64)
10 @code_typed add_numbers(1.0, 2.0) # Specialized cho (Float64, Float64)
11
12 # Xem LLVM IR
13 @code_llvm add_numbers(1, 2)
14
15 # Xem native assembly
16 @code_native add_numbers(1, 2)
```

### 5.1.9.2 Lợi ích của specialization

#### 1. Type-specific optimization:

```

1 # Julia tạo 2 compiled versions khác nhau:
2 sum_values(x::Vector{Int64}) = sum(x) # Optimized cho Int64
3 sum_values(x::Vector{Float64}) = sum(x) # Optimized cho Float64
4
5 # Với Int64: loop với integer arithmetic
6 # Với Float64: có thể dùng SIMD instructions
```

## 2. Inlining và constant propagation:

```

1 @inline square(x) = x * x
2
3 function compute(n)
4     total = 0
5     for i in 1:n
6         total += square(i) # square() được inline
7     end
8     return total
9 end
10
11 # @code_native compute(100) sẽ không có function call cho square

```

## 3. Devirtualization:

Trong OOP truyền thống, method calls thường là **virtual calls** (gián tiếp qua vtable). Julia's specialization cho phép **devirtualization** — chuyển virtual calls thành direct calls:

```

1 abstract type Animal end
2 struct Dog <: Animal end
3 struct Cat <: Animal end
4
5 speak(::Dog) = "Woof!"
6 speak(::Cat) = "Meow!"
7
8 # Khi gọi với concrete type đã biết
9 speak(Dog()) # Direct call, không qua dispatch
10
11 # Khi kiểu chưa biết tại compile time
12 function greet_animal(a::Animal)
13     println(speak(a)) # Dispatch tại runtime
14 end

```

### 5.1.9.3 Chi phí của specialization

Specialization có trade-off:

- **Memory:** Mỗi specialization chiếm bộ nhớ riêng
- **Compile time:** Lần gọi đầu tiên với type mới sẽ chậm do compilation
- **Code bloat:** Quá nhiều specializations có thể làm tăng kích thước binary

**Giải pháp: @nospecialize**

```

1 # Ngăn specialization cho parameter cụ thể
2 function process(@nospecialize(config), data::Vector{Float64})
3     # config không được specialized
4     # data vẫn được specialized
5     return sum(data)
6 end
7
8 # Chỉ có 1 version cho mọi kiểu của config

```

### 5.1.10 Type inference trong dispatch

**Type inference** là quá trình Julia tự động suy luận kiểu của expressions và variables. Đây là nền tảng cho cả multiple dispatch và performance optimization.

#### 5.1.10.1 Sử dụng @code\_warntype

Macro @code\_warntype hiển thị kết quả type inference với color coding:

```
1 function stable_function(x::Int)
2     y = x + 1          # y::Int (stable)
3     z = y * 2.0        # z::Float64 (stable)
4     return z
5 end
6
7 @code_warntype stable_function(10)
8 # Tất cả kiểu đều được suy luận chính xác (màu xanh/đen)
9
10 function unstable_function(x)
11     if x > 0
12         return x      # Trả về kiểu của x
13     else
14         return "negative" # Trả về String
15     end
16 end
17
18 @code_warntype unstable_function(10)
19 # Return type: Union{Int64, String} (màu ĐỎ - type unstable!)
```

#### 5.1.10.2 Type stability patterns

**Type-stable code** là code mà Julia có thể suy luận kiểu chính xác cho mọi biến. Đây là yếu tố quan trọng nhất cho performance.

##### Pattern 1: Consistent return types

```
1 # BAD: Trả về nhiều kiểu khác nhau
2 function bad_abs(x)
3     if x >= 0
4         return x
5     else
6         return -x
7     end
8 end
9
10 # GOOD: Kiểu trả về nhất quán
11 function good_abs(x::T) where T <: Number
12     return x >= 0 ? x : -x # Luôn trả về T
13 end
```

##### Pattern 2: Tránh abstract container types

```
1 # BAD: Vector với element type Any
2 data = [] # Vector{Any}
3 push!(data, 1)
```

```

4 push!(data, 2.0)
5
6 # GOOD: Khai báo element type cụ thể
7 data = Int[] # Vector{Int64}
8 push!(data, 1)
9 push!(data, 2)

```

### Pattern 3: Function barriers cho heterogeneous data

```

1 # Outer function xử lý dispatch, inner function type-stable
2 function process_any(data::Vector{Any})
3     result = 0.0
4     for item in data
5         result += process_item(item) # Dispatch happens here
6     end
7     return result
8 end
9
10 # Inner functions type-stable cho từng kiểu
11 process_item(x::Int) = Float64(x)
12 process_item(x::Float64) = x
13 process_item(x::String) = parse(Float64, x)

```

#### 5.1.11 Design patterns với multiple dispatch

Multiple dispatch cho phép giải quyết các vấn đề thiết kế phần mềm một cách thanh lịch hơn so với OOP truyền thống.

##### 5.1.11.1 The Expression Problem

**Expression Problem** (Wadler, 1998) là thách thức mở rộng hệ thống theo hai chiều:

- Thêm kiểu dữ liệu mới (new data types)
- Thêm operations mới trên các kiểu hiện có

**Hạn chế của OOP (single dispatch):**

- Thêm kiểu mới: dễ (tạo class mới, implement methods)
- Thêm operation mới: khó (phải modify tất cả classes hoặc dùng Visitor pattern phức tạp)

**Julia giải quyết cả hai chiều:**

```

1 # Các kiểu hiện có
2 abstract type Expr end
3 struct Literal <: Expr
4     value::Number
5 end
6 struct Add <: Expr
7     left::Expr
8     right::Expr
9 end
10
11 # Operation 1: evaluate

```

```

12 evaluate(e::Literal) = e.value
13 evaluate(e::Add) = evaluate(e.left) + evaluate(e.right)
14
15 # Thêm kiểu mới - KHÔNG cần modify code cũ
16 struct Multiply <: Expr
17     left::Expr
18     right::Expr
19 end
20 evaluate(e::Multiply) = evaluate(e.left) * evaluate(e.right)
21
22 # Thêm operation mới - KHÔNG cần modify code cũ
23 pretty_print(e::Literal) = string(e.value)
24 pretty_print(e::Add) = "($(pretty_print(e.left)) + $(pretty_print(e.right)))"
25 pretty_print(e::Multiply) = "($(pretty_print(e.left)) *
    ↪ $(pretty_print(e.right)))"
26
27 # Sử dụng
28 expr = Add(Literal(2), Multiply(Literal(3), Literal(4)))
29 evaluate(expr)      # 2 + (3 * 4) = 14
30 pretty_print(expr)  # "(2 + (3 * 4))"

```

#### 5.1.11.2 Holy Trait Pattern

**Holy Trait Pattern** (đặt tên theo Tim Holy, Julia core developer) sử dụng multiple dispatch để implement traits — nhóm các kiểu theo behavior thay vì inheritance.

```

1 # Định nghĩa trait types
2 abstract type IteratorSize end
3 struct HasLength <: IteratorSize end
4 struct IsInfinite <: IteratorSize end
5 struct SizeUnknown <: IteratorSize end
6
7 # Trait function - trả về trait type dựa trên input type
8 IteratorSize{::Type{<:AbstractArray}} = HasLength()
9 IteratorSize{::Type{<:Base.Generator}} = SizeUnknown()
10 IteratorSize{::Type{<:Iterators.Cycle}} = IsInfinite()
11
12 # Behavior dispatch dựa trên trait
13 function collect_safe(iter)
14     return collect_safe(IteratorSize(typeof(iter)), iter)
15 end
16
17 # Methods cho từng trait
18 collect_safe{::HasLength, iter} = collect(iter)
19 collect_safe{::SizeUnknown, iter} = collect(Iterators.take(iter, 1000))
20 collect_safe{::IsInfinite, iter} = error("Cannot collect infinite iterator!")
21
22 # Sử dụng
23 collect_safe([1, 2, 3])           # OK - HasLength
24 collect_safe(x for x in 1:10)     # OK - SizeUnknown, lấy 1000 phần tử đầu
25 collect_safe(Iterators.cycle([1,2])) # ERROR - IsInfinite

```

**Lợi ích của Holy Trait Pattern:**

1. **Extensible:** Có thể thêm trait cho kiểu mới mà không modify code gốc
2. **Zero-cost abstraction:** Trait dispatch được compile away tại compile-time
3. **Orthogonal to inheritance:** Kiểu có thể có nhiều traits không liên quan đến type hierarchy

### 5.1.11.3 Strategy Pattern với Multiple Dispatch

```

1  # Strategy types
2  abstract type SortStrategy end
3  struct QuickSort <: SortStrategy end
4  struct MergeSort <: SortStrategy end
5  struct InsertionSort <: SortStrategy end
6
7  # Generic sorting function
8  function sort_with(strategy::SortStrategy, data::Vector)
9      return sort_impl(strategy, data)
10 end
11
12 # Strategy implementations
13 sort_impl(::QuickSort, data) = sort(data, alg=QuickSort)
14 sort_impl(::MergeSort, data) = sort(data, alg=MergeSort)
15 sort_impl(::InsertionSort, data) = begin
16     # Manual insertion sort for small arrays
17     for i in 2:length(data)
18         key = data[i]
19         j = i - 1
20         while j > 0 && data[j] > key
21             data[j+1] = data[j]
22             j -= 1
23         end
24         data[j+1] = key
25     end
26     return data
27 end
28
29 # Adaptive strategy selection
30 function choose_strategy(data::Vector)
31     n = length(data)
32     if n < 50
33         return InsertionSort()
34     elseif n < 10000
35         return QuickSort()
36     else
37         return MergeSort()
38     end
39 end
40
41 # Usage
42 data = rand(100)
43 strategy = choose_strategy(data)
44 sorted = sort_with(strategy, data)

```

### 5.1.12 So sánh với Visitor Pattern (OOP)

**Visitor Pattern** trong OOP là một cách để thêm operations mới mà không modify classes hiện có. Tuy nhiên, nó có nhiều hạn chế so với multiple dispatch.

#### 5.1.12.1 Visitor Pattern trong OOP

```

1  # Pseudo-code minh họa Visitor Pattern trong OOP style
2  # (Julia không có OOP classes, đây chỉ là minh họa concept)
3
4  # Interface cho Visitor
5  # interface ShapeVisitor {
6  #     visitCircle(Circle c)
7  #     visitRectangle(Rectangle r)
8  # }
9
10 # Mỗi Shape phải có method accept()
11 # class Circle {
12 #     accept(visitor) { visitor.visitCircle(this) }
13 # }
14 # class Rectangle {
15 #     accept(visitor) { visitor.visitRectangle(this) }
16 # }
17
18 # Concrete visitor cho mỗi operation
19 # class AreaVisitor implements ShapeVisitor {
20 #     visitCircle(c) { return PI * c.radius^2 }
21 #     visitRectangle(r) { return r.width * r.height }
22 # }

```

#### 5.1.12.2 Hạn chế của Visitor Pattern

| Vấn đề             | Visitor (OOP)              | Multiple Dispatch (Julia) |
|--------------------|----------------------------|---------------------------|
| Thêm operation mới | Tạo visitor class mới      | Thêm method mới           |
| Thêm data type mới | Modify TẤT CẢ visitors     | Thêm methods cho type mới |
| Double dispatch    | Manual (accept/visit)      | Tự động                   |
| Boilerplate code   | Nhiều (interfaces, accept) | Minimal                   |
| Coupling           | Visitor biết mọi types     | Methods độc lập           |

Bảng 5.2: So sánh Visitor Pattern với Multiple Dispatch

#### 5.1.12.3 Ví dụ equivalent trong Julia

```

1  # Julia: Multiple dispatch thay thế Visitor Pattern
2
3  # Data types
4  abstract type Shape end
5  struct Circle <: Shape
6      radius::Float64
7  end
8  struct Rectangle <: Shape
9      width::Float64

```



```

10     height::Float64
11 end
12 struct Triangle <: Shape
13     base::Float64
14     height::Float64
15 end
16
17 # Operation 1: Area (như AreaVisitor)
18 area(c::Circle) =  $\pi$  * c.radius^2
19 area(r::Rectangle) = r.width * r.height
20 area(t::Triangle) = 0.5 * t.base * t.height
21
22 # Operation 2: Perimeter (như PerimeterVisitor)
23 perimeter(c::Circle) = 2 $\pi$  * c.radius
24 perimeter(r::Rectangle) = 2 * (r.width + r.height)
25 perimeter(t::Triangle) = t.base + 2 * sqrt((t.base/2)^2 + t.height^2)
26
27 # Operation 3: Draw (như DrawVisitor) - thêm mới DỄ DÀNG
28 draw(c::Circle) = println("Drawing circle with radius $(c.radius)")
29 draw(r::Rectangle) = println("Drawing rectangle $(r.width)x$(r.height)")
30 draw(t::Triangle) = println("Drawing triangle base=$(t.base), h=$(t.height)")
31
32 # Thêm shape mới - CHỈ cần thêm methods, KHÔNG modify code cũ
33 struct Polygon <: Shape
34     vertices::Vector{Tuple{Float64, Float64}}
35 end
36
37 area(p::Polygon) = "Polygon area calculation..."
38 perimeter(p::Polygon) = "Polygon perimeter calculation..."
39 draw(p::Polygon) = println("Drawing polygon with $(length(p.vertices))
    ↪ vertices")
40
41 # Sử dụng - KHÔNG cần visitor objects
42 shapes = [Circle(5.0), Rectangle(3.0, 4.0), Triangle(6.0, 4.0)]
43 for s in shapes
44     println("Area: $(area(s)), Perimeter: $(perimeter(s))")
45     draw(s)
46 end

```

#### 5.1.12.4 Khi nào dùng pattern nào?

- **Multiple dispatch (Julia):** Khi cần flexibility trong cả hai chiều — thêm types VÀ operations dễ dàng. Phù hợp với scientific computing, math libraries.
- **Visitor pattern (OOP):** Khi làm việc với ngôn ngữ single-dispatch và data types ổn định (ít thay đổi), nhưng operations thường xuyên được thêm mới.
- **Lưu ý:** Trong Julia, hầu như KHÔNG BAO GIỜ cần Visitor pattern vì multiple dispatch giải quyết vấn đề một cách tự nhiên hơn.

#### 5.1.13 Method introspection

Julia cung cấp nhiều công cụ để kiểm tra và debug dispatch behavior.

### 5.1.13.1 Các hàm introspection quan trọng

```
1 # @which: Xem method nào được gọi
2 @which sqrt(4)           # sqrt(x::Real) in Base at math.jl:...
3 @which 1 + 2.0           # +(x::Integer, y::AbstractFloat) in Base
4
5 # methods(): Liệt kê tất cả methods của một hàm
6 methods(+)               # 190+ methods cho toán tử +!
7
8 # methodswith(): Tìm methods có tham số kiểu cụ thể
9 methodswith(String)      # Tất cả methods nhận String argument
10
11 # applicable(): Kiểm tra method có tồn tại không
12 applicable(sqrt, 4)      # true
13 applicable(sqrt, "a")    # false
14
15 # invoke(): Gọi method cụ thể, bypass dispatch
16 invoke(sqrt, Tuple{Real}, 4) # Gọi sqrt(::Real)
```

### 5.1.13.2 Debug dispatch với @which

```
1 # Ví dụ debug khi không hiểu tại sao method được chọn
2 abstract type Animal end
3 struct Dog <: Animal end
4 struct Cat <: Animal end
5
6 speak(::Animal) = "Some sound"
7 speak(::Dog) = "Woof!"
8
9 # Kiểm tra dispatch
10 @which speak(Dog())      # speak(::Dog) - như mong đợi
11 @which speak(Cat())      # speak(::Animal) - fallback
12
13 # Thêm method cho Cat
14 speak(::Cat) = "Meow!"
15 @which speak(Cat())      # speak(::Cat) - bây giờ dùng method cụ thể
```

## 5.2 Overloading, generic programming

### 5.2.1 Function overloading vs Multiple dispatch

Trong các ngôn ngữ lập trình truyền thống như C++ hay Java, **function overloading** ( nạp chồng hàm) cho phép định nghĩa nhiều hàm cùng tên nhưng khác nhau về số lượng hay kiểu tham số. Quyết định hàm nào được gọi thường xảy ra tại **thời điểm biên dịch** (compile-time). Ngược lại, Julia sử dụng **multiple dispatch** — một cơ chế mạnh mẽ hơn, nơi việc chọn method được thực hiện tại **runtime** dựa trên kiểu của **tất cả** các tham số.

**Sự khác biệt chính:**

- **Function overloading (C++/Java):** Quyết định tại compile-time, chỉ xét chữ ký hàm (signature) trong code nguồn.
- **Multiple dispatch (Julia):** Quyết định tại runtime, xét kiểu thực tế của tất cả arguments khi gọi hàm.

Trong Julia, thay vì gọi là "overloading", chúng ta nói rằng một hàm có nhiều **methods** (phương thức) khác nhau, mỗi method chuyên biệt hóa cho các kiểu tham số cụ thể.

Ví dụ:

```

1  # Định nghĩa nhiều methods cho hàm process
2  function process(x::Int)
3      println("Processing integer: $x")
4      return x * 2
5  end
6
7  function process(x::Float64)
8      println("Processing float: $x")
9      return x / 2
10 end
11
12 function process(x::String)
13     println("Processing string: $x")
14     return uppercase(x)
15 end
16
17 # Julia tự động chọn method phù hợp
18 process(10)           # Output: Processing integer: 10 → 20
19 process(3.14)         # Output: Processing float: 3.14 → 1.57
20 process("hello")      # Output: Processing string: hello → HELLO

```

### 5.2.2 Generic programming với Abstract types

**Generic programming** (lập trình tổng quát) là kỹ thuật viết code có thể hoạt động với nhiều kiểu dữ liệu khác nhau mà không cần viết lại code cho từng kiểu cụ thể. Julia hỗ trợ generic programming thông qua **abstract types** và **type hierarchy**.

**Abstract types** định nghĩa một nhóm các kiểu liên quan mà không chỉ định cấu trúc cụ thể. Các concrete types (kiểu cụ thể) có thể kế thừa từ abstract types bằng toán tử `<:` (subtype).

Ví dụ với abstract types:

```

1  # Định nghĩa abstract type
2  abstract type Vehicle end
3
4  # Các concrete types kế thừa từ Vehicle
5  struct Car <: Vehicle
6      brand::String
7      speed::Float64
8  end
9
10 struct Bike <: Vehicle
11     brand::String
12     speed::Float64
13 end
14
15 # Hàm generic hoạt động với mọi Vehicle
16 function describe(v::Vehicle)
17     println("This is a $(typeof(v)) with speed $(v.speed) km/h")
18 end

```

```
19
20 # Sử dụng
21 car = Car("Toyota", 120.0)
22 bike = Bike("Giant", 25.0)
23
24 describe(car)    # This is a Car with speed 120.0 km/h
25 describe(bike)   # This is a Bike with speed 25.0 km/h
```

Việc sử dụng abstract types giúp code linh hoạt và dễ mở rộng — khi thêm một kiểu `Vehicle` mới (ví dụ `Plane`), hàm `describe` vẫn hoạt động mà không cần sửa đổi.

### 5.2.3 Parametric types (Kiểu tham số)

**Parametric types** (kiểu tham số) cho phép định nghĩa các kiểu dữ liệu có thể hoạt động với nhiều kiểu element khác nhau. Đây là nền tảng của generic programming trong Julia, tương tự như templates trong C++ hay generics trong Java, nhưng mạnh mẽ hơn nhờ multiple dispatch.

Cú pháp cơ bản:

```
1 struct Box{T}
2     value::T
3 end
```

Ở đây, `T` là **type parameter**. Khi tạo instance, Julia sẽ thay thế `T` bằng concrete type:

```
1 # Tạo các Box với kiểu khác nhau
2 box_int = Box(42)           # Box{Int64}
3 box_str = Box("hello")     # Box{String}
4 box_float = Box(3.14)      # Box{Float64}
5
6 println(typeof(box_int))   # Box{Int64}
7 println(typeof(box_str))   # Box{String}
```

**Parametric types với multiple type parameters:**

```
1 struct Pair{T1, T2}
2     first::T1
3     second::T2
4 end
5
6 # Sử dụng
7 p1 = Pair(10, "ten")        # Pair{Int64, String}
8 p2 = Pair(3.14, true)       # Pair{Float64, Bool}
9
10 println(p1.first, " - ", p1.second) # Output: 10 - ten
```

**Lợi ích của parametric types:**

1. **Type safety:** Julia kiểm tra kiểu tại runtime, giúp phát hiện lỗi sớm.
2. **Performance:** Compiler tạo mã tối ưu cho từng concrete type, tránh boxing (đóng gói giá trị với metadata kiểu).
3. **Memory efficiency:** Parametric types cho phép memory layout tối ưu, đặc biệt với immutable structs [?].

### 5.2.4 Where clause và type constraints

Từ khóa `where` trong Julia dùng để khai báo type parameters và áp dụng constraints (ràng buộc) lên chúng. Đây là cơ chế quan trọng để viết generic functions với type safety.

**Cú pháp cơ bản:**

```
1 function identity(x::T) where T
2     return x
3 end
```

Hàm `identity` nhận một argument kiểu `T`, với `T` có thể là bất kỳ kiểu nào. Julia sẽ tự động infer kiểu cụ thể khi gọi hàm.

**Type constraints với subtype operator `<:`**

```
1 # Chỉ chấp nhận Number và các subtype của nó
2 function double(x::T) where T<:Number
3     return 2 * x
4 end
5
6 double(10)           # OK: Int <: Number
7 double(3.14)         # OK: Float64 <: Number
8 double("hello")     # Error: String không phải subtype của Number
```

**Multiple type parameters với constraints:**

```
1 function add_vectors(v1::AbstractVector{T}, v2::AbstractVector{T}) where
2     ↪ T<:Number
3     @assert length(v1) == length(v2) "Vectors must have same length"
4     return [v1[i] + v2[i] for i in eachindex(v1)]
5 end
6
7 # Sử dụng
8 v1 = [1, 2, 3]
9 v2 = [4, 5, 6]
10 println(add_vectors(v1, v2)) # Output: [5, 7, 9]
11
12 # Type safety: cả hai vectors phải cùng element type
13 v3 = [1.0, 2.0, 3.0] # Float64
14 add_vectors(v1, v3)  # Error: Int ≠ Float64
```

**Where clause với multiple constraints:**

```
1 function process(x::T1, y::T2) where {T1<:Number, T2<:AbstractString}
2     println("Number: $x, String: $y")
3 end
4
5 process(42, "hello") # OK
6 process("hello", 42) # Error: không match signature
```

### 5.2.5 Extending Base functions

Một trong những tính năng mạnh mẽ của Julia là khả năng **extend** (mở rộng) các hàm có sẵn trong Base library để hoạt động với custom types. Điều này giúp types mới tích hợp mượt mà vào ecosystem Julia.

**Ví dụ: Extending toán tử `+`**

```
1 struct Vector2D
2     x::Float64
3     y::Float64
4 end
5
6 # Extend toán tử + từ Base
7 import Base: +
8
9 function +(v1::Vector2D, v2::Vector2D)
10     return Vector2D(v1.x + v2.x, v1.y + v2.y)
11 end
12
13 # Sử dụng
14 v1 = Vector2D(1.0, 2.0)
15 v2 = Vector2D(3.0, 4.0)
16 v3 = v1 + v2 # Vector2D(4.0, 6.0)
17
18 println("v3.x = $(v3.x), v3.y = $(v3.y)")
```

### Extending các hàm phổ biến:

```
1 import Base: *, ==, show
2
3 # Scalar multiplication
4 function *(a::Float64, v::Vector2D)
5     return Vector2D(a * v.x, a * v.y)
6 end
7
8 # Equality comparison
9 function ==(v1::Vector2D, v2::Vector2D)
10     return v1.x == v2.x && v1.y == v2.y
11 end
12
13 # Custom display
14 function show(io::IO, v::Vector2D)
15     print(io, "Vector2D($(v.x), $(v.y))")
16 end
17
18 # Sử dụng
19 v = Vector2D(1.0, 2.0)
20 println(2.0 * v) # Vector2D(2.0, 4.0)
21 println(v == Vector2D(1.0, 2.0)) # true
```

### Lưu ý quan trọng:

- Phải `import Base:<function>` trước khi `extend`.
- Chỉ `extend` cho custom types của bạn, không nên `redefine` cho types có sẵn.
- Tuân thủ semantics (ngữ nghĩa) của hàm gốc — ví dụ `+` nên là `commutative` (giao hoán) nếu có thể.

### 5.2.6 Type inference và performance

Julia compiler thực hiện **type inference** (suy luận kiểu) để tự động xác định kiểu của biến và expressions tại compile-time. Điều này là chìa khóa để Julia đạt được performance gần với C/Fortran mà không cần khai báo kiểu tường minh.

**Type stability:** Một hàm được gọi là **type-stable** nếu kiểu output có thể được xác định hoàn toàn từ kiểu của inputs. Type-stable functions cho phép compiler tạo mã tối ưu.

**Ví dụ type-stable function:**

```
1 function safe_sqrt(x::Float64)
2     if x >= 0
3         return sqrt(x)      # Luôn trả về Float64
4     else
5         return 0.0          # Luôn trả về Float64
6     end
7 end
```

**Ví dụ type-unstable function (nên tránh):**

```
1 function unsafe_sqrt(x::Float64)
2     if x >= 0
3         return sqrt(x)      # Trả về Float64
4     else
5         return "error"      # Trả về String --- type instability!
6     end
7 end
```

Hàm `unsafe_sqrt` là type-unstable vì có thể trả về `Float64` hoặc `String` tùy điều kiện. Điều này buộc compiler phải boxing values và làm chậm code.

**Kiểm tra type stability với `@code_warntype`:**

```
1 @code_warntype safe_sqrt(4.0)
2 # → Body::Float64 (type-stable, màu xanh)
3
4 @code_warntype unsafe_sqrt(4.0)
5 # → Body::Union{Float64, String} (type-unstable, màu đỏ)
```

**Best practices cho performance:**

1. Viết type-stable functions — luôn trả về cùng một kiểu.
2. Sử dụng parametric types để compiler có thể specialize code.
3. Tránh boxing bằng cách dùng immutable structs với concrete field types.
4. Sử dụng `@code_warntype` để verify type inference [?].

### 5.2.7 Best practices và pitfalls

**Best practices (Thực hành tốt):**

1. Dùng parametric types cho generic code:

```
1 # □ Good: Generic với type parameter
2 function sum_elements(v::AbstractVector{T}) where T<:Number
3     s = zero(T)
```

```
4     for x in v
5         s += x
6     end
7     return s
8 end
```

2. **Apply type constraints hợp lý:** Sử dụng `T<:AbstractType` để vừa generic vừa type-safe.
3. **Extend Base functions khi cần thiết:** Giúp custom types tích hợp tốt với Julia ecosystem.
4. **Maintain type stability:** Đảm bảo hàm luôn trả về cùng kiểu output.
5. **Document type parameters:** Giải thích ý nghĩa của mỗi type parameter trong docstring.

#### Common pitfalls (Lỗi thường gặp):

1. **Type instability:**

```
1 # □ Bad: Trả về kiểu khác nhau
2 function bad_function(x)
3     if x > 0
4         return x           # Number
5     else
6         return "negative"  # String
7     end
8 end
```

2. **Over-constraining types:**

```
1 # □ Bad: Quá cụ thể
2 function add_numbers(a::Int64, b::Int64)
3     return a + b
4 end
5
6 # □ Good: Generic hơn
7 function add_numbers(a::T, b::T) where T<:Number
8     return a + b
9 end
```

3. **Không sử dụng parametric types:** Dẫn đến boxing và performance kém.
4. **Extend functions không đúng cách:** Redefine Base functions cho types có sẵn có thể gây ambiguity errors.
5. **Confusing multiple dispatch với overloading:** Nhớ rằng Julia dispatch dựa trên kiểu thực tế tại runtime, không phải compile-time.

**Tóm tắt:** Generic programming trong Julia mạnh mẽ nhờ kết hợp parametric types, where clause, multiple dispatch, và type inference. Khi viết generic code, hãy ưu tiên type stability và sử dụng type constraints hợp lý để đạt được cả performance lẫn code flexibility [2].



### 5.2.8 Duck typing trong Julia

**Duck typing** là một khái niệm từ lập trình động: “Nếu nó đi như vịt và kêu như vịt, thì nó là vịt.” Trong Julia, điều này có nghĩa là một object có thể được sử dụng bất cứ đâu, miễn là nó hỗ trợ các phép toán được yêu cầu — không cần khai báo tường minh rằng object thuộc về một interface cụ thể nào[?].

#### Cơ chế hoạt động:

- Julia không yêu cầu khai báo kiểu (type declaration) cho biến.
- Một biến có thể giữ giá trị thuộc bất kỳ kiểu nào trong suốt vòng đời.
- Các hàm generic hoạt động với bất kỳ kiểu nào có phương thức cần thiết.

#### Ví dụ duck typing cơ bản:

```

1  # Biến có thể thay đổi kiểu
2  x = 10                # x là Int64
3  x = "hello"          # x bây giờ là String
4  x = [1, 2, 3]         # x bây giờ là Vector{Int64}
5
6  # Hàm hoạt động với bất kỳ kiểu nào có toán tử +
7  function add_twice(a, b)
8      return a + b + b
9  end
10
11 add_twice(1, 2)        # 5 (Int)
12 add_twice(1.0, 2.0)    # 5.0 (Float64)
13 add_twice("Hello ", "!") # "Hello !" (String)
```

#### Duck typing với collections:

```

1  # Hàm này hoạt động với bất kỳ iterable nào
2  function sum_elements(collection)
3      total = 0
4      for item in collection
5          total += item
6      end
7      return total
8  end
9
10 sum_elements([1, 2, 3])    # 6 (Vector)
11 sum_elements((1, 2, 3))    # 6 (Tuple)
12 sum_elements(1:5)          # 15 (UnitRange)
13 sum_elements(Set{Int}([1, 2, 3])) # 6 (Set)
```

Trong ví dụ trên, `sum_elements` không quan tâm collection cụ thể là gì — chỉ cần nó hỗ trợ iteration qua `for...in`.

#### So sánh với static typing:

- **Static typing (Java/C++):** Phải khai báo interface tường minh, compiler kiểm tra tại compile-time.
- **Duck typing (Julia/Python):** Không cần khai báo interface, kiểm tra tại runtime khi gọi method.

**Ưu điểm của duck typing:**

1. **Flexibility:** Code có thể hoạt động với các kiểu mới mà không cần sửa đổi.
2. **Less boilerplate:** Không cần viết interface declarations.
3. **Rapid prototyping:** Phát triển nhanh hơn trong giai đoạn đầu.

**Nhược điểm:**

1. **Runtime errors:** Lỗi phát hiện muộn hơn (khi chạy).
2. **Documentation burden:** Cần document rõ ràng các method required.
3. **IDE support:** Khó cung cấp autocomplete chính xác.

**5.2.9 Informal interfaces (Giao diện ngầm định)**

Julia không có từ khóa `interface` như Java hay `abstract class` với `pure virtual methods` như C++. Thay vào đó, Julia sử dụng **informal interfaces** — các quy ước (conventions) về những methods mà một kiểu phải implement để hoạt động trong một ngữ cảnh nhất định[?].

**Cơ chế hoạt động:**

- Interfaces được định nghĩa bởi *documentation* và *convention*, không phải compiler.
- Một kiểu “implement” interface bằng cách định nghĩa các methods cần thiết.
- Julia Base và các packages phổ biến định nghĩa nhiều informal interfaces.

**Ví dụ: Iteration interface**

Để một kiểu có thể được iterate qua `for` loop, nó cần implement hai methods:

```
1  # Interface yêu cầu:
2  # iterate(collection) -> (item, state) hoặc nothing
3  # iterate(collection, state) -> (item, state) hoặc nothing
4
5  struct Countdown
6      start::Int
7  end
8
9  # Implement iteration interface
10 function Base.iterate(c::Countdown)
11     c.start <= 0 && return nothing
12     return (c.start, c.start - 1)
13 end
14
15 function Base.iterate(c::Countdown, state)
16     state <= 0 && return nothing
17     return (state, state - 1)
18 end
19
20 # Bây giờ Countdown có thể dùng trong for loop
21 for i in Countdown(5)
22     print(i, " ") # Output: 5 4 3 2 1
23 end
```

```

24
25 # Và với các hàm khác
26 collect(Countdown(3)) # [3, 2, 1]
27 sum(Countdown(5))     # 15

```

### Ví dụ: Indexing interface

Để hỗ trợ `obj[i]` syntax, implement `getindex`:

```

1  struct CircularBuffer{T}
2      data::Vector{T}
3  end
4
5  function Base.getindex(cb::CircularBuffer, i::Int)
6      n = length(cb.data)
7      idx = mod1(i, n) # mod1: modulo bắt đầu từ 1
8      return cb.data[idx]
9  end
10
11 function Base.length(cb::CircularBuffer)
12     return length(cb.data)
13 end
14
15 # Sử dụng
16 cb = CircularBuffer([10, 20, 30])
17 println(cb[1]) # 10
18 println(cb[4]) # 10 (wrap around)
19 println(cb[5]) # 20

```

### Các informal interfaces phổ biến trong Julia:

| Interface  | Methods cần implement                                             | Cho phép                                      |
|------------|-------------------------------------------------------------------|-----------------------------------------------|
| Iteration  | <code>iterate</code>                                              | <code>for</code> loop, <code>collect</code>   |
| Indexing   | <code>getindex</code> , <code>setindex!</code>                    | <code>obj[i]</code> , <code>obj[i] = v</code> |
| Length     | <code>length</code>                                               | <code>length(obj)</code>                      |
| Equality   | <code>==</code> , <code>hash</code>                               | Comparison, Dict keys                         |
| Show       | <code>show</code>                                                 | <code>print</code> , REPL display             |
| Arithmetic | <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> | Math operators                                |

**Best practice:** Khi thiết kế custom type, hãy xem xét implement các informal interfaces phổ biến để type tích hợp tốt với Julia ecosystem.

#### 5.2.10 Traits pattern

**Traits** (đặc điểm) là một design pattern trong Julia cho phép phân loại types dựa trên hành vi hoặc khả năng của chúng, thay vì dựa trên cây kế thừa. Pattern này giải quyết vấn đề Julia không hỗ trợ multiple inheritance[?].

##### Vấn đề cần giải quyết:

Giả sử bạn có nhiều types và muốn phân loại chúng theo một đặc điểm — ví dụ, phân loại assets thành “thanh khoản” (liquid) hoặc “phi thanh khoản” (illiquid). Với single inheritance, điều này khó thực hiện nếu các types đã có abstract type khác.

##### Giải pháp: Traits pattern

##### Bước 1: Định nghĩa traits như types

```

1  # Abstract type cho trait
2  abstract type LiquidityStyle end
3
4  # Các concrete traits
5  struct IsLiquid <: LiquidityStyle end
6  struct IsIlliquid <: LiquidityStyle end

```

### Bước 2: Hàm gán trait cho types

```

1  # Hàm trả về trait của một type
2  # Default: illiquid
3  LiquidityStyle(::Type) = IsIlliquid()
4
5  # Cash và Stock là liquid
6  LiquidityStyle(::Type{<:Cash}) = IsLiquid()
7  LiquidityStyle(::Type{<:Stock}) = IsLiquid()
8
9  # RealEstate và PrivateEquity là illiquid (dùng default)

```

### Bước 3: Dispatch dựa trên trait

```

1  # Hàm entry point
2  function sell(asset)
3      sell(LiquidityStyle(typeof(asset)), asset)
4  end
5
6  # Dispatch theo trait
7  function sell(::IsLiquid, asset)
8      println("Quick sale at market price: $asset")
9  end
10
11 function sell(::IsIlliquid, asset)
12     println("Complex sale, need time to find buyer: $asset")
13 end

```

### Ví dụ hoàn chỉnh:

```

1  # Định nghĩa assets
2  abstract type Asset end
3  struct Cash <: Asset
4      amount::Float64
5  end
6  struct Stock <: Asset
7      ticker::String
8      shares::Int
9  end
10 struct RealEstate <: Asset
11     address::String
12     value::Float64
13 end
14
15 # Định nghĩa traits
16 abstract type LiquidityStyle end
17 struct IsLiquid <: LiquidityStyle end

```

```

18 struct IsIlliquid <: LiquidityStyle end
19
20 # Gán traits
21 LiquidityStyle{::Type} = IsIlliquid()
22 LiquidityStyle{::Type{Cash}} = IsLiquid()
23 LiquidityStyle{::Type{Stock}} = IsLiquid()
24
25 # Trait-based dispatch
26 function market_value(asset)
27     market_value(LiquidityStyle(typeof(asset)), asset)
28 end
29
30 function market_value{::IsLiquid, asset::Cash}
31     return asset.amount # Face value
32 end
33
34 function market_value{::IsLiquid, asset::Stock}
35     return asset.shares * 100.0 # Simplified
36 end
37
38 function market_value{::IsIlliquid, asset}
39     return 0.7 * asset.value # 30% discount for illiquidity
40 end
41
42 # Test
43 cash = Cash(1000.0)
44 stock = Stock("AAPL", 10)
45 house = RealEstate("123 Main St", 500000.0)
46
47 println(market_value(cash)) # 1000.0
48 println(market_value(stock)) # 1000.0
49 println(market_value(house)) # 350000.0

```

#### Ưu điểm của traits pattern:

1. **Orthogonal classification:** Phân loại types theo nhiều chiều khác nhau mà không cần multiple inheritance.
2. **Extensible:** Có thể thêm trait mới cho types có sẵn mà không sửa code gốc.
3. **Performance:** Julia inline và optimize các trait functions hiệu quả.
4. **Computed traits:** Trait có thể được tính toán từ properties của type.

#### 5.2.11 Holy traits (Tim Holy’s trick)

**Holy traits** là một biến thể tinh vi của traits pattern, được đặt tên theo Tim Holy — một trong những core developers của Julia. Pattern này đặc biệt hữu ích khi làm việc với abstract types và type hierarchies phức tạp[?].

##### Ý tưởng cốt lõi:

Thay vì dispatch trực tiếp trên types, Holy traits dispatch trên một “trait function” trả về một “trait type”. Điều này cho phép:

- Gán traits cho toàn bộ cây subtype cùng lúc.

- Override trait cho specific subtypes khi cần.
- Giữ code DRY (Don't Repeat Yourself).

### Ví dụ với AbstractArray:

Julia Base sử dụng Holy traits để phân biệt arrays theo cách lưu trữ (storage):

```
1  # Trait types
2  struct IndexLinear <: IndexStyle end
3  struct IndexCartesian <: IndexStyle end
4
5  # Trait function
6  IndexStyle(::Type{<:Array}) = IndexLinear()
7  IndexStyle(::Type{<:SubArray}) = IndexCartesian()
8
9  # Fallback: inherit từ parent
10 IndexStyle(::Type{A}) where {A<:AbstractArray} =
11     IndexStyle(supertype(A))
12
13 # Sử dụng trait trong generic function
14 function mysum(A::AbstractArray)
15     mysum(IndexStyle(typeof(A)), A)
16 end
17
18 function mysum(::IndexLinear, A)
19     # Optimized path cho linear indexing
20     s = zero(eltype(A))
21     for i in eachindex(A)
22         s += A[i]
23     end
24     return s
25 end
26
27 function mysum(::IndexCartesian, A)
28     # Path cho cartesian indexing
29     s = zero(eltype(A))
30     for I in CartesianIndices(A)
31         s += A[I]
32     end
33     return s
34 end
```

### Ví dụ: Trait cho iteration behavior

```
1  # Định nghĩa traits cho iteration capability
2  abstract type IterationStyle end
3  struct HasLength <: IterationStyle end
4  struct IsInfinite <: IterationStyle end
5  struct SizeUnknown <: IterationStyle end
6
7  # Default: unknown
8  IterationStyle(::Type) = SizeUnknown()
9
10 # Arrays có length
```

```

11 IterationStyle(<:Type{<:AbstractArray}) = HasLength()
12
13 # Ranges có length
14 IterationStyle(<:Type{<:AbstractRange}) = HasLength()
15
16 # Generators có thể infinite
17 struct InfiniteCounter end
18 IterationStyle(<:Type{InfiniteCounter}) = IsInfinite()
19
20 # Generic function sử dụng trait
21 function safe_collect(itr)
22     safe_collect(IterationStyle(typeof(itr)), itr)
23 end
24
25 function safe_collect(<:HasLength, itr)
26     return collect(itr)
27 end
28
29 function safe_collect(<:IsInfinite, itr)
30     error("Cannot collect infinite iterator!")
31 end
32
33 function safe_collect(<:SizeUnknown, itr)
34     # Thu thập với giới hạn an toàn
35     result = []
36     count = 0
37     for x in itr
38         push!(result, x)
39         count += 1
40         count > 10000 && error("Too many elements!")
41     end
42     return result
43 end

```

### Computed traits:

Holy traits có thể được *computed* từ properties của type, không chỉ hard-coded:

```

1 # Trait phụ thuộc vào element type
2 abstract type MutabilityStyle end
3 struct IsMutable <: MutabilityStyle end
4 struct IsImmutable <: MutabilityStyle end
5
6 # Arrays of mutable structs
7 function MutabilityStyle(<:Type{Array{T}}) where T
8     ismutabletype(T) ? IsMutable() : IsImmutable()
9 end
10
11 # Test
12 mutable struct MutablePoint
13     x::Float64
14     y::Float64
15 end
16

```

```

17 struct ImmutablePoint
18     x::Float64
19     y::Float64
20 end
21
22 println(MutabilityStyle(Array{MutablePoint})) # IsMutable()
23 println(MutabilityStyle(Array{ImmutablePoint})) # IsImmutable()

```

Best practices cho Holy traits:

1. **Trait types nên là singletons:** Không có fields, chỉ dùng để dispatch.
2. **Cung cấp sensible default:** Fallback trait cho types không được định nghĩa rõ ràng.
3. **Document trait requirements:** Giải thích mỗi trait có nghĩa gì và behaviors kỳ vọng.
4. **Use `::Type{<:}` pattern:** Cho phép gán trait cho toàn bộ subtype tree.

### 5.2.12 Custom number types — Ví dụ thực tế

Một trong những ứng dụng ấn tượng nhất của generic programming trong Julia là khả năng tạo **custom number types** tích hợp hoàn toàn với arithmetic và linear algebra của Julia.

#### Ví dụ 1: Số hữu tỷ chính xác (Exact Rational)

Julia đã có Rational type, nhưng hãy xem cách implement:

```

1 struct MyRational{T<:Integer} <: Real
2     num::T # Tử số
3     den::T # Mẫu số
4
5     function MyRational{T}(n::T, d::T) where T<:Integer
6         d == 0 && throw(DivideError())
7         g = gcd(n, d)
8         new(div(n, g), div(d, g)) # Rút gọn
9     end
10 end
11
12 MyRational(n::T, d::T) where T<:Integer = MyRational{T}(n, d)
13
14 # Extend Base operations
15 import Base: +, -, *, /, ==, show
16
17 function +(a::MyRational, b::MyRational)
18     n = a.num * b.den + b.num * a.den
19     d = a.den * b.den
20     return MyRational(n, d)
21 end
22
23 function *(a::MyRational, b::MyRational)
24     return MyRational(a.num * b.num, a.den * b.den)
25 end
26
27 function ==(a::MyRational, b::MyRational)
28     return a.num == b.num && a.den == b.den
29 end

```



```

30
31 function show(io::IO, r::MyRational)
32     print(io, "$ (r.num) // $ (r.den) ")
33 end
34
35 # Test
36 r1 = MyRational(1, 2)
37 r2 = MyRational(1, 3)
38 println(r1 + r2)  # 5//6
39 println(r1 * r2)  # 1//6

```

## Ví dụ 2: Dual numbers cho automatic differentiation

```

1  # Dual number:  $a + b\varepsilon$  với  $\varepsilon^2 = 0$ 
2  struct Dual{T<:Real} <: Number
3      value::T      # Giá trị thực
4      deriv::T      # Đạo hàm (hệ số của  $\varepsilon$ )
5  end
6
7  Dual(v::T) where T<:Real = Dual(v, zero(T))
8
9  import Base: +, -, *, /, sin, cos, exp, show
10
11 # Arithmetic
12 +(a::Dual, b::Dual) = Dual(a.value + b.value, a.deriv + b.deriv)
13 -(a::Dual, b::Dual) = Dual(a.value - b.value, a.deriv - b.deriv)
14 *(a::Dual, b::Dual) = Dual(a.value * b.value,
15                             a.value * b.deriv + a.deriv * b.value)
16 /(a::Dual, b::Dual) = Dual(a.value / b.value,
17                             (a.deriv * b.value - a.value * b.deriv) /
18                             ↪ b.value^2)
19
20 # Scalar operations
21 +(a::Dual, b::Real) = Dual(a.value + b, a.deriv)
22 *(a::Dual, b::Real) = Dual(a.value * b, a.deriv * b)
23 *(b::Real, a::Dual) = a * b
24
25 # Transcendental functions
26 sin(d::Dual) = Dual(sin(d.value), cos(d.value) * d.deriv)
27 cos(d::Dual) = Dual(cos(d.value), -sin(d.value) * d.deriv)
28 exp(d::Dual) = Dual(exp(d.value), exp(d.value) * d.deriv)
29
30 show(io::IO, d::Dual) = print(io, "$ (d.value) + $ (d.deriv)  $\varepsilon$ ")
31
32 # Automatic differentiation!
33 function derivative(f, x)
34     dual_x = Dual(x, 1.0) #  $x + 1\varepsilon$ 
35     result = f(dual_x)
36     return result.deriv
37 end
38
39 # Test
40 f(x) = x^2 + 3x + 1

```

```

40 println(derivative(f, 2.0)) # 7.0 (đạo hàm tại x=2: 2x + 3 = 7)
41
42 g(x) = sin(x) * exp(x)
43 println(derivative(g, 0.0)) # 1.0 (cos(0)*exp(0) + sin(0)*exp(0))

```

### Ví dụ 3: Units of measurement

```

1  # Số với đơn vị đo lường
2  struct Quantity{T<:Number, U}
3      value::T
4  end
5
6  # Type alias cho các đơn vị phổ biến
7  const Meter = Quantity{Float64, :m}
8  const Second = Quantity{Float64, :s}
9  const Kilogram = Quantity{Float64, :kg}
10
11 # Constructor helpers
12 meter(v) = Quantity{Float64, :m}(v)
13 second(v) = Quantity{Float64, :s}(v)
14 kilogram(v) = Quantity{Float64, :kg}(v)
15
16 import Base: +, *, show
17
18 # Cộng cùng đơn vị
19 function +(a::Quantity{T,U}, b::Quantity{T,U}) where {T,U}
20     return Quantity{T,U}(a.value + b.value)
21 end
22
23 # Nhân khác đơn vị → đơn vị mới
24 function *(a::Quantity{T,U1}, b::Quantity{T,U2}) where {T,U1,U2}
25     return Quantity{T, (U1,U2)}(a.value * b.value)
26 end
27
28 function show(io::IO, q::Quantity{T,U}) where {T,U}
29     print(io, "$(q.value) $U")
30 end
31
32 # Test
33 d = meter(100.0)
34 t = second(10.0)
35 println(d) # 100.0 m
36 println(t) # 10.0 s
37 # velocity = d / t # Cần implement /
38
39 # Cộng cùng đơn vị OK
40 d2 = meter(50.0)
41 println(d + d2) # 150.0 m

```

### Tại sao custom number types mạnh mẽ trong Julia:

1. **Full integration:** Custom types hoạt động với linear algebra, broadcasting, etc.
2. **Zero overhead:** Julia compiler specializes code cho mỗi concrete type.

3. **Composable:** Có thể nest types (ví dụ: `Matrix{Dual{Float64}}`).

4. **Extensible:** Dễ dàng thêm methods mới cho existing operations.

**Kết luận chương:** Generic programming trong Julia đạt được sự cân bằng tuyệt vời giữa *flexibility* (linh hoạt như Python) và *performance* (nhanh như C). Nhờ duck typing, informal interfaces, traits pattern, và khả năng extend Base functions, lập trình viên có thể viết code generic, type-safe, và hiệu năng cao mà không phải đánh đổi bất kỳ yếu tố nào.

```
1 length(v::AbstractVector) = sum(abs2, v)^0.5
```

## 5.3 Lambda, map/filter/reduce

Lập trình hàm (functional programming) là một phong cách lập trình quan trọng trong Julia, cho phép viết code ngắn gọn và biểu đạt. Trong phần này, chúng ta sẽ tìm hiểu về hàm lambda, các hàm xử lý collection (map/filter/reduce), và các kỹ thuật đánh giá lazy (lazy evaluation) giúp tối ưu hiệu năng.

### 5.3.1 Lambda và hàm ẩn danh

**Hàm ẩn danh** (anonymous function) hay **hàm lambda** là hàm không có tên, thường được định nghĩa ngay tại chỗ để truyền làm tham số cho hàm khác[?]. Julia hỗ trợ hai cú pháp:

```
1 # Cú pháp arrow (->)
2 square = x -> x^2
3 add = (x, y) -> x + y
4
5 # Cú pháp function
6 cube = function(x) x^3 end
```

Hàm lambda rất hữu ích khi cần một hàm đơn giản chỉ dùng một lần, tránh phải định nghĩa hàm có tên riêng biệt.

### 5.3.2 Closures - Hàm đóng

**Closure** là hàm ẩn danh có khả năng *capture* (lưu giữ) biến từ phạm vi bên ngoài[?]. Đây là kỹ thuật lập trình quan trọng được sử dụng rộng rãi trong functional programming, callbacks, và event-driven programming.

#### 5.3.2.1 Cơ chế capture biến

Khi định nghĩa một hàm bên trong hàm khác, hàm bên trong có thể truy cập các biến của hàm bên ngoài. Biến được “đóng lại” (closed over) cùng với hàm:

```
1 function make_adder(n)
2     x -> x + n # closure captures n
3 end
4
5 add5 = make_adder(5)
6 add10 = make_adder(10)
7
8 println(add5(3)) # 8
9 println(add10(3)) # 13
```

Trong ví dụ trên, biến `n` được *captured* bởi hàm lambda bên trong, cho phép mỗi closure giữ trạng thái riêng.

#### Function factory pattern:

Closure cho phép tạo “nhà máy hàm” (function factory) - hàm trả về hàm khác với hành vi tùy chỉnh:

```

1  # Function factory cho phép lũy thừa
2  function power(n)
3      return function(x)
4          x^n
5      end
6  end
7
8  square = power(2)  # x -> x^2
9  cube = power(3)    # x -> x^3
10
11 println(square(4)) # 16
12 println(cube(4))   # 64

```

#### Closure với mutable state:

Closure có thể duy trì và thay đổi trạng thái nội bộ:

```

1  function make_counter(start=0)
2      count = start
3      function()
4          count += 1
5          return count
6      end
7  end
8
9  counter = make_counter(10)
10 println(counter()) # 11
11 println(counter()) # 12
12 println(counter()) # 13

```

#### Ghi chú

Biến `count` được shared giữa closure và outer function. Mỗi lần gọi closure, giá trị `count` được cập nhật và giữ lại cho lần gọi tiếp theo.

#### 5.3.2.2 let blocks - Tránh closure bugs

Một lỗi phổ biến khi dùng closures trong vòng lặp là tất cả closures đều capture cùng một biến:

```

1  # BUG: Tất cả closures đều capture cùng biến i
2  functions = []
3  for i in 1:3
4      push!(functions, () -> i)
5  end
6
7  # Tất cả đều trả về 3!
8  println(functions[1]()) # 3
9  println(functions[2]()) # 3
10 println(functions[3]()) # 3

```

**Giải pháp với let block:**

let tạo một phạm vi mới với biến riêng biệt cho mỗi iteration:

```

1  # CORRECT: Dùng let để capture giá trị tại thời điểm tạo closure
2  functions = []
3  for i in 1:3
4      let j = i
5          push!(functions, () -> j)
6      end
7  end
8
9  println(functions[1]()) # 1
10 println(functions[2]()) # 2
11 println(functions[3]()) # 3

```

**let với multiple bindings:**

```

1  let x = 1, y = 2
2      println(x + y) # 3
3  end
4
5  # x, y không tồn tại ngoài let block

```

**Mẹo**

Luôn dùng let khi tạo closures trong vòng lặp để đảm bảo mỗi closure capture đúng giá trị tại thời điểm tạo.

**5.3.2.3 do blocks - Cú pháp cho anonymous functions**

Nhiều hàm trong Julia nhận một hàm làm tham số đầu tiên. do block cung cấp cú pháp rõ ràng hơn cho những trường hợp này:

```

1  # Cú pháp thông thường
2  map(x -> begin
3      result = x^2
4      result + 1
5  end, [1, 2, 3])
6
7  # Cú pháp do block - dễ đọc hơn
8  map([1, 2, 3]) do x
9      result = x^2
10     result + 1
11 end
12 # Kết quả: [2, 5, 10]

```

**do block với multiple arguments:**

```

1  # reduce với do block
2  reduce([1, 2, 3, 4]) do acc, x
3      acc + x^2
4  end
5  # Kết quả: 1 + 4 + 9 + 16 = 30

```

```

6
7 # filter với do block phức tạp
8 filter(1:10) do x
9     isprime = true
10    for i in 2:isqrt(x)
11        if x % i == 0
12            isprime = false
13            break
14        end
15    end
16    x > 1 && isprime
17 end
18 # Kết quả: [2, 3, 5, 7]

```

### Ứng dụng phổ biến - File handling:

```

1 # open() với do block tự động đóng file
2 open("data.txt", "w") do io
3     println(io, "Line 1")
4     println(io, "Line 2")
5 end # file tự động được đóng
6
7 # Đọc file an toàn
8 content = open("data.txt") do io
9     read(io, String)
10 end

```

#### Ghi chú

Khi dùng do block với `open()`, file được tự động đóng khi block kết thúc, kể cả khi có exception xảy ra. Đây là pattern quan trọng để tránh resource leaks.

#### 5.3.2.4 Higher-order functions

**Higher-order function** là hàm nhận hàm khác làm tham số hoặc trả về hàm. Julia hỗ trợ đầy đủ higher-order functions như một ngôn ngữ functional programming[?]:

```

1 # Hàm nhận hàm làm tham số
2 function apply_twice(f, x)
3     f(f(x))
4 end
5
6 println(apply_twice(x -> x + 1, 5)) # 7
7 println(apply_twice(x -> x * 2, 3)) # 12
8
9 # Hàm trả về hàm (function composition)
10 function compose(f, g)
11     x -> f(g(x))
12 end
13
14 add1 = x -> x + 1
15 times2 = x -> x * 2
16

```

```

17 add1_then_times2 = compose(times2, add1)
18 println(add1_then_times2(5))  # (5+1)*2 = 12

```

### Function composition operator:

Julia cung cấp toán tử  $\circ$  (nhập bằng `\circ` + Tab) cho function composition:

```

1 f = x -> x + 1
2 g = x -> x * 2
3 h = x -> x^2
4
5 # Composition: h(g(f(x)))
6 composed = h ∘ g ∘ f
7
8 println(composed(3))  # ((3+1)*2)^2 = 64

```

#### 5.3.2.5 Callback pattern

**Callback** là hàm được truyền vào hàm khác để được gọi sau khi một sự kiện hoặc thao tác hoàn thành:

```

1 # Hàm xử lý với callback
2 function process_data(data, on_success, on_error)
3     try
4         result = sum(data) / length(data)
5         on_success(result)
6     catch e
7         on_error(e)
8     end
9 end
10
11 # Sử dụng callbacks
12 process_data([1, 2, 3, 4, 5],
13     result -> println("Average: $result"),
14     error -> println("Error: $error")
15 )
16 # Output: Average: 3.0

```

### Event handler pattern:

```

1 # Simple event system
2 mutable struct EventEmitter
3     handlers::Dict{Symbol, Vector{Function}}
4 end
5
6 EventEmitter() = EventEmitter{Dict{Symbol, Vector{Function}}}()
7
8 function on!(emitter::EventEmitter, event::Symbol, handler::Function)
9     if !haskey(emitter.handlers, event)
10         emitter.handlers[event] = Function[]
11     end
12     push!(emitter.handlers[event], handler)
13 end
14

```

```

15 function emit!(emitter::EventEmitter, event::Symbol, data)
16     if haskey(emitter.handlers, event)
17         for handler in emitter.handlers[event]
18             handler(data)
19         end
20     end
21 end
22
23 # Sử dụng
24 events = EventEmitter()
25 on!(events, :data, x -> println("Received: $x"))
26 on!(events, :data, x -> println("Processing: $(x*2)"))
27
28 emit!(events, :data, 42)
29 # Output:
30 # Received: 42
31 # Processing: 84

```

### 5.3.2.6 Decorator pattern

**Decorator** là pattern sử dụng higher-order functions để mở rộng hành vi của hàm khác mà không thay đổi code gốc:

```

1 # Decorator đo thời gian thực thi
2 function timed(f)
3     function(args...; kwargs...)
4         start = time()
5         result = f(args...; kwargs...)
6         elapsed = time() - start
7         println("Execution time: $(elapsed)s")
8         return result
9     end
10 end
11
12 # Hàm gốc
13 function slow_sum(n)
14     total = 0
15     for i in 1:n
16         total += i
17     end
18     return total
19 end
20
21 # Áp dụng decorator
22 timed_sum = timed(slow_sum)
23 result = timed_sum(1_000_000)
24 # Output: Execution time: 0.023s

```

**Logging decorator:**

```

1 function logged(f, name="function")
2     function(args...; kwargs...)
3         println("[$name] Called with args: $args")

```



```

4         result = f(args...; kwargs...)
5         println("[\$name] Returned: \$result")
6         return result
7     end
8 end
9
10 # Áp dụng
11 add = logged((x, y) -> x + y, "add")
12 add(3, 5)
13 # Output:
14 # [add] Called with args: (3, 5)
15 # [add] Returned: 8

```

### Memoization decorator:

```

1 function memoize(f)
2     cache = Dict{<}<()
3     function(args...)
4         if haskey(cache, args)
5             return cache[args]
6         end
7         result = f(args...)
8         cache[args] = result
9         return result
10    end
11 end
12
13 # Fibonacci không tối ưu
14 fib_slow(n) = n <= 1 ? n : fib_slow(n-1) + fib_slow(n-2)
15
16 # Fibonacci với memoization
17 fib = memoize(function(n)
18     n <= 1 ? n : fib(n-1) + fib(n-2)
19 end)
20
21 println(fib(40)) # Nhanh nhờ cache

```

#### Cảnh báo

Khi dùng memoization, cần cẩn thận với:

- Bộ nhớ: Cache có thể tăng vô hạn với nhiều input khác nhau
- Pure functions: Chỉ memoize hàm thuần (không có side effects)
- Key comparison: Args phải comparable và hashable

### 5.3.2.7 Tổng kết closures

| Khái niệm      | Cú pháp                             | Use case                              |
|----------------|-------------------------------------|---------------------------------------|
| Closure cơ bản | <code>x -&gt; x + n</code>          | Function factory, state encapsulation |
| let block      | <code>let x = v; ... end</code>     | Tránh capture bug trong loops         |
| do block       | <code>f(args) do x; ... end</code>  | Anonymous function dễ đọc             |
| Callback       | <code>f(data, on_done)</code>       | Event handling, async operations      |
| Decorator      | <code>wrapped = decorator(f)</code> | Logging, timing, memoization          |

### 5.3.3 map, filter, reduce

Julia cung cấp ba hàm cơ bản để xử lý collections:

#### 5.3.3.1 map - Áp dụng hàm cho từng phần tử

map áp dụng một hàm cho từng phần tử trong collection và trả về collection mới:

```

1 # map với lambda
2 map(x -> x^2, [1, 2, 3, 4, 5]) # [1, 4, 9, 16, 25]
3
4 # map với hàm có sẵn
5 map(sqrt, [1, 4, 9, 16]) # [1.0, 2.0, 3.0, 4.0]
6
7 # map với nhiều arguments
8 map(+, [1, 2, 3], [10, 20, 30]) # [11, 22, 33]
```

#### 5.3.3.2 filter - Lọc phần tử thỏa điều kiện

filter trả về collection mới chỉ chứa các phần tử thỏa mãn điều kiện:

```

1 # Lọc số > 3
2 filter(x -> x > 3, [1, 2, 3, 4, 5, 6]) # [4, 5, 6]
3
4 # Dùng predicates có sẵn
5 filter(iseven, [1, 2, 3, 4, 5, 6]) # [2, 4, 6]
6 filter(isodd, [1, 2, 3, 4, 5, 6]) # [1, 3, 5]
```

#### 5.3.3.3 reduce - Rút gọn về một giá trị

reduce áp dụng một toán tử nhị phân (binary operator) để kết hợp tất cả phần tử thành một giá trị duy nhất:

```

1 # Tính tổng
2 reduce(+, [1, 2, 3, 4, 5]) # 15
3
4 # Tính tích
5 reduce(*, [1, 2, 3, 4, 5]) # 120
6
7 # Với lambda tùy chỉnh
8 reduce((x, y) -> x * 10 + y, [1, 2, 3, 4]) # 1234
9
10 # Với giá trị khởi tạo
11 reduce(+, [1, 2, 3, 4]; init=100) # 110
```

### 5.3.4 mapreduce - Tối ưu hóa hiệu năng

mapreduce kết hợp map và reduce trong một bước duy nhất, tránh tạo mảng trung gian, do đó tiết kiệm bộ nhớ và tăng tốc đáng kể[?]:

```
1 # Tính tổng bình phương
2 mapreduce(x -> x^2, +, [1, 2, 3, 4, 5]) # 55
3
4 # Tương đương nhưng chậm hơn (tạo mảng trung gian)
5 reduce(+, map(x -> x^2, [1, 2, 3, 4, 5])) # 55
```

mapreduce luôn được ưu tiên khi cần kết hợp map và reduce, đặc biệt với dữ liệu lớn.

### 5.3.5 Array comprehensions - Biểu thức tạo mảng

**Array comprehension** là cú pháp ngắn gọn để tạo mảng mới từ một iterable, sử dụng *đánh giá ngay* (eager evaluation):

```
1 # Comprehension cơ bản
2 [x^2 for x in 1:5] # [1, 4, 9, 16, 25]
3
4 # Với điều kiện
5 [x for x in 1:10 if x % 2 == 0] # [2, 4, 6, 8, 10]
6
7 # Nested comprehension (2D)
8 [x * y for x in 1:3, y in 1:3]
9 # 3x3 Matrix: [1 2 3; 2 4 6; 3 6 9]
```

Array comprehension tạo ra mảng ngay lập tức và cấp phát bộ nhớ cho toàn bộ kết quả.

### 5.3.6 Generator expressions - Biểu thức sinh

**Generator expression** có cú pháp tương tự comprehension nhưng không dùng ngoặc vuông, tạo ra một iterator thay vì mảng[?]. Đây là *đánh giá trì hoãn* (lazy evaluation):

```
1 # Generator (lazy)
2 gen = (x^2 for x in 1:5)
3
4 # Collect để chuyển thành mảng
5 collect(gen) # [1, 4, 9, 16, 25]
6
7 # sum trực tiếp không tạo mảng trung gian
8 sum(x^2 for x in 1:5) # 55
```

Generator chỉ tạo từng phần tử khi cần, không cấp phát bộ nhớ cho toàn bộ kết quả.

### 5.3.7 Lazy vs Eager evaluation

Sự khác biệt giữa eager và lazy evaluation có tác động lớn đến hiệu năng:

```
1 # Eager (array comprehension): cấp phát 8MB bộ nhớ
2 eager = [x^2 for x in 1:1_000_000]
3
4 # Lazy (generator): chỉ 16 bytes
```

```

5 lazy = (x^2 for x in 1:1_000_000)
6
7 # Cả hai cho cùng kết quả khi reduce
8 sum(eager) == sum(lazy) # true

```

### Khi nào dùng Lazy:

- Dữ liệu lớn, muốn tiết kiệm bộ nhớ
- Chỉ xử lý từng phần tử một lần
- Pipeline xử lý dữ liệu phức tạp

### Khi nào dùng Eager:

- Cần truy cập ngẫu nhiên vào phần tử
- Cần duyệt qua collection nhiều lần
- Dữ liệu nhỏ, không lo bộ nhớ

## 5.3.8 Pipeline operator và do blocks

### 5.3.8.1 Pipeline operator |>

Pipeline operator |> cho phép xâu chuỗi các hàm một cách dễ đọc:

```

1 # Pipeline với functional style
2 result = [1, 2, 3, 4, 5] |>
3     x -> filter(isodd, x) |>
4     x -> map(y -> y^2, x) |>
5     sum
6 # result = 35 (1^2 + 3^2 + 5^2)

```

### 5.3.8.2 do blocks

do blocks cung cấp cú pháp dễ đọc hơn cho hàm ẩn danh phức tạp:

```

1 # map với do block
2 result = map([1, 2, 3, 4, 5]) do x
3     if x % 2 == 0
4         x^2
5     else
6         x^3
7     end
8 end
9 # result = [1, 4, 27, 16, 125]
10
11 # filter với do block
12 result = filter([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]) do x
13     x % 2 == 0 && x > 5
14 end
15 # result = [6, 8, 10]

```

do block đặc biệt hữu ích khi lambda function chứa nhiều dòng code hoặc logic phức tạp.

### 5.3.9 @chain macro - Pipeline nâng cao

Pipeline operator `|>` có hạn chế: chỉ truyền được vào argument đầu tiên. Macro `@chain` từ package `Chain.jl` giải quyết vấn đề này một cách linh hoạt[?]:

```

1 using Chain
2
3 # @chain cho phép pipeline phức tạp
4 result = @chain "hello" begin
5     uppercase           # "HELLO"
6     reverse             # "OLLEH"
7     occursin("OL", _)  # true - _ là placeholder
8 end
9
10 # So sánh với |> (khó làm được)
11 # "hello" |> uppercase |> reverse |> ???
12 # occursin cần argument thứ 2!
```

#### Cú pháp @chain:

- Kết quả của mỗi expression tự động thành argument đầu tiên của expression tiếp theo
- Dùng `_` (underscore) để chỉ định vị trí argument khác
- Hỗ trợ multi-line với `begin...end` block

```

1 using Chain
2
3 # Data processing pipeline
4 data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
5
6 result = @chain data begin
7     filter(iseven, _)    # [2, 4, 6, 8, 10]
8     map(x -> x^2, _)     # [4, 16, 36, 64, 100]
9     sum                  # 220
10    _ / length(data)     # 22.0 (average of squares)
11 end
```

#### Mẹo

`@chain` đặc biệt hữu ích trong data science workflows với `DataFrames.jl`, cho phép viết queries giống SQL style.

### 5.3.10 Iterators.jl - Lazy operations

Module `Iterators` (từ `Base`) cung cấp các hàm tạo iterators lazy, không cấp phát bộ nhớ cho kết quả trung gian[?]:

#### 5.3.10.1 Các hàm Iterators phổ biến

```

1 # Iterators.map - lazy version của map
2 lazy_squares = Iterators.map(x -> x^2, 1:1_000_000)
3 # Không cấp phát bộ nhớ!
4 sum(lazy_squares) # Tính khi cần
```

```

5
6  # Iterators.filter - lazy version của filter
7  lazy_evens = Iterators.filter(iseven, 1:1_000_000)
8  first(lazy_evens, 5)  # [2, 4, 6, 8, 10]
9
10 # Iterators.take - lấy n phần tử đầu
11 first_10 = Iterators.take(1:1_000_000, 10)
12 collect(first_10)  # [1, 2, ..., 10]
13
14 # Iterators.drop - bỏ n phần tử đầu
15 skip_10 = Iterators.drop(1:20, 10)
16 collect(skip_10)  # [11, 12, ..., 20]

```

### 5.3.10.2 Chaining lazy operations

```

1  # Pipeline với lazy operations
2  result = 1:1_000_000 |>
3    x -> Iterators.filter(iseven, x) |>
4    x -> Iterators.map(y -> y^2, x) |>
5    x -> Iterators.take(x, 100) |>
6    sum
7  # Chỉ tính 100 phần tử đầu, không tạo mảng trung gian!

```

### 5.3.10.3 Iterators nâng cao

```

1  # Iterators.flatten - làm phẳng nested iterables
2  nested = [[1, 2], [3, 4], [5, 6]]
3  flat = Iterators.flatten(nested)
4  collect(flat)  # [1, 2, 3, 4, 5, 6]
5
6  # Iterators.cycle - lặp vô hạn
7  cycled = Iterators.cycle([1, 2, 3])
8  collect(Iterators.take(cycled, 7))  # [1, 2, 3, 1, 2, 3, 1]
9
10 # Iterators.repeated - lặp một giá trị
11 repeated = Iterators.repeated(42, 5)
12 collect(repeated)  # [42, 42, 42, 42, 42]
13
14 # Iterators.partition - chia thành chunks
15 chunks = Iterators.partition(1:10, 3)
16 collect(chunks)  # [[1,2,3], [4,5,6], [7,8,9], [10]]
17
18 # Iterators.enumerate - index + value
19 for (i, v) in enumerate(["a", "b", "c"])
20   println("$i: $v")
21 end
22 # 1: a
23 # 2: b
24 # 3: c
25
26 # Iterators.zip - kết hợp nhiều iterables
27 for (x, y) in zip([1, 2, 3], ["a", "b", "c"])
28   println("$x -> $y")

```

```

29 end
30 # 1 -> a
31 # 2 -> b
32 # 3 -> c

```

### Ghi chú

Iterators lazy chỉ tính giá trị khi được yêu cầu (lazy evaluation). Điều này cho phép làm việc với dữ liệu lớn hoặc vô hạn mà không hết bộ nhớ.

## 5.3.11 So sánh với Python functional tools

Julia và Python đều hỗ trợ lập trình hàm, nhưng có nhiều khác biệt quan trọng:

### 5.3.11.1 Lambda syntax

| Tính năng         | Julia                           | Python                          |
|-------------------|---------------------------------|---------------------------------|
| Lambda cơ bản     | <code>x -&gt; x^2</code>        | <code>lambda x: x**2</code>     |
| Nhiều arguments   | <code>(x, y) -&gt; x + y</code> | <code>lambda x, y: x + y</code> |
| Multi-line lambda | Có (do block)                   | Không (phải dùng def)           |
| Type annotations  | Có                              | Không (trong lambda)            |

```

1 # Julia - lambda có thể multi-line với do block
2 result = map([1, 2, 3]) do x
3     temp = x^2
4     temp + 1
5 end

```

```

# Python - lambda chỉ được 1 expression
# Phải dùng def cho multi-line
def process(x):
    temp = x**2
    return temp + 1
result = list(map(process, [1, 2, 3]))

```

### 5.3.11.2 map, filter, reduce

| Tính năng          | Julia                  | Python                        |
|--------------------|------------------------|-------------------------------|
| map return type    | Array                  | Iterator (lazy)               |
| filter return type | Array                  | Iterator (lazy)               |
| reduce location    | Base                   | <code>functools.reduce</code> |
| mapreduce          | Có (built-in)          | Không                         |
| Broadcasting       | <code>f.(array)</code> | Không có                      |

```

1 # Julia - map trả về Array ngay
2 squares = map(x -> x^2, [1, 2, 3]) # [1, 4, 9]
3
4 # Broadcasting - cú pháp ngắn gọn hơn
5 squares = (x -> x^2).( [1, 2, 3]) # [1, 4, 9]
6 squares = [1, 2, 3] .^ 2 # [1, 4, 9]

```

```
# Python - map trả về iterator (lazy)
squares = map(lambda x: x**2, [1, 2, 3]) # <map object>
squares_list = list(squares) # [1, 4, 9]

# List comprehension thường được ưa thích hơn
squares = [x**2 for x in [1, 2, 3]] # [1, 4, 9]
```

### 5.3.11.3 Hiệu năng

```
1 # Julia - map được compile thành native code
2 # Hiệu năng tương đương C loop
3 @time sum(map(x -> x^2, 1:10_000_000))
4 # ~0.05s
5
6 # Broadcasting còn nhanh hơn
7 @time sum((1:10_000_000) .^ 2)
8 # ~0.02s

# Python - map chậm hơn đáng kể
import time
start = time.time()
sum(map(lambda x: x**2, range(1, 10_000_001)))
print(time.time() - start)
# ~2.5s (chậm hơn 50-100x)

# NumPy nhanh hơn nhưng cần import
import numpy as np
np.sum(np.arange(1, 10_000_001)**2)
# ~0.1s
```

### 5.3.11.4 Tổng hợp so sánh

| Khía cạnh       | Julia | Python | Ghi chú                        |
|-----------------|-------|--------|--------------------------------|
| Hiệu năng       |       |        | Julia nhanh hơn 50-100x cho FP |
| Syntax ngắn gọn |       |        | Tương đương                    |
| Lazy by default |       |        | Python map/filter lazy         |
| Broadcasting    |       |        | Julia có cú pháp f. ()         |
| Ecosystem       |       |        | Python có nhiều libs hơn       |
| Learning curve  |       |        | Python dễ học hơn              |

#### Mẹo

##### Khi nào chọn Julia cho functional programming:

- Cần hiệu năng cao (scientific computing, ML)
- Làm việc với arrays lớn (broadcasting)
- Cần type stability và JIT compilation

##### Khi nào Python phù hợp hơn:



- Prototype nhanh, scripting
- Ecosystem libs phong phú (web, automation)
- Team đã quen Python

### 5.3.12 Best practices

- Dùng **mapreduce** thay vì `reduce(map(...))` để tối ưu hiệu năng
- Dùng **generator** cho dữ liệu lớn để tiết kiệm bộ nhớ
- Dùng **do blocks** khi lambda phức tạp, tăng tính dễ đọc
- **Kết hợp pipeline** `|>` cho functional chains dễ hiểu
- **Cẩn thận với reduce** trên toán tử không kết hợp (`-`, `/`)
- **Profile trước khi tối ưu** - đôi khi comprehension rõ ràng hơn generator

## 5.4 Tóm tắt chương

- **Multiple Dispatch:** Julia chọn method dựa trên kiểu của **TẤT CẢ** các đối số, không chỉ đối số đầu tiên như OOP truyền thống. Điều này cho phép thiết kế API linh hoạt và mở rộng.
- **Method Specialization:** Compiler tự động sinh mã tối ưu cho mỗi tổ hợp kiểu cụ thể, đạt hiệu năng tương đương C/Fortran trong khi giữ cú pháp động.
- **Generic Programming:** Kiểu tham số hóa (`Vector{T}`) kết hợp với `where` clauses cho phép viết code tổng quát với ràng buộc kiểu chặt chẽ.
- **Functional Programming:** Julia hỗ trợ đầy đủ lambda (`x -> x^2`), higher-order functions (`map`, `filter`, `reduce`), và function composition (`∘`).

## 5.5 Bài tập

### 5.5.1 Bài tập cơ bản (\*)

Bài tập: Multiple dispatch cơ bản

Định nghĩa function `mo_ta` với các methods khác nhau:

```
mo_ta(x::Int) = "Số nguyên: $x"
mo_ta(x::Float64) = "Số thực: $x"
mo_ta(x::String) = "Chuỗi: $x"
mo_ta(x::Vector) = "Vector có $(length(x)) phần tử"
```

# Test

```
mo_ta(42)           # => "Số nguyên: 42"
mo_ta(3.14)         # => "Số thực: 3.14"
mo_ta("Julia")      # => "Chuỗi: Julia"
```

Thêm methods cho: `Bool`, `Complex`, `Dict`

### Bài tập: Lambda functions cơ bản

Viết các lambda functions:

# 1. Bình phương

`binh_phuong = x -> x^2`

# 2. Kiểm tra số chẵn

`la_chan = x -> x % 2 == 0`

# 3. Tính tổng hai số

`tong = (a, b) -> a + b`

# Áp dụng với map

`map(binh_phuong, 1:5) # => [1, 4, 9, 16, 25]`

`filter(la_chan, 1:10) # => [2, 4, 6, 8, 10]`

### Bài tập: Map

Cho vector điểm sinh viên:

`diem = [7.5, 8.0, 6.5, 9.0, 5.5, 8.5, 7.0]`

Sử dụng map/filter/reduce để:

1. Chuyển điểm sang thang 4.0 (chia 2.5)
2. Lọc các điểm  $\geq 7.0$
3. Tính tổng và trung bình các điểm
4. Đếm số điểm  $\geq 8.0$

### Bài tập: Do block syntax

Viết lại các đoạn code sau dùng do block:

# Trước

`map(x -> x^2 + 2x + 1, 1:5)`

# Sau (dùng do block)

`map(1:5) do x`

`x^2 + 2x + 1`

`end`

# Bài tập: Viết lại với do block

`filter(x -> x > 0 && x % 2 == 0, -10:10)`

`reduce((a, b) -> a > b ? a : b, [3, 1, 4, 1, 5])`

**Bài tập: Function composition**

Sử dụng composition operator:

```
# Định nghĩa các functions
tang_gap_doi = x -> 2x
cong_mot = x -> x + 1
binh_phuong = x -> x^2

# Compose: f  g nghĩa là f(g(x))
h = binh_phuong  cong_mot  tang_gap_doi
h(3)  # => (2*3 + 1)^2 = 49

# Pipe operator: x |> f |> g
3 |> tang_gap_doi |> cong_mot |> binh_phuong

Tạo pipeline: đọc file -> lọc dòng -> đếm từ
```

**5.5.2 Bài tập trung bình (\*\*)****Bài tập: Multiple dispatch - Phép toán hình học**

Định nghĩa function `khoang_cach` với multiple dispatch:

```
struct Diem2D
    x::Float64
    y::Float64
end

struct Diem3D
    x::Float64
    y::Float64
    z::Float64
end

# Implement khoang_cach cho:
# - Hai Diem2D
# - Hai Diem3D
# - Diem2D và Diem3D (chuyển 2D thành 3D ở vị trí z=0)
```

### Bài tập: Higher-order functions - Decorator pattern

Viết higher-order function `voi_thoi_gian`:

```
function voi_thoi_gian(f)
  function wrapped(args...)
    t1 = time()
    result = f(args...)
    t2 = time()
    println("Thời gian: $(t2 - t1)s")
    return result
  end
end
```

# Ứng dụng

```
slow_sum = voi_thoi_gian(sum)
slow_sum(1:1000000)
```

Thêm decorator: `voi_log`, `voi_cache` (memoization)

### Bài tập: Generic programming với constraints

Viết generic function với type constraints:

```
# Tính tích chập
function tinh_tong(v::Vector{T}) where T <: Number
  reduce(+, v; init=zero(T))
end
```

# Test

```
tinh_tong([1, 2, 3])           # OK
tinh_tong([1.5, 2.5, 3.5])    # OK
tinh_tong(["a", "b"])         # Error!
```

# Bài tập: Viết `tinh_tich`, `tinh_tu` với constraints

## 5.5.3 Bài tập nâng cao (\*\*\*)

### Bài tập: Method ambiguity resolution

Giải quyết method ambiguity:

```
xu_ly(x::Int, y::Number) = "Int, Number"
xu_ly(x::Number, y::Int) = "Number, Int"
```

# Gọi `xu_ly(1, 2)` gây ambiguity!

# Thêm method để giải quyết:

```
xu_ly(x::Int, y::Int) = "Int, Int"
```

Tạo hệ thống dispatch cho: Matrix operations (add, multiply) với các type combinations

**Bài tập: Mini-project: Expression Evaluator với dispatch**

Xây dựng expression evaluator sử dụng multiple dispatch:

```
abstract type Expr end

struct Num <: Expr
    value::Float64
end

struct BinOp <: Expr
    op::Symbol
    left::Expr
    right::Expr
end

# Implement: eval_expr(e::Num), eval_expr(e::BinOp)
# Hỗ trợ: +, -, *, /

# Test: (3 + 4) * 2
expr = BinOp(:*, BinOp(:+, Num(3), Num(4)), Num(2))
eval_expr(expr) # => 14.0

Mở rộng: Thêm UnaryOp (-, sqrt), Variables
```

**Đọc thêm**

- Bezanson et al. (2017): "Julia: A Fresh Approach to Numerical Computing"
- Julia Documentation: Methods - <https://docs.julialang.org/en/v1/manual/methods/>
- Julia Documentation: Functions - <https://docs.julialang.org/en/v1/manual/functions/>



## Chương 6

# Gói và module

### Mục tiêu học tập

- Tạo và quản lý modules
- Sử dụng Package Manager (Pkg)
- Làm việc với environments cho reproducibility
- Publish packages lên Julia registry

Trong Julia, **modules** (mô-đun) và **packages** (gói) là hai khái niệm quan trọng để tổ chức và chia sẻ code. Chapter này giới thiệu cách tạo modules, quản lý packages, và làm việc với environments để đảm bảo tính tái tạo (reproducibility) của dự án.

### 6.1 Modules và tổ chức code

#### 6.1.1 Module là gì?

Module là một **namespace** (không gian tên) nhóm các functions, types và constants liên quan. Module cung cấp global scope riêng biệt, cho phép tái sử dụng tên mà không xung đột [5].

**Cú pháp định nghĩa module:**

```
1 module MyMath
2
3 # Định nghĩa functions
4 add(a, b) = a + b
5 multiply(a, b) = a * b
6 square(x) = x^2
7
8 end # module MyMath
```

Module MyMath tạo namespace riêng. Các hàm add, multiply, square thuộc namespace MyMath.

**Sử dụng module:**

```
1 # Truy cập với qualified name
2 MyMath.add(2, 3)           # 5
3 MyMath.multiply(4, 5)     # 20
4 MyMath.square(7)          # 49
```

### 6.1.2 Export và public API

Keyword `export` định nghĩa public API của module — các tên được `export` có thể dùng trực tiếp sau khi `using`:

```
1 module Statistics
2
3 export mean, median
4 # variance không được export
5
6 mean(xs) = sum(xs) / length(xs)
7 median(xs) = sort(xs)[div(length(xs), 2) + 1]
8 variance(xs) = sum((x - mean(xs))^2 for x in xs) / length(xs)
9
10 end
```

Sử dụng exported names:

```
1 using .Statistics
2
3 mean([1, 2, 3, 4, 5])           # 3.0 □ exported
4 median([1, 2, 3, 4, 5])         # 3 □ exported
5 # variance([1,2,3])           # □ Error: not exported
6
7 # Phải dùng qualified name
8 Statistics.variance([1,2,3]) # 2.0 □
```

**Best practice:** Chỉ export những functions/types thuộc public API. Các implementation details không nên export.

### 6.1.3 Local modules vs registered packages

Julia phân biệt hai loại modules:

**1. Local modules** — modules được định nghĩa trong file hiện tại hoặc `include()`:

```
1 # File: mymath.jl
2 module MyMath
3 export add
4 add(a, b) = a + b
5 end
6
7 # File: main.jl
8 include("mymath.jl") # Load file
9 using .MyMath         # □ Chú ý dấu chấm trước MyMath!
10 add(2, 3)             # 5
```

**Lưu ý:** Local modules phải có **dấu chấm (.) prefix** khi `using/.import`: `using .MyMath`. Dấu chấm báo cho Julia đây là local module, không phải registered package.

**2. Registered packages** — packages đã đăng ký trong Julia registry, cài đặt qua Pkg:

```
1 using DataFrames # Không có dấu chấm – registered package
2 using Plots       # Registered package
```



### 6.1.4 Submodules

Module có thể chứa submodules (module con) để tổ chức code phức tạp:

```

1 module MathLib
2
3 module Arithmetic
4 export add, subtract
5 add(a, b) = a + b
6 subtract(a, b) = a - b
7 end
8
9 module Geometry
10 export area_circle, area_rectangle
11 area_circle(r) =  $\pi$  * r^2
12 area_rectangle(w, h) = w * h
13 end
14
15 end # module MathLib

```

Sử dụng submodules:

```

1 using .MathLib.Arithmetic
2 add(10, 5) # 15
3
4 using .MathLib.Geometry
5 area_circle(3.0) # 28.27...

```

### 6.1.5 Relative imports trong submodules

Khi làm việc với submodules, Julia cung cấp cú pháp **relative imports** sử dụng dấu chấm để truy cập parent modules và sibling modules[5].

Cú pháp dấu chấm:

- `.Module` — module cùng cấp (sibling) hoặc local module
- `..Module` — module ở parent level
- `...Module` — module ở grandparent level

Ví dụ cấu trúc nested modules:

```

1 module TopLevel
2
3 module Utils
4 export helper
5 helper() = "I'm a helper"
6 end
7
8 module Core
9 # Import từ sibling module Utils
10 using ..Utils # Hai chấm: lên một cấp rồi xuống Utils
11 # Tương đương: using TopLevel.Utils
12
13 function main()

```

```

14     println(helper()) # Sử dụng helper từ Utils
15 end
16 end
17
18 module SubCore
19     module Deep
20         # Import từ grandparent's sibling
21         using ....Utils # Bốn chấm: lên 3 cấp rồi xuống Utils
22
23         function deep_function()
24             helper()
25         end
26     end
27 end
28
29 end # module TopLevel

```

Giải thích relative path:

| Từ module | Đến module | Cú pháp         |
|-----------|------------|-----------------|
| Core      | Utils      | using ..Utils   |
| Deep      | Utils      | using ....Utils |
| Deep      | Core       | using ...Core   |

Ví dụ thực tế: Package structure

```

1 # File: src/MyPackage.jl
2 module MyPackage
3
4     include("utils.jl") # Defines Utils submodule
5     include("core.jl") # Defines Core submodule
6     include("api.jl") # Defines API submodule
7
8 end
9
10 # File: src/core.jl
11 module Core
12     using ..Utils # Import từ sibling Utils
13     export process
14     process(x) = Utils.transform(x) |> Utils.validate
15 end
16
17 # File: src/api.jl
18 module API
19     using ..Core # Import từ sibling Core
20     using ..Utils # Import từ sibling Utils
21     export run_pipeline
22     run_pipeline(data) = Core.process(data)
23 end

```

### Mẹo

**Best practice:** Sử dụng relative imports trong package development để tránh hardcode package name. Điều này giúp rename package dễ dàng hơn.

### 6.1.6 Re-exporting từ modules khác

Đôi khi bạn muốn module A re-export symbols từ module B, để users chỉ cần using A mà có thể dùng được symbols từ B.

#### Pattern 1: Manual re-export

```

1 module InnerModule
2 export inner_function
3 inner_function() = "From inner"
4 end
5
6 module OuterModule
7 # Import và re-export
8 using .InnerModule
9 export inner_function # Re-export symbol từ InnerModule
10
11 # Thêm functions riêng
12 export outer_function
13 outer_function() = "From outer"
14 end
15
16 # Usage:
17 using .OuterModule
18 inner_function() # Works! (re-exported)
19 outer_function() # Works!
```

#### Pattern 2: Re-export với Reexport.jl

```

1 module DataTools
2
3 using Reexport
4 @reexport using Statistics # Re-export toàn bộ Statistics
5
6 end
7
8 # Usage:
9 using DataTools
10 mean([1,2,3]) # Works! (từ Statistics, re-exported)
```

#### Pattern 3: Facade module

```

1 # Tạo unified API từ nhiều submodules
2 module MyLibrary
3
4 include("parsing.jl") # module Parsing
5 include("processing.jl") # module Processing
6 include("output.jl") # module Output
7
8 # Re-export public API từ tất cả submodules
9 using .Parsing: parse_input, validate_input
10 using .Processing: process_data, transform
11 using .Output: write_output, format_result
12
13 export parse_input, validate_input
```

```
14 export process_data, transform
15 export write_output, format_result
16
17 end
18
19 # User chỉ cần:
20 using MyLibrary
21 # Có access đến tất cả public functions
```

#### Cảnh báo

**Cẩn thận với re-export:** Re-export quá nhiều symbols có thể gây namespace pollution. Chỉ re-export những symbols thực sự thuộc public API.

### 6.1.7 Module constants và globals pattern

Julia có quy tắc đặc biệt về constants và global variables trong modules.

#### 6.1.7.1 const declarations

const khai báo binding cố định — tên không thể reassign sang giá trị khác type:

```
1 module Config
2
3 # Compile-time constants
4 const VERSION = "1.0.0"
5 const MAX_THREADS = 8
6 const PI_APPROX = 3.14159
7
8 # Không thể reassign const
9 # VERSION = "2.0.0" # Error!
10
11 end
```

#### Tại sao dùng const?

- **Performance:** Julia compiler có thể inline const values
- **Type stability:** Compiler biết type không đổi
- **Clarity:** Đánh dấu rõ giá trị không nên thay đổi

#### 6.1.7.2 Mutable globals với Ref

Khi cần global state có thể thay đổi, dùng Ref thay vì plain variable:

```
1 module Counter
2
3 # BAD: Plain mutable global (type unstable)
4 count = 0
5
6 # GOOD: Ref wrapper (type stable)
7 const counter = Ref{Int}(0)
8
```

```

9  function increment()
10     counter[] += 1
11 end
12
13 function get_count()
14     return counter[]
15 end
16
17 function reset()
18     counter[] = 0
19 end
20
21 end

```

### Tại sao Ref?

```

1  # Plain global: type có thể thay đổi -> type unstable
2  global_var = 0          # Int
3  global_var = "hello"   # String -> không optimize được
4
5  # Ref: container type cố định -> type stable
6  const ref_var = Ref{Int}(0)
7  ref_var[] = 100        # Chỉ value thay đổi, type giữ nguyên

```

#### 6.1.7.3 Pattern: Configuration module

```

1  module AppConfig
2
3  # Immutable settings (compile-time)
4  const APP_NAME = "MyApp"
5  const VERSION = v"1.2.3"
6
7  # Mutable settings (runtime configurable)
8  const debug_mode = Ref{Bool}(false)
9  const log_level = Ref{Symbol}(:info)
10 const max_connections = Ref{Int}(100)
11
12 # Accessors
13 is_debug() = debug_mode[]
14 get_log_level() = log_level[]
15
16 # Mutators (có validation)
17 function set_debug(value::Bool)
18     debug_mode[] = value
19 end
20
21 function set_log_level(level::Symbol)
22     level in (:debug, :info, :warn, :error) ||
23         throw(ArgumentError("Invalid log level: $level"))
24     log_level[] = level
25 end
26
27 # Initialize từ environment

```

```
28 function __init__()
29     if haskey(ENV, "DEBUG")
30         debug_mode[] = ENV["DEBUG"] == "true"
31     end
32 end
33
34 end
```

#### 6.1.7.4 Thread-safe globals

Khi dùng multi-threading, cần đồng bộ hóa access đến globals:

```
1 module ThreadSafeCounter
2
3 using Base.Threads
4
5 const counter = Ref{Int}(0)
6 const lock = ReentrantLock()
7
8 function increment()
9     lock(lock) do
10         counter[] += 1
11     end
12 end
13
14 # Hoặc dùng Atomic cho simple counters
15 const atomic_counter = Atomic{Int}(0)
16
17 function atomic_increment()
18     atomic_add!(atomic_counter, 1)
19 end
20
21 end
```

#### 6.1.8 Import aliasing và selective imports

Julia cho phép đổi tên (alias) khi import để tránh xung đột hoặc dùng tên ngắn hơn.

##### 6.1.8.1 Aliasing với as keyword (Julia 1.6+)

```
1 # Alias module name
2 import DataFrames as DF
3 DF.DataFrame(a=[1,2,3])
4
5 # Alias function name
6 import Statistics: mean as avg, std as stdev
7 avg([1,2,3,4,5]) # 3.0
8 stdev([1,2,3,4,5]) # 1.58...
9
10 # Alias khi có xung đột
11 import Base: parse
12 import JSON: parse as json_parse
13
```

```

14 parse(Int, "42")           # Base.parse
15 json_parse("{\"a\":1}")    # JSON.parse

```

### 6.1.8.2 Selective import patterns

Import tất cả exported:

```

1 using Statistics           # Import mean, median, std, var, etc.

```

Import cụ thể:

```

1 using Statistics: mean, median # Chỉ import 2 functions

```

Import chỉ module name:

```

1 import Statistics          # Chỉ import tên Statistics
2 Statistics.mean([1,2,3])   # Phải dùng qualified name

```

Import để extend:

```

1 import Base: show, +, *   # Import để định nghĩa methods mới

```

### 6.1.8.3 Xử lý name conflicts

Khi hai modules export cùng tên:

```

1 module A
2 export process
3 process(x) = "A: $x"
4 end
5
6 module B
7 export process
8 process(x) = "B: $x"
9 end
10
11 # Conflict!
12 using .A
13 using .B # Warning: process được export bởi cả A và B
14
15 # Solutions:
16
17 # 1. Selective import với alias
18 using .A: process as process_a
19 using .B: process as process_b
20
21 # 2. Qualified access
22 import .A
23 import .B
24 A.process("test")
25 B.process("test")
26
27 # 3. Import only từ một module
28 using .A
29 import .B # B.process cần qualified name

```

## Mẹo

**Convention:** Với packages phổ biến, Julia community thường dùng abbreviations chuẩn:

- `import DataFrames as DF`
- `import LinearAlgebra as LA`
- `import Statistics as Stats`

### 6.1.9 Module introspection

Julia cung cấp các functions để inspect nội dung của modules tại runtime.

#### 6.1.9.1 Liệt kê names trong module

```

1 module Demo
2 export public_func
3 public_func() = "public"
4 private_func() = "private"
5 const MY_CONST = 42
6 end
7
8 # Xem exported names
9 names(Demo)  # [:Demo, :public_func]
10
11 # Xem tất cả names (kể cả private)
12 names(Demo, all=true)
13 # [:Demo, :MY_CONST, :eval, :include, :private_func, :public_func]
```

#### 6.1.9.2 Kiểm tra module membership

```

1 # Kiểm tra name có trong module không
2 :public_func in names(Demo)  # true
3 :private_func in names(Demo)  # false (không exported)
4 :private_func in names(Demo, all=true)  # true
5
6 # Lấy value từ module bằng tên
7 getfield(Demo, :MY_CONST)  # 42
8 getfield(Demo, :public_func)  # function
```

#### 6.1.9.3 parentmodule và module hierarchy

```

1 module Outer
2 module Inner
3 struct MyType end
4 end
5 end
6
7 # Tìm parent module
8 parentmodule(Outer.Inner)  # Outer
9 parentmodule(Outer)        # Main
10
11 # Tìm module chứa type/function
```



```

12 parentmodule(Outer.Inner.MyType) # Outer.Inner
13 parentmodule(sin)                 # Base
14
15 # Kiểm tra fullname
16 fullname(Outer.Inner) # (:Main, :Outer, :Inner)

```

#### 6.1.9.4 Module path và location

```

1 # Đường dẫn file định nghĩa module
2 pathof(DataFrames)
3 # "/home/user/.julia/packages/DataFrames/..."
4
5 # Package directory
6 pkgdir(DataFrames)
7 # "/home/user/.julia/packages/DataFrames/abc123"
8
9 # Version của package
10 using Pkg
11 Pkg.status("DataFrames")

```

## 6.2 Using vs Import

Julia cung cấp hai cách để đưa modules vào scope: `using` và `import`. Hiểu rõ sự khác biệt giúp viết code rõ ràng và tránh xung đột namespace.

### 6.2.1 using statement

`using ModuleName` đưa tất cả **exported names** vào current namespace, cho phép sử dụng trực tiếp:

```

1 module Colors
2 export red, green, blue
3 red() = "Red color"
4 green() = "Green color"
5 blue() = "Blue color"
6 yellow() = "Yellow color" # Not exported
7 end
8
9 using .Colors
10 red() # "Red color" □
11 green() # "Green color" □
12 blue() # "Blue color" □
13 # yellow() # □ Error: not defined (not exported)
14
15 # Non-exported names cần qualified access
16 Colors.yellow() # "Yellow color" □

```

**using với selective import:**

```

1 using .Colors: red, green # Chỉ import red và green
2 red() # □
3 green() # □
4 # blue() # □ Error: not imported

```

### 6.2.2 import statement

import ModuleName đưa module vào scope nhưng **không tự động import exported names**:

```
1 import .Colors
2 # red()           # □ Error: not defined
3 Colors.red()      # □ Must use qualified name
4 Colors.green()    # □
```

**import với specific names:**

```
1 import .Colors: red, green
2 red()           # □ Imported explicitly
3 green()         # □ Imported explicitly
```

### 6.2.3 Extending functions từ modules

Điểm khác biệt quan trọng: import cho phép **extend** (mở rộng) functions từ module, còn using thì không [5].

**Ví dụ: Extending Base.show**

```
1 struct Point
2     x::Float64
3     y::Float64
4 end
5
6 # □ ĐÚNG: import rồi extend
7 import Base: show
8 function show(io::IO, p::Point)
9     print(io, "Point($(p.x), $(p.y))")
10 end
11
12 # □ SAI: using không cho phép extend
13 # using Base: show
14 # function show(io::IO, p::Point) # Error!
15 #     ...
16 # end
```

**Ví dụ thực tế: Custom + operator**

```
1 struct Vector2D
2     x::Float64
3     y::Float64
4 end
5
6 # Extend Base.+ để cộng hai vectors
7 import Base: +
8 function +(v1::Vector2D, v2::Vector2D)
9     Vector2D(v1.x + v2.x, v1.y + v2.y)
10 end
11
12 v1 = Vector2D(1.0, 2.0)
13 v2 = Vector2D(3.0, 4.0)
14 v1 + v2 # Vector2D(4.0, 6.0)
```

### 6.2.4 Comparison table: using vs import

| Đặc điểm                     | using                      | import                |
|------------------------------|----------------------------|-----------------------|
| Đưa exported names vào scope | Tự động                    | Phải explicit         |
| Qualified access cần thiết?  | Không (cho exported names) | Có (mặc định)         |
| Cho phép extend functions?   | Không                      | Có                    |
| Selective import             | using Mod: f, g            | import Mod: f, g      |
| Use case chính               | Sử dụng functionality      | Mở rộng functionality |

#### Rule of thumb:

- Dùng `using` khi bạn **sử dụng** module (end-user code)
- Dùng `import` khi bạn **mở rộng** module (library/package development)

## 6.3 Module lifecycle và precompilation

Module trong Julia có lifecycle riêng với các giai đoạn quan trọng: định nghĩa, initialization, và precompilation. Hiểu rõ lifecycle giúp viết modules hiệu quả và tránh bugs khó debug.

### 6.3.1 Module initialization với `__init__()`

`__init__()` là function đặc biệt được Julia tự động gọi khi module được load lần đầu. Function này hữu ích cho các tác vụ chỉ nên thực hiện một lần khi runtime[5].

#### Cú pháp:

```

1 module MyModule
2
3 export greet
4
5 # Code ở đây chạy khi module được ĐỊNH NGHĨA (compile time)
6 const COMPILER_TIME = now()
7
8 # __init__ chạy khi module được LOAD (runtime)
9 function __init__()
10     global RUNTIME_VALUE = setup_something()
11     println("MyModule initialized at runtime!")
12 end
13
14 greet() = "Hello from MyModule!"
15
16 end # module

```

#### Phân biệt compile time vs runtime:

| Thời điểm           | Code thông thường        | <code>__init__()</code> |
|---------------------|--------------------------|-------------------------|
| Khi nào chạy?       | Precompilation (compile) | Load time (runtime)     |
| Chạy bao nhiêu lần? | Một lần khi compile      | Mỗi lần load module     |
| Có thể truy cập gì? | Static data, types       | Runtime resources       |

### 6.3.1.1 Use cases cho `__init__()`

#### 1. Khởi tạo global state từ environment:

```
1 module Config
2
3 const API_KEY = Ref{String}()
4
5 function __init__()
6     # Đọc từ environment variable tại runtime
7     API_KEY[] = get(ENV, "MY_API_KEY", "default_key")
8 end
9
10 get_api_key() = API_KEY[]
11
12 end
```

#### 2. Đăng ký callbacks/hooks:

```
1 module Logging
2
3 function __init__()
4     # Đăng ký atexit handler
5     atexit() do
6         println("Application shutting down...")
7         flush_logs()
8     end
9 end
10
11 flush_logs() = println("Logs flushed!")
12
13 end
```

#### 3. Khởi tạo C libraries:

```
1 module CWrapper
2
3 using Libdl
4
5 const libhandle = Ref{Ptr{Cvoid}}()
6
7 function __init__()
8     # Load shared library tại runtime
9     libhandle[] = Libdl.dlopen("mylib.so")
10 end
11
12 function cleanup()
13     Libdl.dlclose(libhandle[])
14 end
15
16 end
```

#### 4. Random number generator seeding:

```
1 module RandomUtils
2
```

```

3 using Random
4
5 function __init__()
6     # Seed RNG với current time (khác nhau mỗi lần load)
7     Random.seed!(time_ns())
8 end
9
10 random_id() = rand(1:1_000_000)
11
12 end

```

#### Cảnh báo

Những gì **KHÔNG** nên làm trong `__init__()`:

- Định nghĩa types/structs (phải ở top-level)
- Heavy computation (làm chậm load time)
- File I/O không cần thiết
- Network calls (có thể fail)

#### 6.3.1.2 `__init__()` và precompilation

Khi module được precompile, code trong `__init__()` **không** được chạy. Nó chỉ chạy khi module thực sự được load:

```

1 module Demo
2
3     # Chạy khi precompile
4     println("Compiling Demo...") # In ra một lần khi precompile
5
6     function __init__()
7         # Chạy mỗi lần load
8         println("Loading Demo...") # In ra mỗi lần using Demo
9     end
10
11 end
12
13 # Session 1: First time
14 using Demo
15 # Output: Compiling Demo... (nếu chưa precompile)
16 #           Loading Demo...
17
18 # Session 2: Second time (đã precompile)
19 using Demo
20 # Output: Loading Demo... (không có "Compiling...")

```

#### 6.3.2 Precompilation

**Precompilation** là quá trình Julia compile module thành native code và cache để load nhanh hơn trong các sessions sau[5].

### 6.3.2.1 Cách precompilation hoạt động

#### Workflow:

1. Lần đầu using MyPackage: Julia compile và tạo cache file (.ji)
2. Lần sau using MyPackage: Julia load từ cache (nhanh hơn nhiều)
3. Nếu source thay đổi: Julia recompile và update cache

#### Cache location:

```
1  # Cache files nằm trong
2  ~/.julia/compiled/v1.10/MyPackage/
3
4  # Ví dụ:
5  ~/.julia/compiled/v1.10/DataFrames/abc123.ji
```

### 6.3.2.2 Điều khiển precompilation

#### Bật precompilation (mặc định):

```
1  module MyModule
2  # Precompilation enabled by default (Julia 1.9+)
3
4  export my_function
5  my_function(x) = x^2
6
7  end
```

#### Tắt precompilation:

```
1  module MyModule
2
3  __precompile__(false) # Tắt precompilation
4
5  # Useful khi:
6  # - Module có side effects tại compile time
7  # - Module phụ thuộc vào runtime environment
8  # - Debug precompilation issues
9
10 end
```

### 6.3.2.3 Precompile statements (Julia 1.9+)

Julia 1.9+ hỗ trợ precompile statements để compile thêm methods vào cache:

```
1  module FastMath
2
3  export fast_sum
4
5  fast_sum(xs::Vector{Float64}) = sum(xs)
6  fast_sum(xs::Vector{Int64}) = sum(xs)
7
8  # Precompile cả hai methods
9  precompile(fast_sum, (Vector{Float64},))
```

```

10 precompile(fast_sum, (Vector{Int64},))
11
12 end

```

**Lợi ích:** Giảm time-to-first-execution (TTFX) — code chạy nhanh ngay từ lần gọi đầu tiên.

#### 6.3.2.4 PrecompileTools.jl

Package `PrecompileTools.jl` tự động hóa việc precompile:

```

1 module MyPackage
2
3 using PrecompileTools
4
5 export process_data
6
7 process_data(x::Float64) = x^2 + 2x + 1
8 process_data(x::Int) = x^2 + 2x + 1
9
10 # Automatically precompile common call patterns
11 @setup_workload begin
12     # Code here runs during precompilation
13     @compile_workload begin
14         process_data(1.0)
15         process_data(1)
16     end
17 end
18
19 end

```

**@setup\_workload vs @compile\_workload:**

- @setup\_workload: Setup code (không precompile)
- @compile\_workload: Code được precompile vào cache

#### 6.3.2.5 Troubleshooting precompilation

**Common issues:**

##### 1. Precompilation error:

```

1 # Error: Package X failed to precompile
2 # Solution: Clear cache
3 rm -rf ~/.julia/compiled/v1.10/X/

```

##### 2. Stale cache:

```

1 # Force recompilation
2 using Pkg
3 Pkg.precompile() # Recompile all packages

```

##### 3. Debug precompilation:

```

1 # Set environment variable
2 ENV["JULIA_DEBUG"] = "loading"
3 using MyPackage # Verbose output

```

### 6.3.3 Best practices cho module lifecycle

#### DO:

- Dùng `__init__()` cho runtime initialization (env vars, C libs)
- Dùng `const` cho compile-time constants
- Dùng `Ref` cho mutable globals được set trong `__init__`
- Precompile common code paths với `PrecompileTools.jl`

#### DON'T:

- Đừng định nghĩa types trong `__init__()`
- Đừng rely vào compile-time values cho runtime state
- Đừng tắt precompilation trừ khi thật sự cần thiết
- Đừng store paths/files trong `const` (có thể khác giữa systems)

#### Mẹo

**Rule of thumb:** Nếu giá trị cố định (số, string literal) → `const`. Nếu giá trị phụ thuộc runtime → `__init__()`.

## 6.4 Package structure

Package là tập hợp modules với cấu trúc thư mục chuẩn và metadata files. Hiểu package structure giúp tạo và maintain packages đúng cách.

### 6.4.1 Package filesystem layout

Một Julia package có cấu trúc thư mục chuẩn [4]:

```

1 MyPackage/
2 |— Project.toml           # Package metadata và dependencies
3 |— Manifest.toml         # Dependency resolution (auto-generated)
4 |— src/
5 |   |— MyPackage.jl      # Main module file (tên phải trùng package)
6 |   |— types.jl          # Type definitions
7 |   └— utils.jl          # Utility functions
8 |— test/
9 |   └— runtests.jl       # Test suite
10 |— docs/
11 |   └— make.jl           # Documentation builder
12 └— README.md             # Project documentation
```

**File quan trọng nhất: `src/MyPackage.jl`**

```

1 # File: src/MyPackage.jl
2 module MyPackage
3
4 # Export public API
```



```

5 export useful_function, ImportantType
6
7 # Include other source files
8 include("types.jl")
9 include("utils.jl")
10
11 # Main module code
12 useful_function(x) = x^2 + 2x + 1
13
14 end # module MyPackage

```

**Lưu ý:** Main module file **phải trùng tên** với package. Package `MyPackage` → file `src/MyPackage.jl` → module `MyPackage`.

### 6.4.2 Project.toml — Package metadata

`Project.toml` định nghĩa package metadata và **direct dependencies** (phụ thuộc trực tiếp) [4]:

```

1 # File: Project.toml
2 name = "MyPackage"
3 uuid = "12345678-1234-1234-1234-123456789abc" # Unique ID
4 authors = ["Your Name <you@example.com>"]
5 version = "0.1.0"
6
7 [deps]
8 DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
9 Plots = "91a5bcd-d55d7-5caf-9e0b-520d859cae80"
10
11 [compat]
12 julia = "1.6"
13 DataFrames = "1.3, 1.4"
14 Plots = "1"

```

#### Các sections quan trọng:

- `[deps]`: Direct dependencies với UUIDs
- `[compat]`: Version constraints cho Julia và packages
- `version`: Package version (semantic versioning)

**UUID (Universally Unique Identifier):** Mỗi package cần UUID duy nhất để phân biệt. Julia tự động sinh UUID khi tạo package mới.

### 6.4.3 File organization best practices

#### Tổ chức `src/` directory:

```

1 src/
2 └─ MyPackage.jl      # Main module + exports
3 └─ types.jl          # Type definitions (structs, abstract types)
4 └─ constants.jl      # Constants và configuration
5 └─ utils.jl          # Utility functions
6 └─ core.jl           # Core algorithms
7 └─ io.jl             # Input/output operations

```

**Thứ tự include files trong MyPackage.jl:**

```
1 module MyPackage
2
3 # 1. Types trước (vì other files phụ thuộc types)
4 include("types.jl")
5
6 # 2. Constants
7 include("constants.jl")
8
9 # 3. Utility functions
10 include("utils.jl")
11
12 # 4. Core functionality
13 include("core.jl")
14
15 # 5. I/O operations sau cùng
16 include("io.jl")
17
18 end
```

**Best practices:**

- Mỗi file < 300-500 dòng code
- Types và dependencies ở đầu
- Tránh circular dependencies giữa files
- Dùng include() một lần cho mỗi file

**6.4.4 PkgTemplates.jl — Tự động tạo package**

Thay vì tạo thủ công từng file, PkgTemplates.jl tự động sinh toàn bộ package structure với best practices được tích hợp sẵn [?].

**Cài đặt:**

```
1 using Pkg
2 Pkg.add("PkgTemplates")
```

**Tạo package cơ bản:**

```
1 using PkgTemplates
2
3 # Template đơn giản
4 t = Template()
5 t("MyAwesomePackage") # Tạo package trong ~/.julia/dev/
```

**Tạo package với full configuration:**

```
1 using PkgTemplates
2
3 t = Template(;
4     user = "username",           # GitHub username
5     authors = ["Your Name"],     # Authors
```

```

6     dir = "~/projects",           # Output directory
7     julia = v"1.6",              # Minimum Julia version
8     plugins = [
9         Git(;                     # Git initialization
10             manifest = true,      # Include Manifest.toml
11             ssh = true            # Use SSH for GitHub
12         ),
13         License(; name = "MIT"), # License file
14         Readme(;                 # README.md
15             inline_badges = true
16         ),
17         Tests(;                  # test/runtests.jl
18             project = true        # Separate test environment
19         ),
20         Documenter{GitHubActions}(), # Documentation setup
21         GitHubActions(;          # CI/CD workflows
22             x86 = false,         # Skip 32-bit
23             linux = true,
24             osx = false,
25             windows = false
26         ),
27         Codecov(),               # Code coverage
28         CompatHelper(),          # Auto-update compat
29         TagBot()                 # Auto-release tags
30     ]
31 )
32
33 t("MyAwesomePackage")

```

### Structure được tạo:

```

1  MyAwesomePackage/
2  +-- .github/
3  |   +-- workflows/
4  |       +-- CI.yml           # GitHub Actions CI
5  |       +-- CompatHelper.yml # Auto-update dependencies
6  |       +-- TagBot.yml       # Auto-release
7  +-- docs/
8  |   +-- make.jl              # Documentation builder
9  |   +-- Project.toml         # Docs dependencies
10 |   +-- src/
11 |       +-- index.md          # Documentation home
12 +-- src/
13 |   +-- MyAwesomePackage.jl  # Main module
14 +-- test/
15 |   +-- Project.toml         # Test dependencies
16 |   +-- runtests.jl          # Test suite
17 +-- .gitignore
18 +-- LICENSE
19 +-- Project.toml
20 +-- Manifest.toml
21 +-- README.md

```

**Ghi chú**

**Best practice:** Luôn dùng `PkgTemplates.jl` thay vì tạo thủ công. Template đảm bảo structure chuẩn và tích hợp sẵn CI/CD, documentation, testing.

**6.4.5 Semantic Versioning và Compat**

Julia packages sử dụng **Semantic Versioning** (SemVer) để quản lý version [?]:

MAJOR.MINOR.PATCH — Ví dụ: 1.3.2

**Quy tắc version:**

- MAJOR: Breaking changes (API thay đổi, không tương thích ngược)
- MINOR: New features (tính năng mới, tương thích ngược)
- PATCH: Bug fixes (sửa lỗi, không thay đổi API)

**Ví dụ version progression:**

```
1 0.1.0 # Initial development
2 0.2.0 # New feature added
3 0.2.1 # Bug fix
4 1.0.0 # First stable release (public API)
5 1.1.0 # New feature (backward compatible)
6 1.1.1 # Bug fix
7 2.0.0 # Breaking change (API change)
```

**[compat] section trong Project.toml:**

[compat] định nghĩa version constraints cho dependencies:

```
1 [compat]
2 julia = "1.6" # Julia >= 1.6.0, < 2.0.0
3 DataFrames = "1.3" # >= 1.3.0, < 2.0.0
4 CSV = "0.10" # >= 0.10.0, < 0.11.0
5 Plots = "1.25, 1.26, 1.27" # Chi 3 minor versions nay
6 HTTP = "0.9, 1" # 0.9.x hoặc 1.x
```

**Compat syntax:**

| Syntax  | Ý nghĩa                 | Ví dụ                |
|---------|-------------------------|----------------------|
| 1.2     | $\geq 1.2.0, < 2.0.0$   | 1.2.0, 1.3.5, 1.99.0 |
| 1.2.3   | $\geq 1.2.3, < 2.0.0$   | 1.2.3, 1.5.0, 1.99.0 |
| 1, 2    | Union: 1.x.x hoặc 2.x.x | OR condition         |
| = 1.2.3 | Exactly 1.2.3           | Pinned (avoid!)      |

**Cảnh báo**

**Warning:** Tránh dùng `= 1.2.3` (exact version). Nó ngăn dependency resolution và gây conflicts với packages khác.

**Update compat tự động với CompatHelper:**

```
1 # .github/workflows/CompatHelper.yml được tạo bởi PkgTemplates
2 # CompatHelper tự động tạo PRs khi dependencies có version mới
```

### 6.4.6 Documenter.jl — Documentation Generator

Documenter.jl là standard tool để tạo documentation cho Julia packages. Nó sinh static HTML từ Markdown và docstrings [?].

#### Setup documentation:

```

1 # docs/Project.toml
2 [deps]
3 Documenter = "e30172f5-a6a5-5a46-863b-614d45cd2de4"
4 MyPackage = "... " # Your package UUID
5
6 [compat]
7 Documenter = "1"
```

#### docs/make.jl — Build script:

```

1 using Documenter
2 using MyPackage
3
4 makedocs(;
5     modules = [MyPackage],
6     authors = "Your Name",
7     repo = "https://github.com/user/MyPackage.jl",
8     sitename = "MyPackage.jl",
9     format = Documenter.HTML(;
10         prettyurls = get(ENV, "CI", nothing) == "true",
11         canonical = "https://user.github.io/MyPackage.jl"
12     ),
13     pages = [
14         "Home" => "index.md",
15         "Getting Started" => "getting_started.md",
16         "API Reference" => "api.md"
17     ]
18 )
19
20 # Deploy to GitHub Pages
21 deploydocs(;
22     repo = "github.com/user/MyPackage.jl",
23     devbranch = "main"
24 )
```

#### Docstrings trong source code:

```

1 """
2     useful_function(x::Number) -> Number
3
4     Compute the square plus double plus one of `x`.
5
6     # Arguments
7     - `x::Number`: Input value
8
9     # Returns
10    - `Number`: Result of  $x^2 + 2x + 1$ 
11
```

```

12 # Examples
13 ```jldoctest
14 julia> useful_function(3)
15 16
16 ```
17
18 See also: [another_function](@ref)
19 """
20 function useful_function(x::Number)
21     return x^2 + 2x + 1
22 end

```

### Build documentation locally:

```

1 $ cd docs/
2 $ julia --project=. make.jl
3 # Output: docs/build/ (HTML files)

```

#### Mẹo

**Tip:** Dùng `LiveServer.jl` để preview documentation trong browser với auto-reload:

```

1 using LiveServer
2 serve(dir="docs/build")
3 # Mở http://localhost:8000

```

## 6.4.7 CI/CD với GitHub Actions

**CI (Continuous Integration)** tự động chạy tests mỗi khi push code. **CD (Continuous Deployment)** tự động deploy documentation và release packages.

**.github/workflows/CI.yml — Test workflow:**

```

1 name: CI
2 on:
3   push:
4     branches: [main]
5   pull_request:
6     branches: [main]
7
8 jobs:
9   test:
10     runs-on: ubuntu-latest
11     strategy:
12       matrix:
13         julia-version: ['1.6', '1.10', 'nightly']
14     steps:
15       - uses: actions/checkout@v4
16       - uses: julia-actions/setup-julia@v1
17       with:
18         version: ${{ matrix.julia-version }}
19       - uses: julia-actions/cache@v1
20       - uses: julia-actions/julia-buildpkg@v1
21       - uses: julia-actions/julia-runtest@v1
22       - uses: julia-actions/julia-processcoverage@v1

```

```

23     - uses: codecov/codecov-action@v3
24     with:
25         files: lcov.info

```

### Key actions:

- `julia-actions/setup-julia`: Cài Julia với version cụ thể
- `julia-actions/cache`: Cache compiled packages (faster CI)
- `julia-actions/julia-buildpkg`: Build package
- `julia-actions/julia-runtest`: Run `test/runtests.jl`
- `julia-actions/julia-processcoverage`: Generate coverage report

### `.github/workflows/TagBot.yml` — Auto-release:

```

1 name: TagBot
2 on:
3   issue_comment:
4     types: [created]
5   workflow_dispatch:
6
7 jobs:
8   TagBot:
9     runs-on: ubuntu-latest
10    steps:
11      - uses: JuliaRegistries/TagBot@v1
12      with:
13        token: ${{ secrets.GITHUB_TOKEN }}

```

### TagBot workflow:

1. Register package với Julia General Registry
2. Khi registration được merge, TagBot tự động:
  - Tạo Git tag (v1.0.0)
  - Tạo GitHub Release
  - Generate release notes từ commits

### Test locally trước khi push:

```

1 $ cd MyPackage/
2 $ julia --project=.
3 julia> using Pkg
4 julia> Pkg.test() # Runs test/runtests.jl

```

## 6.4.8 Publishing package to General Registry

Để publish package lên Julia General Registry (để users có thể `Pkg.add("YourPackage")`):

### Bước 1: Chuẩn bị package

- Package phải có `Project.toml` với `name`, `uuid`, `version`

- Phải có [compat] cho Julia và tất cả dependencies
- Code phải trên public GitHub repository
- Nên có tests và documentation

### Bước 2: Register với Registrator.jl

Comment trên GitHub issue hoặc commit:

```
1 @JuliaRegistrator register
```

Hoặc dùng web interface: <https://juliahub.com/ui/Registrator>

### Bước 3: Đợi review

- RegistryCI bot kiểm tra package
- AutoMerge tự động merge sau 3 ngày nếu pass
- Maintainers có thể merge sớm hơn

### Bước 4: Release new versions

```
1 # Update version trong Project.toml
2 version = "1.1.0"
3
4 # Commit và push
5 $ git add Project.toml
6 $ git commit -m "Bump version to 1.1.0"
7 $ git push
8
9 # Comment để register new version
10 @JuliaRegistrator register
```

#### Quan trọng

**Naming convention:** Julia packages nên có tên kết thúc bằng .jl trên GitHub (e.g., MyPackage.jl), nhưng package name trong Project.toml không có .jl (e.g., name = "MyPackage").

## 6.5 Quản lý môi trường (Environments)

Julia's environment system đảm bảo tính tái tạo (reproducibility) và quản lý dependencies chặt chẽ. Đây là một trong những điểm mạnh của Julia so với Python/R.

### 6.5.1 Environments là gì?

**Environment** là một tập hợp độc lập các packages cho một project. Mỗi project có environment riêng, tránh xung đột phiên bản giữa projects [4].

Environment được định nghĩa bởi hai files:

- Project.toml: Direct dependencies (dependencies trực tiếp)
- Manifest.toml: Full dependency graph (tất cả dependencies kể cả transitive)



### 6.5.2 Project.toml vs Manifest.toml

| Thuộc tính      | Project.toml                        | Manifest.toml                                  |
|-----------------|-------------------------------------|------------------------------------------------|
| Mục đích        | Khai báo direct dependencies        | Ghi lại exact versions resolved                |
| Nội dung        | Package names + version constraints | Full dependency graph + exact versions + UUIDs |
| Edit thủ công?  | Có (add/rm packages)                | Không (auto-generated)                         |
| Commit vào Git? | Luôn commit                         | Commit cho reproducibility                     |
| Human-readable? | Dễ đọc                              | Partially (auto-generated)                     |

#### Ví dụ Project.toml:

```

1 [deps]
2 DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
3 CSV = "336ed68f-0bac-5ca0-87d4-7b16caf5d00b"
4
5 [compat]
6 DataFrames = "1.3, 1.4"
7 CSV = "0.10"
```

#### Ví dụ Manifest.toml (excerpt):

```

1 [[DataFrames]]
2 deps = ["Compat", "DataAPI", "Future", ...]
3 git-tree-sha1 = "..."
4 uuid = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
5 version = "1.3.4"
6
7 [[DataAPI]]
8 git-tree-sha1 = "..."
9 uuid = "9a962f9c-6df0-11e9-0e5d-c546b8b5ee8a"
10 version = "1.10.0"
11 # ... tất cả transitive dependencies
```

Manifest.toml chứa **tất cả** dependencies (bao gồm dependencies của dependencies), đảm bảo exact reproducibility.

### 6.5.3 Working với environments

#### 1. Activate environment — Chuyển sang environment của project:

```

1 # Trong REPL, nhấn ] để vào Pkg mode
2 (@v1.10) pkg> activate .
3 Activating project at `./path/to/project`
4
5 (project) pkg> # Prompt thay đổi → đã activate
```

Hoặc trong code:

```

1 using Pkg
2 Pkg.activate(".") # Activate environment ở current directory
```

#### 2. Add packages — Thêm dependencies:

```
1 (project) pkg> add DataFrames CSV Plots
2   Resolving package versions...
3   Updating `/path/to/project/Project.toml`
4   + DataFrames v1.3.4
5   + CSV v0.10.4
6   + Plots v1.29.0
7   Updating `/path/to/project/Manifest.toml`
8   # ... full dependency resolution
```

### 3. Remove packages — Xóa dependencies:

```
1 (project) pkg> rm Plots
2   Updating `/path/to/project/Project.toml`
3   - Plots v1.29.0
4   Updating `/path/to/project/Manifest.toml`
5   # ... dependencies của Plots cũng bị xóa nếu không còn dùng
```

### 4. Instantiate environment — Cài đặt tất cả packages từ Manifest.toml:

```
1 (project) pkg> instantiate
2   # Downloads và installs tất cả packages theo exact versions
```

instantiate dùng khi clone project từ Git — cài đặt exact environment từ Manifest.toml.

### 5. Update packages — Cập nhật dependencies lên latest compatible versions:

```
1 (project) pkg> update
2   # Updates packages theo [compat] constraints
```

### 6. Status — Xem danh sách packages:

```
1 (project) pkg> status
2 Status `/path/to/project/Project.toml`
3 [336ed68f] CSV v0.10.4
4 [a93c6f00] DataFrames v1.3.4
```

## 6.5.4 Reproducibility workflow

### Workflow cho reproducible research/development:

#### Bước 1: Tạo project environment

```
1 $ cd my-project/
2 $ julia
3
4 julia> using Pkg
5 julia> Pkg.activate(".")
6   Activating new project at `/path/to/my-project`
7
8 (@my-project) pkg> add DataFrames CSV
```

#### Bước 2: Commit cả Project.toml và Manifest.toml vào Git

```
1 $ git add Project.toml Manifest.toml
2 $ git commit -m "Add DataFrames and CSV dependencies"
```

#### Bước 3: Collaborators clone và instantiate

```

1 $ git clone https://github.com/user/my-project.git
2 $ cd my-project/
3 $ julia
4
5 julia> using Pkg
6 julia> Pkg.activate(".")
7 julia> Pkg.instantiate() # Installs exact versions from Manifest.toml

```

**Kết quả:** Tất cả collaborators có **exact same environment** → reproducible results.

### 6.5.5 Global environment vs Project environments

Julia có **global environment** mặc định (@v1.10) nhưng **không nên dùng** cho projects:

**Anti-pattern: Dùng global environment**

```

1 (@v1.10) pkg> add DataFrames Plots # ❌ Add vào global
2 # Problem: Tất cả projects dùng chung → version conflicts!

```

**Best practice: Project-specific environments**

```

1 (@v1.10) pkg> activate .
2 (my-project) pkg> add DataFrames Plots # ❌ Add vào project environment
3 # Each project độc lập → no conflicts!

```

### 6.5.6 Stacked Environments và LOAD\_PATH

Julia cho phép **stacking** (xếp chồng) nhiều environments, tạo nên một hệ thống linh hoạt để quản lý packages ở nhiều cấp độ [4].

**LOAD\_PATH** — Vector xác định thứ tự tìm kiếm packages:

```

1 julia> LOAD_PATH
2 3-element Vector{String}:
3  "@"           # Current active environment
4  "@v#.#"       # Default environment (@v1.10)
5  "@stdlib"     # Julia standard library

```

**Ý nghĩa các entries:**

- "@" — Environment đang active (project hiện tại)
- "@v#.#" — Global environment cho Julia version (ví dụ: @v1.10)
- "@stdlib" — Standard library (luôn available)

**Xem paths thực tế:**

```

1 julia> Base.load_path()
2 3-element Vector{String}:
3  "/home/user/myproject/Project.toml"
4  "/home/user/.julia/environments/v1.10/Project.toml"
5  "/home/user/julia-1.10/share/julia/stdlib/v1.10"

```

**Stacking workflow** — Khi using Package:

1. Julia tìm trong **active project** trước
2. Nếu không có, tìm trong **global environment**
3. Cuối cùng, tìm trong **stdlib**

**Mẹo**

**Tip:** Dùng stacking để có **development tools** (Revise, OhMyREPL) trong global environment mà không pollute project environments.

**Ví dụ: Setup global với development tools**

```

1 # Trong default environment
2 (@v1.10) pkg> add Revise OhMyREPL BenchmarkTools
3
4 # Trong project environment
5 (myproject) pkg> add DataFrames CSV
6 # Revise vẫn available từ global environment!
```

**Custom LOAD\_PATH:**

```

1 # Thêm shared environment
2 push!(LOAD_PATH, "/path/to/shared/Project.toml")
3
4 # Hoặc trong startup.jl
5 # ~/.julia/config/startup.jl
6 push!(LOAD_PATH, "@team") # Symbolic entry
```

**6.5.7 Manifest.toml Deep Dive**

Manifest.toml là **exact snapshot** của toàn bộ dependency graph, đảm bảo reproducibility hoàn toàn [4].

**Cấu trúc Manifest.toml:**

```

1 # File header - Metadata
2 julia_version = "1.10.0"
3 manifest_format = "2.0"
4 project_hash = "abc123..."
5
6 # Dependencies section
7 [[deps.DataFrames]]
8 deps = ["Compat", "DataAPI", "InvertedIndices", "IteratorInterfaceExtensions",
9         "Missings", "PooledArrays", "PrecompileTools", "PrettyTables",
10        "Printf", "Reexport", "SortingAlgorithms", "Statistics",
11        "TableTraits", "Tables", "Unicode"]
12 git-tree-sha1 = "04c738083f29f86e62c8afc341f0967d8717bdb8"
13 uuid = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
14 version = "1.6.1"
15
16 [[deps.Compat]]
17 deps = ["Base64", "Dates", "DelimitedFiles", "Distributed",
18        ↪ "InteractiveUtils",
19        "LibGit2", "Libdl", "LinearAlgebra", "Markdown", "Mmap", "Pkg",
20        "Printf", "REPL", "Random", "SHA", "Serialization", "SharedArrays",
21        "Sockets", "SparseArrays", "Statistics", "Test", "UUIDs", "Unicode"]
22 git-tree-sha1 = "c99c7b8d3b09d1a4e5f8b7c8c94b8c95a7e8a0f6"
23 uuid = "34da2185-b29b-5c13-b0c7-acf172513d20"
24 version = "4.10.0"
```

**Các fields quan trọng:**

- `julia_version`: Julia version khi resolve dependencies
- `manifest_format`: Format version của Manifest
- `deps`: Danh sách dependencies của package
- `git-tree-sha1`: SHA hash của Git tree (unique identifier)
- `uuid`: Universally Unique Identifier của package
- `version`: Exact version đã resolved

**Khi nào cần regenerate Manifest?**

```

1 # Delete và regenerate từ Project.toml
2 (project) pkg> rm Manifest.toml
3 (project) pkg> resolve # Hoặc instantiate
4
5 # Hoặc update tất cả
6 (project) pkg> update

```

**Cảnh báo**

**Cảnh báo:** Deleting Manifest.toml có thể dẫn đến **khác version** packages! Chỉ làm khi cần thiết.

**6.5.8 Advanced Reproducibility Practices**

Đảm bảo **reproducibility** là quan trọng cho scientific computing và production systems.

**1. Compat bounds trong Project.toml**

Luôn specify `[compat]` để giới hạn compatible versions:

```

1 [deps]
2 DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
3 CSV = "336ed68f-0bac-5ca0-87d4-7b16caf5d00b"
4
5 [compat]
6 DataFrames = "1.3, 1.4, 1.5, 1.6" # Chấp nhận versions 1.3.x - 1.6.x
7 CSV = "0.10" # Chấp nhận versions 0.10.x
8 julia = "1.8" # Minimum Julia version

```

**Compat syntax:**

- `"1.2"` — `>=1.2.0, <2.0.0` (semver)
- `"1.2.3"` — `>=1.2.3, <1.3.0`
- `"1.2, 1.3, 1.4"` — Explicit list of compatible minor versions
- `">=1.2, <2.0"` — Explicit range

**2. Pin specific versions khi cần**

```
1 # Pin package to exact version
2 (project) pkg> pin DataFrames@1.3.4
3   Pinning DataFrames@1.3.4
4
5 # Unpin để cho phép updates
6 (project) pkg> free DataFrames
7   Freeing DataFrames
```

### 3. Seed random cho reproducibility

```
1 # Luôn set seed cho random operations
2 using Random
3 Random.seed!(42)
4
5 # Document Julia version và key package versions
6 println("Julia $(VERSION)")
7 using Pkg
8 println("DataFrames $(Pkg.installed()["DataFrames"])")
```

### 4. Docker cho complete reproducibility

```
# Dockerfile
FROM julia:1.10

WORKDIR /app
COPY Project.toml Manifest.toml ./

RUN julia -e 'using Pkg; Pkg.activate("."); Pkg.instantiate()'

COPY . .
CMD ["julia", "main.jl"]
```

### 5. CI/CD với matrix testing

```
# .github/workflows/CI.yml
jobs:
  test:
    strategy:
      matrix:
        julia-version: ['1.8', '1.9', '1.10']
        os: [ubuntu-latest, macos-latest, windows-latest]
    steps:
      - uses: julia-actions/setup-julia@v1
        with:
          version: ${matrix.julia-version}
      - uses: julia-actions/julia-buildpkg@v1
      - uses: julia-actions/julia-runttest@v1
```

## 6.5.9 Troubleshooting Environments

### Problem 1: Package not found sau khi activate

```

1 # Check current environment
2 julia> Base.active_project()
3 "/home/user/myproject/Project.toml"
4
5 # Verify package is in environment
6 (myproject) pkg> status
7 # Nếu package không có -> add lại
8 (myproject) pkg> add MissingPackage

```

### Problem 2: Version conflict

```

1 # Xem dependency tree
2 (project) pkg> status --manifest
3
4 # Resolve conflicts
5 (project) pkg> update
6 # Hoặc adjust [compat] bounds trong Project.toml

```

### Problem 3: Precompilation failed

```

1 # Clear precompile cache
2 (project) pkg> precompile
3 # Hoặc restart Julia và thử lại
4
5 # Nuclear option: Delete compiled cache
6 # rm -rf ~/.julia/compiled/v1.10/PackageName/

```

### Problem 4: Manifest out of sync

```

1 # Khi clone project mới, luôn instantiate
2 (project) pkg> instantiate
3
4 # Nếu vẫn lỗi, resolve lại
5 (project) pkg> resolve

```

#### Quan trọng

**Golden Rule:** Luôn commit cả `Project.toml` VÀ `Manifest.toml` vào Git. `Project.toml` cho **flexibility**, `Manifest.toml` cho **exact reproducibility**.

## 6.6 Tạo package với PkgTemplates

`PkgTemplates.jl` tự động hóa việc tạo package với cấu trúc chuẩn, CI/CD setup, và documentation [4].

### 6.6.1 PkgTemplates workflow

#### Bước 1: Cài đặt PkgTemplates

```

1 using Pkg
2 Pkg.add("PkgTemplates")

```

#### Bước 2: Tạo template với options

```
1 using PkgTemplates
2
3 t = Template(;
4     user="YourGitHubUsername",
5     authors="Your Name <you@example.com>",
6     julia=v"1.6",           # Minimum Julia version
7     plugins=[
8         Git(; manifest=true),      # Git initialization
9         GitHubActions(),          # CI với GitHub Actions
10        Documenter{GitHubActions}(), # Documentation
11    ]
12 )
```

### Bước 3: Generate package

```
1 t("MyNewPackage")
```

**Kết quả:** Package được tạo tại ~/.julia/dev/MyNewPackage/:

```
1 MyNewPackage/
2 |— .git/                # Git repository initialized
3 |— .github/
4 |   |— workflows/
5 |       |— CI.yml        # GitHub Actions CI
6 |       |— TagBot.yml    # Automatic release tagging
7 |— docs/
8 |   |— make.jl          # Documentation builder
9 |   |— src/
10 |       |— index.md     # Documentation homepage
11 |— src/
12 |   |— MyNewPackage.jl  # Main module
13 |— test/
14 |   |— runtests.jl      # Test suite
15 |— Project.toml         # Package metadata
16 |— README.md            # Auto-generated README
17 |— LICENSE              # License file
```

#### 6.6.2 Generated package structure

**File `src/MyNewPackage.jl`:**

```
1 module MyNewPackage
2
3 greet() = print("Hello World!")
4
5 end # module MyNewPackage
```

**File `test/runtests.jl`:**

```
1 using MyNewPackage
2 using Test
3
4 @testset "MyNewPackage.jl" begin
5     # Write your tests here.
6 end
```



**File Project.toml:**

```

1 name = "MyNewPackage"
2 uuid = "...
3 authors = ["Your Name <you@example.com>"]
4 version = "0.1.0"
5
6 [compat]
7 julia = "1.6"
8
9 [extras]
10 Test = "8dfed614-e22c-5e08-85e1-65c5234f0b40"
11
12 [targets]
13 test = ["Test"]

```

**6.6.3 Develop package locally****Add package vào environment để develop:**

```

1 (@v1.10) pkg> develop ~/.julia/dev/MyNewPackage
2 # Or shorter:
3 (@v1.10) pkg> dev MyNewPackage

```

**Test package:**

```

1 (MyNewPackage) pkg> test
2     Testing MyNewPackage
3     Status `~/tmp/...`
4     Testing Running tests...
5 Test Summary:      | Pass  Total
6 MyNewPackage.jl  |     0     0
7     Testing MyNewPackage tests passed

```

**Build documentation:**

```

1 $ cd ~/.julia/dev/MyNewPackage/docs/
2 $ julia make.jl
3 # Generates HTML documentation in docs/build/

```

**6.7 Semantic Versioning và Compatibility**

Julia packages tuân theo **Semantic Versioning (SemVer)** để quản lý phiên bản và đảm bảo tương thích ngược<sup>[4]</sup>.

**6.7.1 Semantic Versioning basics**

**Format:** MAJOR.MINOR.PATCH

| Component | Khi nào tăng                             | Ví dụ         |
|-----------|------------------------------------------|---------------|
| MAJOR     | Breaking changes (API không tương thích) | 1.0.0 → 2.0.0 |
| MINOR     | New features (backward compatible)       | 1.0.0 → 1.1.0 |
| PATCH     | Bug fixes (no API changes)               | 1.0.0 → 1.0.1 |

**Ví dụ thực tế:**

```

1 # Package MyMath v1.2.3
2 # MAJOR=1: API ổn định kể từ v1.0
3 # MINOR=2: 2 features mới kể từ v1.0
4 # PATCH=3: 3 bug fixes kể từ v1.2.0

```

**6.7.2 0.x versions (Pre-release)**

Packages với version 0.x.y được coi là **pre-release** — API có thể thay đổi bất cứ lúc nào:

```

1 version = "0.1.0" # Pre-release: API không ổn định
2 version = "0.2.0" # Có thể có breaking changes từ 0.1.0
3 version = "1.0.0" # Stable release: API commitment bắt đầu

```

**Rule of thumb:**

- 0.x.y: Development phase, breaking changes allowed
- 1.0.0+: Stable phase, follow SemVer strictly

**6.7.3 [compat] section chi tiết**

[compat] trong Project.toml định nghĩa version constraints cho dependencies:

```

1 [compat]
2 julia = "1.6" # Julia >= 1.6.0, < 2.0.0
3 DataFrames = "1.3" # DataFrames >= 1.3.0, < 2.0.0
4 CSV = "0.10" # CSV >= 0.10.0, < 0.11.0 (0.x rule)
5 Plots = "1.25, 1.26" # Plots 1.25.x OR 1.26.x
6 HTTP = "0.9, 1" # HTTP 0.9.x OR 1.x.x

```

**Compat syntax patterns:**

| Syntax     | Meaning                                           |
|------------|---------------------------------------------------|
| "1.3"      | $\geq 1.3.0, < 2.0.0$                             |
| "1.3.2"    | $\geq 1.3.2, < 2.0.0$                             |
| "0.10"     | $\geq 0.10.0, < 0.11.0$ (0.x special rule)        |
| "1.3, 1.4" | 1.3.x OR 1.4.x (multiple ranges)                  |
| "^1.3"     | Caret: $\geq 1.3.0, < 2.0.0$ (same as "1.3")      |
| "~1.3"     | Tilde: $\geq 1.3.0, < 1.4.0$ (patch updates only) |

**0.x special rule:** Với versions  $< 1.0$ , MINOR được coi như MAJOR:

```

1 [compat]
2 Package = "0.10" # >= 0.10.0, < 0.11.0 (NOT < 1.0.0!)
3 Package = "0.10.2" # >= 0.10.2, < 0.11.0

```

**6.7.4 Version bounds best practices****DO:**

- Specify [compat] cho **tất cả** direct dependencies
- Specify Julia version: `julia = "1.6"` (minimum supported)

- Test với cả lower và upper bounds của constraints
- Update [compat] khi thêm features cần newer versions

#### DON'T:

- Đừng dùng exact versions: "`= 1.3.2`" (quá restrictive)
- Đừng bỏ qua [compat] — gây conflicts cho users
- Đừng set bounds quá rộng mà không test

#### Cảnh báo

**Package registration yêu cầu [compat]:** Julia General registry từ chối packages không có [compat] entries cho tất cả dependencies.

### 6.7.5 Breaking changes và deprecation

Khi cần thay đổi API, tuân theo deprecation workflow:

#### Bước 1: Deprecate old API (v1.x)

```

1 # Old function - deprecate but keep working
2 function old_api(x)
3     Base.depwarn("`old_api(x)` is deprecated, use `new_api(x)` instead",
4         ↳ :old_api)
5     return new_api(x)
6 end
7
8 # New function
9 function new_api(x)
10     # New implementation
11 end

```

#### Bước 2: Remove deprecated API (v2.0)

```

1 # Version 2.0.0 - remove old_api completely
2 # function old_api(x) ... end # REMOVED
3
4 function new_api(x)
5     # Only new API exists
6 end

```

#### Deprecation timeline:

- v1.4.0: Add new\_api, deprecate old\_api
- v1.5.0, v1.6.0: Keep both (users migrate)
- v2.0.0: Remove old\_api (breaking change)

## 6.8 Documentation với Documenter.jl

Documenter.jl là standard tool để build documentation cho Julia packages<sup>[4]</sup>. Nó generates HTML documentation từ docstrings và Markdown files.

### 6.8.1 Setup Documenter.jl

#### Bước 1: Add Documenter as dev dependency

```
1 (MyPackage) pkg> add Documenter --build
2 # Hoặc add vào [extras] trong Project.toml
```

#### Bước 2: Tạo docs structure

```
1 MyPackage/
2 |— docs/
3 |   |— make.jl           # Build script
4 |   |— Project.toml      # Docs-specific dependencies
5 |   └─ src/
6 |       |— index.md      # Homepage
7 |       |— tutorial.md   # Tutorial
8 |       └─ api.md        # API reference
9 |— src/
10 |   └─ MyPackage.jl
11 └─ Project.toml
```

#### Bước 3: Create docs/make.jl

```
1 # File: docs/make.jl
2 using Documenter
3 using MyPackage
4
5 makedocs(
6     sitename = "MyPackage.jl",
7     format = Documenter.HTML(
8         prettyurls = get(ENV, "CI", nothing) == "true"
9     ),
10    modules = [MyPackage],
11    pages = [
12        "Home" => "index.md",
13        "Tutorial" => "tutorial.md",
14        "API Reference" => "api.md",
15    ]
16 )
17
18 # Deploy to GitHub Pages (optional)
19 deploydocs(
20     repo = "github.com/username/MyPackage.jl.git",
21 )
```

### 6.8.2 Writing documentation

#### docs/src/index.md — Homepage:

```
1 # MyPackage.jl
2
3 Welcome to MyPackage.jl documentation!
4
5 ## Installation
6
```

```

7  ```julia
8  using Pkg
9  Pkg.add("MyPackage")
10  ```
11
12  ## Quick Start
13
14  ```julia
15  using MyPackage
16  result = useful_function(42)
17  ```
18
19  ## Features
20
21  - Feature 1: Description
22  - Feature 2: Description

```

#### docs/src/api.md — API Reference:

```

1  # API Reference
2
3  ## Public API
4
5  ```@docs
6  useful_function
7  ImportantType
8  ```
9
10 ## Index
11
12 ```@index
13 ```

```

### 6.8.3 Docstrings for Documenter

Documenter extracts docstrings từ source code. Write comprehensive docstrings:

```

1  """
2      compute_stats(data::Vector{<:Number}; normalize=false)
3
4  Compute statistical summary of numeric data.
5
6  # Arguments
7  - `data::Vector{<:Number}`: Input data vector
8  - `normalize::Bool=false`: If true, normalize results to [0,1]
9
10 # Returns
11 - `NamedTuple` with fields:
12   - `mean`: Arithmetic mean
13   - `std`: Standard deviation
14   - `min`, `max`: Minimum and maximum values
15
16 # Examples

```

```
17  ```jldoctest
18  julia> stats = compute_stats([1, 2, 3, 4, 5])
19  (mean = 3.0, std = 1.5811388300841898, min = 1, max = 5)
20
21  julia> stats.mean
22  3.0
23  ```
24
25  # See also
26  - [`mean`](@ref): Compute mean only
27  - [`std`](@ref): Compute standard deviation only
28  """
29  function compute_stats(data::Vector{<:Number}; normalize=false)
30      m = mean(data)
31      s = std(data)
32      mn, mx = extrema(data)
33
34      if normalize
35          m, s, mn, mx = (m - mn) / (mx - mn), s / (mx - mn), 0.0, 1.0
36      end
37
38      return (mean=m, std=s, min=mn, max=mx)
39  end
```

#### Docstring sections:

- **Signature:** Function signature với types
- **Description:** One-line summary
- **Arguments:** Parameters với types và descriptions
- **Returns:** Return value description
- **Examples:** jldoctest blocks (tested by Documenter)
- **See also:** Links to related functions

#### 6.8.4 Building documentation locally

```
1  # Terminal
2  $ cd MyPackage/docs/
3  $ julia --project=. make.jl
4
5  # Output in docs/build/
6  $ open build/index.html # macOS
7  $ xdg-open build/index.html # Linux
```

#### docs/Project.toml:

```
1  [deps]
2  Documenter = "e30172f5-a6a5-5a46-863b-614d45cd2de4"
3  MyPackage = "... " # UUID của package
4
5  [compat]
6  Documenter = "1"
```

### 6.8.5 Doctest — Testing documentation examples

jldoctest blocks trong docstrings được **tự động test** bởi Documenter:

```

1  """
2      add(a, b)
3
4  Add two numbers.
5
6  # Examples
7  ```jldoctest
8  julia> add(2, 3)
9  5
10
11 julia> add(-1, 1)
12 0
13 ```
14 """
15 add(a, b) = a + b

```

#### Run doctests:

```

1 # In docs/make.jl
2 makedocs (
3     ...,
4     doctest = true, # Enable doctest (default)
5 )

```

#### Mẹo

**Doctests = Living documentation:** Examples trong docstrings được verify mỗi lần build docs. Nếu code thay đổi mà docs không update → build fails.

## 6.9 CI/CD với GitHub Actions

Continuous Integration (CI) tự động chạy tests và build docs mỗi khi push code. Julia ecosystem sử dụng **GitHub Actions** làm standard CI platform.

### 6.9.1 Basic CI workflow

PkgTemplates.jl tự động tạo CI workflow. File .github/workflows/CI.yml:

```

1 # File: .github/workflows/CI.yml
2 name: CI
3 on:
4   push:
5     branches:
6       - main
7     tags: ['*']
8   pull_request:
9
10 jobs:
11   test:

```

```

12   name: Julia ${{ matrix.version }} - ${{ matrix.os }}
13   runs-on: ${{ matrix.os }}
14   strategy:
15     matrix:
16       version:
17         - '1.6'      # LTS
18         - '1'        # Latest stable
19         - 'nightly'
20       os:
21         - ubuntu-latest
22         - windows-latest
23         - macos-latest
24   steps:
25     - uses: actions/checkout@v4
26     - uses: julia-actions/setup-julia@v1
27       with:
28         version: ${{ matrix.version }}
29     - uses: julia-actions/cache@v1
30     - uses: julia-actions/julia-buildpkg@v1
31     - uses: julia-actions/julia-runtest@v1

```

### Workflow giải thích:

- **Trigger:** Push to main, tags, hoặc pull requests
- **Matrix:** Test trên multiple Julia versions × OS combinations
- **Steps:** Checkout → Setup Julia → Cache → Build → Test

## 6.9.2 Documentation deployment

Thêm job để build và deploy docs lên GitHub Pages:

```

1  # Thêm vào .github/workflows/CI.yml
2  docs:
3    name: Documentation
4    runs-on: ubuntu-latest
5    steps:
6      - uses: actions/checkout@v4
7      - uses: julia-actions/setup-julia@v1
8        with:
9          version: '1'
10     - name: Install dependencies
11       run: |
12         julia --project=docs -e '
13           using Pkg
14           Pkg.develop(PackageSpec(path=pwd()))
15           Pkg.instantiate()'
16     - name: Build and deploy
17       env:
18         GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
19         DOCUMENTER_KEY: ${{ secrets.DOCUMENTER_KEY }}
20       run: julia --project=docs docs/make.jl

```



**Setup DOCUMENTER\_KEY:**

```

1 # Generate SSH key for Documenter
2 $ julia -e 'using DocumenterTools; DocumenterTools.genkeys()'
3 # Copy public key to GitHub repo → Settings → Deploy keys
4 # Copy private key to GitHub repo → Settings → Secrets → DOCUMENTER_KEY

```

**6.9.3 Code coverage**

Thêm code coverage reporting với Codecov:

```

1 # Thêm vào CI.yml sau julia-runtest
2     - uses: julia-actions/julia-processcoverage@v1
3     - uses: codecov/codecov-action@v3
4     with:
5         files: lcov.info

```

**Coverage badge trong README:**

```

1 # README.md
2 [[codecov]](https://codecov.io/gh/username/MyPackage.jl/branch/main/graph/badge.svg)

```

**6.9.4 TagBot — Automatic releases**

TagBot tự động tạo GitHub releases khi package version được register:

```

1 # File: .github/workflows/TagBot.yml
2 name: TagBot
3 on:
4   issue_comment:
5     types:
6       - created
7   workflow_dispatch:
8 jobs:
9   TagBot:
10    if: github.event_name == 'workflow_dispatch' || github.actor ==
11        ↪ 'JuliaTagBot'
12    runs-on: ubuntu-latest
13    steps:
14      - uses: JuliaRegistries/TagBot@v1
15      with:
16        token: ${ secrets.GITHUB_TOKEN }

```

**Workflow:**

1. Developer bumps version trong `Project.toml`
2. Register new version với General registry
3. JuliaTagBot comments on PR
4. TagBot workflow tạo GitHub release với changelog

### 6.9.5 CompatHelper — Automatic compat updates

CompatHelper tự động update [compat] bounds khi dependencies release new versions:

```
1 # File: .github/workflows/CompatHelper.yml
2 name: CompatHelper
3 on:
4   schedule:
5     - cron: '0 0 * * *' # Daily at midnight
6   workflow_dispatch:
7 jobs:
8   CompatHelper:
9     runs-on: ubuntu-latest
10    steps:
11      - uses: julia-actions/CompatHelper@v1
12        with:
13          token: ${ secrets.GITHUB_TOKEN }
```

**Khi dependency (e.g., DataFrames) releases v1.5.0:**

1. CompatHelper detects new version
2. Creates PR updating [compat] từ "1.3, 1.4" → "1.3, 1.4, 1.5"
3. CI runs tests với new version
4. Developer reviews và merges PR

## 6.10 Publishing Package to General Registry

Julia General registry là official package registry. Publishing cho phép users cài đặt package với `Pkg.add("YourPackage")`.

### 6.10.1 Pre-registration checklist

Trước khi register package, đảm bảo:

- ☐ `Project.toml` có đầy đủ: name, uuid, version, authors
- ☐ [compat] entries cho **tất cả** dependencies
- ☐ [compat] `julia = "1.x"` specified
- ☐ Tests pass trên CI (multiple Julia versions)
- ☐ Documentation available (README hoặc Documenter.jl)
- ☐ LICENSE file present
- ☐ Package name follows conventions (UpperCamelCase, no "Julia" suffix)

## 6.10.2 Registrator workflow

### Bước 1: Install Registrator GitHub App

- Go to: <https://github.com/apps/julia-registrator>
- Install for your repository

### Bước 2: Trigger registration

Option A — Comment trên commit:

```
1 # Trên GitHub, comment trên commit muốn register:
2 @JuliaRegistrator register
```

Option B — Dùng Registrator.jl locally:

```
1 using Registrator
2 Registrator.register("path/to/MyPackage")
```

### Bước 3: Review và merge

- Registrator tạo PR trên JuliaRegistries/General
- AutoMerge bot checks compliance (3 ngày cho new packages)
- PR được auto-merge sau waiting period
- Package available via `Pkg.add("MyPackage")`

## 6.10.3 Version bumping workflow

Khi release new version:

### Bước 1: Update version trong Project.toml

```
1 # Project.toml
2 name = "MyPackage"
3 uuid = "..."
4 version = "1.2.0" # Bump từ 1.1.0
```

### Bước 2: Commit và push

```
1 $ git add Project.toml
2 $ git commit -m "Bump version to 1.2.0"
3 $ git push origin main
```

### Bước 3: Register new version

```
1 # Comment trên commit:
2 @JuliaRegistrator register
```

### Bước 4: TagBot creates release

- TagBot tự động tag commit: `v1.2.0`
- Creates GitHub release với changelog

### 6.10.4 Package naming conventions

#### Good names:

- DataFrames — Descriptive, UpperCamelCase
- HTTP — Acronym OK
- CSV — Short, clear
- Plots — Noun, describes functionality

#### Avoid:

- MyPackage.jl — Don't include ".jl" in package name
- JuliaPlots — Don't prefix with "Julia"
- my\_package — Use UpperCamelCase, not snake\_case
- Utilities — Too generic

#### Mẹo

**Naming tip:** Package name nên trả lời được câu hỏi "This package does what?" trong 1-2 words.

## 6.11 Testing Best Practices

Comprehensive testing đảm bảo package reliability. Julia's testing ecosystem dựa trên Test standard library.

### 6.11.1 Test file organization

```

1 MyPackage/
2 |— test/
3 |   |— runtests.jl      # Main test entry point
4 |   |— test_core.jl    # Core functionality tests
5 |   |— test_io.jl      # I/O tests
6 |   |— test_utils.jl   # Utility tests
7 |— src/
8   |— MyPackage.jl

```

#### test/runtests.jl:

```

1 using MyPackage
2 using Test
3
4 @testset "MyPackage.jl" begin
5     include("test_core.jl")
6     include("test_io.jl")
7     include("test_utils.jl")
8 end

```

#### test/test\_core.jl:

```

1 @testset "Core functionality" begin
2     @testset "useful_function" begin
3         @test useful_function(2) == 9      # 2^2 + 2*2 + 1 = 9
4         @test useful_function(0) == 1      # Edge case
5         @test useful_function(-1) == 0     # Negative input
6     end
7
8     @testset "Type handling" begin
9         @test useful_function(2.0) isa Float64
10        @test useful_function(big(2)) isa BigInt
11    end
12 end

```

### 6.11.2 Test macros

```

1 using Test
2
3 # Basic assertions
4 @test 1 + 1 == 2                # Equality
5 @test 1.0 ≈ 1.0 + 1e-10        # Approximate equality
6 @test 1 < 2                     # Comparison
7 @test "hello" isa String       # Type check
8
9 # Exception testing
10 @test_throws DomainError sqrt(-1) # Expects exception
11 @test_throws ArgumentError func(-1)
12
13 # Broken tests (known failures)
14 @test_broken broken_function() == 42 # Skip but track
15
16 # Skip tests
17 @test_skip expensive_test()         # Skip entirely
18
19 # Logging tests
20 @test_logs (:warn, "message") warn_function()

```

### 6.11.3 Property-based testing

Dùng random inputs để test properties thay vì specific values:

```

1 @testset "Property-based tests" begin
2     # Commutativity: add(a, b) == add(b, a)
3     for _ in 1:100
4         a, b = rand(), rand()
5         @test add(a, b) ≈ add(b, a)
6     end
7
8     # Idempotency: normalize(normalize(x)) == normalize(x)
9     for _ in 1:100
10        x = randn(10)
11        @test normalize(normalize(x)) ≈ normalize(x)
12    end
13 end

```

```
14     # Inverse: decode(encode(x)) == x
15     for _ in 1:100
16         x = rand(1:1000)
17         @test decode(encode(x)) == x
18     end
19 end
```

#### 6.11.4 Test fixtures và setup

```
1 @testset "Database tests" begin
2     # Setup - chạy trước mỗi testset
3     db = create_test_database()
4
5     @testset "Insert operations" begin
6         @test insert!(db, "key", "value") === nothing
7         @test haskey(db, "key")
8     end
9
10    @testset "Query operations" begin
11        insert!(db, "test", 42)
12        @test get(db, "test") == 42
13    end
14
15    # Teardown - cleanup
16    close(db)
17    rm(db.path)
18 end
```

#### 6.11.5 Running tests

```
1 # Từ package directory
2 (MyPackage) pkg> test
3
4 # Với verbose output
5 (MyPackage) pkg> test --verbose
6
7 # Test specific file
8 julia> include("test/test_core.jl")
9
10 # Với coverage
11 (MyPackage) pkg> test --coverage
```

#### Mẹo

**Test-driven development (TDD):** Viết tests trước khi viết implementation. Tests define expected behavior → implementation makes tests pass.

## 6.12 Best practices

### 6.12.1 Module và package organization

DO:

- Dùng **UpperCamelCase** cho module/package names: `MyPackage`, `DataFrames`
- Keep modules focused — một module một responsibility
- Export chỉ public API, giữ internals private
- Tổ chức code theo dependency order: `types` → `utils` → `core` → `io`
- Dùng meaningful file names: `types.jl`, `utils.jl`, `io.jl`

#### **DON'T:**

- Đừng export mọi thứ — giữ internal functions private
- Đừng tạo deep nesting submodules — flat structure dễ maintain
- Đừng dùng global environment cho projects — dùng project-specific environments
- Đừng ignore `Manifest.toml` trong Git — commit nó cho reproducibility

### 6.12.2 Environment management

#### **Best practices:**

- **Luôn activate** project environment trước khi làm việc
- **Commit both** `Project.toml` và `Manifest.toml` vào Git
- Dùng `[compat]` để specify version constraints
- Run `]test` trước khi commit để đảm bảo tests pass
- Dùng `]update` cẩn thận — có thể break compatibility

#### **Common pitfalls:**

- Quên activate environment → add packages vào global
- Không commit `Manifest.toml` → không reproducible
- Quên run `]instantiate` sau clone project
- Dùng `]add` khi đã có `Manifest.toml` → overwrite versions

### 6.12.3 Version control

#### **File nên commit vào Git:**

- `Project.toml` — Luôn luôn
- `Manifest.toml` — Luôn luôn (cho reproducibility)
- `src/`, `test/`, `docs/` — Source code
- `README.md`, `LICENSE` — Documentation

#### **File nên ignore (`.gitignore`):**

- `Manifest.toml` — CHỈ ignore nếu library package (không phải application)

- docs/build/ — Generated documentation
- \*.jl.cov, \*.jl.mem — Coverage và profiling outputs

**Lưu ý:** Applications (executables) nên commit `Manifest.toml`, còn libraries (reusable packages) có thể không commit (để users tự resolve versions).

#### 6.12.4 Documentation

##### Document your package:

- README.md: Giới thiệu, installation, quick start
- Docstrings: Document tất cả exported functions/types
- Examples: Practical examples trong docs/ hoặc examples/
- Tests: Tests vừa là tests vừa là examples

##### Docstring example:

```
1  """
2      mean(xs::Vector{<:Number})
3
4  Compute the arithmetic mean of a vector.
5
6  # Examples
7  ```jldoctest
8  julia> mean([1, 2, 3, 4, 5])
9  3.0
10  ```
11  """
12  mean(xs) = sum(xs) / length(xs)
```

#### 6.12.5 Tóm tắt

- **Modules** tạo namespaces, **packages** là distributable collections of modules
- Dùng `export` để define public API
- `using` cho end-user code, `import` cho extending functions
- `Project.toml` khai báo dependencies, `Manifest.toml` locks exact versions
- Luôn dùng project-specific environments, không dùng global environment
- Commit cả `Project.toml` và `Manifest.toml` cho reproducibility
- Dùng `PkgTemplates.jl` để tạo packages với structure chuẩn

Chapter này giới thiệu Julia's module system và package management. Chapter tiếp theo sẽ đi sâu về hiệu năng và profiling.



## 6.13 Bài tập

### 6.13.1 Bài tập cơ bản (\*)

#### Bài tập: Tạo module đơn giản

Tạo module `TinhToan` với các functions:

```
module TinhToan
    export cong, tru, nhan, chia

    cong(a, b) = a + b
    tru(a, b) = a - b
    nhan(a, b) = a * b
    chia(a, b) = a / b

    # Private function (không export)
    _kiem_tra(x) = x != 0
end

# ửS ụng
using .TinhToan
cong(3, 4) # => 7
```

Thêm functions: *luythua, canbac\_hai, gia\_trituyet\_doi*

#### Bài tập: Import và using

Giải thích sự khác biệt và demo:

```
# Cách 1: using – import tất cả exported symbols
using Statistics

# Cách 2: import – chỉ import module name
import Statistics
Statistics.mean([1, 2, 3])

# Cách 3: using với specific symbols
using Statistics: mean, std

# Cách 4: import với renaming
import Statistics: mean as tb
```

Bài tập: Tạo file với 2 modules, demo các cách import

**Bài tập: Package Manager cơ bản**

Thực hành các lệnh Pkg:

```
using Pkg
```

```
# 1. Xem status
```

```
Pkg.status()
```

```
# 2. Add package
```

```
Pkg.add("DataFrames")
```

```
# 3. Remove package
```

```
Pkg.rm("DataFrames")
```

```
# 4. Update packages
```

```
Pkg.update()
```

```
# 5. Xem installed packages
```

```
Pkg.installed()
```

Bài tập: Install và sử dụng: CSV, JSON3, Plots

**Bài tập: Project environment**

Tạo project mới với environment:

```
# Terminal
```

```
julia --project=MyProject
```

```
# ăHoc trong REPL
```

```
] activate MyProject
```

```
] add DataFrames CSV
```

```
] status
```

```
# Xem files được ạo
```

```
# – Project.toml: dependencies
```

```
# – Manifest.toml: exact versions
```

Bài tập: Tạo project "DataAnalysis" với 3 packages

**Bài tập: Include và file organization**

Tổ chức code thành nhiều files:

```
# src/MyModule.jl
module MyModule
    include("utils.jl")
    include("core.jl")
    include("io.jl")

    export main_function
end

# src/utils.jl
helper1() = ...
helper2() = ...

# src/core.jl
main_function() = helper1() + helper2()
```

Bài tập: Tổ chức calculator thành 3 files

**6.13.2 Bài tập trung bình (\*\*)****Bài tập: Tạo package structure**

Tạo package theo chuẩn Julia:

```
# Cấu trúc thư mục
MyPackage/
  Project.toml
  src/
    MyPackage.jl
  test/
    runtests.jl
  README.md

# Tạo ví dụ PkgTemplates
using PkgTemplates
t = Template(;
    user="username",
    plugins=[Git(), Tests()]
)
t("MyPackage")
```

Bài tập: Tạo package "VectorUtils" với 5 functions

## Bài tập: Unit testing

Viết tests cho package:

```
# test/runtests.jl
using Test
using MyPackage

@testset "Basic operations" begin
    @test cong(2, 3) == 5
    @test tru(5, 3) == 2
    @test_throws DivideError chia(1, 0)
end

@testset "Edge cases" begin
    @test cong(0, 0) == 0
    @test nhan(-1, -1) == 1
end

# Run tests
] test MyPackage
```

Bài tập: Viết  $\geq 10$  tests cho VectorUtils package

## Bài tập: Reproducible environments

Setup environment cho reproducibility:

```
# 1. ạo project
] activate MyProject
] add DataFrames@1.6 CSV@0.10

# 2. Pin versions
] pin DataFrames

# 3. Instantiate từ existing project
] activate .
] instantiate

# 4. Resolve conflicts
] resolve
```

Bài tập: Clone project từ GitHub, instantiate, run

## 6.13.3 Bài tập nâng cao (\*\*\*)

## Bài tập: Package documentation

Thêm documentation cho package:

```
"""
```

```
    cong(a, b)
```

Tính tổng hai số.

```
# Arguments
```

```
– `a::Number`: số thứ nhất
```

```
– `b::Number`: số thứ hai
```

```
# Returns
```

```
– Tổng của a và b
```

```
# Examples
```

```
```julia
```

```
julia> cong(2, 3)
```

```
5
```

```
```
```

```
"""
```

```
function cong(a, b)
```

```
    return a + b
```

```
end
```

```
# Build docs với Documenter.jl
```

Bài tập: Viết docs cho toàn bộ VectorUtils package

**Bài tập: Mini-project: Publish package**

Publish package lên GitHub:

```
# 1. ạo GitHub repo
# 2. Push code
git init && git add . && git commit -m "Initial"
git remote add origin <url>
git push -u origin main

# 3. Register gói General registry (optional)
# ăHoc install ựngc ếtíp ừt GitHub:
] add https://github.com/user/MyPackage.jl

# 4. ạo release gói version tag
git tag v0.1.0
git push --tags
```

Bài tập: Publish VectorUtils lên GitHub, tạo README với badges

**Đọc thêm**

- Julia Documentation: Modules - <https://docs.julialang.org/en/v1/manual/modules/>
- Julia Documentation: Pkg - <https://pkgdocs.julialang.org/>
- PkgTemplates.jl - <https://github.com/invenia/PkgTemplates.jl>
- Documenter.jl - <https://documenter.juliadocs.org/>

## Chương 7

# I/O và xử lý file

### Mục tiêu học tập

- Sử dụng thành thạo các hàm I/O cơ bản: `print`, `println`, `@show`
- Hiểu và áp dụng file operations với `open`, `read`, `write`
- Làm việc với CSV và DataFrames cho data analysis
- Xử lý JSON data với `JSON3.jl`
- Serialization với `JLD2.jl` cho Julia-native storage
- Kết nối databases với `SQLite.jl` và `LibPQ.jl`

### 7.1 Standard I/O và REPL

Julia's vision on input/output là **stream-oriented**, tức là đọc hoặc viết streams of bytes. Chương này sẽ giới thiệu các loại streams khác nhau, bao gồm file streams, network streams, và standard I/O.

#### 7.1.1 Standard input và output

Julia cung cấp các constants `stdin`, `stdout`, và `stderr` để tương tác với standard I/O:

```
1 # stdout - standard output stream
2 println(stdout, "Hello, Julia!") # Writes to stdout
3
4 # stderr - standard error stream
5 println(stderr, "Warning: something happened")
6
7 # stdin - standard input stream
8 # print("Enter your name: ")
9 # name = readline(stdin)
```

#### 7.1.2 Các hàm output cơ bản

Julia cung cấp nhiều hàm để output dữ liệu:

```
1 # print - không xuống dòng
2 print("Hello")
3 print(" World")  # Output: Hello World (same line)
4
5 # println - có xuống dòng
6 println("First line")
7 println("Second line")
8
9 # @show - hiển thị expression và value (debugging)
10 x = 42
11 @show x          # x = 42
12 @show 2x + 1     # 2x + 1 = 85
13
14 # display - pretty-printing cho complex objects
15 A = [1 2; 3 4]
16 display(A)
```

### 7.1.2.1 Formatting với Printf

Module Printf cung cấp C-style formatting:

```
1 using Printf
2
3 # @printf - print formatted output
4 @printf("Pi is approximately %.4f\n", π)
5 # Output: Pi is approximately 3.1416
6
7 # @sprintf - return formatted string
8 s = @sprintf("Value: %d, Ratio: %.2f%%", 42, 85.5)
9 println(s)  # Value: 42, Ratio: 85.50%
10
11 # Common format specifiers:
12 # %d - integer
13 # %f - floating point
14 # %e - scientific notation
15 # %s - string
16 # %x - hexadecimal
17 @printf("Int: %d, Float: %8.3f, Sci: %e\n", 42, 3.14159, 1.23e-10)
```

### 7.1.3 String interpolation

Julia hỗ trợ string interpolation với \$:

```
1 name = "Julia"
2 version = 1.10
3 println("Welcome to $name version $version!")
4
5 # Complex expressions với $()
6 x, y = 3, 4
7 println("The sum of $x and $y is $(x + y)")
8
9 # Escape $ với \$
10 println("Price: \$100")  # Output: Price: $100
```



### 7.1.4 Input từ user

Đọc input từ standard input:

```

1 # readline - đọc một dòng (including newline)
2 print("Enter your name: ")
3 name = readline()
4 println("Hello, $name!")
5
6 # read - đọc bytes
7 # c = read(stdin, Char) # Read single character
8
9 # readuntil - đọc cho đến delimiter
10 # data = readuntil(stdin, '.')

```

## 7.2 File operations cơ bản

### 7.2.1 Mở và đóng files

Function open là cách cơ bản nhất để làm việc với files:

```

1 # Open file for reading (default mode)
2 io = open("data.txt")
3 content = read(io, String)
4 close(io)
5
6 # IMPORTANT: Always close files to avoid resource leaks
7 # If file writing is buffered, not closing risks incomplete writes

```

#### 7.2.1.1 File modes

Julia hỗ trợ các modes sau:

| Mode | Description                   |
|------|-------------------------------|
| "r"  | Read (default)                |
| "r+" | Read and write                |
| "w"  | Write, create, truncate       |
| "w+" | Read, write, create, truncate |
| "a"  | Write, create, append         |
| "a+" | Read, write, create, append   |

Bảng 7.1: File access modes trong Julia

```

1 # Write mode - creates or overwrites file
2 io = open("output.txt", "w")
3 write(io, "Hello, World!\n")
4 close(io)
5
6 # Append mode - adds to existing file
7 io = open("output.txt", "a")
8 write(io, "Another line\n")
9 close(io)

```

### 7.2.2 Do-block pattern (Recommended)

Pattern `do...end` tự động close file khi block kết thúc, ngay cả khi có exception:

```
1  # Reading with do-block (RECOMMENDED)
2  open("data.txt") do io
3      for line in eachline(io)
4          println(line)
5      end
6  end # File automatically closed here
7
8  # Writing with do-block
9  open("output.txt", "w") do io
10     println(io, "Line 1")
11     println(io, "Line 2")
12 end
13
14 # This is equivalent to:
15 # try
16 #     io = open("data.txt")
17 #     # ... do work ...
18 # finally
19 #     close(io)
20 # end
```

#### Ghi chú

**Best Practice:** Luôn sử dụng `do-block` pattern khi làm việc với files. Pattern này đảm bảo file được close đúng cách, tránh resource leaks và data corruption.

### 7.2.3 Reading files

Julia cung cấp nhiều functions để đọc files:

```
1  # read entire file as String
2  content = read("data.txt", String)
3
4  # read as bytes (Vector{UInt8})
5  bytes = read("data.txt")
6
7  # read file line by line
8  lines = readlines("data.txt")
9  for line in lines
10     println(line)
11 end
12
13 # eachline - lazy iterator (memory efficient)
14 for line in eachline("large_file.txt")
15     # Process each line without loading entire file
16     process(line)
17 end
18
19 # readline - đọc một dòng
```

```

20 open("data.txt") do io
21     first_line = readline(io)
22     second_line = readline(io)
23 end

```

### 7.2.4 Writing files

```

1  # write - low-level write (returns bytes written)
2  open("output.txt", "w") do io
3      write(io, "Raw bytes\n") # Returns 10
4  end
5
6  # print/println - formatted output
7  open("output.txt", "w") do io
8      println(io, "First line")
9      print(io, "No newline")
10     println(io, " - continued")
11 end
12
13 # writedlm - write delimited data (requires DelimitedFiles)
14 using DelimitedFiles
15 data = [1 2 3; 4 5 6]
16 writedlm("matrix.csv", data, ',')

```

### 7.2.5 File system operations

```

1  # Check file existence
2  isfile("data.txt")      # true if file exists
3  isdir("mydir")          # true if directory exists
4  ispath("something")     # true if path exists
5
6  # File information
7  stat("data.txt")        # File statistics
8  filesize("data.txt")    # Size in bytes
9  mtime("data.txt")       # Modification time
10
11 # Path manipulation
12 joinpath("dir", "subdir", "file.txt") # Cross-platform path
13 dirname("/path/to/file.txt")          # "/path/to"
14 basename("/path/to/file.txt")          # "file.txt"
15 splitext("file.txt")                   # ("file", ".txt")
16
17 # Directory operations
18 mkdir("newdir")                        # Create directory
19 mkpath("a/b/c")                        # Create nested directories
20 readdir(".")                            # List directory contents
21 rm("file.txt")                          # Remove file
22 rm("dir", recursive=true)               # Remove directory recursively

```

## 7.3 CSV và DataFrames

Làm việc với tabular data là task phổ biến trong data science. Julia cung cấp packages mạnh mẽ cho việc này.

### 7.3.1 CSV.jl - Đọc và ghi CSV files

```
1 using CSV, DataFrames
2
3 # Read CSV file into DataFrame
4 df = CSV.read("data.csv", DataFrame)
5
6 # With options
7 df = CSV.read("data.csv", DataFrame,
8     header = true,           # First row is header
9     delim = ',',           # Delimiter character
10    missingstring = "NA",    # String representing missing
11    types = Dict{:age => Int, :score => Float64}
12 )
13
14 # Write DataFrame to CSV
15 CSV.write("output.csv", df)
16
17 # Write with options
18 CSV.write("output.csv", df,
19     delim = ';',
20     header = ["Name", "Age", "Score"]
21 )
```

### 7.3.2 DataFrames.jl cơ bản

DataFrame là cấu trúc dữ liệu dạng bảng, tương tự Pandas trong Python:

```
1 using DataFrames
2
3 # Create DataFrame
4 df = DataFrame(
5     name = ["Alice", "Bob", "Charlie"],
6     age = [25, 30, 35],
7     score = [85.5, 92.0, 78.5]
8 )
9
10 # Access columns
11 df.name           # Column as vector
12 df[:, :name]      # Same, explicit syntax
13 df[:, "name"]     # String indexing
14 df[:, 1]          # By column number
15
16 # Access rows
17 df[1, :]          # First row
18 df[1:3, :]        # First 3 rows
19
20 # Filter rows
```

```

21 df[df.age .> 25, :]          # Age > 25
22 subset(df, :score => x -> x .> 80)  # Score > 80
23
24 # Add column
25 df.passed = df.score .>= 80
26
27 # Basic statistics
28 describe(df)

```

### 7.3.3 Data manipulation với DataFrames

```

1  using DataFrames, Statistics
2
3  # Sort
4  sort(df, :age)              # Ascending
5  sort(df, :age, rev=true)    # Descending
6  sort(df, [:name, :age])     # Multiple columns
7
8  # Select columns
9  select(df, :name, :score)
10 select(df, Not(:age))       # All except age
11 select(df, r"^s")           # Regex: starts with 's'
12
13 # Transform columns
14 transform(df, :score => (x -> x ./ 100) => :normalized)
15
16 # Group and aggregate
17 gdf = groupby(df, :passed)
18 combine(gdf,
19     :score => mean => :avg_score,
20     :age => maximum => :max_age,
21     nrow => :count
22 )
23
24 # Join DataFrames
25 df2 = DataFrame(name = ["Alice", "Bob"], dept = ["IT", "HR"])
26 innerjoin(df, df2, on = :name)
27 leftjoin(df, df2, on = :name)

```

### 7.3.4 Missing values

Julia sử dụng `missing` để represent missing data:

```

1  using DataFrames
2
3  # DataFrame with missing values
4  df = DataFrame(
5      x = [1, 2, missing, 4],
6      y = [missing, 2.0, 3.0, 4.0]
7  )
8
9  # Check for missing
10 ismissing(df.x[3])          # true

```

```
11 ismissing.(df.x)           # Vector of bools
12
13 # Handle missing
14 dropmissing(df)             # Remove rows with any missing
15 dropmissing(df, :x)         # Remove rows with missing in :x
16 coalesce.(df.x, 0)         # Replace missing with 0
17
18 # Skip missing in calculations
19 using Statistics
20 mean(skipmissing(df.x))     # Mean ignoring missing
```

## 7.4 JSON processing

JSON (JavaScript Object Notation) là format phổ biến cho data exchange.

### 7.4.1 JSON3.jl - High-performance JSON

```
1 using JSON3
2
3 # Parse JSON string
4 json_str = "\"{\"name\": \"Julia\", \"version\": 1.10, \"features\": [\"fast\",
   ↪  \"dynamic\"]}\""
5 data = JSON3.read(json_str)
6
7 # Access parsed data
8 data.name           # "Julia"
9 data[:version]      # 1.10
10 data.features[1]    # "fast"
11
12 # Parse JSON file
13 data = open("config.json") do io
14     JSON3.read(io)
15 end
16
17 # Or simply
18 data = JSON3.read(read("config.json", String))
```

### 7.4.2 Converting Julia objects to JSON

```
1 using JSON3
2
3 # Dict to JSON
4 d = Dict("name" => "Alice", "age" => 25)
5 json_str = JSON3.write(d)
6 println(json_str)  # {"name": "Alice", "age": 25}
7
8 # Named tuple to JSON
9 data = (name = "Bob", scores = [85, 90, 95])
10 JSON3.write(data)
11
12 # Write to file
13 open("output.json", "w") do io
```

```

14     JSON3.write(io, data)
15 end
16
17 # Pretty print
18 JSON3.pretty(data)

```

### 7.4.3 Structured JSON với StructTypes

Để parse JSON vào custom structs:

```

1  using JSON3, StructTypes
2
3  struct Person
4      name::String
5      age::Int
6      email::Union{String, Nothing}
7  end
8
9  # Define how to construct Person from JSON
10 StructTypes.StructType{::Type{Person}} = StructTypes.Struct()
11
12 # Parse JSON into struct
13 json_str = "\"{\\name\\: \\Alice\\, \\age\\: 25, \\email\\: \\alice@example.com\\}\""
14 person = JSON3.read(json_str, Person)
15
16 println(person.name)    # Alice
17 println(person.age)     # 25
18
19 # Convert struct to JSON
20 JSON3.write(person)

```

## 7.5 Serialization với JLD2

Khi cần lưu Julia objects một cách native (không convert sang text format).

### 7.5.1 JLD2.jl - Julia Data Format

```

1  using JLD2
2
3  # Save variables
4  x = [1, 2, 3]
5  y = Dict{"a" => 1, "b" => 2}
6  @save "data.jld2" x y
7
8  # Load variables
9  @load "data.jld2" x y
10 println(x)    # [1, 2, 3]
11
12 # Save with custom names
13 jldsave("data.jld2"; myarray=x, mydict=y)
14
15 # Load specific variables

```

```

16 data = load("data.jld2")
17 data["myarray"]
18
19 # Check contents without loading
20 keys(load("data.jld2"))

```

## 7.5.2 JLD2 với custom types

```

1 using JLD2
2
3 struct MyData
4     values::Vector{Float64}
5     metadata::Dict{String, Any}
6 end
7
8 data = MyData([1.0, 2.0, 3.0], Dict("source" => "experiment"))
9
10 # Save custom struct
11 @save "mydata.jld2" data
12
13 # Load back
14 @load "mydata.jld2" data
15 println(data.values)      # [1.0, 2.0, 3.0]
16 println(data.metadata)    # Dict("source" => "experiment")

```

### Cảnh báo

**Serialization Limitations:** Serialization of Julia objects is designed for **short-term storage**. Serialization và deserialization chỉ an toàn khi sử dụng **cùng version Julia** với **cùng packages và versions**. Với long-term storage, prefer text formats như JSON hoặc CSV.

## 7.6 Database connections

Julia có thể kết nối với nhiều database systems.

### 7.6.1 SQLite.jl - Embedded database

SQLite là database dạng file, không cần server:

```

1 using SQLite, DataFrames
2
3 # Create or open database
4 db = SQLite.DB("mydata.db") # Creates file if not exists
5 # db = SQLite.DB()          # In-memory database
6
7 # Create table
8 SQLite.execute(db, """
9     CREATE TABLE IF NOT EXISTS users (
10         id INTEGER PRIMARY KEY,
11         name TEXT NOT NULL,
12         email TEXT UNIQUE,

```



```

13         age INTEGER
14     )
15     """)
16
17     # Insert data
18     SQLite.execute(db, "INSERT INTO users (name, email, age) VALUES (?, ?, ?)",
19         ("Alice", "alice@example.com", 25))
20
21     # Query data
22     result = DBInterface.execute(db, "SELECT * FROM users")
23     df = DataFrame(result)
24
25     # Parameterized queries (safe from SQL injection)
26     stmt = SQLite.Stmt(db, "SELECT * FROM users WHERE age > ?")
27     result = DBInterface.execute(stmt, (20,))

```

### 7.6.2 LibPQ.jl - PostgreSQL

Kết nối với PostgreSQL server:

```

1  using LibPQ, DataFrames
2
3  # Connect to PostgreSQL
4  conn = LibPQ.Connection("host=localhost dbname=mydb user=postgres
   ↪ password=secret")
5
6  # Execute query
7  result = execute(conn, "SELECT * FROM products WHERE price > 100")
8  df = DataFrame(result)
9
10 # Parameterized query
11 result = execute(conn,
12     "SELECT * FROM products WHERE category = \$1 AND price < \$2",
13     ["electronics", 500]
14 )
15
16 # Insert data
17 execute(conn,
18     "INSERT INTO products (name, price) VALUES (\$1, \$2)",
19     ["New Product", 99.99]
20 )
21
22 # Always close connection
23 close(conn)

```

### 7.6.3 DBInterface - Unified database interface

```

1  using DBInterface
2
3  # DBInterface provides common interface across databases
4  # Works with SQLite, LibPQ, MySQL, ODBC, etc.
5
6  # Common operations:

```

```
7  # DBInterface.execute(conn, sql) - Execute query
8  # DBInterface.prepare(conn, sql) - Prepare statement
9  # DBInterface.close!(result)      - Close result set
10
11 # Convert any result to DataFrame
12 using DataFrames
13 df = DataFrame(DBInterface.execute(conn, "SELECT * FROM table"))
```

## 7.7 Các format khác

### 7.7.1 Excel files với XLSX.jl

```
1  using XLSX, DataFrames
2
3  # Read Excel file
4  df = DataFrame(XLSX.readtable("data.xlsx", "Sheet1"))
5
6  # Read specific range
7  data = XLSX.readdata("data.xlsx", "Sheet1!A1:C10")
8
9  # Write to Excel
10 XLSX.writetable("output.xlsx", collect(eachcol(df)), names(df))
```

### 7.7.2 Parquet files với Parquet.jl

Parquet là format hiệu quả cho big data:

```
1  using Parquet, DataFrames
2
3  # Read Parquet
4  df = DataFrame(Parquet.read_parquet("data.parquet"))
5
6  # Write Parquet
7  Parquet.write_parquet("output.parquet", df)
```

### 7.7.3 Arrow format

Apache Arrow cho high-performance data exchange:

```
1  using Arrow, DataFrames
2
3  # Write Arrow file
4  Arrow.write("data.arrow", df)
5
6  # Read Arrow file
7  df = DataFrame(Arrow.Table("data.arrow"))
8
9  # Arrow is excellent for:
10 # - Inter-process communication
11 # - Memory-mapped files
12 # - Zero-copy reads
```

## 7.8 Best practices

### 7.8.1 Error handling cho I/O

```

1  # Always handle potential errors
2  try
3      data = read("important.txt", String)
4  catch e
5      if isa(e, SystemError)
6          println("File not found or permission denied")
7      else
8          rethrow(e)
9      end
10 end
11
12 # Check before operations
13 if isfile("data.txt")
14     content = read("data.txt", String)
15 else
16     println("File does not exist")
17 end

```

### 7.8.2 Working with large files

```

1  # DON'T: Load entire file into memory
2  # content = read("huge_file.txt", String)
3
4  # DO: Process line by line
5  open("huge_file.txt") do io
6      for line in eachline(io)
7          process(line)
8      end
9  end
10
11 # DO: Use memory-mapped files
12 using Mmap
13 io = open("huge_binary.dat")
14 data = Mmap.mmap(io, Vector{Float64}, 1_000_000)
15 # data is now accessible without loading into RAM

```

### 7.8.3 Temporary files

```

1  # Create temporary file
2  tmpfile = tempname()
3  write(tmpfile, "temporary data")
4  # Use the file...
5  rm(tmpfile)
6
7  # Or use mktempdir
8  mktempdir() do tmpdir
9      filepath = joinpath(tmpdir, "data.txt")
10     write(filepath, "data")
11     # Directory and contents auto-deleted after block

```

### 7.8.4 Tóm tắt

- **Standard I/O:** `print`, `println`, `@show`, `readline`
- **File operations:** Luôn dùng do-block pattern để ensure proper cleanup
- **CSV/DataFrames:** `CSV.jl` + `DataFrames.jl` là combo chuẩn cho tabular data
- **JSON:** `JSON3.jl` cho high-performance JSON processing
- **Serialization:** `JLD2.jl` cho Julia-native storage (short-term only)
- **Databases:** `SQLite.jl` (embedded), `LibPQ.jl` (PostgreSQL)
- **Best practices:** Handle errors, process large files lazily, use temp files properly

Chapter này giới thiệu Julia's I/O system và các packages phổ biến cho data handling. Chapter tiếp theo sẽ đi sâu về hiệu năng và profiling.

## 7.9 Bài tập

### 7.9.1 Bài tập cơ bản

#### Bài tập: File reading

Viết function đọc file text và đếm số từ trong file. Function nhận filename và trả về số từ.

#### Bài tập: CSV processing

Đọc file CSV chứa dữ liệu sinh viên (name, score), tính điểm trung bình và tìm sinh viên có điểm cao nhất.

### 7.9.2 Bài tập nâng cao

#### Bài tập: JSON API client

Viết function fetch data từ JSON file, filter theo điều kiện, và save kết quả ra file mới.

#### Bài tập: Database CRUD

Tạo SQLite database với table "products", implement các operations: Create, Read, Update, Delete.

### 7.9.3 Bài tập thách thức

#### Bài tập: Log file analyzer

Viết chương trình xử lý log file lớn (>1GB) một cách hiệu quả: parse timestamps, count error types, generate summary report.

## Đọc thêm

- Julia Documentation: I/O and Network - <https://docs.julialang.org/en/v1/base/io-network/>
- CSV.jl Documentation - <https://csv.juliadata.org/>
- DataFrames.jl Documentation - <https://dataframes.juliadata.org/>
- JSON3.jl Documentation - <https://github.com/quinnj/JSON3.jl>



## Chương 8

# Metaprogramming

### Mục tiêu học tập

- Hiểu cấu trúc biểu thức (Expressions) và cây cú pháp trừu tượng (AST) trong Julia
- Sử dụng quoting với ``` và quote blocks
- Thực hiện interpolation với `$` trong expressions
- Tạo và sử dụng macros với `@macro`
- Hiểu generated functions và khi nào nên dùng
- Sử dụng các macros phổ biến: `@time`, `@benchmark`, `@assert`

### 8.1 Giới thiệu Metaprogramming

**Metaprogramming** là kỹ thuật viết code để tạo ra hoặc thao tác code khác. Julia có hỗ trợ metaprogramming mạnh mẽ nhờ vào việc mã nguồn Julia được biểu diễn dưới dạng cấu trúc dữ liệu của chính Julia.

#### 8.1.1 Tại sao cần Metaprogramming?

Metaprogramming cho phép:

- **Code generation:** Tự động sinh code lặp lại
- **Domain-Specific Languages (DSLs):** Tạo ngôn ngữ chuyên biệt cho vấn đề cụ thể
- **Performance optimization:** Tối ưu code tại compile-time
- **Code introspection:** Phân tích và kiểm tra code

```
# Ví dụ: Thay vì viết lặp lại
add_1(x) = x + 1
add_2(x) = x + 2
add_3(x) = x + 3
```

```
# Ta có thể dùng metaprogramming để sinh tự động
for i in 1:3
```

```
@eval $(Symbol("add_", i))(x) = x + $i
end

add_2(10)  # => 12
```

## 8.2 Expressions và Abstract Syntax Tree (AST)

Mọi chương trình Julia đều được parse thành cây cú pháp trừu tượng (AST). AST là cấu trúc dữ liệu biểu diễn cấu trúc cú pháp của code.

### 8.2.1 Expr - Kiểu dữ liệu Expression

Trong Julia, Expr là kiểu dữ liệu chứa biểu thức:

```
# ạo expression bằng quoting
ex = :(1 + 2)
typeof(ex)  # => Expr
```

```
# Xem cấu trúc của expression
dump(ex)

# Expr
#   head: Symbol call
#   args: Array{Any}((3,))
#     1: Symbol +
#     2: Int64 1
#     3: Int64 2
```

Một Expr có hai thành phần chính:

- head: Symbol chỉ loại expression (như `:call`, `:if`, `:for`)
- args: Mảng chứa các thành phần con

### 8.2.2 Visualizing AST với dump()

```
# Expression phức tạp hơn
ex = :(x = 2 * y + 1)
dump(ex)

# Expr
#   head: Symbol =
#   args: Array{Any}((2,))
#     1: Symbol x
#     2: Expr
#       head: Symbol call
#       args: Array{Any}((3,))
#         1: Symbol +
#         2: Expr
#           head: Symbol call
#           args: Array{Any}((3,))
```



```
#          1: Symbol *
#          2: Int64 2
#          3: Symbol y
#          3: Int64 1
```

### 8.2.3 Symbols

Symbol là kiểu dữ liệu đại diện cho tên biến hoặc operators:

```
# ặTo Symbol
s = :hello
typeof(s) # => Symbol

# Symbol trong expression
ex = :(x + y)
ex.args[2] # => :x (Symbol)

# So sánh Symbols
:hello == :hello # => true
:hello == :world # => false

# ặTo Symbol động
Symbol("variable_", 1) # => :variable_1
```

## 8.3 Quoting: Tạo Expressions

**Quoting** là cách tạo expression mà không thực thi nó.

### 8.3.1 Cú pháp :()

Cách đơn giản nhất để quote một expression:

```
# Quote ột expression đơn ảgin
ex1 = :(1 + 2)

# Quote ớvi ềnnhiu statements thì dùng quote block
ex2 = quote
  x = 1
  y = 2
  x + y
end

# Trong REPL
julia> 3 * 5          # Evaluate => 15
julia> :(3 * 5)       # Quote => :(3 * 5)
```

### 8.3.2 Quote blocks

Cho expressions nhiều dòng:

```
ex = quote
  function square(x)
    return x * x
  end
end

# Xem cấu trúc
dump(ex)
# Expr
#   head: Symbol block
#   args: Array{Any}((2,))
#       1: LineNumberNode ...
#       2: Expr (function definition)
```

### 8.3.3 Meta.parse() - Parse từ String

Chuyển đổi string thành expression:

```
# Parse string thành expression
ex = Meta.parse("1 + 2 * 3")
dump(ex)

# Parse function definition
code = """
function greet(name)
  println("Hello , ", name)
end
"""
ex = Meta.parse(code)

# Lưu ý: Meta.parse chỉ parse một expression
# Dùng Meta.parseall cho nhiều expressions
```

## 8.4 Interpolation với \$

**Interpolation** cho phép chèn giá trị vào expression.

### 8.4.1 Interpolation cơ bản

```
# Interpolate giá trị
a = 1
b = 2
ex = :($a + $b)
dump(ex)
```

```
# Expr (args ứa giá ịtr 1 và 2, không ảphi symbols)

# Interpolate vào quote block
x = 10
ex = quote
  result = $x * 2
  println(result)
end
```

### 8.4.2 Interpolate Expressions

```
# Có ểth interpolate ảc expression
inner_ex = :(x + y)
outer_ex = :($inner_ex * 2)
# => :((x + y) * 2)

# Splatting óvi $... cho ềnhieu arguments
args = [:a, :b, :c]
ex = :(f($args...))
# => :(f(a, b, c))
```

### 8.4.3 Interpolation nhiều cấp

```
# Interpolate trong nested quotes (ầcn $$ cho ỗmi level)
value = 5
ex = quote
  inner = quote
    result = $$value
  end
end

# ựThc ết: ảHn ếch dùng quá 2 levels — ắt khó debug!
```

## 8.5 *eval()* và Runtime Code Generation

`eval()` thực thi expression tại runtime.

### 8.5.1 Sử dụng `eval()`

```
# Evaluate expression đơn ảgin
ex = :(1 + 2)
eval(ex) # => 3

# Evaluate óvi ếbin
x = 10
ex = :(x * 2)
```

```
eval(ex)  # => 20 (dùng x ở global scope)
```

```
# Định nghĩa function động
ex = :(double(x) = 2x)
eval(ex)
double(5)  # => 10
```

### 8.5.2 @eval macro

Tiện lợi hơn khi cần interpolate:

```
# Sinh nhiều functions
for op in [:+, :-, :*, :/]
    fname = Symbol("apply_", op)
    @eval $fname(a, b) = $op(a, b)
end
```

```
apply_+(1, 2)  # => 3
apply_*(3, 4)  # => 12
```

```
# Sinh structs
for name in [:Point2D, :Point3D]
    @eval struct $name
        x::Float64
        y::Float64
    end
end
```

### 8.5.3 Lưu ý về eval()

#### Cảnh báo về eval()

- `eval()` chạy trong **global scope** của module
- Không nên dùng trong performance-critical code
- Có thể gây security issues nếu eval user input
- Thường nên dùng macros thay thế

## 8.6 Macros

**Macros** là công cụ metaprogramming mạnh nhất trong Julia. Chúng nhận expressions làm input và trả về expression mới để được compiled.

### 8.6.1 Định nghĩa Macro cơ bản

```
# Macro đơn giản
macro sayhello()
```

```

        return :(println("Hello , World!"))
    end

    @sayhello # In ra: Hello , World!

# Macro óvi arguments
macro sayhello(name)
    return :(println("Hello , ", $name))
end

@sayhello "Julia" # In ra: Hello , Julia

```

### 8.6.2 Macro vs Function

| Đặc điểm       | Function | Macro                  |
|----------------|----------|------------------------|
| Thời điểm chạy | Runtime  | Compile-time           |
| Input          | Giá trị  | Expressions            |
| Output         | Giá trị  | Expression (được eval) |
| Syntax         | $f(x)$   | $@f\ x$ hoặc $@f(x)$   |
| Scope          | Lexical  | Hygiene rules          |

### 8.6.3 Xem Macro Expansion với @macroexpand

```

macro twice(ex)
    quote
        $ex
        $ex
    end
end

# Xem macro ãs sinh ra code gì
@macroexpand @twice println("Hello")
# quote
#     println("Hello")
#     println("Hello")
# end

# ựThc thi
@twice println("Hello")
# Hello
# Hello

```

### 8.6.4 Macro Hygiene

Julia tự động đổi tên biến trong macros để tránh xung đột:

```
macro setx(val)
```

```
    quote
      x = $val
    end
end

x = 10
@setx 20
println(x) # Vẫn là 10! (macro dùng ếbin khác)

# Để thực ự modify ếbin ngoài, dùng esc()
macro setx_real(val)
  esc(quote
    x = $val
  end)
end

x = 10
@setx_real 20
println(x) # Bây ờgi là 20
```

### 8.6.5 esc() - Escape Hygiene

```
# esc() làm cho expression được eval trong caller's scope
macro define_fn(name, body)
  esc(quote
    function $name()
      $body
    end
  end)
end

@define_fn my_function begin
  println("This is my function")
end

my_function() # In ra: This is my function
```

### 8.6.6 Macro với nhiều Arguments

```
# Macro timing code execution
macro timed(ex)
  quote
    local t0 = time()
    local result = $(esc(ex))
    local elapsed = time() - t0
    println("Elapsed time: ", elapsed, " seconds")
  end
end
```

```

        result
    end
end

@timed sleep(1)
# Elapsed time: 1.00234 seconds

# Macro óvi keyword-style arguments
macro debug(level, msg)
    quote
        if DEBUG_LEVEL >= $level
            println("[DEBUG $( $level)] ", $msg)
        end
    end
end
end

```

## 8.7 Generated Functions

**Generated functions** cho phép tạo specialized code dựa trên types của arguments tại compile-time.

### 8.7.1 Định nghĩa Generated Function

```

@generated function doubled(x)
    if x <: AbstractFloat
        return :(x + x) # Floating point: dùng addition
    else
        return :(2 * x) # Integer: dùng multiplication
    end
end

doubled(1.5)    # Dùng x + x
doubled(3)      # Dùng 2 * x

```

### 8.7.2 Use Cases cho Generated Functions

```

# Unroll loops ựa trên tuple size
@generated function mysum(t::NTuple{N, T}) where {N, T}
    ex = :(t[1])
    for i in 2:N
        ex = :($ex + t[$i])
    end
    return ex
end

mysum((1, 2, 3, 4)) # Compiled thành: t[1] + t[2] + t[3] + t[4]

```

```
# Type-specialized printing
@generated function type_info(x::T) where T
    msg = "Value is of type $T"
    return :(println(msg, ":", x))
end

type_info(42)          # Value is of type Int64: 42
type_info(3.14)        # Value is of type Float64: 3.14
```

### 8.7.3 Hạn chế của Generated Functions

#### Lưu ý quan trọng

- Body của generated function KHÔNG được gọi impure functions
- Không được access mutable global state
- Chỉ có thể dựa vào types, không phải values
- Debugging khó hơn macros
- Nên dùng khi performance thực sự critical

## 8.8 Các Macros Phổ biến trong Julia

### 8.8.1 @time và @elapsed

```
# @time: In ờthi gian và memory allocation
@time sum(rand(1000000))
# 0.002341 seconds (2 allocations: 7.629 MiB)
```

```
# @elapsed: ỉCh ảtr ềv ờthi gian (seconds)
t = @elapsed sum(rand(1000000))
println("Time: $t seconds")
```

```
# @allocated: ảTr ềv bytes allocated
bytes = @allocated sum(rand(1000000))
println("Allocated: $bytes bytes")
```

### 8.8.2 @benchmark (từ BenchmarkTools.jl)

```
using BenchmarkTools

# Benchmark chính xác (ạchy ềnhiu ảln)
@benchmark sum(rand(1000))
# BenchmarkTools.Trial: 10000 samples with 10 evals.
# Range (min – max): 714.700 ns – 152.934 s
```



```
# ỚVi setup
@benchmark sum(data) setup=(data=rand(1000))

# So sánh implementations
@btime sort(data) setup=(data=rand(100))
@btime sort!(copy(data)) setup=(data=rand(100))
```

### 8.8.3 @assert và @test

```
# @assert: ỚKim tra condition, throw error Ớnu false
x = 5
@assert x > 0 "x must be positive"
@assert x < 10

# @test (Ớt Test.jl): Cho unit testing
using Test

@test 1 + 1 == 2
@test sqrt(4) == 2.0
@test_throws DomainError sqrt(-1)

# Test set
@testset "Math tests" begin
    @test 2 + 2 == 4
    @test 3 * 3 == 9
end
```

### 8.8.4 @show và @info

```
# @show: Debug printing Ớvi tên Ớbin
x = 42
y = "hello"
@show x y
# x = 42
# y = "hello"

# @info, @warn, @error: Logging
@info "Starting computation" n=1000
@warn "Large dataset" size=1e9
@error "Failed to connect" server="localhost"
```

### 8.8.5 @inline và @noinline

```
# ỚGi ý compiler inline function
@inline function fast_add(a, b)
    a + b
```

```
end

# ãNgn inlining (ũhu ích cho profiling)
@noinline function tracked_function(x)
    # Code here...
end
```

### 8.8.6 @simd và @inbounds

```
# @simd: Cho phép SIMD vectorization
function sum_simd(arr)
    s = zero(eltype(arr))
    @simd for x in arr
        s += x
    end
    return s
end

# @inbounds: ỏB bounds checking (nguy ẻhim!)
function fast_access(arr, indices)
    s = 0
    @inbounds for i in indices
        s += arr[i]
    end
    return s
end
```

## 8.9 Thực hành Metaprogramming

### 8.9.1 Ví dụ: Tạo DSL cho Math Expressions

```
# DSL để symbolic differentiation
macro diff(ex, var)
    diff_expr(ex, var)
end

function diff_expr(ex::Symbol, var::Symbol)
    ex == var ? 1 : 0
end

function diff_expr(ex::Number, var::Symbol)
    0
end

function diff_expr(ex::Expr, var::Symbol)
    if ex.head == :call
```

```

    op = ex.args[1]
    if op == :+
      :($ (diff_expr(ex.args[2], var)) + $ (diff_expr(ex.args[3], var)))
    elsif op == :*
      a, b = ex.args[2], ex.args[3]
      :($ (diff_expr(a, var)) * $b + $a * $ (diff_expr(b, var)))
    end
  end
end

# ửS ụng
@diff(x * x + x, x)  # => 2x + 1 (ụng expression)

```

### 8.9.2 Ví dụ: Macro cho Memoization

```

macro memoize(fn_def)
  # Parse function definition
  fn_name = fn_def.args[1].args[1]
  fn_args = fn_def.args[1].args[2:end]
  fn_body = fn_def.args[2]

  cache_name = Symbol("__cache_", fn_name)

  esc(quote
    const $cache_name = Dict()

    function $fn_name($ (fn_args...))
      key = ($ (fn_args...) ,)
      if haskey($cache_name, key)
        return $cache_name[key]
      end
      result = $fn_body
      $cache_name[key] = result
      return result
    end
  end)
end

# ửS ụng
@memoize function fib(n)
  n <= 1 ? n : fib(n-1) + fib(n-2)
end

@time fib(40)  # Nhanh ờnh memoization

```

### 8.9.3 Ví dụ: Auto-generate Getters/Setters

```
macro with_accessors(struct_def)
  # To struct + getters/setters t động
  name = struct_def.args[2]
  fields = struct_def.args[3].args

  accessors = Expr[]
  for field in fields
    if field.isa Expr && field.head == :(::)
      fname = field.args[1]
      push!(accessors, :(
        $(Symbol("get_", fname))(obj::$name) = obj.$fname
      ))
      push!(accessors, :(
        $(Symbol("set_", fname, "!="))(obj::$name, val) =
          (obj.$fname = val)
      ))
    end
  end
end

quote
  $(esc(struct_def))
  $(map(esc, accessors)...)
end

end

@with_accessors mutable struct Person
  name::String
  age::Int
end

p = Person("Alice", 30)
get_name(p)      # => "Alice"
set_age!(p, 31)  # Modify age
```

## Bài tập thực hành

### 8.9.4 Bài tập cơ bản

#### Bài tập: Quote và Eval

Tạo một expression đại diện cho phép tính  $(a + b) * c$ , sau đó gán giá trị cho các biến và eval.

**Bài tập: Inspect AST**

Dùng `dump()` để phân tích cấu trúc AST của expression: `if x > 0 println("positive") else println("non-positive") end`

**8.9.5 Bài tập nâng cao****Bài tập: Debug Macro**

Viết macro `@debug` in ra tên biến và giá trị cho debugging, ví dụ `@debug x y z` in ra `x = 1, y = 2, z = 3`.

**Bài tập: Retry Macro**

Viết macro `@retry n expr` thử thực hiện `expr` tối đa `n` lần nếu có exception.

**8.9.6 Bài tập thách thức****Bài tập: Unit Conversion DSL**

Tạo macro cho DSL chuyển đổi đơn vị: `@convert 5km to m` trả về 5000.

**Đọc thêm**

- Julia Documentation: Metaprogramming - <https://docs.julialang.org/en/v1/manual/metaprogramming/>
- *Hands-On Design Patterns with Julia* - Chapter 4: Macros and Metaprogramming
- MacroTools.jl - <https://github.com/FluxML/MacroTools.jl>
- Cassette.jl - Advanced metaprogramming - <https://github.com/JuliaLabs/Cassette.jl>



## Phần III

# Phần III — Tính toán hiệu suất cao





## Chương 9

# Hiệu năng và profiling

### Mục tiêu học tập

- Benchmarking chính xác với BenchmarkTools.jl
- Hiểu và áp dụng type stability với @code\_warntype
- Quản lý memory allocation và garbage collection
- Sử dụng profiling tools để tìm bottlenecks
- Áp dụng optimization techniques và best practices

### 9.1 Benchmarking với BenchmarkTools.jl

Trước khi tối ưu hóa code, chúng ta cần đo lường hiệu năng một cách chính xác. Như câu nói nổi tiếng: “Premature optimization is the root of all evil” (Donald Knuth). Chương này sẽ hướng dẫn cách đo lường, phân tích và tối ưu hiệu năng Julia code một cách có hệ thống.

#### 9.1.1 Tại sao cần benchmarking chính xác?

Benchmarking không chính xác có thể dẫn đến những kết luận sai lầm về hiệu năng code. Một số nguyên nhân gây ra kết quả benchmarking không chính xác:

- **JIT compilation:** Lần chạy đầu tiên của function bao gồm thời gian compile, không phản ánh execution time thực sự
- **Global variables:** Biến global làm compiler không thể infer types, gây slowdown đáng kể
- **Noise từ hệ thống:** Background processes, garbage collection có thể ảnh hưởng timing
- **Cache effects:** Memory caching có thể làm lệch kết quả giữa các lần chạy

#### Ghi chú

Julia compiler tạo specialized code cho mỗi combination của argument types. Lần gọi đầu tiên của function với types mới sẽ trigger compilation, làm thời gian chạy lâu hơn bình thường.

### 9.1.2 Macro @time và các hạn chế

Julia cung cấp macro @time để đo thời gian thực thi:

```

1  # Đo thời gian và memory allocation
2  function sum_squares(n)
3      total = 0.0
4      for i in 1:n
5          total += i^2
6      end
7      return total
8  end
9
10 # Lần 1: Bao gồm compilation time
11 @time sum_squares(10^6)
12 # => 0.045123 seconds (bao gồm compile)
13
14 # Lần 2: Chỉ execution time
15 @time sum_squares(10^6)
16 # => 0.002341 seconds (đã compiled)
```

@time hiển thị:

- Thời gian thực thi (seconds)
- Số allocations và tổng memory allocated
- Phần trăm thời gian dành cho garbage collection

Các macro timing khác:

```

1  # @elapsed - Trả về thời gian (seconds) thay vì print
2  elapsed = @elapsed sum_squares(10^6)
3  println("Took $elapsed seconds")
4
5  # @allocated - Trả về số bytes allocated
6  bytes = @allocated sum_squares(10^6)
7  println("Allocated $bytes bytes")
8
9  # @timed - Trả về named tuple với đầy đủ thông tin
10 result = @timed sum_squares(10^6)
11 println("Value: $(result.value)")
12 println("Time: $(result.time) seconds")
13 println("Bytes: $(result.bytes)")
14 println("GC time: $(result.gc_time) seconds")
```

#### Cảnh báo

@time chỉ chạy code một lần, nên kết quả có thể bị ảnh hưởng bởi noise. Để có accurate benchmarking, cần chạy nhiều lần và lấy thống kê.

### 9.1.3 BenchmarkTools.jl — Benchmarking chuyên nghiệp

Package BenchmarkTools.jl cung cấp các macro cho accurate benchmarking:

```

1 using Pkg
2 Pkg.add("BenchmarkTools")
3
4 using BenchmarkTools
5
6 # @benchmark - Chạy nhiều lần, thống kê chi tiết
7 @benchmark sum_squares(10^6)
8 # BenchmarkTools.Trial: 1000 samples
9 # Range (min ... max): 2.1 ms ... 3.5 ms
10 # Memory estimate: 0 bytes
11 # Allocs estimate: 0
12
13 # @btime - Phiên bản ngắn gọn của @benchmark
14 @btime sum_squares(10^6)
15 # => 2.134 ms (0 allocations: 0 bytes)

```

**Interpolation với \$:** Để tránh global variable overhead, dùng \$ để interpolate:

```

1 data = rand(1000)
2
3 # SAI: data là global variable -> type instability
4 @btime sum(data)
5 # => 45.234 us (chậm do global lookup)
6
7 # ĐÚNG: Dùng $ để interpolate giá trị
8 @btime sum($data)
9 # => 1.234 us (nhanh hơn nhiều!)

```

**Setup phase:** Với setup argument để chuẩn bị data:

```

1 # Setup được chạy trước mỗi sample nhưng không tính vào timing
2 @btime sort($data) setup=(data = rand(1000))

```

#### 9.1.4 Đọc kết quả benchmark

```

1 result = @benchmark sum_squares(10^6)
2
3 # Truy cập statistics
4 minimum(result)      # Thời gian nhanh nhất
5 median(result)        # Giá trị trung vị (ổn định)
6 mean(result)          # Giá trị trung bình
7 maximum(result)       # Thời gian chậm nhất
8
9 # Memory statistics
10 result.memory         # Bytes allocated
11 result.allocs         # Số allocations

```

#### Mẹo

Dùng minimum hoặc median để đánh giá performance. mean và maximum thường bị ảnh hưởng bởi outliers (GC, OS scheduling).

### 9.1.5 So sánh implementations

```
1  # Hai implementations khác nhau
2  function sum_v1(arr)
3      total = 0.0
4      for x in arr
5          total += x
6      end
7      return total
8  end
9
10 function sum_v2(arr)
11     return reduce(+, arr; init=0.0)
12 end
13
14 # So sánh
15 data = rand(10000)
16 println("Version 1:")
17 @btime sum_v1($data)
18
19 println("Version 2:")
20 @btime sum_v2($data)
21
22 println("Built-in sum:")
23 @btime sum($data)
```

## 9.2 Type stability và @code\_warntype

Type stability là một trong những khái niệm quan trọng nhất để viết Julia code hiệu quả. Khi code type-stable, Julia compiler có thể tạo optimized machine code tương đương với C hoặc Fortran.

### 9.2.1 Type stability là gì?

**Định nghĩa:** Một function được gọi là *type-stable* khi kiểu trả về chỉ phụ thuộc vào *types* của arguments, không phụ thuộc vào *values* của chúng.

```
1  # TYPE-STABLE: Return type luôn là typeof(x)
2  function double_stable(x)
3      return 2 * x
4  end
5
6  double_stable(5)      # => 10 (Int64)
7  double_stable(5.0)    # => 10.0 (Float64)
8
9  # TYPE-UNSTABLE: Return type phụ thuộc vào VALUE của x
10 function pos_unstable(x)
11     if x < 0
12         return 0      # Int64
13     else
14         return x      # typeof(x) - có thể là Float64
15     end
16 end
```

```

17
18 pos_unstable(-2.5)    # => 0 (Int64)
19 pos_unstable(2.5)     # => 2.5 (Float64)

```

### Tại sao type stability quan trọng?

Khi function type-stable, Julia compiler có thể:

- Infer types tại compile time mà không cần chạy code
- Generate specialized machine code cho mỗi type combination
- Avoid boxing values (wrapping trong container)
- Enable aggressive optimizations như inlining, SIMD

#### Cảnh báo

Type instability có thể làm code chậm đi 10–100 lần so với type-stable version, vì Julia phải dùng dynamic dispatch tại runtime.

### 9.2.2 Phát hiện type instability với @code\_warntype

Macro @code\_warntype hiển thị type inference results, highlight những chỗ type không xác định:

```

1  # Function type-unstable
2  function pos_bad(x)
3      if x < 0
4          return 0
5      else
6          return x
7      end
8  end
9
10 @code_warntype pos_bad(2.5)

```

Output sẽ hiển thị:

- `Body::Union{Float64, Int64}` — Return type không xác định (đỏ = warning)
- Các biến với type Any hoặc `Union{...}` là dấu hiệu type instability

#### Mẹo

Trong output của @code\_warntype: màu đỏ (hoặc UNION) = type instability cần fix. Màu xanh/trắng = type đã xác định, tốt.

### 9.2.3 Các nguyên nhân phổ biến của type instability

#### 1. Trả về types khác nhau dựa trên value:

```

1  # BAD: Trả về Int hoặc String
2  function describe(x)
3      if x > 0
4          return x
5      else

```

```
6         return "negative" # String!
7     end
8 end
9
10 # GOOD: Luôn trả về cùng type
11 function describe_fixed(x)
12     if x > 0
13         return string(x) # String
14     else
15         return "negative" # String
16     end
17 end
```

## 2. Sử dụng global variables:

```
1 # BAD: global_data không có type annotation
2 global_data = [1, 2, 3]
3
4 function sum_global_bad()
5     return sum(global_data) # Type của global_data = Any
6 end
7
8 # GOOD 1: Khai báo type cho global
9 const GLOBAL_DATA::Vector{Int} = [1, 2, 3]
10
11 # GOOD 2: Pass data as argument
12 function sum_data(data::Vector{Int})
13     return sum(data)
14 end
```

## 3. Empty containers không có type:

```
1 # BAD: Empty array không có type
2 function build_array_bad(n)
3     arr = [] # Vector{Any}
4     for i in 1:n
5         push!(arr, i)
6     end
7     return arr
8 end
9
10 # GOOD: Specify element type
11 function build_array_good(n)
12     arr = Int[] # Vector{Int}
13     for i in 1:n
14         push!(arr, i)
15     end
16     return arr
17 end
```

### 9.2.4 Strategies để đạt type stability

#### 1. Dùng **zero(x)** và **one(x)**:

```

1  # BAD: Hardcoded 0 là Int64
2  function pos_bad(x)
3      if x < 0
4          return 0 # Int64, không match Float64 input
5      else
6          return x
7      end
8  end
9
10 # GOOD: zero(x) trả về 0 với cùng type như x
11 function pos_good(x)
12     if x < 0
13         return zero(x) # Float64(0.0) nếu x là Float64
14     else
15         return x
16     end
17 end
18
19 @code_warntype pos_good(2.5) # Body::Float64 - green!

```

## 2. Dùng Union splitting (Julia 1.8+):

```

1  # Julia có thể optimize small Unions
2  function maybe_sqrt(x)
3      if x >= 0
4          return sqrt(x)
5      else
6          return nothing # Union{Float64, Nothing}
7      end
8  end
9
10 # Julia 1.8+ handles Union{T, Nothing} efficiently

```

## 3. Function barriers:

```

1  # BAD: Type instability trong hot loop
2  function process_bad(data)
3      result = [] # Any[]
4      for x in data
5          push!(result, x^2)
6      end
7      return sum(result)
8  end
9
10 # GOOD: Barrier function với typed inner loop
11 function process_good(data)
12     result = similar(data) # Same type as input
13     return _process_inner!(result, data)
14 end
15
16 function _process_inner!(result, data)
17     for i in eachindex(data)
18         result[i] = data[i]^2

```

```

19     end
20     return sum(result)
21 end

```

### 9.2.5 Các công cụ phân tích khác

```

1  # @code_llvm - Xem LLVM IR
2  @code_llvm sum_squares(1000)
3
4  # @code_native - Xem assembly code
5  @code_native sum_squares(1000)
6
7  # @code_typed - Xem typed AST
8  @code_typed sum_squares(1000)
9
10 # @code_lowered - Xem lowered AST
11 @code_lowered sum_squares(1000)

```

#### Mẹo

Thứ tự phân tích: @code\_warntype (type stability) → @code\_llvm (optimization) → @code\_native (final assembly). Bắt đầu với @code\_warntype.

## 9.3 Memory management và allocation

Memory allocation là một trong những bottlenecks phổ biến nhất trong Julia code. Hiểu cách Julia quản lý memory giúp viết code hiệu quả hơn.

### 9.3.1 Stack vs Heap allocation

Julia sử dụng hai vùng nhớ chính:

- **Stack:** Nhanh, tự động giải phóng khi function return. Dùng cho small, fixed-size values (primitives, small immutable structs)
- **Heap:** Chậm hơn, cần garbage collection. Dùng cho arrays, mutable structs, large objects

```

1  # Stack allocation - nhanh
2  function stack_example()
3      x = 5          # Int64 - stack
4      y = 3.14       # Float64 - stack
5      z = (1, 2, 3)  # Tuple - stack (immutable, small)
6      return x + y
7  end
8
9  # Heap allocation - chậm hơn
10 function heap_example()
11     arr = [1, 2, 3, 4, 5] # Array - heap
12     dict = Dict{"a" => 1} # Dict - heap
13     return sum(arr)
14 end
15

```



```

16 @btime stack_example() # ~1 ns, 0 allocations
17 @btime heap_example()  # ~50 ns, 2 allocations

```

### Ghi chú

Immutable structs nhỏ (< 16 bytes) thường được allocate trên stack. Mutable structs luôn allocate trên heap.

### 9.3.2 Đo lường allocations

```

1  # @allocated - Đếm bytes allocated
2  bytes = @allocated sum([1, 2, 3, 4, 5])
3  println("Allocated $bytes bytes")
4
5  # @time hiển thị cả allocations
6  @time begin
7      data = rand(1000000)
8      result = sum(data)
9  end
10 # => 0.005 seconds (2 allocations: 7.629 MiB)
11
12 # BenchmarkTools cho accurate measurement
13 using BenchmarkTools
14 @btime sum(rand(1000))
15 # => 1.234 μs (1 allocation: 7.94 KiB)

```

### 9.3.3 Garbage Collection (GC)

Julia sử dụng garbage collector tự động để giải phóng memory không còn được sử dụng.

```

1  # Xem GC statistics
2  GC.gc() # Force garbage collection
3  println(Base.gc_num()) # GC statistics
4
5  # Tạm disable GC (cẩn thận!)
6  GC.enable(false)
7  # ... performance-critical code ...
8  GC.enable(true)
9  GC.gc()

```

#### Khi nào GC chạy?

- Khi heap memory vượt threshold (mặc định 300MB)
- Khi gọi `GC.gc()` explicitly
- Tại allocation sites khi cần

### Cảnh báo

GC có thể gây “stop-the-world” pauses, làm gián đoạn execution. Giảm allocations = giảm GC pressure = code nhanh hơn và ổn định hơn.

### 9.3.4 Strategies để giảm allocations

#### 1. Preallocate buffers:

```
1  # BAD: Tạo array mới mỗi lần gọi
2  function compute_bad(n)
3      result = zeros(n)  # Allocate mỗi lần
4      for i in 1:n
5          result[i] = i^2
6      end
7      return result
8  end
9
10 # GOOD: Preallocate và reuse
11 function compute_good!(result, n)
12     for i in 1:n
13         result[i] = i^2
14     end
15     return result
16 end
17
18 # Usage
19 buffer = zeros(1000)
20 @btime compute_bad(1000)          # 1.5 μs, 1 allocation
21 @btime compute_good!($buffer, 1000) # 0.5 μs, 0 allocations
```

#### 2. In-place operations (functions ending with !):

```
1  data = rand(1000)
2
3  # BAD: Tạo new array
4  sorted = sort(data)          # Allocates new array
5
6  # GOOD: Sort in-place
7  sort!(data)                  # Modifies data directly, no allocation
8
9  # Các in-place functions phổ biến:
10 # push!, pop!, append!, empty!
11 # sort!, reverse!, shuffle!
12 # mul!, add!, sub! (LinearAlgebra)
```

#### 3. Sử dụng views thay vì copies:

```
1  matrix = rand(1000, 1000)
2
3  # BAD: Tạo copy của column
4  col = matrix[:, 1]          # Allocates new array
5  @btime sum($matrix[:, 1])    # 1.5 μs, 1 allocation
6
7  # GOOD: View không copy data
8  col_view = @view matrix[:, 1]
9  @btime sum(@view $matrix[:, 1]) # 0.5 μs, 0 allocations
10
```

```

11 # Hoặc dùng @views macro cho cả block
12 @views begin
13     a = matrix{1:10, 1:10}
14     b = matrix{:, 5}
15     c = sum(a) + sum(b)
16 end

```

#### 4. Avoid string concatenation in loops:

```

1 # BAD: String concatenation tạo new strings
2 function concat_bad(n)
3     s = ""
4     for i in 1:n
5         s = s * string(i) # New allocation mỗi lần
6     end
7     return s
8 end
9
10 # GOOD: Dùng IOBuffer
11 function concat_good(n)
12     io = IOBuffer()
13     for i in 1:n
14         print(io, i)
15     end
16     return String(take!(io))
17 end
18
19 @btime concat_bad(1000) # 500 μs, 1000 allocations
20 @btime concat_good(1000) # 20 μs, 2 allocations

```

#### 5. StaticArrays cho small fixed-size arrays:

```

1 using StaticArrays
2
3 # Regular array - heap allocated
4 regular = [1.0, 2.0, 3.0]
5
6 # StaticArray - stack allocated
7 static = SVector{3, Float64}(1.0, 2.0, 3.0)
8
9 function dot_product(a, b)
10     return sum(a .* b)
11 end
12
13 @btime dot_product($regular, $regular) # 50 ns, 1 allocation
14 @btime dot_product($static, $static) # 2 ns, 0 allocations

```

#### Mẹo

StaticArrays.jl rất hữu ích cho vectors/matrices nhỏ (< 100 elements). Với arrays lớn, regular Array hiệu quả hơn.

## 9.4 Profiling tools và techniques

Profiling giúp xác định những phần code chiếm nhiều thời gian nhất (bottlenecks), từ đó tập trung optimization efforts hiệu quả.

### 9.4.1 Built-in profiling với @profile

Julia cung cấp profiler trong standard library:

```

1  using Profile
2
3  # Function để profile
4  function expensive_computation(n)
5      result = 0.0
6      for i in 1:n
7          for j in 1:n
8              result += sin(i * j)
9          end
10     end
11     return result
12 end
13
14 # Chạy function 1 lần để compile (tránh profile JIT)
15 expensive_computation(10)
16
17 # Profile
18 @profile expensive_computation(1000)
19
20 # Xem kết quả
21 Profile.print()
22
23 # Clear profile data
24 Profile.clear()

```

Đọc kết quả `Profile.print()`:

```

1  # Output format:
2  # [samples] file:line function
3  #   80 expensive_computation(n::Int64)
4  #   45 @Base sin(x::Float64)
5  #   35 expensive_computation loop body

```

Số samples càng cao = function đó chiếm càng nhiều thời gian.

#### Ghi chú

Profiler hoạt động bằng sampling: định kỳ kiểm tra xem code đang chạy ở đâu. Chạy code lâu hơn sẽ cho kết quả chính xác hơn.

### 9.4.2 ProfileView.jl — Visualization với Flame Graphs

Flame graphs giúp visualize profiling results trực quan hơn:

```

1 using Pkg
2 Pkg.add("ProfileView")
3
4 using Profile, ProfileView
5
6 # Profile function
7 @profile expensive_computation(1000)
8
9 # Mở flame graph viewer
10 ProfileView.view()

```

### Đọc Flame Graph:

- Trục X: Thời gian (width của bar = thời gian chiếm)
- Trục Y: Call stack (bottom = entry point, top = deepest call)
- Màu: Thường ngẫu nhiên, giúp phân biệt functions
- Hover: Hiện thị function name và source location

#### 9.4.3 ProfileSVG.jl và PProf.jl

```

1 # ProfileSVG - Export flame graph ra SVG file
2 using ProfileSVG
3 @profile expensive_computation(1000)
4 ProfileSVG.save("profile.svg")
5
6 # PProf - Interactive profiler (tương tự Google's pprof)
7 using PProf
8 @profile expensive_computation(1000)
9 pprof() # Mở web interface tại localhost:57599

```

#### 9.4.4 Memory profiling với `--track-allocation`

Để tìm xem memory được allocate ở đâu:

```

1 # Chạy Julia với --track-allocation=user
2 # julia --track-allocation=user script.jl
3
4 # Trong script:
5 function allocating_function(n)
6     result = zeros(n) # Allocation ở đây
7     for i in 1:n
8         result[i] = i^2
9     end
10    return result
11 end
12
13 # Sau khi chạy, Julia tạo file .mem với annotations:
14 #         - function allocating_function(n)
15 # 8000000    result = zeros(n) # 8MB allocated
16 #         -     for i in 1:n

```

### 9.4.5 TimerOutputs.jl — Detailed timing

```

1  using TimerOutputs
2
3  const to = TimerOutput()
4
5  function complex_pipeline(data)
6      @timeit to "preprocessing" begin
7          processed = preprocess(data)
8      end
9
10     @timeit to "computation" begin
11         result = heavy_compute(processed)
12     end
13
14     @timeit to "postprocessing" begin
15         final = postprocess(result)
16     end
17
18     return final
19 end
20
21 complex_pipeline(rand(1000))
22
23 # Print timing breakdown
24 show(to)
25 # -----
26 #              Time           Allocations
27 #              -----
28 # Total:           50.3ms          10.5 MiB
29 #  preprocessing  10.2ms          2.1 MiB
30 #   computation   35.1ms          7.2 MiB
31 #  postprocessing   5.0ms          1.2 MiB

```

### 9.4.6 Identifying và prioritizing bottlenecks

**Quy tắc 80/20:** Thường 80% thời gian dành cho 20% code. Tập trung optimize những phần đó trước.

```

1  # Workflow tìm bottlenecks:
2  # 1. Profile toàn bộ program
3  @profile main_function()
4  Profile.print(mincount=50) # Chỉ hiển thị functions với >= 50 samples
5
6  # 2. Xác định top functions
7  # Tìm functions chiếm > 10% total time
8
9  # 3. Drill down vào từng function
10 # Profile riêng function đó với input lớn hơn
11
12 # 4. Optimize và verify
13 # Sau mỗi optimization, benchmark lại

```

**Mẹo**

Optimize the right thing: Profile trước, optimize sau. Đừng đoán — đo lường. “Premature optimization is the root of all evil” — Donald Knuth.

## 9.5 Optimization techniques

Sau khi xác định bottlenecks qua profiling, áp dụng các kỹ thuật optimization sau.

### 9.5.1 Avoiding global variables

Global variables là nguyên nhân phổ biến nhất làm Julia code chậm:

```

1  # BAD: Global variable - type không xác định
2  x = 10
3  data = rand(1000)
4
5  function sum_global()
6      total = 0.0
7      for i in 1:x # x là global -> type = Any
8          total += data[i] # data là global -> type = Any
9      end
10     return total
11 end
12
13 @btime sum_global() # Rất chậm!
14
15 # GOOD 1: Pass as arguments
16 function sum_args(data, n)
17     total = 0.0
18     for i in 1:n
19         total += data[i]
20     end
21     return total
22 end
23
24 @btime sum_args($data, 10) # Nhanh hơn nhiều
25
26 # GOOD 2: Dùng const cho globals không đổi
27 const X = 10
28 const DATA = rand(1000)
29
30 function sum_const()
31     total = 0.0
32     for i in 1:X
33         total += DATA[i]
34     end
35     return total
36 end
37
38 @btime sum_const() # Nhanh như version với arguments
```

**Cảnh báo**

Với `const`, giá trị có thể thay đổi nhưng type không được thay đổi. Nếu cần thay đổi type, dùng `Ref` hoặc `pass as argument`.

**9.5.2 Loop optimization****1. Loop order — Column-major:**

Julia arrays là column-major (giống Fortran, khác C). Iterate theo columns trước:

```

1 matrix = rand(1000, 1000)
2
3 # BAD: Row-major order (cache unfriendly)
4 function sum_rowmajor(m)
5     total = 0.0
6     for i in 1:size(m, 1)      # Row
7         for j in 1:size(m, 2)  # Column
8             total += m[i, j]
9         end
10    end
11    return total
12 end
13
14 # GOOD: Column-major order (cache friendly)
15 function sum_colmajor(m)
16     total = 0.0
17     for j in 1:size(m, 2)      # Column first
18         for i in 1:size(m, 1)  # Row second
19             total += m[i, j]
20         end
21     end
22     return total
23 end
24
25 @btime sum_rowmajor($matrix) # ~5 ms
26 @btime sum_colmajor($matrix) # ~1 ms (5x faster!)

```

**2. Dùng @simd và @inbounds:**

```

1 function dot_naive(a, b)
2     result = 0.0
3     for i in eachindex(a)
4         result += a[i] * b[i]
5     end
6     return result
7 end
8
9 function dot_optimized(a, b)
10    result = 0.0
11    @inbounds @simd for i in eachindex(a)
12        result += a[i] * b[i]
13    end
14    return result

```



```

15 end
16
17 a, b = rand(10000), rand(10000)
18 @btime dot_naive($a, $b)      # 5.2 μs
19 @btime dot_optimized($a, $b)  # 1.1 μs

```

- @inbounds: Bỏ bounds checking (chỉ dùng khi chắc chắn không out-of-bounds)
- @simd: Hint cho compiler dùng SIMD vectorization

### 3. Loop fusion với broadcasting:

```

1 a, b, c = rand(1000), rand(1000), rand(1000)
2
3 # BAD: Tạo temporary arrays
4 function compute_bad(a, b, c)
5     t1 = a .+ b      # Temporary
6     t2 = t1 .* c      # Temporary
7     return sum(t2)
8 end
9
10 # GOOD: Fused với @.
11 function compute_good(a, b, c)
12     return sum(@. (a + b) * c) # No temporaries
13 end
14
15 @btime compute_bad($a, $b, $c) # 2.5 μs, 2 allocations
16 @btime compute_good($a, $b, $c) # 1.2 μs, 0 allocations

```

### 9.5.3 Multiple dispatch cho specialization

Tận dụng multiple dispatch để tạo specialized methods:

```

1 # Generic fallback
2 function process(x)
3     # Slow generic implementation
4     return sum(x)
5 end
6
7 # Specialized for Vector{Float64}
8 function process(x::Vector{Float64})
9     # Fast BLAS implementation
10    return BLAS.asum(length(x), x, 1)
11 end
12
13 # Specialized for small StaticArrays
14 using StaticArrays
15 function process(x::SVector{N, Float64}) where N
16     # Unrolled for small arrays
17     result = 0.0
18     @inbounds for i in 1:N
19         result += abs(x[i])
20     end
21     return result
22 end

```

### 9.5.4 Algorithmic improvements

Đôi khi thay đổi algorithm quan trọng hơn micro-optimization:

```

1  # O(n^2) - Quadratic
2  function find_duplicates_naive(arr)
3      duplicates = []
4      for i in 1:length(arr)
5          for j in i+1:length(arr)
6              if arr[i] == arr[j]
7                  push!(duplicates, arr[i])
8              end
9          end
10     end
11     return unique(duplicates)
12 end
13
14 # O(n) - Linear với Set
15 function find_duplicates_fast(arr)
16     seen = Set{eltype(arr)}()
17     duplicates = Set{eltype(arr)}()
18     for x in arr
19         if x in seen
20             push!(duplicates, x)
21         else
22             push!(seen, x)
23         end
24     end
25     return collect(duplicates)
26 end
27
28 data = rand(1:1000, 10000)
29 @btime find_duplicates_naive($data)  # 50 ms
30 @btime find_duplicates_fast($data)   # 0.5 ms (100x faster!)

```

#### Mẹo

Trước khi micro-optimize, hãy xem xét: Có algorithm tốt hơn không?  $O(n \log n)$  vs  $O(n^2)$  quan trọng hơn constant factor.

## 9.6 Best practices và common pitfalls

### 9.6.1 Performance Do's and Don'ts

DO:

- Benchmark trước khi optimize (dùng BenchmarkTools.jl)
- Profile để tìm bottlenecks thực sự
- Viết type-stable functions
- Dùng const cho global constants

- Preallocate buffers và dùng in-place operations (!)
- Iterate column-major cho matrices
- Dùng @view thay vì slicing
- Dùng StaticArrays cho small fixed-size arrays

**DON'T:**

- Optimize mà không đo lường trước
- Dùng global variables trong hot loops
- Tạo temporary arrays không cần thiết
- Bỏ qua type instability warnings
- Dùng abstract types trong containers (Vector{Any})
- Concatenate strings trong loops với \*
- Dùng push! khi biết trước size (dùng preallocate)

**9.6.2 Performance tips quick reference**

| Issue             | Bad                           | Good                                     |
|-------------------|-------------------------------|------------------------------------------|
| Global vars       | <code>x = 10; f() = x</code>  | <code>const X = 10</code> hoặc argument  |
| Type instability  | <code>x &gt; 0 ? x : 0</code> | <code>x &gt; 0 ? x : zero(x)</code>      |
| Untyped container | <code>arr = []</code>         | <code>arr = Int[]</code>                 |
| Slicing           | <code>m[:, 1]</code>          | <code>@view m[:, 1]</code>               |
| String concat     | <code>s *= str</code>         | <code>IOBuffer</code>                    |
| Loop order        | <code>for i, for j</code>     | <code>for j, for i</code> (column-major) |
| Temporary arrays  | <code>a .+ b .* c</code>      | <code>@. a + b * c</code>                |

Bảng 9.1: Performance anti-patterns và solutions

**9.6.3 Readability vs Performance trade-offs**

Không phải lúc nào cũng cần optimize tối đa. Cân nhắc:

- **Hot path:** Code chạy nhiều lần (inner loops, frequently called functions) — optimize mạnh
- **Cold path:** Code chỉ chạy một lần (initialization, config loading) — ưu tiên readability
- **Prototype vs Production:** Prototype có thể chậm, production cần optimize

```

1 # READABLE (cold path - OK)
2 function load_config(filename)
3     content = read(filename, String)
4     return JSON.parse(content)
5 end
6
7 # OPTIMIZED (hot path - necessary)
8 function process_frame!(buffer, data)
9     @inbounds @simd for i in eachindex(data)

```

```

10         buffer[i] = transform(data[i])
11     end
12 end

```

### Ghi chú

“Make it work, make it right, make it fast” — Kent Beck. Đầu tiên viết code đúng và dễ hiểu, sau đó optimize những phần thực sự cần thiết.

## 9.7 Tóm tắt chương

- **Benchmarking:** Sử dụng `BenchmarkTools.jl` với `@benchmark` và `@btime` để đo lường hiệu năng chính xác. Luôn dùng `$` để interpolate variables, tránh global variable overhead.
- **Type stability:** Code type-stable cho phép Julia compiler tạo optimized machine code. Dùng `@code_warntype` để phát hiện type instability, fix bằng `zero(x)`, typed containers, và function barriers.
- **Memory management:** Giảm allocations bằng cách preallocate buffers, dùng in-place operations (!), views thay vì slicing, và `StaticArrays` cho small arrays. Tránh string concatenation trong loops.
- **Profiling:** Dùng `Profile.jl` và `ProfileView.jl` (flame graphs) để tìm bottlenecks. `TimerOutputs.jl` cho detailed timing breakdown. Profile trước, optimize sau.
- **Optimization techniques:** Tránh global variables, iterate column-major, dùng `@simd` và `@inbounds`, loop fusion với `@..`. Algorithmic improvements thường quan trọng hơn micro-optimization.

## 9.8 Bài tập

### 9.8.1 Bài tập cơ bản

#### Bài tập: Benchmarking cơ bản

Viết hai function tính tổng bình phương các số từ 1 đến n:

1. Version 1: Dùng loop với accumulator
2. Version 2: Dùng sum với generator expression

Benchmark cả hai với `@btime` và  $n = 10^6$ . Giải thích kết quả.

**Bài tập: Phát hiện type instability**

Function sau đây type-unstable. Tìm nguyên nhân bằng @code\_warntype và fix:

```

1 function safe_sqrt(x)
2     if x < 0
3         return "Error: negative input"
4     else
5         return sqrt(x)
6     end
7 end

```

**Bài tập: Giảm allocations**

Function sau allocate nhiều memory không cần thiết. Viết lại để đạt 0 allocations:

```

1 function process_data(data)
2     filtered = filter(x -> x > 0, data)
3     squared = map(x -> x^2, filtered)
4     return sum(squared)
5 end

```

**9.8.2 Bài tập nâng cao****Bài tập: Profiling và optimization**

Cho function sau:

```

1 function matrix_operation(n)
2     A = rand(n, n)
3     B = rand(n, n)
4     result = zeros(n, n)
5     for i in 1:n
6         for j in 1:n
7             for k in 1:n
8                 result[i, j] += A[i, k] * B[k, j]
9             end
10        end
11    end
12    return result
13 end

```

1. Profile function với  $n = 500$
2. Tìm bottleneck chính
3. Optimize (hint: loop order, BLAS)
4. Verify speedup bằng benchmark

**Bài tập: Memory optimization với views**

Viết function `column_sums(matrix)` tính tổng mỗi cột của matrix mà không allocate memory cho intermediate results. So sánh với version dùng slicing `matrix[:, j]`.

### 9.8.3 Bài tập thách thức

#### Bài tập: Full optimization pipeline

Cho đoạn code tính ma trận khoảng cách giữa các điểm:

```

1  function distance_matrix_slow(points)
2      n = length(points)
3      D = []
4      for i in 1:n
5          row = []
6          for j in 1:n
7              d = sqrt(sum((points[i] .- points[j]).^2))
8              push!(row, d)
9          end
10         push!(D, row)
11     end
12     return D
13 end

```

Yêu cầu: Optimize để đạt speedup ít nhất 50x cho 1000 điểm 3D. Các bước:

1. Benchmark version gốc
2. Profile và xác định bottlenecks
3. Fix type instability
4. Giảm allocations
5. Dùng efficient data structures
6. Verify final speedup

#### Bài tập: Real-world optimization

Tìm một function trong project của bạn (hoặc viết simulation đơn giản như Monte Carlo). Áp dụng quy trình optimization đầy đủ: benchmark → profile → optimize → verify. Document mỗi bước và speedup đạt được.

### Đọc thêm

- Julia Documentation: Performance Tips — <https://docs.julialang.org/en/v1/manual/performance-tips/>
- BenchmarkTools.jl documentation — <https://juliaci.github.io/BenchmarkTools.jl/stable/>
- ProfileView.jl và flame graphs — <https://github.com/timholly/ProfileView.jl>
- StaticArrays.jl cho small arrays — <https://github.com/JuliaArrays/StaticArrays.jl>
- Julia High Performance (2nd Edition) — Avik Sengupta, Packt Publishing

## Chương 10

# Tính toán song song và GPU

### Mục tiêu học tập

- Hiểu các khái niệm parallel computing trong Julia
- Sử dụng multithreading với `Threads.@threads`
- Distributed computing với `Distributed.jl`
- GPU programming với `CUDA.jl`
- So sánh performance và chọn strategy phù hợp

### 10.1 Giới thiệu parallel computing

#### 10.1.1 Tại sao cần parallel computing?

Trong thời đại hiện nay, phần cứng máy tính đã phát triển theo hướng đa lõi (multi-core) thay vì tăng tốc độ xung nhịp đơn lẻ. CPU desktop phổ thông có 8–16 lõi, server có thể lên đến 64–128 lõi, và GPU có hàng nghìn CUDA cores. Để tận dụng tối đa sức mạnh phần cứng này, chúng ta cần **parallel computing**—khả năng thực hiện nhiều tính toán đồng thời.

**Các lý do chính cần parallel computing:**

- **Tốc độ:** Giảm thời gian tính toán từ giờ xuống phút, từ ngày xuống giờ
- **Quy mô:** Xử lý dữ liệu lớn không vừa trong bộ nhớ một máy
- **Hiệu quả:** Tận dụng tài nguyên phần cứng đắt tiền
- **Thực tế:** Nhiều bài toán khoa học đòi hỏi tính toán song song

Julia được thiết kế từ đầu với parallel computing trong tâm trí, cung cấp các công cụ mạnh mẽ và dễ sử dụng cho việc lập trình song song<sup>1</sup>.

#### 10.1.2 Parallel computing models trong Julia

Julia hỗ trợ ba mô hình parallel computing chính, mỗi mô hình phù hợp với các trường hợp sử dụng khác nhau:

##### 1. Multithreading (Đa luồng):

---

<sup>1</sup>Julia Documentation: Parallel Computing, <https://docs.julialang.org/en/v1/manual/parallel-computing/>

- Nhiều threads chia sẻ cùng một không gian bộ nhớ
- Phù hợp cho parallel loops và tác vụ CPU-intensive
- Overhead thấp, communication nhanh qua shared memory
- Module: `Base.Threads`

## 2. Distributed Computing (Tính toán phân tán):

- Nhiều processes với không gian bộ nhớ riêng biệt
- Có thể chạy trên nhiều máy qua network
- Phù hợp cho dữ liệu lớn và tính toán scale-out
- Module: `Distributed.jl`

## 3. GPU Computing:

- Sử dụng hàng nghìn cores của GPU
- Phù hợp cho SIMD workloads (matrix operations, deep learning)
- Throughput cực cao cho các tác vụ data-parallel
- Packages: `CUDA.jl`, `AMDGPU.jl`, `Metal.jl`

### 10.1.3 Khi nào dùng model nào?

Việc chọn đúng parallel computing model ảnh hưởng lớn đến hiệu năng và độ phức tạp code:

| Tiêu chí      | Threads         | Distributed | GPU         |
|---------------|-----------------|-------------|-------------|
| Shared memory | Có              | Không       | Có (device) |
| Cross-machine | Không           | Có          | Không       |
| Overhead      | Thấp            | Cao         | Trung bình  |
| Best for      | CPU-bound loops | Big data    | Matrix ops  |
| Complexity    | Thấp            | Trung bình  | Cao         |

Bảng 10.1: So sánh các mô hình parallel computing

#### Hướng dẫn chọn:

- Dùng **Threads** khi: Dữ liệu vừa trong RAM, tác vụ CPU-intensive, cần latency thấp
- Dùng **Distributed** khi: Dữ liệu lớn hơn RAM, cần nhiều máy, fault tolerance
- Dùng **GPU** khi: Nhiều phép tính giống nhau trên dữ liệu lớn, matrix/tensor operations

## 10.2 Multithreading với Threads

### 10.2.1 Setup Julia với multithreading

Để sử dụng multithreading trong Julia, bạn cần khởi động Julia với số threads mong muốn. Có hai cách chính:

#### Cách 1: Sử dụng command-line option:



```

1 # Khởi động Julia với 4 threads
2 julia -t4
3 # hoặc
4 julia --threads=4
5
6 # Sử dụng tất cả cores có sẵn
7 julia -t auto

```

### Cách 2: Sử dụng environment variable:

```

1 # Trong terminal (trước khi chạy Julia)
2 export JULIA_NUM_THREADS=4
3 julia

```

Kiểm tra số threads hiện tại:

```

1 # Check number of threads
2 julia> Threads.nthreads()
3 4
4
5 # Xem thread ID hiện tại
6 julia> Threads.threadid()
7 1

```

#### Ghi chú

Trong VS Code với Julia extension, bạn có thể cấu hình số threads trong settings: Julia: Num Threads<sup>a</sup>.

<sup>a</sup>2023-Julia-For-Data-Analysis.PDF.P399

### 10.2.2 Threads.@threads cho parallel loops

Macro @threads là cách đơn giản nhất để song song hóa vòng lặp:

```

1 using Base.Threads
2
3 # Parallel loop cơ bản
4 Threads.@threads for i in 1:10
5     println("Thread $(Threads.threadid()): processing $i")
6 end

```

#### Ví dụ thực tế - Tính tổng song song:

```

1 function parallel_sum(arr)
2     n = length(arr)
3     nthreads = Threads.nthreads()
4     partial_sums = zeros(nthreads)
5
6     Threads.@threads for i in 1:n
7         tid = Threads.threadid()
8         partial_sums[tid] += arr[i]
9     end
10
11     return sum(partial_sums)

```

```
12 end
13
14 # Test
15 data = rand(10_000_000)
16 @time result = parallel_sum(data)
```

### So sánh performance:

```
1 using BenchmarkTools
2
3 # Sequential version
4 function sequential_sum(arr)
5     total = 0.0
6     for x in arr
7         total += x
8     end
9     return total
10 end
11
12 data = rand(10_000_000)
13
14 @btime sequential_sum($data) # ~ 15 ms
15 @btime parallel_sum($data)  # ~ 4 ms (4 threads)
```

### 10.2.3 Thread safety và race conditions

Khi nhiều threads truy cập cùng một biến, có thể xảy ra **race condition**—kết quả phụ thuộc vào thứ tự thực thi không xác định.

#### Ví dụ race condition:

```
1 # SAI - Race condition!
2 counter = 0
3 Threads.@threads for i in 1:1000
4     global counter += 1 # Nhiều threads ghi đồng thời
5 end
6 println(counter) # Kết quả không xác định, < 1000
```

#### Giải pháp 1: Atomic operations

```
1 # ĐÚNG - Sử dụng Atomic
2 counter = Threads.Atomic{Int}(0)
3 Threads.@threads for i in 1:1000
4     Threads.atomic_add!(counter, 1)
5 end
6 println(counter[]) # Luôn = 1000
```

#### Giải pháp 2: Locks

```
1 # ĐÚNG - Sử dụng Lock
2 counter = 0
3 lock = ReentrantLock()
4
5 Threads.@threads for i in 1:1000
6     lock(lock) do
7         global counter += 1
8     end
9 end
```

```

8     end
9 end
10 println(counter) # Luôn = 1000

```

### Giải pháp 3: Thread-local storage (khuyến dùng)

```

1 # TỐT NHẤT - Mỗi thread có biến riêng
2 function safe_sum_threaded(n)
3     partial = zeros(Threads.nthreads())
4     Threads.@threads for i in 1:n
5         partial[Threads.threadid()] += 1
6     end
7     return sum(partial)
8 end

```

#### Cảnh báo

Tránh thực hiện I/O operations (print, file write) trong multithreaded code vì có thể gây ra output không nhất quán<sup>a</sup>.

<sup>a</sup>2019-Julia-High-Performance.PDF.P166

## 10.2.4 Best practices cho multithreading

### 1. Tránh false sharing:

```

1 # SAI - False sharing (các threads ghi vào vùng nhớ kề nhau)
2 result = zeros(Threads.nthreads())
3 Threads.@threads for i in 1:n
4     result[Threads.threadid()] += compute(i)
5 end
6
7 # TỐT HƠN - Padding để tránh cache line conflicts
8 struct PaddedFloat
9     value::Float64
10    padding::NTuple{7, Float64} # 64-byte cache line
11 end

```

### 2. Load balancing với @threads :static vs :dynamic:

```

1 # Static scheduling (mặc định) - phân chia đều từ đầu
2 Threads.@threads :static for i in 1:n
3     work(i)
4 end
5
6 # Dynamic scheduling - phân chia động khi workload không đều
7 Threads.@threads :dynamic for i in 1:n
8     variable_work(i) # Thời gian khác nhau mỗi iteration
9 end

```

### 3. Đảm bảo thread-safe cho collections:

```

1 # Sử dụng Channel cho thread-safe queue
2 ch = Channel{Int}(100)
3
4 # Producer threads

```

```
5 Threads.@spawn for i in 1:50
6     put!(ch, i)
7 end
8
9 # Consumer thread
10 Threads.@spawn while isready(ch)
11     item = take!(ch)
12     process(item)
13 end
```

## 10.3 Distributed computing

### 10.3.1 Giới thiệu Distributed.jl

Distributed computing trong Julia sử dụng mô hình **multi-process** thay vì multi-thread. Mỗi worker process có không gian bộ nhớ riêng biệt, giao tiếp qua message passing. Điều này mang lại:

- **Isolation:** Một process crash không ảnh hưởng đến các process khác
- **Scalability:** Có thể chạy trên nhiều máy qua network
- **No race conditions:** Không có shared memory nên không có data races
- **Fault tolerance:** Dễ dàng restart workers bị lỗi

```
1 using Distributed
2
3 # Master process (myid() = 1) điều phối workers (myid() > 1)
4 println("Master process ID: ", myid())
```

### 10.3.2 Creating và managing workers

Thêm workers:

```
1 using Distributed
2
3 # Thêm 4 local workers
4 addprocs(4)
5
6 # Kiểm tra workers hiện có
7 workers() # [2, 3, 4, 5]
8 nworkers() # 4
9
10 # Xem tất cả processes (bao gồm master)
11 procs() # [1, 2, 3, 4, 5]
```

Thêm workers trên remote machines:

```
1 # Qua SSH (cần SSH access)
2 addprocs([("user@host1", 2), ("user@host2", 2)])
3
4 # Với custom options
5 addprocs(4, exeflags="--project")
```

Xóa workers:

```

1 # Xóa workers cụ thể
2 rmprocs(5)
3
4 # Xóa tất cả workers
5 rmprocs(workers())

```

**Chạy code trên tất cả workers:**

```

1 using Distributed
2 addprocs(4)
3
4 # @everywhere chạy code trên tất cả processes
5 @everywhere println("Hello from $(myid())")
6
7 # Load package trên tất cả workers
8 @everywhere using LinearAlgebra
9
10 # Define function trên tất cả workers
11 @everywhere function compute(x)
12     return x^2 + sin(x)
13 end

```

### 10.3.3 Remote calls và futures

**@spawnat - Chạy code trên worker cụ thể:**

```

1 # Chạy trên worker 2
2 future = @spawnat 2 begin
3     sleep(1)
4     return rand(3, 3)
5 end
6
7 # Lấy kết quả (blocking)
8 result = fetch(future)

```

**@spawn - Chạy trên worker bất kỳ:**

```

1 # Julia tự chọn worker
2 future = @spawn expensive_computation(data)
3 result = fetch(future)

```

**remotecall - Lower-level API:**

```

1 # remotecall(func, worker_id, args...)
2 future = remotecall(sum, 2, [1, 2, 3, 4, 5])
3 fetch(future) # 15
4
5 # remotecall_fetch - call và fetch trong một bước
6 result = remotecall_fetch(sum, 2, [1, 2, 3, 4, 5])

```

### 10.3.4 Data movement giữa workers

**@fetch - Lấy dữ liệu từ remote:**

```
1  # Chạy và fetch kết quả
2  result = @fetch begin
3      return rand(1000)
4  end
```

#### pmap - Parallel map:

```
1  @everywhere function process_item(x)
2      sleep(0.1) # Simulate work
3      return x^2
4  end
5
6  # pmap phân phối work tự động cho workers
7  data = 1:100
8  results = pmap(process_item, data)
```

#### @distributed - Distributed loop:

```
1  # Distributed reduction
2  total = @distributed (+) for i in 1:1000000
3      sin(i)
4  end
5
6  # Distributed với local aggregation
7  @distributed for i in 1:100
8      # Mỗi worker xử lý một phần
9      process_chunk(i)
10 end
```

### 10.3.5 Parallel patterns: map-reduce

#### Pattern 1: Parallel Map

```
1  # Xử lý files song song
2  @everywhere function analyze_file(filename)
3      data = read(filename, String)
4      return length(data), count('\n', data)
5  end
6
7  files = readdir("data/", join=true)
8  results = pmap(analyze_file, files)
```

#### Pattern 2: Map-Reduce

```
1  # Word count distributed
2  @everywhere function count_words(text)
3      words = split(lowercase(text))
4      counts = Dict{String, Int}()
5      for w in words
6          counts[w] = get(counts, w, 0) + 1
7      end
8      return counts
9  end
10
11 @everywhere function merge_counts(d1, d2)
```

```

12     result = copy(d1)
13     for (k, v) in d2
14         result[k] = get(result, k, 0) + v
15     end
16     return result
17 end
18
19 # Map phase
20 partial_counts = pmap(count_words, text_chunks)
21 # Reduce phase
22 total_counts = reduce(merge_counts, partial_counts)

```

#### SharedArrays - Shared memory cho same-machine workers:

```

1 using SharedArrays
2
3 # Tạo shared array accessible bởi tất cả workers
4 S = SharedArray{Float64}(1000, 1000)
5
6 # Mỗi worker cập nhật phần của mình
7 @distributed for i in 1:1000
8     for j in 1:1000
9         S[i, j] = compute(i, j)
10    end
11 end

```

## 10.4 GPU computing với CUDA.jl

### 10.4.1 Giới thiệu GPU programming

GPU (Graphics Processing Unit) ban đầu được thiết kế cho đồ họa, nhưng kiến trúc của nó—hàng nghìn cores nhỏ làm việc song song—rất phù hợp cho các tác vụ data-parallel như:

- Ma trận và linear algebra operations
- Deep learning training và inference
- Monte Carlo simulations
- Image và signal processing
- Scientific computing (physics, chemistry simulations)

#### So sánh CPU vs GPU:

| Đặc điểm   | CPU                       | GPU                   |
|------------|---------------------------|-----------------------|
| Cores      | 4-64 (mạnh)               | 1000-10000 (yếu hơn)  |
| Best for   | Sequential, complex logic | Parallel, simple ops  |
| Memory     | RAM lớn (128GB+)          | VRAM nhỏ hơn (8-80GB) |
| Latency    | Thấp                      | Cao                   |
| Throughput | Trung bình                | Rất cao               |

Julia hỗ trợ GPU computing qua các packages:

- CUDA.jl — NVIDIA GPUs (phổ biến nhất)

- `AMDGPU.jl` — AMD GPUs
- `Metal.jl` — Apple Silicon (M1/M2/M3)
- `oneAPI.jl` — Intel GPUs

### 10.4.2 Setup CUDA.jl

#### Yêu cầu:

- NVIDIA GPU với CUDA capability 3.5+
- NVIDIA driver đã cài đặt
- CUDA Toolkit (CUDA.jl có thể tự động download)

#### Cài đặt:

```
1 using Pkg
2 Pkg.add("CUDA")
```

#### Kiểm tra installation:

```
1 using CUDA
2
3 # Kiểm tra CUDA có hoạt động không
4 CUDA.functional() # true nếu OK
5
6 # Xem GPU info
7 CUDA.versioninfo()
8
9 # Xem GPU device
10 CUDA.device() # CuDevice(0): NVIDIA GeForce RTX 3080
```

#### Ghi chú

Sử dụng lệnh `nvidia-smi` trong terminal để kiểm tra trạng thái GPU và driver version<sup>a</sup>.

<sup>a</sup>2019-Julia-High-Performance.PDF.P137

### 10.4.3 CuArray và basic operations

CuArray là array type sống trên GPU memory. Hầu hết operations trên Array đều hoạt động với CuArray:

#### Tạo CuArray:

```
1 using CUDA
2
3 # Chuyển Array từ CPU sang GPU
4 A_cpu = rand(1000, 1000)
5 A_gpu = CuArray(A_cpu)
6
7 # Tạo trực tiếp trên GPU
8 B_gpu = CUDA.rand(1000, 1000)
9 C_gpu = CUDA.zeros(1000, 1000)
10 D_gpu = CUDA.ones(1000, 1000)
```



**Operations tự động chạy trên GPU:**

```

1 using CUDA, LinearAlgebra
2
3 A = CUDA.rand(1000, 1000)
4 B = CUDA.rand(1000, 1000)
5
6 # Tất cả operations chạy trên GPU
7 C = A * B           # Matrix multiplication
8 D = A .+ B          # Element-wise addition
9 E = sin.(A)          # Broadcasting
10 F = A'              # Transpose
11 det_A = det(A)      # Determinant

```

**Chuyển kết quả về CPU:**

```

1 # Collect kết quả về CPU
2 result_cpu = Array{C}
3 # hoặc
4 result_cpu = collect(C)

```

**Benchmark so sánh CPU vs GPU:**

```

1 using CUDA, BenchmarkTools
2
3 n = 5000
4 A_cpu = rand{Float32, n, n}
5 B_cpu = rand{Float32, n, n}
6 A_gpu = CuArray(A_cpu)
7 B_gpu = CuArray(B_cpu)
8
9 # CPU matrix multiplication
10 @btime $A_cpu * $B_cpu; # ~500 ms
11
12 # GPU matrix multiplication
13 @btime CUDA.@sync $A_gpu * $B_gpu; # ~5 ms (100x faster!)

```

**10.4.4 Writing custom CUDA kernels**

Khi cần control chi tiết hơn, bạn có thể viết custom CUDA kernels:

**Kernel cơ bản:**

```

1 using CUDA
2
3 # Kernel function - chạy trên mỗi thread
4 function add_kernel!(C, A, B)
5     i = threadIdx().x + (blockIdx().x - 1) * blockDim().x
6     if i <= length(A)
7         @inbounds C[i] = A[i] + B[i]
8     end
9     return nothing
10 end
11
12 # Sử dụng kernel
13 n = 1024

```

```
14 A = CUDA.rand(n)
15 B = CUDA.rand(n)
16 C = similar(A)
17
18 # Launch kernel với 256 threads/block
19 threads = 256
20 blocks = cld(n, threads)
21 @cuda threads=threads blocks=blocks add_kernel!(C, A, B)
22
23 # Đợi GPU hoàn thành
24 CUDA.synchronize()
```

### 2D kernel cho matrix operations:

```
1 function matrix_add_kernel!(C, A, B)
2     i = threadIdx().x + (blockIdx().x - 1) * blockDim().x
3     j = threadIdx().y + (blockIdx().y - 1) * blockDim().y
4
5     if i <= size(A, 1) && j <= size(A, 2)
6         @inbounds C[i, j] = A[i, j] + B[i, j]
7     end
8     return nothing
9 end
10
11 # Launch 2D kernel
12 A = CUDA.rand(1024, 1024)
13 B = CUDA.rand(1024, 1024)
14 C = similar(A)
15
16 threads = (16, 16)
17 blocks = (cld(size(A,1), 16), cld(size(A,2), 16))
18 @cuda threads=threads blocks=blocks matrix_add_kernel!(C, A, B)
```

#### 10.4.5 Memory management trên GPU

GPU memory là tài nguyên quý, cần quản lý cẩn thận:

##### Kiểm tra memory usage:

```
1 # Xem memory hiện tại
2 CUDA.memory_status()
3
4 # Chi tiết hơn
5 CUDA.available_memory() # bytes còn trống
6 CUDA.total_memory()    # tổng VRAM
```

##### Giải phóng memory:

```
1 # Xóa reference và garbage collect
2 A = CUDA.rand(10000, 10000)
3 A = nothing
4 GC.gc()
5 CUDA.reclaim() # Force reclaim GPU memory
```

##### Tránh memory transfers không cần thiết:

```

1 # SAI - Transfer mỗi iteration
2 for i in 1:1000
3     A_gpu = CuArray(A_cpu) # Transfer 1000 lần!
4     result = process(A_gpu)
5 end
6
7 # ĐÚNG - Transfer một lần
8 A_gpu = CuArray(A_cpu) # Transfer 1 lần
9 for i in 1:1000
10     result = process(A_gpu) # Reuse GPU data
11 end

```

Sử dụng Float32 thay vì Float64:

```

1 # Float32 nhanh hơn 2x trên consumer GPUs
2 A = CUDA.rand(Float32, 1000, 1000) # Khuyến dùng
3
4 # Float64 cần cho precision cao (scientific computing)
5 B = CUDA.rand(Float64, 1000, 1000) # Chậm hơn

```

## 10.5 Performance comparison

### 10.5.1 Benchmarking parallel code

Để đánh giá hiệu quả parallel code, chúng ta cần đo lường các metrics quan trọng:

**Speedup (S):**

$$S = \frac{T_{\text{sequential}}}{T_{\text{parallel}}} \quad (10.1)$$

Speedup lý tưởng với  $n$  cores là  $n$ , nhưng thực tế thường thấp hơn do overhead.

**Efficiency (E):**

$$E = \frac{S}{n} = \frac{T_{\text{sequential}}}{n \times T_{\text{parallel}}} \quad (10.2)$$

Efficiency 100% nghĩa là sử dụng tối ưu tất cả cores. Thực tế thường 60–80%.

**Scalability testing:**

```

1 using BenchmarkTools
2
3 function benchmark_scaling(func, data, max_threads)
4     results = Dict{Int, Float64}()
5
6     for n in [1, 2, 4, 8, 16]
7         if n <= max_threads
8             # Set threads (cần restart Julia với -t n)
9             time = @belapsed $func($data)
10            results[n] = time
11        end
12    end
13
14    # Tính speedup
15    base_time = results[1]
16    for (n, t) in results
17        speedup = base_time / t

```

```

18     efficiency = speedup / n * 100
19     println("Threads: $n, Speedup: $(round(speedup, digits=2))x,
    ↪     Efficiency: $(round(efficiency, digits=1))%")
20 end
21 end

```

### 10.5.2 Threads vs Distributed vs GPU

Mỗi model có strengths và weaknesses khác nhau:

| Workload        | Threads  | Distributed | GPU           |
|-----------------|----------|-------------|---------------|
| CPU-bound loops | Tốt      | Trung bình  | Không phù hợp |
| Matrix multiply | Tốt      | Trung bình  | Xuất sắc      |
| Large datasets  | Giới hạn | Xuất sắc    | Giới hạn VRAM |
| I/O intensive   | Không    | Tốt         | Không         |
| Deep learning   | Không    | Trung bình  | Xuất sắc      |
| Monte Carlo     | Tốt      | Xuất sắc    | Tốt           |

Bảng 10.2: So sánh performance theo loại workload

### 10.5.3 Amdahl's Law và scaling limits

**Amdahl's Law** cho biết giới hạn lý thuyết của speedup:

$$S_{\max} = \frac{1}{(1-p) + \frac{p}{n}} \quad (10.3)$$

Trong đó:

- $p$  = phần trăm code có thể parallelize
- $n$  = số processors
- $S_{\max}$  = speedup tối đa

**Ví dụ:** Nếu 90% code parallelizable ( $p = 0.9$ ):

- 4 cores:  $S = \frac{1}{0.1+0.9/4} = 3.08\times$
- 16 cores:  $S = \frac{1}{0.1+0.9/16} = 6.4\times$
- $\infty$  cores:  $S = \frac{1}{0.1} = 10\times$  (giới hạn!)

#### Ghi chú

Luôn tối ưu phần sequential trước khi parallelize. 10% code không parallel có thể giới hạn speedup tối đa là 10x, bất kể bạn có bao nhiêu cores.

## 10.6 Real-world applications và best practices

### 10.6.1 Case study: Monte Carlo simulation

Monte Carlo simulation là ứng dụng lý tưởng cho parallel computing vì mỗi simulation độc lập:

**Ước tính  $\pi$  với Monte Carlo:**

```

1  # Sequential version
2  function monte_carlo_pi_seq(n)
3      inside = 0
4      for _ in 1:n
5          x, y = rand(), rand()
6          if x^2 + y^2 <= 1.0
7              inside += 1
8          end
9      end
10     return 4.0 * inside / n
11 end
12
13 # Threaded version
14 function monte_carlo_pi_threads(n)
15     inside = zeros{Int, Threads.nthreads()}
16     Threads.@threads for i in 1:n
17         x, y = rand(), rand()
18         if x^2 + y^2 <= 1.0
19             inside[Threads.threadid()] += 1
20         end
21     end
22     return 4.0 * sum(inside) / n
23 end
24
25 # Distributed version
26 using Distributed
27 @everywhere function monte_carlo_chunk(n)
28     inside = 0
29     for _ in 1:n
30         x, y = rand(), rand()
31         if x^2 + y^2 <= 1.0
32             inside += 1
33         end
34     end
35     return inside
36 end
37
38 function monte_carlo_pi_distributed(n, nworkers)
39     chunk_size = n ÷ nworkers
40     futures = [@spawnat w monte_carlo_chunk(chunk_size) for w in workers()]
41     total_inside = sum(fetch.(futures))
42     return 4.0 * total_inside / n
43 end

```

### 10.6.2 Case study: Matrix operations

So sánh matrix multiplication trên các platforms:

```

1  using LinearAlgebra, BenchmarkTools
2  using CUDA
3
4  n = 4096
5  A_cpu = rand{Float32, n, n}

```

```
6 B_cpu = rand(Float32, n, n)
7
8 # CPU với BLAS (đã multi-threaded)
9 @btime $A_cpu * $B_cpu; # ~200 ms (8 threads)
10
11 # GPU
12 A_gpu = CuArray(A_cpu)
13 B_gpu = CuArray(B_cpu)
14 @btime CUDA.@sync $A_gpu * $B_gpu; # ~3 ms (66x faster!)
15
16 # Kết luận: GPU xuất sắc cho matrix operations lớn
```

### 10.6.3 Hybrid parallelism

Kết hợp nhiều strategies cho maximum performance:

```
1 # Hybrid: Distributed workers, mỗi worker dùng multi-threading
2 using Distributed
3 addprocs(4)
4
5 @everywhere begin
6     using Base.Threads
7
8     function hybrid_compute(data_chunk)
9         result = zeros(length(data_chunk))
10         Threads.@threads for i in eachindex(data_chunk)
11             result[i] = expensive_computation(data_chunk[i])
12         end
13         return result
14     end
15 end
16
17 # Phân chia data cho workers, mỗi worker dùng threads
18 data = rand(1_000_000)
19 chunks = Iterators.partition(data, 250_000)
20 results = pmap(hybrid_compute, collect(chunks))
21 final_result = vcat(results...)
```

### 10.6.4 Common pitfalls và debugging

#### 1. Race Conditions:

```
1 # SAI - Race condition
2 counter = 0
3 Threads.@threads for i in 1:1000
4     global counter += 1 # Không atomic!
5 end
6 # Kết quả < 1000
7
8 # ĐÚNG - Dùng Atomic
9 counter = Threads.Atomic{Int}(0)
10 Threads.@threads for i in 1:1000
11     Threads.atomic_add!(counter, 1)
12 end
```

## 2. Deadlocks:

```

1 # SAI - Potential deadlock
2 lock1 = ReentrantLock()
3 lock2 = ReentrantLock()
4
5 # Thread 1: lock1 rồi lock2
6 # Thread 2: lock2 rồi lock1
7 # => Deadlock!
8
9 # ĐÚNG - Luôn lock theo thứ tự nhất quán

```

## 3. False Sharing:

```

1 # SAI - False sharing (cache line contention)
2 results = zeros(Threads.nthreads())
3 Threads.@threads for i in 1:n
4     results[Threads.threadid()] += 1
5 end
6
7 # ĐÚNG - Padding để tránh cache conflicts
8 # Hoặc dùng thread-local variables

```

## 4. GPU Memory Errors:

```

1 # SAI - Out of memory
2 huge_array = CUDA.rand(100000, 100000) # 40GB!
3
4 # ĐÚNG - Kiểm tra memory trước
5 if sizeof(Float32) * 100000^2 < CUDA.available_memory()
6     huge_array = CUDA.rand(100000, 100000)
7 else
8     println("Not enough GPU memory!")
9 end

```

### 10.6.5 Best practices summary

#### General Guidelines:

1. **Profile first:** Xác định bottleneck trước khi parallelize
2. **Start simple:** Bắt đầu với @threads hoặc pmap
3. **Measure:** Luôn benchmark để verify speedup
4. **Consider overhead:** Parallelism có cost, không phải lúc nào cũng nhanh hơn

#### Threads:

- Dùng thread-local storage để tránh race conditions
- Tránh I/O trong threaded code
- Dùng @threads :dynamic khi workload không đều

#### Distributed:

- Minimize data transfer giữa workers
- Dùng `pmap` cho embarrassingly parallel tasks
- `SharedArrays` cho same-machine workers

**GPU:**

- Batch operations để amortize transfer cost
- Dùng `Float32` khi có thể (2x faster)
- Avoid scalar indexing trên `CuArrays`

## 10.7 Tóm tắt chương

- **Multithreading:** Sử dụng `Threads.@threads` để song song hóa vòng lặp trên cùng một máy với shared memory. Cần chú ý thread safety và race conditions.
- **Distributed Computing:** `Distributed.jl` cho phép chạy code trên nhiều processes/machines. Sử dụng `pmap`, `@distributed`, và `@spawnat` cho parallel tasks.
- **GPU Computing:** `CUDA.jl` cung cấp GPU acceleration cho matrix operations và data-parallel workloads. `CuArray` tự động chạy operations trên GPU.
- **Performance:** Amdahl's Law giới hạn speedup. Luôn benchmark và profile trước khi parallelize.
- **Best Practices:** Chọn model phù hợp với workload, minimize data movement, handle race conditions cẩn thận.

## 10.8 Bài tập

### 10.8.1 Bài tập cơ bản

#### Bài tập: Multithreading cơ bản

Viết hàm `parallel_sum(arr)` sử dụng `@threads` để tính tổng một array. So sánh performance với `sum()` built-in.

```

1 function parallel_sum(arr)
2     # TODO: Implement với @threads
3     # Hint: Dùng partial sums cho mỗi thread
4 end
5
6 # Test
7 data = rand(10_000_000)
8 @btime sum($data)
9 @btime parallel_sum($data)
```



**Bài tập: Distributed workers**

Thiết lập 4 distributed workers và sử dụng pmap để tính bình phương của các số từ 1 đến 1000.

```
1 using Distributed
2 # TODO: addprocs, define function @everywhere, use pmap
```

**10.8.2 Bài tập nâng cao****Bài tập: GPU matrix operations**

So sánh thời gian thực hiện matrix multiplication ( $1000 \times 1000$ ) trên CPU và GPU. Tính speedup ratio.

```
1 using CUDA, BenchmarkTools
2 # TODO: Create matrices, benchmark CPU vs GPU
3 # Report: speedup = time_cpu / time_gpu
```

**Bài tập: Parallel map-reduce**

Implement word count distributed sử dụng map-reduce pattern với pmap.

```
1 @everywhere function count_words(text)
2     # TODO: Return Dict{String, Int} với word counts
3 end
4
5 function merge_counts(d1, d2)
6     # TODO: Merge two dictionaries
7 end
8
9 # Test với multiple text chunks
```

**10.8.3 Bài tập thách thức****Bài tập: Hybrid parallelism**

Implement Monte Carlo simulation ước tính  $\pi$  sử dụng hybrid approach: distributed workers với mỗi worker sử dụng multi-threading. So sánh với pure distributed và pure threaded versions.

```
1 # Target: 1 billion samples
2 # Compare: sequential vs threads vs distributed vs hybrid
3 # Report: time, speedup, và estimated pi value
```

**Bài tập: Custom CUDA kernel**

Viết custom CUDA kernel để thực hiện element-wise operation:  $C[i] = A[i]^2 + B[i]^2 + 2A[i]B[i]$ .

```
1 function custom_kernel!(C, A, B)
2     # TODO: Implement kernel
3 end
4 # Verify kết quả so với CPU implementation
```

**Đọc thêm**

- Julia Documentation: Parallel Computing — <https://docs.julialang.org/en/v1/manual/parallel-computing/>
- CUDA.jl Documentation — <https://cuda.juliagpu.org/>
- Julia High Performance (2nd Edition) — Avik Sengupta, Packt Publishing
- JuliaGPU Organization — <https://juliagpu.org/>
- Distributed Computing in Julia — <https://docs.julialang.org/en/v1/manual/distributed-computing/>

# Lời cảm ơn

Tôi xin chân thành cảm ơn TS. Nguyễn Hoài Đức, người đã tạo cơ hội và tin tưởng giao cho tôi, Lâm Thanh Phong, cựu sinh viên và hiện đang theo học chương trình Thạc sĩ ngành Hệ thống thông tin quản lý — chuyên ngành Trí tuệ nhân tạo (Artificial Intelligence) tại Trường Đại học Ngân hàng, được tham gia đóng góp biên soạn một phần nhỏ vào cuốn sách này.

Đây là một trải nghiệm quý báu giúp tôi không chỉ nâng cao kiến thức về ngôn ngữ lập trình Julia mà còn rèn luyện kỹ năng viết học thuật và nghiên cứu khoa học. Tôi hy vọng những đóng góp của mình sẽ góp phần nhỏ bé vào việc phổ biến Julia đến cộng đồng lập trình viên và sinh viên Việt Nam.

*Lâm Thanh Phong*

*Thành phố Hồ Chí Minh, 2025*



## Phụ lục A

# Tham chiếu lệnh Julia REPL

Julia REPL (Read-Eval-Print Loop) là môi trường tương tác chính để làm việc với Julia. Phần này tổng hợp các lệnh và phím tắt quan trọng nhất.

### A.1 Các chế độ REPL

Julia REPL có bốn chế độ chính:

| Chế độ       | Phím kích hoạt | Prompt       |
|--------------|----------------|--------------|
| Julia mode   | (mặc định)     | julia>       |
| Help mode    | ?              | help?>       |
| Package mode | ]              | (@v1.x) pkg> |
| Shell mode   | ;              | shell>       |

Bảng A.1: Các chế độ của Julia REPL

Để quay về Julia mode từ bất kỳ chế độ nào, nhấn Backspace hoặc Ctrl+C.

### A.2 Phím tắt điều hướng

| Phím tắt | Chức năng                              |
|----------|----------------------------------------|
| Ctrl+C   | Hủy lệnh đang chạy / Xóa dòng hiện tại |
| Ctrl+D   | Thoát REPL (khi dòng trống)            |
| Ctrl+L   | Xóa màn hình                           |
| Ctrl+R   | Tìm kiếm ngược lịch sử                 |
| Ctrl+S   | Tìm kiếm xuôi lịch sử                  |
| Up/Down  | Duyệt lịch sử lệnh                     |
| Tab      | Tự động hoàn thành                     |
| Tab Tab  | Hiển thị tất cả gợi ý                  |

Bảng A.2: Phím tắt điều hướng REPL

### A.3 Phím tắt chỉnh sửa

### A.4 Lệnh Package Manager

Trong chế độ Package (nhấn ]):

| Phím tắt | Chức năng                    |
|----------|------------------------------|
| Ctrl+A   | Di chuyển đầu dòng           |
| Ctrl+E   | Di chuyển cuối dòng          |
| Ctrl+K   | Xóa từ con trỏ đến cuối dòng |
| Ctrl+U   | Xóa từ đầu dòng đến con trỏ  |
| Ctrl+W   | Xóa từ trước con trỏ         |
| Alt+D    | Xóa từ sau con trỏ           |
| Ctrl+Y   | Dán văn bản đã xóa           |

Bảng A.3: Phím tắt chỉnh sửa

| Lệnh            | Chức năng                     |
|-----------------|-------------------------------|
| add PackageName | Cài đặt package               |
| rm PackageName  | Gỡ bỏ package                 |
| up              | Cập nhật tất cả packages      |
| up PackageName  | Cập nhật package cụ thể       |
| st hoặc status  | Hiển thị packages đã cài      |
| activate .      | Kích hoạt project environment |
| instantiate     | Cài packages từ Project.toml  |
| precompile      | Biên dịch trước packages      |
| gc              | Dọn dẹp packages không dùng   |

Bảng A.4: Lệnh Package Manager

## A.5 Biến đặc biệt REPL

| Biến | Ý nghĩa                                 |
|------|-----------------------------------------|
| ans  | Kết quả của biểu thức cuối cùng         |
| _    | Alias cho ans (trong một số trường hợp) |
| ;    | Kết thúc lệnh, ẩn output                |

Bảng A.5: Biến đặc biệt trong REPL

## A.6 Macro hữu ích trong REPL

```

1  # Đo thời gian thực thi
2  @time sum(1:1000000)
3
4  # Đo chi tiết với allocations
5  @btime sum(1:1000000)  # Cần BenchmarkTools
6
7  # Xem kiểu dữ liệu
8  @typeof [1, 2, 3]
9
10 # Xem code được sinh ra
11 @code_lowered sum(1:10)
12 @code_typed sum(1:10)
13 @code_llvm sum(1:10)
14 @code_native sum(1:10)
15
16 # Xem documentation
17 @doc sum

```

## Phụ lục B

# Packages thông dụng

Julia có hệ sinh thái packages phong phú. Dưới đây là các packages quan trọng nhất theo từng lĩnh vực.

### B.1 Xử lý dữ liệu

| Package       | Mô tả                                   |
|---------------|-----------------------------------------|
| DataFrames.jl | Cấu trúc dữ liệu dạng bảng (như pandas) |
| CSV.jl        | Đọc/ghi file CSV hiệu suất cao          |
| JSON.jl       | Parse và sinh JSON                      |
| XLSX.jl       | Đọc/ghi file Excel                      |
| JLD2.jl       | Lưu trữ dữ liệu Julia native            |
| Arrow.jl      | Apache Arrow format                     |

Bảng B.1: Packages xử lý dữ liệu

### B.2 Trực quan hóa

| Package         | Mô tả                                   |
|-----------------|-----------------------------------------|
| Plots.jl        | Interface thống nhất cho nhiều backends |
| Makie.jl        | Đồ họa hiệu suất cao, GPU-accelerated   |
| Gadfly.jl       | Grammar of Graphics (như ggplot2)       |
| VegaLite.jl     | Vega-Lite visualizations                |
| UnicodePlots.jl | Vẽ đồ thị trong terminal                |

Bảng B.2: Packages trực quan hóa

| Package                  | Mô tả                            |
|--------------------------|----------------------------------|
| LinearAlgebra            | Đại số tuyến tính (stdlib)       |
| Statistics               | Thống kê cơ bản (stdlib)         |
| DifferentialEquations.jl | Giải phương trình vi phân        |
| Optim.jl                 | Tối ưu hóa                       |
| JuMP.jl                  | Mô hình hóa tối ưu               |
| Distributions.jl         | Phân phối xác suất               |
| SpecialFunctions.jl      | Hàm đặc biệt (Bessel, Gamma,...) |

Bảng B.3: Packages khoa học

| Package        | Mô tả                                  |
|----------------|----------------------------------------|
| Flux.jl        | Deep learning framework                |
| MLJ.jl         | Machine learning framework tổng hợp    |
| Knet.jl        | Deep learning với auto-differentiation |
| ScikitLearn.jl | Wrapper cho scikit-learn               |
| Turing.jl      | Probabilistic programming              |

Bảng B.4: Packages Machine Learning

### B.3 Khoa học và toán học

### B.4 Machine Learning

### B.5 Web và Networking

### B.6 Testing và Development

### B.7 Parallel và GPU Computing



| Package       | Mô tả                    |
|---------------|--------------------------|
| HTTP.jl       | HTTP client và server    |
| Genie.jl      | Full-stack web framework |
| Oxygen.jl     | Micro web framework      |
| WebSockets.jl | WebSocket support        |

Bảng B.5: Packages Web

| Package           | Mô tả                           |
|-------------------|---------------------------------|
| Test              | Unit testing (stdlib)           |
| BenchmarkTools.jl | Micro-benchmarking chính xác    |
| Revise.jl         | Hot-reload code khi phát triển  |
| Debugger.jl       | Interactive debugger            |
| ProfileView.jl    | Visualization profiling results |
| JET.jl            | Static code analysis            |

Bảng B.6: Packages Development

| Package               | Mô tả                          |
|-----------------------|--------------------------------|
| Distributed           | Distributed computing (stdlib) |
| CUDA.jl               | NVIDIA GPU computing           |
| AMDGPU.jl             | AMD GPU computing              |
| Metal.jl              | Apple Silicon GPU              |
| KernelAbstractions.jl | Portable GPU kernels           |
| ThreadsX.jl           | Parallel algorithms            |

Bảng B.7: Packages Parallel Computing



# Phụ lục C

## Ký hiệu Unicode trong Julia

Julia hỗ trợ Unicode một cách toàn diện, cho phép sử dụng các ký hiệu toán học trực tiếp trong code. Dưới đây là các ký hiệu thường dùng và cách nhập chúng trong REPL.

### C.1 Chữ cái Hy Lạp

| Ký hiệu    | Tên     | LaTeX                 | Ký hiệu  | Tên   | LaTeX               |
|------------|---------|-----------------------|----------|-------|---------------------|
| $\alpha$   | alpha   | <code>\alpha</code>   | $\nu$    | nu    | <code>\nu</code>    |
| $\beta$    | beta    | <code>\beta</code>    | $\xi$    | xi    | <code>\xi</code>    |
| $\gamma$   | gamma   | <code>\gamma</code>   | $\pi$    | pi    | <code>\pi</code>    |
| $\delta$   | delta   | <code>\delta</code>   | $\rho$   | rho   | <code>\rho</code>   |
| $\epsilon$ | epsilon | <code>\epsilon</code> | $\sigma$ | sigma | <code>\sigma</code> |
| $\zeta$    | zeta    | <code>\zeta</code>    | $\tau$   | tau   | <code>\tau</code>   |
| $\eta$     | eta     | <code>\eta</code>     | $\phi$   | phi   | <code>\phi</code>   |
| $\theta$   | theta   | <code>\theta</code>   | $\chi$   | chi   | <code>\chi</code>   |
| $\lambda$  | lambda  | <code>\lambda</code>  | $\psi$   | psi   | <code>\psi</code>   |
| $\mu$      | mu      | <code>\mu</code>      | $\omega$ | omega | <code>\omega</code> |

Bảng C.1: Chữ cái Hy Lạp thường

Chữ cái Hy Lạp in hoa: thêm hậu tố viết hoa, ví dụ `\Gamma`  $\rightarrow$   $\Gamma$ , `\Delta`  $\rightarrow$   $\Delta$ .

### C.2 Ký hiệu toán học

| Ký hiệu              | Tên            | LaTeX input                     |
|----------------------|----------------|---------------------------------|
| $\infty$             | Vô cực         | <code>\infty</code>             |
| $\partial$           | Đạo hàm riêng  | <code>\partial</code>           |
| $\nabla$             | Nabla/Gradient | <code>\nabla</code>             |
| $\sum$               | Tổng           | <code>\sum</code>               |
| $\prod$              | Tích           | <code>\prod</code>              |
| $\int$               | Tích phân      | <code>\int</code>               |
| $\sqrt{\phantom{x}}$ | Căn bậc hai    | <code>\sqrt{\phantom{x}}</code> |

Bảng C.2: Ký hiệu toán học

| Ký hiệu  | Nghĩa       | LaTeX               | Ký hiệu     | Nghĩa               | LaTeX                  |
|----------|-------------|---------------------|-------------|---------------------|------------------------|
| $\leq$   | $\leq$      | <code>\leq</code>   | $\approx$   | Xấp xỉ              | <code>\approx</code>   |
| $\geq$   | $\geq$      | <code>\geq</code>   | $\equiv$    | Đồng nhất           | <code>\equiv</code>    |
| $\neq$   | Khác        | <code>\neq</code>   | $\sim$      | Tương tự            | <code>\sim</code>      |
| $\in$    | Thuộc       | <code>\in</code>    | $\subset$   | Tập con             | <code>\subset</code>   |
| $\notin$ | Không thuộc | <code>\notin</code> | $\subseteq$ | Tập con $\subseteq$ | <code>\subseteq</code> |

Bảng C.3: Ký hiệu quan hệ

| Ký hiệu           | Tên               | LaTeX input                                    |
|-------------------|-------------------|------------------------------------------------|
| $\rightarrow$     | Mũi tên phải      | <code>\rightarrow</code> hoặc <code>\to</code> |
| $\leftarrow$      | Mũi tên trái      | <code>\leftarrow</code>                        |
| $\leftrightarrow$ | Mũi tên hai chiều | <code>\leftrightarrow</code>                   |
| $\Rightarrow$     | Suy ra            | <code>\Rightarrow</code>                       |
| $\Leftrightarrow$ | Tương đương       | <code>\Leftrightarrow</code>                   |
| $\mapsto$         | Ánh xạ            | <code>\mapsto</code>                           |

Bảng C.4: Ký hiệu mũi tên

C.3 Ký hiệu quan hệ

C.4 Mũi tên

C.5 Subscript và Superscript

Julia cho phép dùng subscript và superscript Unicode cho tên biến:

```
1 # Subscript: _0, _1, _2, ... _9, _a, _b, ...
2 x□ = 0      # \_0 + Tab
3 x□ = 1      # \_1 + Tab
4 x□ = 10     # \_n + Tab
5
6 # Superscript: ^0, ^1, ^2, ...
7 x² = 4      # \^2 + Tab
8 x³ = 8      # \^3 + Tab
9
10 # Kết hợp
11 a□□ = 5     # a\_i + Tab + \_j + Tab
```

C.6 Cách nhập Unicode trong REPL

- 1. Gõ dấu backslash (\) theo sau là tên LaTeX
- 2. Nhấn Tab để chuyển thành ký hiệu Unicode
- 3. Ví dụ: `\alpha + Tab`  $\rightarrow \alpha$

```
1 # Ví dụ sử dụng Unicode trong code thực tế
2 function sphere_volume(r)
3     V = (4/3) * π * r^3
4     return V
5 end
6
7 # Phương trình vật lý
```

```
8  E = m * c^2  # Tương đương E = m * c^2
9
10 # Tọa độ
11 Δx = x□ - x□
12 Δt = t□ - t□
13 v = Δx / Δt
```



## Phụ lục D

# Chuyển đổi từ Python sang Julia

Chương này hướng dẫn lập trình viên Python chuyển sang Julia, so sánh cú pháp và cách tiếp cận giữa hai ngôn ngữ.

### D.1 So sánh cú pháp cơ bản

| Thao tác           | Python       | Julia          |
|--------------------|--------------|----------------|
| Comment            | # comment    | # comment      |
| Multi-line comment | '''...'''    | #= ... =#      |
| Print              | print("Hi")  | println("Hi")  |
| Range              | range(1, 10) | 1:9            |
| List/Array         | [1, 2, 3]    | [1, 2, 3]      |
| Dictionary         | {"a": 1}     | Dict{"a" => 1} |
| Tuple              | (1, 2, 3)    | (1, 2, 3)      |
| Boolean            | True, False  | true, false    |
| None/Nothing       | None         | nothing        |

Bảng D.1: So sánh cú pháp cơ bản

### D.2 Indexing và Slicing

| Thao tác       | Python   | Julia        |
|----------------|----------|--------------|
| Phần tử đầu    | arr[0]   | arr[1]       |
| Phần tử cuối   | arr[-1]  | arr[end]     |
| Slice          | arr[1:3] | arr[2:3]     |
| Slice từ đầu   | arr[:3]  | arr[1:3]     |
| Slice đến cuối | arr[2:]  | arr[3:end]   |
| Bước nhảy      | arr[::2] | arr[1:2:end] |

Bảng D.2: So sánh Indexing (Python 0-based, Julia 1-based)

**Lưu ý quan trọng:** Julia dùng 1-based indexing, Python dùng 0-based.

| Python                                                                                                       | Julia                                                                                                                 |
|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <pre> if x &gt; 0:     print("positive") elif x &lt; 0:     print("negative") else:     print("zero") </pre> | <pre> if x &gt; 0     println("positive") elseif x &lt; 0     println("negative") else     println("zero") end </pre> |
| <pre> for i in range(5):     print(i) </pre>                                                                 | <pre> for i in 0:4     println(i) end </pre>                                                                          |
| <pre> while x &gt; 0:     x -= 1 </pre>                                                                      | <pre> while x &gt; 0     x -= 1 end </pre>                                                                            |

Bảng D.3: So sánh Control Flow

## D.3 Control Flow

## D.4 Hàm

## D.5 Comprehensions

## D.6 Classes vs Structs

## D.7 Sử dụng thư viện Python trong Julia

Julia có thể gọi code Python thông qua package `PyCall.jl`:

```

1  using Pkg
2  Pkg.add("PyCall")
3
4  using PyCall
5
6  # Import module Python
7  np = pyimport("numpy")
8  pd = pyimport("pandas")
9
10 # Sử dụng numpy
11 arr = np.array([1, 2, 3, 4, 5])
12 println(np.mean(arr)) # 3.0
13
14 # Sử dụng pandas
15 df = pd.DataFrame(Dict("a" => [1,2,3], "b" => [4,5,6]))
16 println(df.describe())
17

```



| Python                                                                        | Julia                                                                                  |
|-------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <pre>def add(a, b):     return a + b</pre>                                    | <pre>function add(a, b)     return a + b end # hoặc add(a, b) = a + b</pre>            |
| <pre># Lambda f = lambda x: x**2</pre>                                        | <pre># Anonymous function f = x -&gt; x^2</pre>                                        |
| <pre># Default args def greet(name="World"):     print(f"Hello {name}")</pre> | <pre># Default args function greet(name="World")     println("Hello \$name") end</pre> |

Bảng D.4: So sánh định nghĩa hàm

| Python                                                      | Julia                                                          |
|-------------------------------------------------------------|----------------------------------------------------------------|
| <pre># List comprehension [x**2 for x in range(10)]</pre>   | <pre># Array comprehension [x^2 for x in 0:9]</pre>            |
| <pre># Dict comprehension {x: x**2 for x in range(5)}</pre> | <pre># Dict comprehension Dict{x =&gt; x^2 for x in 0:4}</pre> |
| <pre># Generator (x**2 for x in range(10))</pre>            | <pre># Generator (x^2 for x in 0:9)</pre>                      |

Bảng D.5: So sánh Comprehensions

```

18 # Gọi hàm Python tùy chỉnh
19 py"""
20 def python_function(x):
21     return x ** 2 + 1
22 """
23 result = py"python_function"(5)
24 println(result) # 26

```

## D.8 Khác biệt quan trọng cần nhớ

1. **Indexing:** Julia là 1-based, Python là 0-based
2. **End keyword:** Julia dùng end để kết thúc blocks
3. **Mutability:** Mặc định struct trong Julia là immutable

| Python (OOP)                                                                                                                                                                                                                                                                    | Julia (Multiple Dispatch)                                                                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class Point:     def __init__(self, x, y):         self.x = x         self.y = y      def distance(self, other):         dx = self.x - other.x         dy = self.y - other.y         return sqrt(dx**2 + dy**2)  p1 = Point(0, 0) p2 = Point(3, 4) p1.distance(p2) </pre> | <pre> struct Point     x::Float64     y::Float64 end  function distance(p1::Point, p2::Point)     dx = p1.x - p2.x     dy = p1.y - p2.y     return sqrt(dx^2 + dy^2) end  p1 = Point(0, 0) p2 = Point(3, 4) distance(p1, p2) </pre> |

Bảng D.6: So sánh OOP Python với Multiple Dispatch Julia

4. **Broadcasting:** Julia dùng `.` để broadcast: `sin.(arr)`
5. **Scope:** Biến trong loop Julia có local scope
6. **String interpolation:** Julia dùng `$`, Python dùng f-string
7. **Division:** `5/2` trong Julia cho `2.5`, trong Python 3 cũng vậy
8. **Power:** Julia dùng `^`, Python dùng `**`
9. **Column-major:** Julia lưu matrix theo cột (như Fortran), Python/NumPy theo hàng

# Tài liệu tham khảo

- [1] J. Bezanson, S. Karpinski, V. B. Shah, and A. Edelman, “Julia: A fast dynamic language for technical computing,” in *Lang.NEXT 2012*, 2012.
- [2] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, “Julia: A fresh approach to numerical computing,” *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017.
- [3] J. M. Perkel, “Julia: Come for the syntax, stay for the speed,” *Nature*, vol. 572, pp. 141–142, 2019.
- [4] Julia Language, “The julia language documentation,” 2024.
- [5] Julia Language, “Modules — julia documentation,” 2024.